

FPGA STORY 《工具篇 I》

静态时序分析 & TimeQuest

芯驿电子科技（上海）有限公司

[Http://www.alinx.cn](http://www.alinx.cn)

ALINX 系列教程

追求精品 宁缺莫乱



黑金动力社区
荣誉呈献
www.heijin.org

目录

前言.....	3
第一章：TimeQuest 静态时序分析模型.....	4
1.1 模型的基础单位.....	4
1.2 理想的建立关系值和保持关系值.....	6
1.3 物理中的建立关系过程和建立余量.....	9
1.4 令人误会的 建立关系 和 建立余量。.....	14
1.5 建立余量.....	15
1.6 保持余量.....	22
总结：.....	32
第二章：模型角色，网表概念，时序报告.....	34
2.1 TimeQuest 模型的努力.....	34
2.2 SDC 的网表（ Net Lists ）基本单位.....	35
2.3 TimeQuest 模型的简单实例.....	38
实验一： 入门实验.....	38
2.4 进一步看看 TimeQuest 模型如何分析时序.....	42
2.5 建立 sdc 文件与网表.....	47
2.6 约束时钟.....	56
2.7 时序报告.....	60
总结：.....	68
第三章：TimeQuest 扫盲文.....	69
3.1 余量的概念.....	69
3.2 延迟因数：内部延迟因数.....	73
3.3 延迟因数：外部延迟因数.....	77
3.4 第一层与第二层时间要求.....	81
3.5 山寨 TimeQuest 模型.....	87
3.6 约束命令和约束行为.....	90
总结：.....	91
第四章：内部延迟与其他.....	92
4.1 PLL 与约束命令.....	92
4.2 约束 PLL 时钟.....	94
实验二 vga 模块的物理时序分析.....	94
4.3 延迟怪兽①.....	101
实验三 延迟怪兽的暴动.....	103
Delay_monster.v.....	104
4.4 延迟怪兽②.....	112
实验四 约束延迟怪兽.....	112
4.5 Set Multicycle Path 的噩梦.....	122
4.6 不可理喻的 Set False Path.....	128
实验五 延迟怪兽受死吧！.....	129

总结：	132
第五章：网表质量与外部模型	133
5.1 网表质量的概念	133
5.2 Fmax 评估值	140
5.3 外部模型①	145
5.4 外部模型②	155
5.5 推导外包资料的公式	164
5.6 外部模型的 input/output 约束指令	171
实验六 假想实验 input/output delay 约束	172
总结：	179
第六章：物理时钟与外部模型	180
6.1 物理时钟①	180
实验七：set_clock_latency 告诉 TimeQuest 外部时钟延迟信息	184
6.2 物理时钟②	191
实验八 时钟抖动与 set_clock_latency	191
6.3 物理时钟③	195
实验九 时钟抖动与 input/output 约束命令	203
6.4 物理时钟④	207
实验十 时间抖动与 set_clock_uncertainly	207
6.5 无奈的 set_max_delay 与 set_min_delay	213
总结：	216
第七章：供源时钟与其他	217
7.1 供源时钟①	217
实验十一 供源时钟与 Create Generated Clock	217
7.2 供源时钟②	224
实验十二： 供源时钟与寄存器与 create_generated_clock	225
7.3 供源时钟③	230
实验十三 供源时钟与 pll 与 create generated clock	231
7.4 引脚电容性与上下坡信号	236
7.5 介电参数与延迟时间	241
7.6 IO 口	242
实验十四 分析和约束 io 口外部模型	242
7.7 SDRAM 实例	247
实验十五 SDRAM 约束实例	249
总结：	260
后语	261

前言

这是笔者用两年构思准备一年之久的笔记，其实这也是笔者的另一种挑战。写《工具篇 I》不像写《Verilog HDL 那些事儿》系列的笔记一样，只要针对原理和 HDL 内容作出解释即可，虽然《Verilog HDL 那些事儿》夹杂着许多笔者对 Verilog 的独特见解，不过这些内容都可以透过想象力来弥补。然而《工具篇 I》需要一定的基础才能书写。

两年前，编辑《时序篇》之际，笔者忽然对 TimeQuest 产生兴趣，可是笔者当时却就连时序是什么也不懂，更不明白时序有理想和物理之分，为此笔者先着手理想时序的研究。一年后，虽然已掌握解理想时序，但是笔者始终觉得理想时序和 TimeQuest 之间缺少什么，这种感觉就像磁极不会没有原因就相互吸引着？于是漫长的思考就开始了 ... 在不知不觉中就写出《整合篇》。

HDL 描述的模块是软模型，modelsim 仿真的软模型是理想时序。换之，软模型经过综合器总综合以后就会成为硬模型，也是俗称的网表。而 TimeQuest 分析的对象就是硬模型的物理时序。理想时序与物理时序虽然与物理时序有显明的区别，但它们却有黏糊的关系，就像南极和北极的磁性一样相互作用着。

编辑《工具篇 I》的过程不也是一番风顺，其中也有搁浅或者灵感耗尽的情况。《工具篇 I》给笔者最具挑战的地方就是如何将抽象的概念，将其简化并且用语言和图形表达出来。读者们可要知道《工具篇 I》使用许多不曾出现在常规书的用词与概念 ... 但是，不曾出现并不代表它们不复存在，反之如何定义与实例化它们让笔者兴奋到夜夜失眠。

《工具篇 I》的书写方式依然继承笔者往常的笔记风格，内容排版方面虽然给人次序不一的感觉，不过笔者认为这种次序对学习有最大的帮助。编辑《工具篇 I》辛苦归辛苦，但是笔者却很热衷，心情好比小时候研究新玩具一般，一边好奇一边疑惑，一边学习一边记录。完成它让笔者有莫大的愉快感，想必那是笔者久久不失的童心吧！？

(akuei2 上 30-09-2013)

第一章：TimeQuest 静态时序分析模型

兜兜转转，然后又转转兜兜，最终还是需要站在 TimeQuest 的大门口。先自白一下，接下来的故事都是一位傻子对 TimeQuest 的理解，这个傻子的脑容量有限，处理能力不好，空白的地方都会用想象力自圆。

TimeQuest 是 Altera - Quartus II 旗下的东西，其中它（TimeQuest）用了一个“**时序分析模型**”作为物理时序的分析“框子”或者“数学模型”。在这里笔者需要强调一下，TimeQuest 中的“**时序**”和 Verilog HDL 中出现的时序是两个东西。所谓 HDL 的时序是指“**模块的活动记录**”等理想的波形；反之 TimeQuest 中出现的时序是指“**物理上路径，各种延迟问题**”的不理想波形。

让我们暂时把 Verilog HDL 忘了吧 ... 一般上分析物理时序有用一套方法称为“**静态时序分析**”的方法，关于静态时序分析网上有太多的解释了，但是 TimeQuest 有属于自己一套“**分析时序的模型**”，所以我们可以“**无视**”其它。

1.1 模型的基础单位

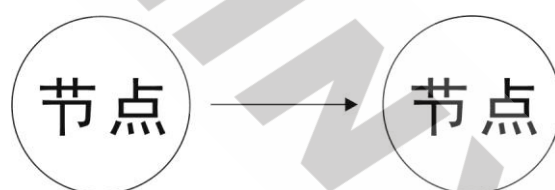


图 1.1.1 模型的基础

TimeQuest 的时序分析模型（以下简称模型）的基础，基本上需要两个节点才能发挥，如图 1.1.1 所示。节点的定义很模糊，**因为节点本身没有个性，它处在什么就变什么** ... 如果用 TimeQuest 的话来讲，将一对节点**放在 fpga 内部，节点就会化身成为寄存器**。这也节点最常见的姿态。所以说，暂时让我们把节点看成寄存器吧！

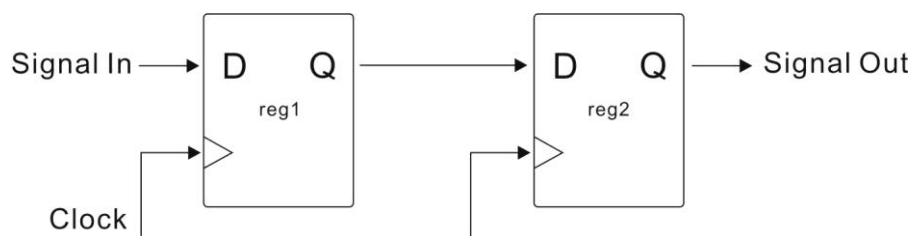


图 1.1.2 寄存器为节点

图 1.1.2 是把另一个节点看成寄存器，亦即寄存器 1（reg1）和寄存器 2（reg2）。寄存器 1 和寄存器 2 共享同样的时钟信号。Signal（信号）从寄存器 1 的 D 进入，再由寄存器 1

的 Q 游走，然后从寄存器 2 的 D 进入，最后再经寄存器 2 的 Q 出走。上述的寄存器活动，按“理想时序”会产生图 1.1.3 的时序图。

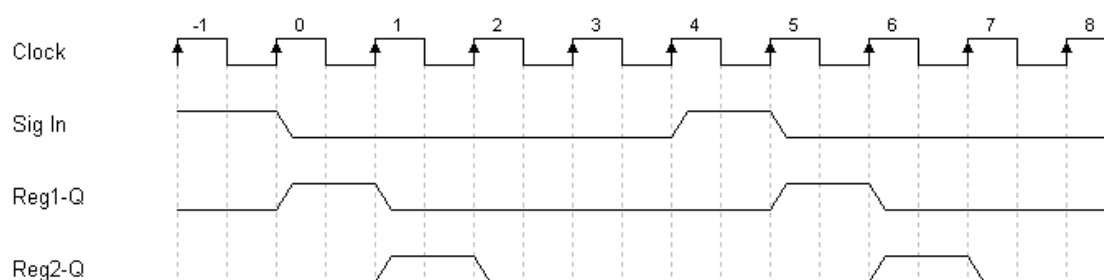


图 1.1.3 理想时序

如图 1.1.3 所示 ... 所谓理想——就是“**不存在任何物理上的延迟**”，T-1 之际是初始化状态，Sig In 的初值为 1，然后在 T0 的时候寄存器 1 读取 Sig In 的过去值，亦即逻辑 1。在 T0 的未来寄存器 1 输出逻辑 1 长达一个时钟周期。在 T1 的时候，寄存器 2 读取 Reg1-Q 的过去值，亦即逻辑 1，然后在 T1 的未来寄存器 2 拉高输出长达一个时钟周期。

根据“模块沟通”而言，每一个寄存器至少需要消耗 1 个时钟，在此 Sig In 经过两个寄存器，也就是它延迟了 2 两个时钟，从最开始到最后的 Reg2-Q 输出。

“理想时序和 TimeQuest 模型到底有什么关系？”——笔者一直在苦恼这个问题。要完全理解 TimeQuest 的模型，理想时序作的角色有“参考”和“比较”的作用。在前面笔者说过了，TimeQuest 的模型需要一对节点才能成立，这是铁一般的基础，但是 TimeQuest 模型它才不在乎时序的整个过程，而它只在乎“Reg1 发送数据给 Reg2 然后 Reg2 读取数据这一段时间”而已。

TimeQuest 模型中有两个关键关系，亦即建立关系和保持关系。老实说要很好的明白它们之间的定义，还是真是不简单，笔者也耗了很长的时间去整理思想，然后才能得出零星的总结。在此，我们需要借助“理想时序”的力量，唯有如此才能更好了解 TimeQuest 模型。

1.2 理想的建立关系值和保持关系值

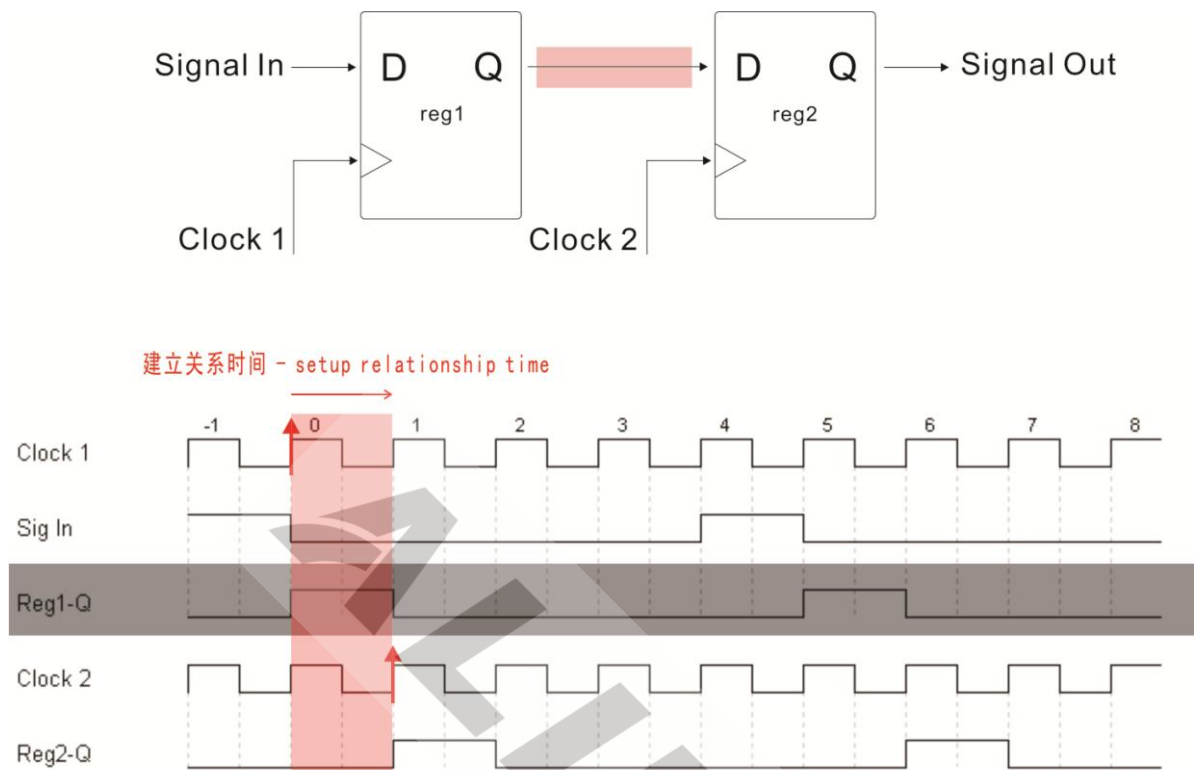


图 1.2.1 理想的建立关系值

图 1.2.1 的一对节点,该 reg1 和 reg2 都有自己专属的时钟信号,亦即 Clock1 和 Clock2。在 T0 的时候,reg1 决定向 reg2 发送数据,同学们请注意时序图的箭头“→”,该数据长达一个周期时间,这段时间也称为“建立关系值”或者“建立关系时间”。

“建立关系值”的定义是指——在 T0 的时候 Clock1 的上升沿使得 reg1 给 reg2 输出数据,这个上升沿也称为“启动沿 - lunch edge”。在 T1 的时候,Clock2 的上升沿使得 reg2 保存 reg1 发过来的数据,这个 Clock2 的上升沿也称为“锁存沿 - latch edge”然而该启动沿与锁存沿之间的长度就是“建立关系值|建立关系时间”。

在理想的状态下,如果 reg1 和 reg2 使用相同频率的时钟,那么:

建立关系值 = 时钟周期

假设 Clock1 和 Clock2 都是 20Mhz 频率,那么建立关系值是 50ns。

既然有建立关系值当然也有保存关系值,但是关于保存关系值的概念确实会比较头疼,笔者抓到头秃了 ... 要理解保存关系值,基本上又要从锁存时间开明白。所谓锁存时间就是 reg2 用来锁存 reg1 发送过来数据所需的“最小时间”。在理想的状态下,锁存时

间几乎是 0 或者 瞬间。

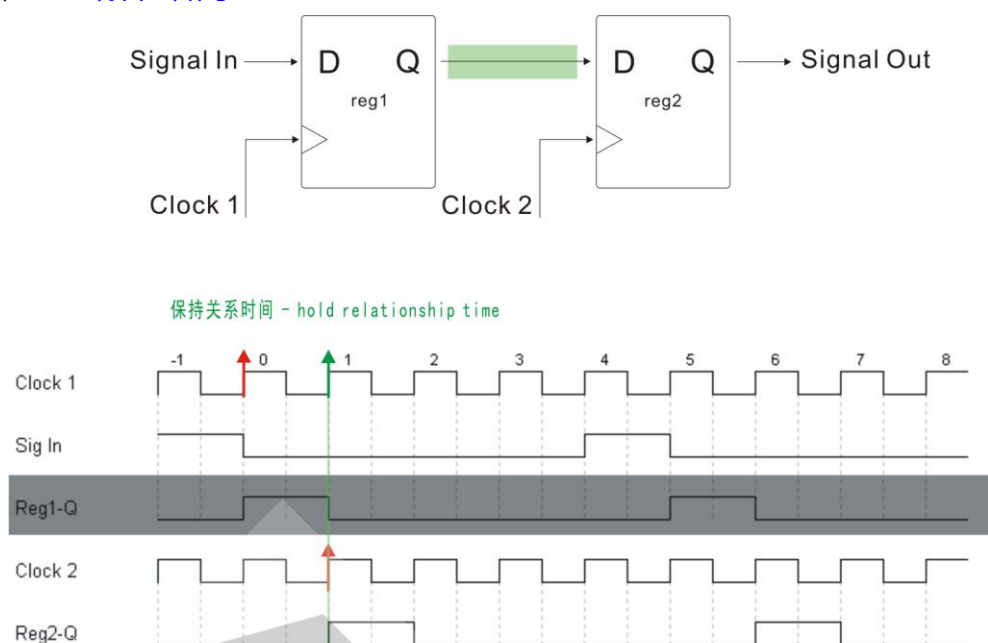


图 1.2.2 保持关系值

如图 1.2.1 所示，在 T_0 的时候 Clock1 的启动沿（红色的箭头↑）使得 reg1 向 reg2 发送数据。然后在 T_1 的时候（我们先看 Clock2 的红色键头↑）Clock2 的锁存沿使得 reg2 保存 reg1 发送过来的数据。

保持关系的定义是指——Clock1 的另一个上升沿来到之前（Clock1 的绿色箭头），reg2 要最快的速度将数据读取。换句话说，reg1 在 T_0 向 reg2 发送的数据，而 reg2 乘数据还没有更新之前，reg2 必须迅速的将 reg1 在 T_0 发送过来的数据保存起来。（在理想的时序下 reg2 的锁存所需的时间是 0）

所以说，我们可以这样决定保持关系值，亦即“Clock1 的下一个启动沿 至 当前 Clock2 的锁存沿 之间的长度”。在理想的状态下，如果 reg1 和 reg2 使用相同的时钟频率，那么一定：

保持关系值 = 0ns

在这里，笔者需要再强调一下，在实际的物理情况下不可能会有 0ns 的保持关系值，然而 reg2 锁存数据所使用的时间也不会是 0ns。同学们需要好好的记住，理想时序仅是一种直观的表达而已。

有些朋友会纳闷道：“建立关系值和想保持关系值到底在 TimeQuest 模型中扮演什么有角色？”呵呵，用处可真大了 ... 笔者根据自己的理解，它们有 3 大用处：

- （一）对物理时序分析有直观的表达
- （二）简化 TimeQuest 模型分析

(三) 为物理时序取等价关系图



1.3 物理中的建立关系过程和建立余量

建立余量是什么？在建立关系的过程中，“**正值的建立余量**”代表了“**两个节点之中的建立关系是合格**”。让我们一步一步来理解吧：

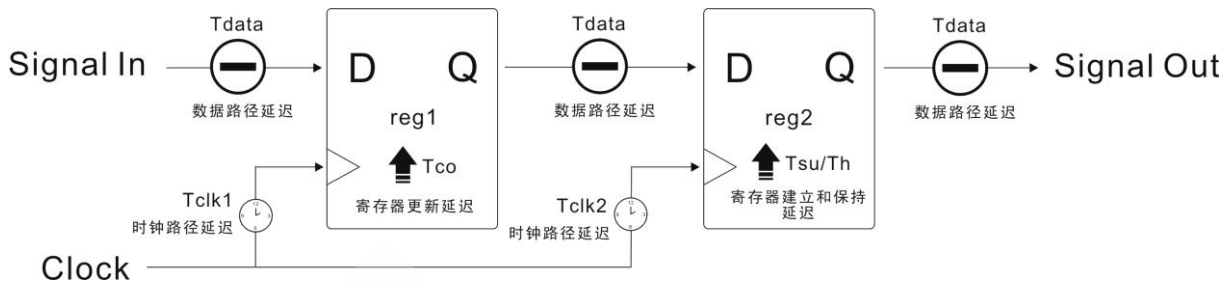


图 1.3.1 物理上的延迟

图 1.3.1 所示，有一对节点上出现许多骇人的东西，这就是我们所谓的物理时序，而那些骇人的东西就是内部延迟因数。物理时序和理想时序就像水火不容的好兄弟，不过它们再怎么糟，身上也有同样的血亲（黏糊关系）。**物理时序的产生是 Verilog HDL 经过综合后（FPGA 布线完成后）才会产生**，然而理想时序是 Verilog HDL 所描述模块会出现的“**活动记录**”或者“**沟通记录**”。

我们来看看图 1.3.1 有什么样的延迟因数呢？Tclk1 和 Tclk2 本是时钟路径延迟，别名 clock path；Tdata 是指数据路径延迟，也称 data path；Tco 是寄存器发送数据时需要的“**最小持续（更新）时间**”，Tsu 是寄存器在锁存数据前时需要的“**最小准备时间**”，反之 Th 是寄存器在锁存数据过后需要的“**最小确保时间**”，而 Tco，Tsu 和 Th 也称为**寄存器特性**。

在图 1.3.1 出现的延迟因数会大大的影响时序表现，尤其是“**建立关系和保持关系**”。不过，不同的延迟因数都会出现在特定的场合。换句话说，当我们分析建立余量的时候，必定会出现某个特定的延迟因素；然后，在我们分析保持余量的时候，又会某个特定的延迟因数。

好了，我们先来看看在建立关系的过程中，会出现什么延迟因数呢？

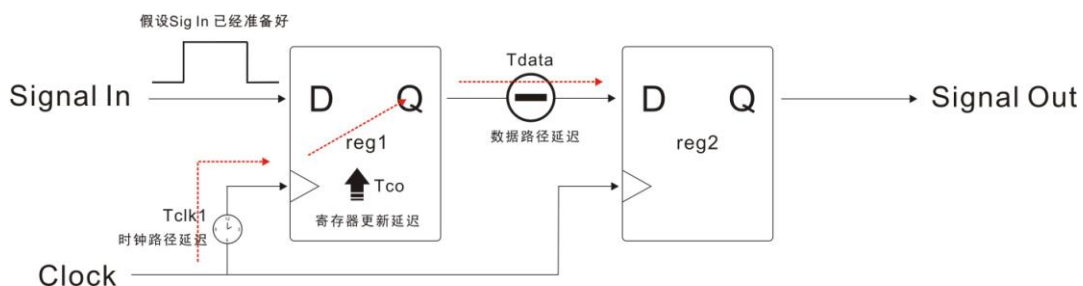


图 1.3.2 延迟因素影响建立关系

在前面笔者已经说过，计算建立关系的长度（建立关系值）需要“先”考虑 reg1 向 reg2 发送数据的启动沿。理想情况 reg1 会无视一切延迟因数，反之物理情况 reg1 发送数据给 reg2 的时候就必须考虑 $T_{clk1} + T_{co} + T_{data}$ 的等延迟。

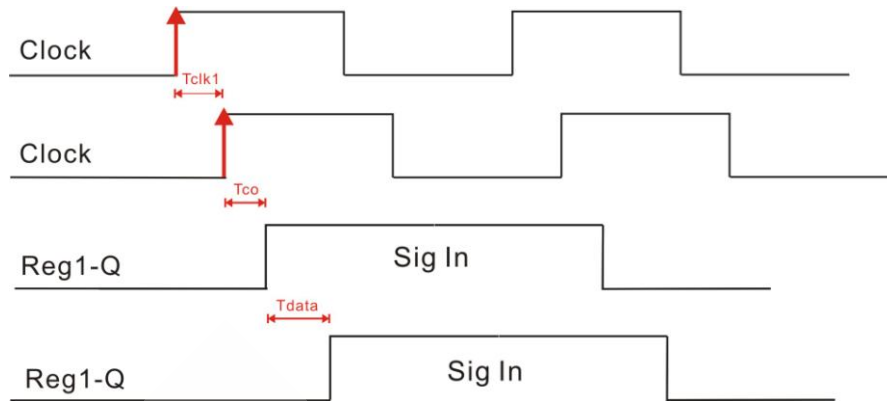


图 1.3.3 被延迟因素影响（物理状态）

图 1.3.3 是 reg1 发送数据时受延迟因数影响的经过 ... Clock 的上升沿来到 reg1 之前，clock 受到 T_{clk1} 的延迟 ... 当 clock 的启动沿（红色箭头）触发 reg1 reg1 向 reg2 发送数据之际，reg1 还要需要 T_{co} 的热身时间（更新数据）。当经过 T_{co} 的延迟之后，数据终于从 Reg1-Q 发送出去了 ...

可以在它前面的还有 T_{data} 的延迟。 T_{data} 延迟的产生源除了路径本身的延迟以外，最主要还是“组合逻辑”。最后 reg1 向 reg2 发送的数据经过 T_{data} 的延迟之后就抵达 reg2 的入口（reg2-D）。

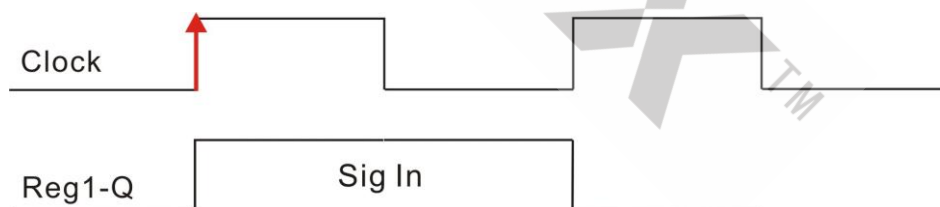


图 1.3.4 理想状态

相反的图 1.3.4 是理想时序而里边不包含何延迟。当 Clock 的启动沿触发 reg1 以后，reg1 就会立即输出数据。而已，还有锁存沿这一方。如果用了一条公式评估 $T_{clk1} + T_{co} + T_{data}$ 对启动沿的影响：

数据抵达时间 Data Arrival Time = 启动沿时间 + T_{clk1} 延迟 + T_{co} 延迟 + T_{data} 延迟

这公式的定义很简单：

“在分析建立关系的过程中，假设 reg1 向 reg2 在某个启动沿 T 发送数据，当经过 $T_{clk1} + T_{co} + T_{data}$ 等延迟因素，而该启动沿 T 会产生多少位移呢？”

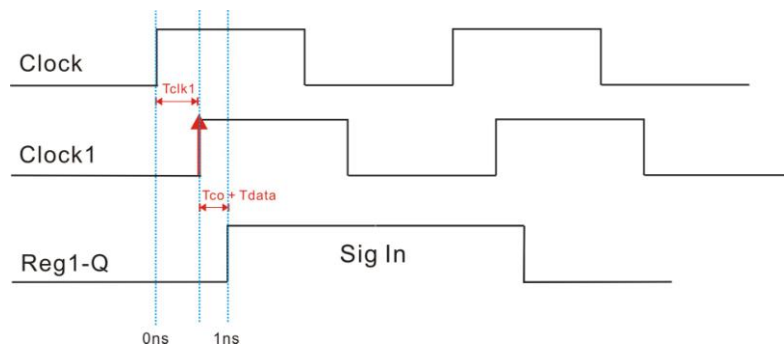


图 1.3.5 例子

举个例子，假设 reg1 在 0ns 的时候向 reg2 发送数据，而 $T_{clk1} + T_{co} + T_{data}$ 等总和延迟时间是 1ns，那么启动沿会被位移至 1ns，如图 1.3.5 所示。在前面笔者说过，如果要分析建立关系值，单是评估延迟因数对启动沿的影响是不足的，因为造就建立关系，除了启动沿以外，还有锁存方（reg2）的锁存沿。

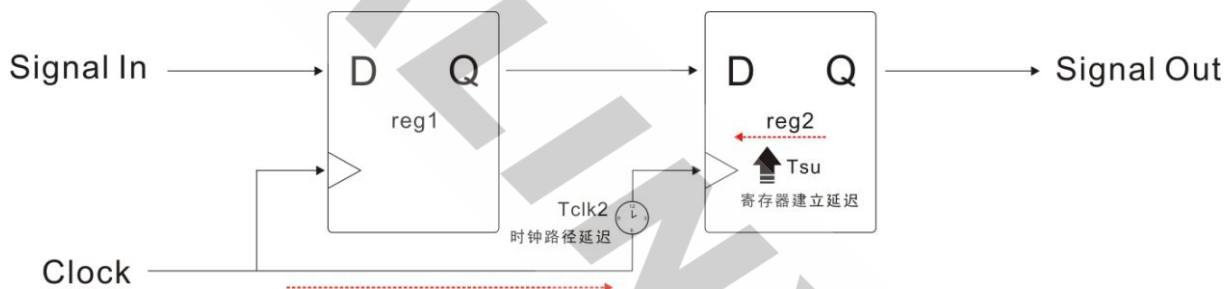


图 1.3.6 锁存沿受延迟因数的影响

要明白锁存方如何造就建立关系的结果（建立关系值），我们考虑考虑锁存方的寄存器特性还有锁存方的时钟路径，如图 1.3.6 所示的 T_{clk2} 和 T_{su} 。 T_{clk2} 是锁存沿抵达 reg2 之前，时钟路径的延迟。反之 T_{su} 是在 reg2 要锁存从 reg1 发来的数据之前，需要的最小准备时间。

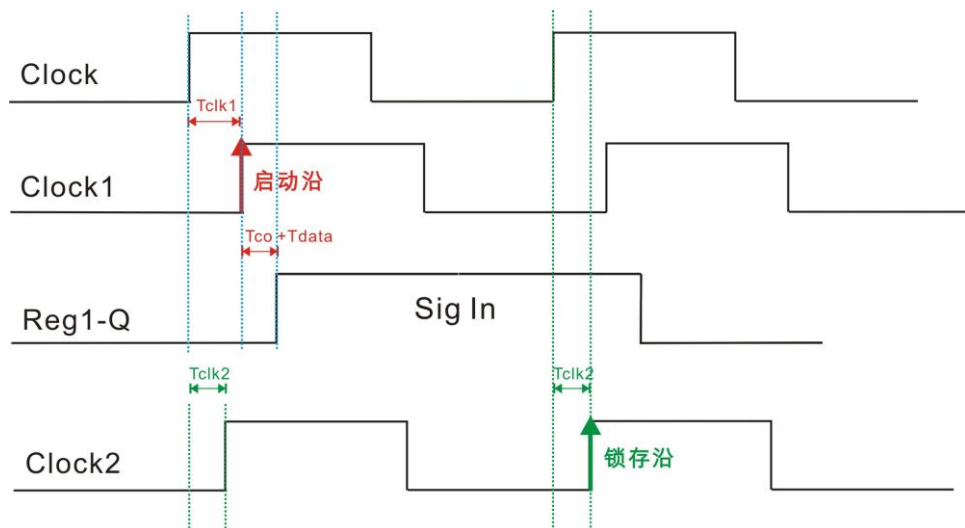


图 1.3.7

图 1.3.7 是建立关系考虑锁存方的物理时序。Clock 是源时钟 ,又或者理想时钟 ,而 Clock1 是经过 T_{clk1} 延迟以后, reg1 的时钟源 (注意那个启动沿); Clock2 是经过 T_{clk2} 延迟以后, reg2 的时钟源 (注意那个锁存沿), 而 T_{clk2} 延迟造成了 reg2 时钟源的位移。clock2 经过 T_{clk2} 延迟之后 reg2 终于迎来了自己的锁存沿 ... 接下我们要注意的家伙是 T_{su} , 亦即锁存方的寄存性特性。

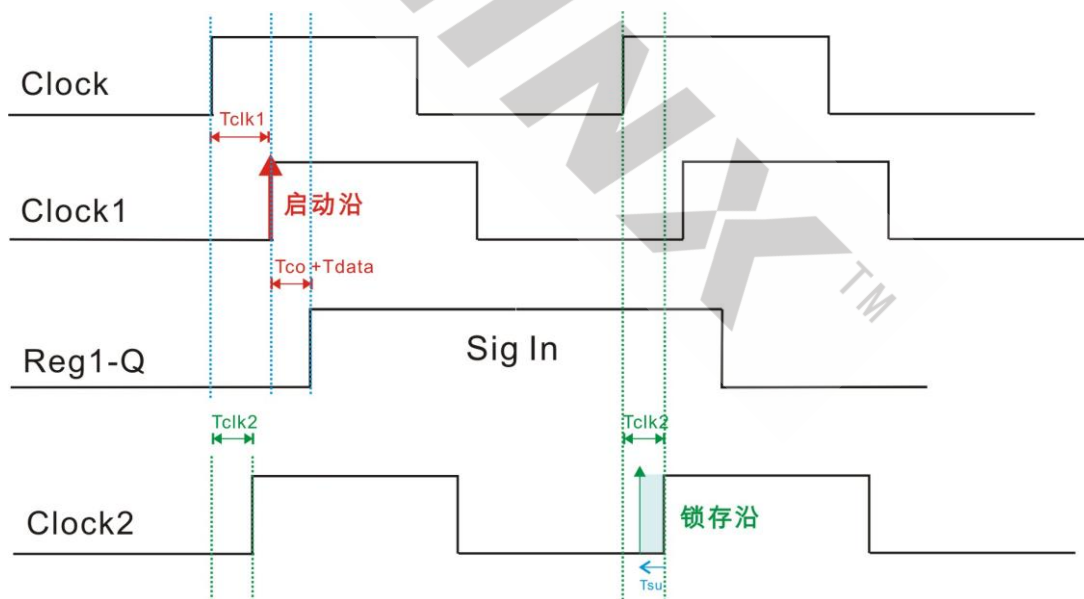


图 1.3.8 T_{su} 延迟因素的影响

T_{su} 的定义是指“寄存器 2 在锁存之前需要的最小准备时间”。如图 1.3.6 所示, T_{su} 的箭头方向与 T_{clk2} 的箭头方向是相冲的, 这也使得 T_{su} 将锁存沿左移, 随之造就图 1.3.8 的相冲结果。同样 TimeQuest 模型也用了一条公式来评估 锁存沿受 T_{clk2} 和 T_{su} 的影响:

数据锁存（读取|保存）时间 Data Required Time = 锁存沿时间 + Tclk2 - Tsu

这一条公式的定义很简单：

“在分析建立关系的过程中，锁存时间就是 reg2 欲读取数据之际，而锁存沿受 Tclk2 影响影响以后会右移几点，接着 Tsu 再将锁存沿左移几点”

假设一个例子：

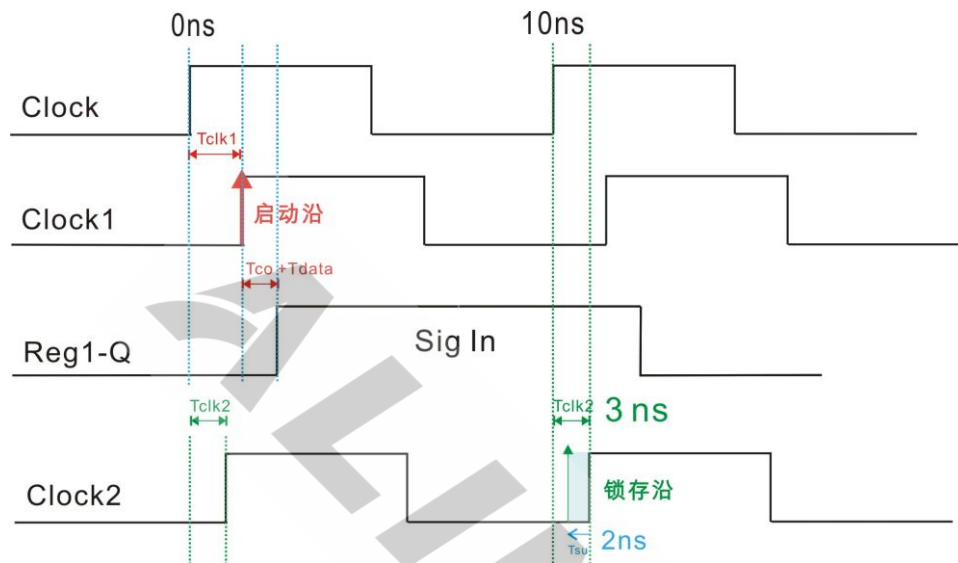


图 1.3.9 例子

如图 1.1.9 所示，在理想的状态下 ... 锁存沿的时间原本是 10ns，但是经 Tclk2 的延迟向右位移几点，成为 $10\text{ns} + 3\text{ns} = 13\text{ns}$ 。然后锁存沿又受到 Tsu 影响所以又向左移几点，成为 $13\text{ns} - 2\text{ns} = 11\text{ns}$ ，最终 reg2 的锁存时间是 11ns。

1.4 令人误会的 建立关系 和 建立余量。

还没有进入建立余量这个主题之前笔者有责任，简单的刷新刷新一下那些封尘已久的记忆还有一误点。建立关系被 TimeQuest 称为 "Setup Relationship", 事实上这个 "Setup Relationship" 是评估 (计算) 建立余量 (setup lack) 的一个过程。但是，不知曾经何时，在很多文章中 Setup Relationship 的定义忽然模糊起来...

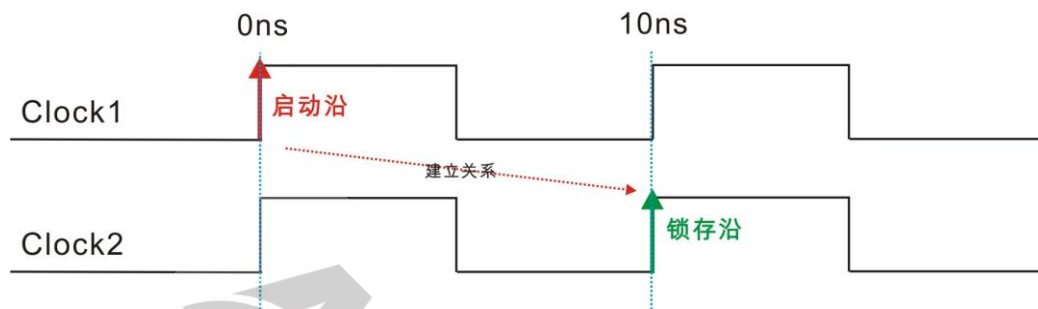


图 1.4.1 理想建立关系值

如图 1.4.1 所示，在理想的状态下“建立关系值”是 10ns ... 那么问题来了，在这里我们需要严重的注意这个“10ns”，而这个“10ns”是一个建立关系值，而 Setup Relationship 是指“分析建立关系的过程”... 结果很明显：

1. 这个“10ns” ≠ Setup Relationship。
2. 这个“10ns” 是一个理想值而已。
3. 这个“10ns” 理想值，在 Setup Relationship 过程是“建立余量”的最好结果！

虽说，上述的内容仅是笔者的想法而已，但是笔者始终认为，建立关系只是一个过程，建立关系值是一个有用的理想值，而这个理想值是建立余量最理想的结果。

1.5 建立余量

建立余量英文是 setup slack，它是 TimeQuest 模型中重要的结果之一，正值的建立余量使得两个寄存器有合格的建立关系。建立余量实际上一点也不难理解，让笔者慢慢解释吧：

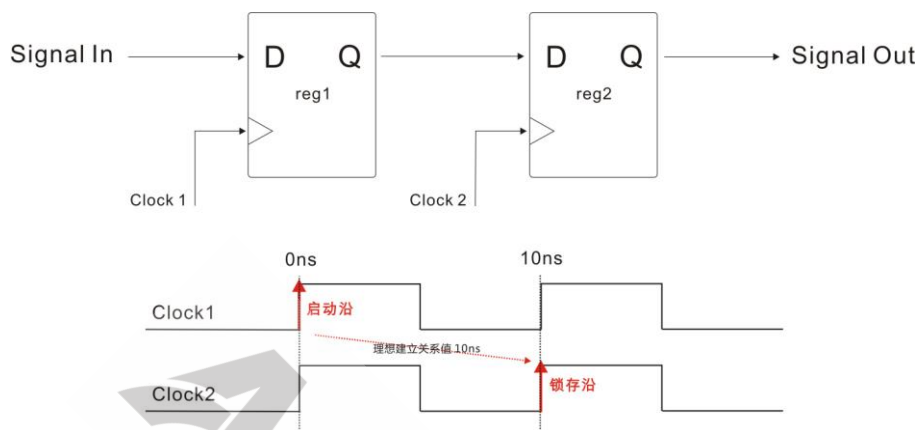


图 1.5.1 理想时序图

图 1.5.1 有一对节点和理想的建立关系过程，寄存器 1 和寄存器 2 各自都是用相同的源时钟，无论 Clock1 还是 Clock2 的时钟周期都是 10ns。在图 1.5.1 中，Clock1 和 Clock2 之前的建立关系值是 10ns。

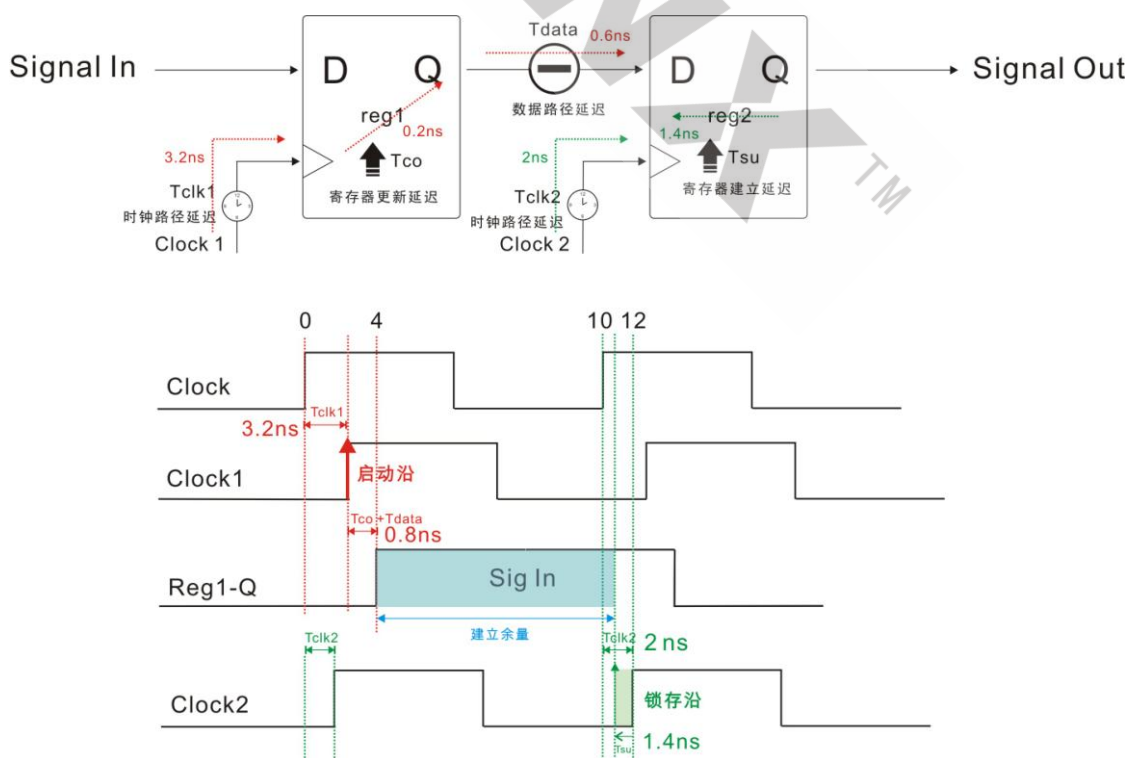


图 1.5.2 物理的时序图

图 1.5.2 是物理的建立关系过程，寄存器 1 和寄存器 2 也使用拥有相同属性的时钟源。而图 1.5.2 和图 1.5.1 之间的区别是，图 1.5.2 有一大堆的延迟。我们接下来的工作就是要取得建立余量。

首先要取得 Clock1 的启动沿受延迟的影响，而 TimeQuest 模型用了一条公式评估启动沿受 $T_{clk1} + T_{co} + T_{data}$ 的影响：

$$\begin{aligned}\text{数据抵达时间 Time Arrival Time} &= \text{启动沿时间} + T_{clk1} + T_{co} + T_{data} \\ &= 0\text{ns} + 3.2\text{ns} + 0.2\text{ns} + 0.6\text{ns} \\ &= 4\text{ns}\end{aligned}$$

接下来我们要取得 Clock2 的锁存沿受延迟的影响。同样 TimeQuest 模型也使用了一条公式评估 锁存沿受 $T_{clk2} + T_{su}$ 的影响：

$$\begin{aligned}\text{数据锁存（获取读取）时间 Time Required Time} &= \text{锁存沿时间} + T_{clk2} - T_{su} \\ &= 10\text{ns} + 2\text{ns} - 1.4\text{ns} \\ &= 10.6\text{ns}\end{aligned}$$

最后我们要用 数据抵达时间 和 数据锁存时间 取得建立余量：

$$\begin{aligned}\text{建立余量} &= \text{数据锁存时间} - \text{数据抵达(获取读取)时间} \\ \text{Setup Slack} &= \text{Time Required Time} - \text{Time Arrival Time} \\ &= 10.6\text{ns} - 4\text{ns} \\ &= 6.6\text{ns}\end{aligned}$$

其中建立余量 6.6ns 是在图 1.5.2 的“蓝色框图的部分”。在这里，建立余量是“正值”，亦即“reg1 和 reg2 之间的建立关系是合格”。可能读者会觉得纳闷：“建立余量正值和 reg1 与 reg2 之间的建立关系合不合格到底有什么关系？”呵呵，这真是一个好问题 ... 就让笔者慢慢解释：

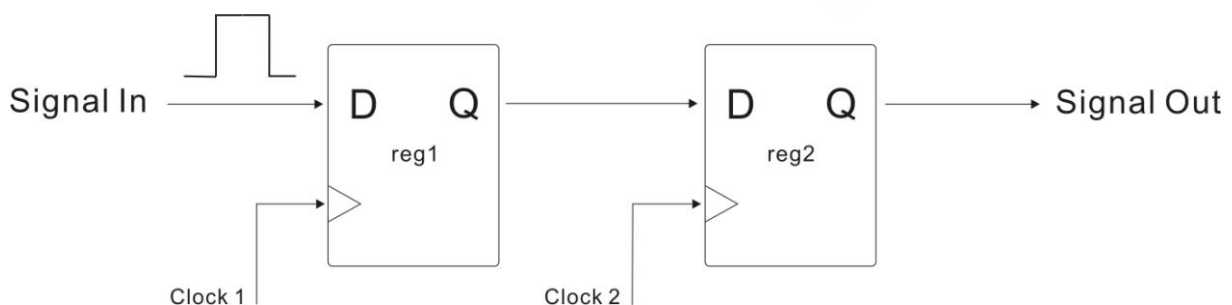


图 1.5.3 一个例子

图 1.5.3 是一个典型的例子，就是 reg1 向 reg2 发送数据，reg2 从 reg1 锁存数据。我们先假设 reg1 和 reg2 拥有相同性质的时钟源，然后 reg2 包含一个“可以随意调整”的 T_{su} 。

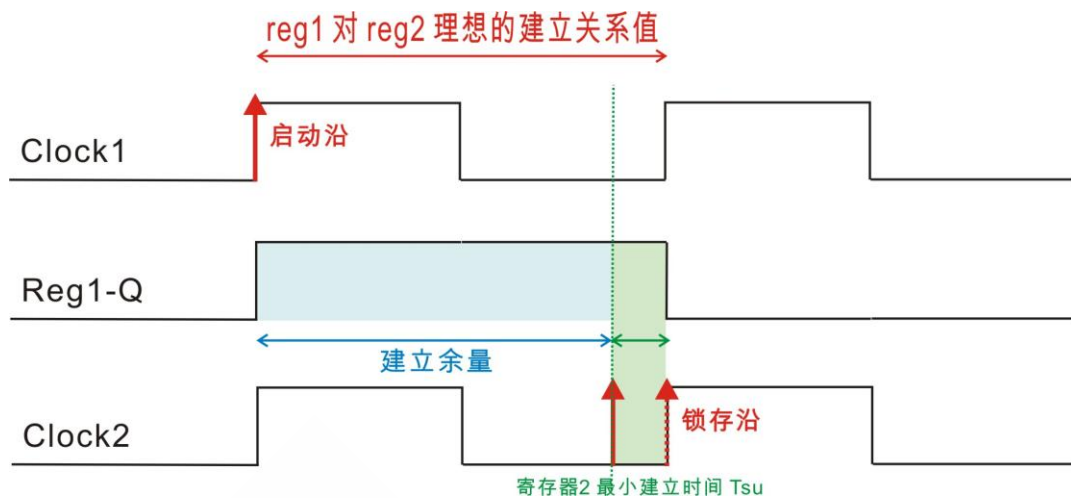


图 1.5.4 加上 Tsu 延迟因素

Tsu 表示了锁存方的寄存器在读取数据之前所需要的“最小准备时间”，图 1.5.4 中的理想建立关系值是 10ns，假设为 2ns ... 锁存沿从原本 10ns 的位置返回 8ns 的位置，那么建立余量从 10ns 慢慢减至 8ns。结果建立余量还是正值，而锁存沿还顶着数据。

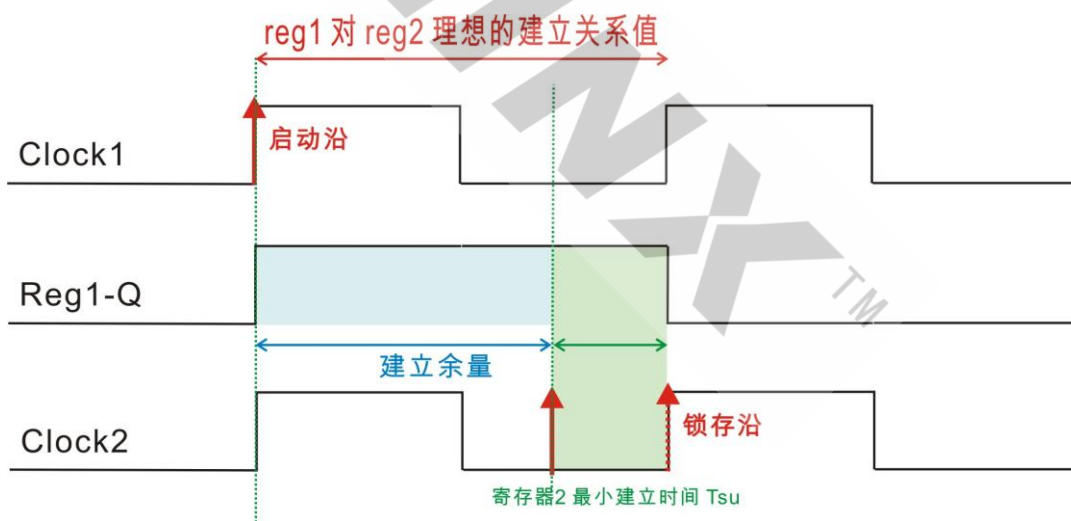


图 1.5.5 延迟因素 Tsu 调大一点

如图 1.5.5 所示，假设在把 Tsu 调大至 3ns，锁存沿从原本 10ns 的位置返回 7ns 的位置，那么建立余量也从原本的 8ns 的再降至 7ns，不过建立余量还是正值。换言之，只要建立余量还是正值，寄存器 2 读取 Reg1-Q 的数据也是没有问题的。

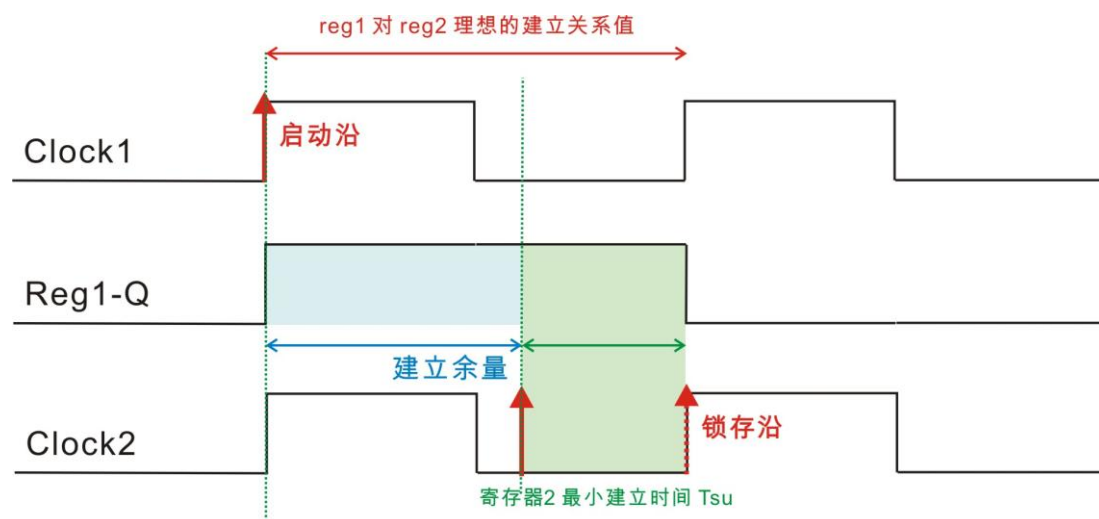


图 1.5.6 还是把 T_{su} 调大一点

图 1.5.6 中的 T_{su} 再再假设调大至 4ns，锁存沿从原本 10ns 的位置返回 6ns 的位置，那么建立余量也从原本的 7ns 降至 6ns。不过建立余量的结果还是正值，如果寄存器 2 锁存数据，也是没问题的说 ... 总结说，**只要建立余量大于 0，那么两个寄存器之间的建立关系就没有问题。**

经过几番理解，读者是不是对“建立关系值”有一点头绪了？在理想的情况下，建立余量一定是理想的建立关系值。此外，这个理想的建立关系值还有简化公式的作用：

在前面，我们知道为了计算建立余的公式是：

$$\text{建立余量} = \text{数据锁存（获取|读取）} - \text{数据抵达时间}$$

其中数据锁存（获取|读取）时间的公式是：

$$\text{数据锁存（获取|读取）时间} = \text{锁存沿时间} + T_{clk2} - T_{su}$$

读者知道吗？锁存沿时间基本上就是“理想的建立关系值”。同学们可能会觉得笔者在胡扯，很好！就让笔者尝试用几个例子来解释这一点。

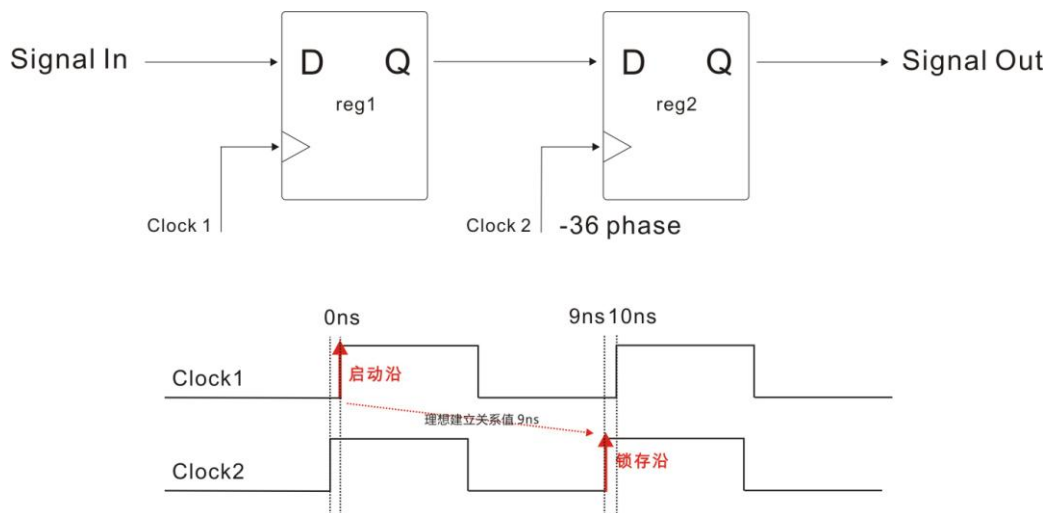


图 1.5.7 Clock2 位移 -36 度

图 1.5.7 中的 Clock1 没有变化，换之 Clock2 位移 -36 度，亦即 Clock2 比起 Clock1 提早 1ns。Clock2 移位 -36 度又意味着什么呢？它意味着 Clock1 和 Clock2 的理想建立关系值已经改变了，结果为 9ns。

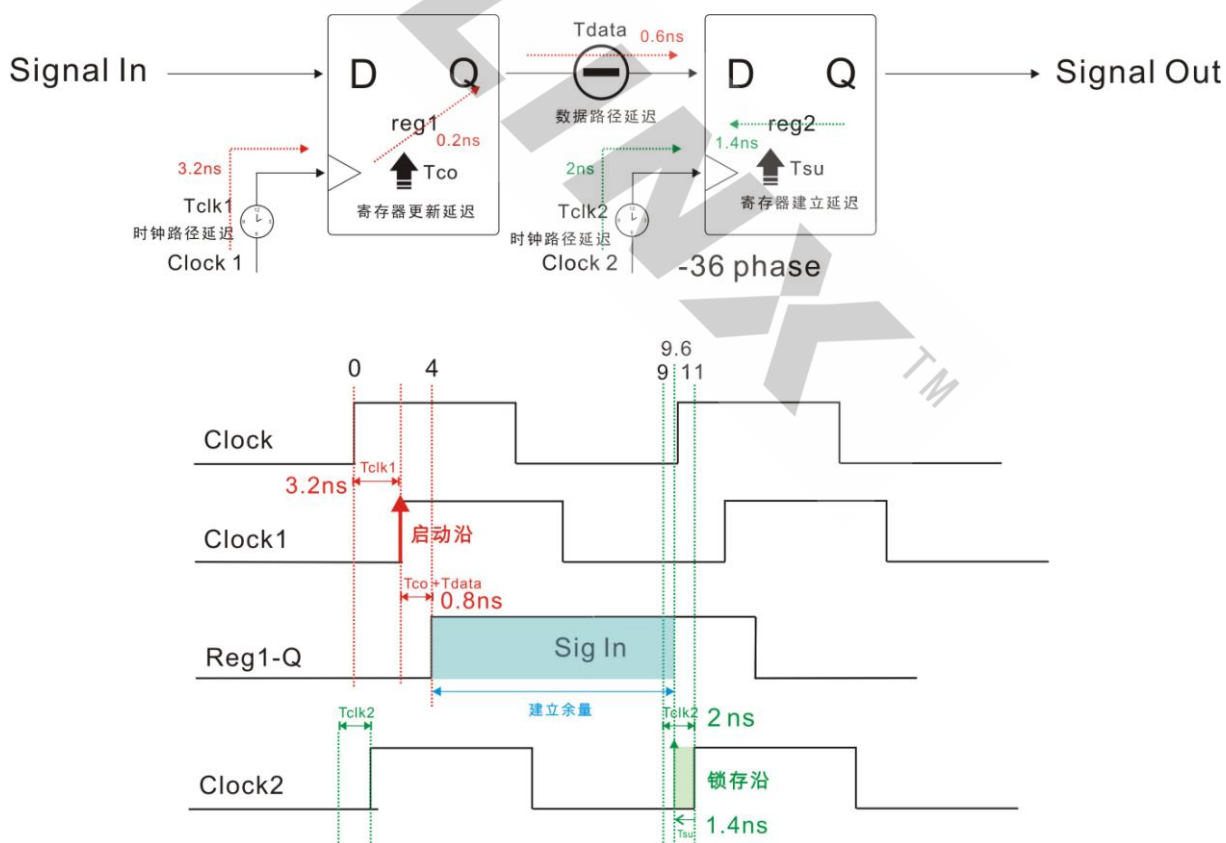


图 1.5.8 Clock2 经过-36 度位移的物理时序图

图 1.5.8 是图 1.5.7 物理的建立关系过程，在前面我们知道 Clock1 和 Clock2 之间的理

想建立关系值是 9ns ...

$$\begin{aligned}\text{数据抵达时间 Time Arrival Time} &= \text{启动沿时间} + T_{clk1} + T_{co} + T_{data} \\ &= 0\text{ns} + 3.2\text{ns} + 0.2\text{ns} + 0.6\text{ns} \\ &= 4\text{ns}\end{aligned}$$

$$\begin{aligned}\text{数据锁存 (获取|读取) 时间 Time Required Time} &= \text{锁存沿时间} + T_{clk2} - T_{su} \\ &= 9\text{ns} + 2\text{ns} - 1.4\text{ns} \\ &= 9.6\text{ns}\end{aligned}$$

$$\begin{aligned}\text{建立余量} &= \text{数据锁存时间} - \text{数据抵达(获取|读取)时间} \\ \text{Setup Slack} &= \text{Time Required Time} - \text{Time Arrival Time} \\ &= 9.6\text{ns} - 4\text{ns} \\ &= 5.6\text{ns}\end{aligned}$$

我们取得 5.6ns 的建立余量, 亦即建立关系合格。在这里稍微注意一下锁存沿时间的 9ns, 结果是不是和理想建立关系值的 9ns 一模一样呢? 读者: “慢着慢着 ... 我差点就要被笔者忽悠了 ... 笔者这样说是不是太扯了! ? 还是不相信吗? 那好, 笔者再举另一个例子, 这回请看好呀。

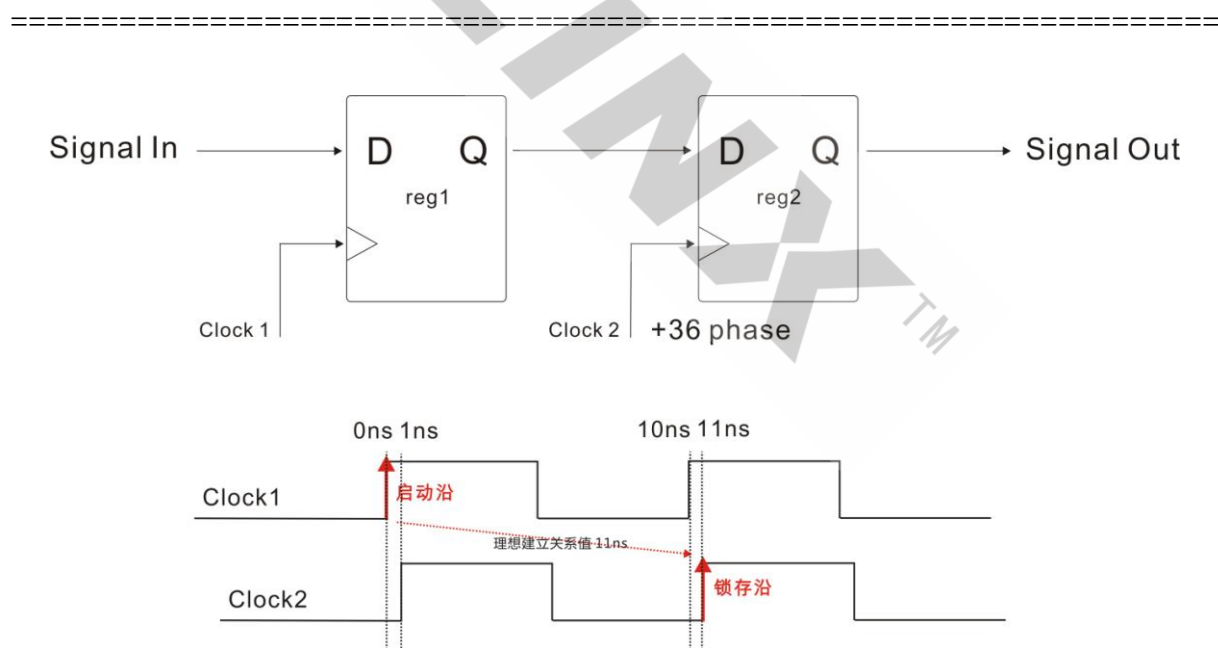


图 1.5.9 Clock2 位移+36 度

图 1.5.9 是 Clock2 位移+36°, 也就是延迟了 1ns, 这也使得 Clock1 对应 Clock2 的理想建立关系值是 11ns。

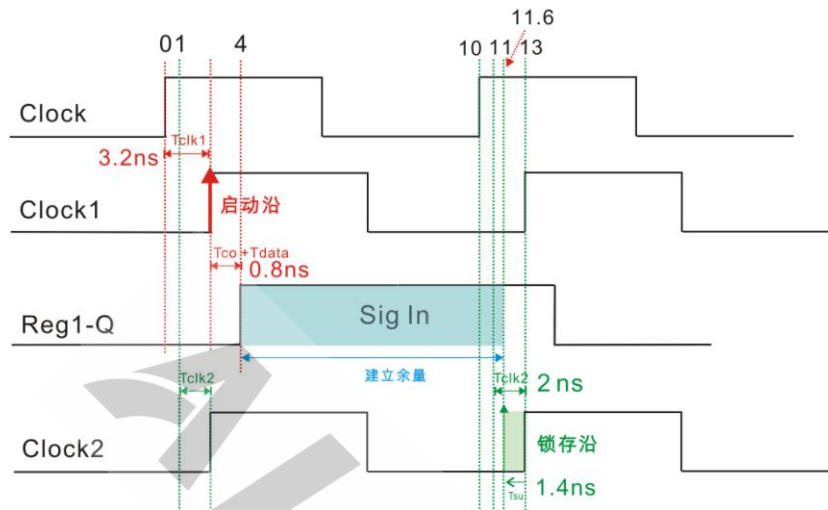
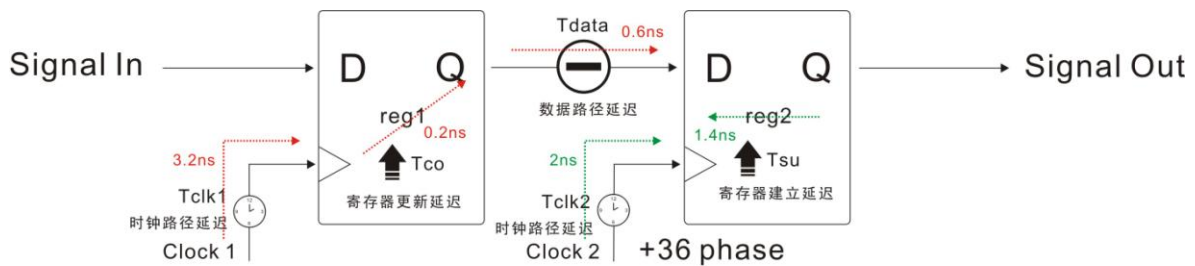


图 1.5.10 Clock2 经过+36 度位移的物理时序图

图 1.5.10 是 Clock2 经过+36 度位移后的物理建立关系过程，然而理想的建立关系值是 11ns。在这里，我们计算建立余量之余，也看看理想建立关系值是不是和**锁存沿时间**等价？

$$\begin{aligned}
 \text{数据抵达时间 Time Arrival Time} &= \text{启动沿时间} + T_{\text{clk1}} + T_{\text{co}} + T_{\text{data}} \\
 &= 0\text{ns} + 3.2\text{ns} + 0.2\text{ns} + 0.6\text{ns} \\
 &= 4\text{ns}
 \end{aligned}$$

$$\begin{aligned}
 \text{数据锁存（获取读取）时间 Time Required Time} &= \text{锁存沿时间} + T_{\text{clk2}} - T_{\text{su}} \\
 &= 11\text{ns} + 2\text{ns} - 1.4\text{ns} \\
 &= 11.6\text{ns}
 \end{aligned}$$

$$\begin{aligned}
 \text{建立余量} &= \text{数据锁存时间} - \text{数据抵达(获取|读取)时间} \\
 \text{Setup Slack} &= \text{Time Required Time} - \text{Time Arrival Time} \\
 &= 11.6\text{ns} - 4\text{ns} \\
 &= 7.6\text{ns}
 \end{aligned}$$

有没有看到？**理想建立关系值**和**锁存沿时间**在评估数据锁存时间是等价的关系 ... 当然理想建立关系值的作用不仅而已，它作用还偶用来“评估”建立余量。在原理上“**建立余量是不可能超过理想建立关系值**”，如果超过了，那么两个节点（寄存器）之间的建立关系就违规，或者不合格化了。

1.6 保持余量

“保持时间”在寄存器的概念中是表示：寄存器在读取某个数据以后，需用一段最小时间来“确保数据锁存”的稳定。然而“保持余量”的定义是指：在两个节点（寄存器）之间，在分析保持关系的过程中“到底有多少剩时间可以提供给寄存器用来确保已存数据的稳定”。嗯 ... 同学们是不是开始抓狂了？那么我们来看一副图 ...

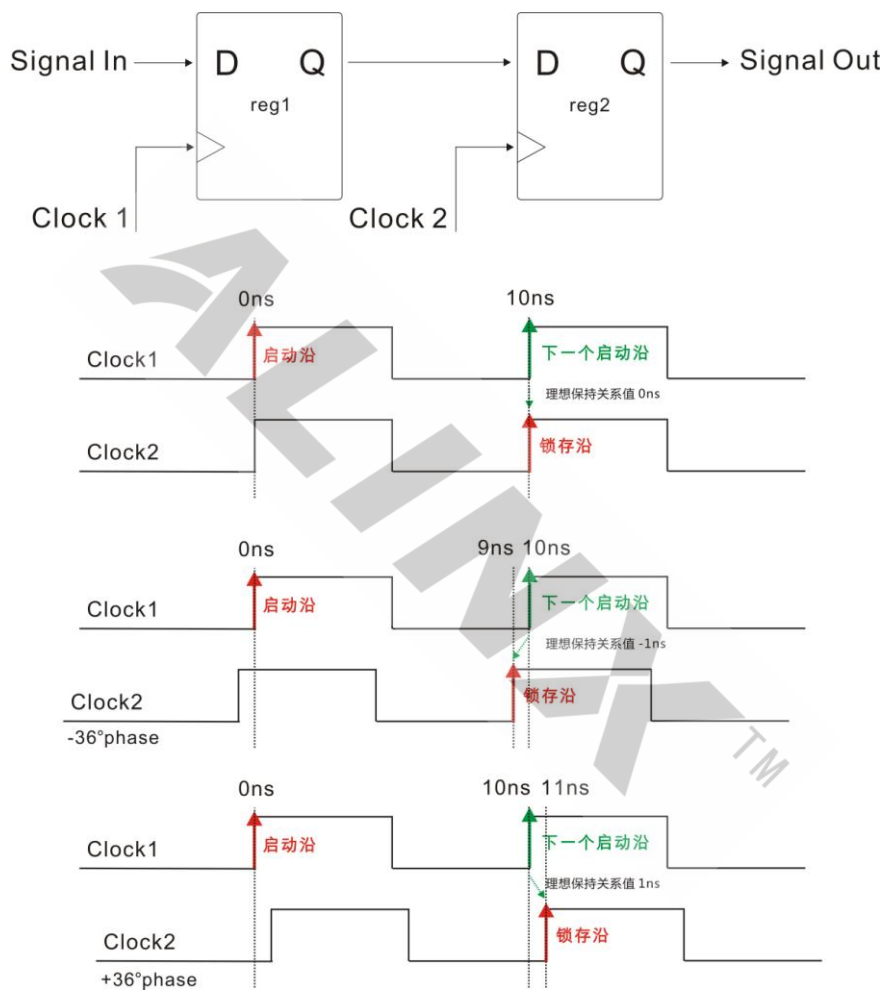


图 1.6.1 不同的理想保持关系值

让我们先回忆一下，如何取得“理想保持关系值” ... 亦即下一个启动沿和当前锁存沿之间的距离。图 1.6.1 中呈现不同的理想保持关系值，笔者曾说过 ... 只有当 Clock1 和 Clock2 使用相同性质的时钟的时候，理想的保持关系值才是 0ns。当 Clock2 有-36 度的位移，亦即 Clock2 比起 Clock1 提早 1ns，那么理想保持关系值是 -1ns。又当 Clock2 有+36 度的位移，亦即 Clock2 比起 Clock1 延迟 1ns，那么理想保持关系值是 1ns。

其中“这个 0ns，-1ns，+1ns 理想保持关系值”到底对取得保持余量有什么作用呢？

在这里笔者需要强调一下！保持余量在英文中 Hold Slack，然而保持关系 Hold Relationship 是求出两个寄存器（节点）之间的保持余量的过程。

1. 这个 “0ns , -1ns , 1ns” $\neq$ Hold Relationship。
2. 这个 “0ns , -1ns , 1ns” 是一个理想值而已。
3. 这个 “0ns , -1ns , 1ns” 理想值，在 Hold Relationship 的过程中“保持余量”的参考值而已！

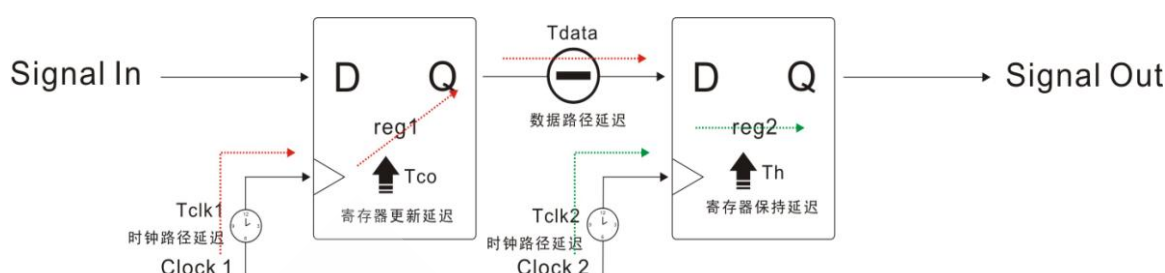


图 1.6.2 在物理保持关系中出现的延迟因数

笔者也在前面好像说过：“保持关系是指寄存器 1 在还没有更新输出之前，寄存器 2 用最快速的速度读取从寄存器 1 发送过来的数据”。在理想的时序图中“理想的保存时间”都是 0ns，然而“理想保持关系值”却不是“理想的保存时间”。理想保持关系值是指，在理想状态下，在两个寄存器之间得到的一个可以参考的值而已。

图 1.6.2 是物理建立关系过程，在两个寄存器（节点）之间的保持关系中有许多延迟因数。Tclk1 是寄存器 1 时钟源的路径延迟，Tco 是寄存器 1 更新数据的时候，最小的消耗时间，Tdata 是寄存器 1 向寄存器 2 发送数据时最小的数据路劲延迟，Tclk2 是寄存器 2 时钟源的路径延迟，最后的 Th 是寄存器 2 用来确保已存数据的最小时间。

TimeQuest 模型也用了一条公式评估 启动沿受 $Tclk1 + Tco + Tdata$ 延迟因数的影响：

数据保持时间 Data Hold Time = 启动沿 + $Tclk1 + Tco + Tdata$ + 数据周期时间

再来 TimeQuest 模型同样也用一条公式评估 锁存沿受 $Tclk2 + Th$ 延迟因素的影响：

数据锁存（读取/获取）时间 Data Required Time = 锁存沿 + $Tclk2 + Th$

不知同学们有没有注意到，图 1.6.2 中的 Th 出现微妙的箭头方向？在前面的内容中，如果箭头是 $\leftarrow$ 会使得锁存沿向左位移，如果箭头是 $\rightarrow$ ，是不是会使得锁存沿向右位移呢？呵呵 ... 接下来我们来看物理时序图：

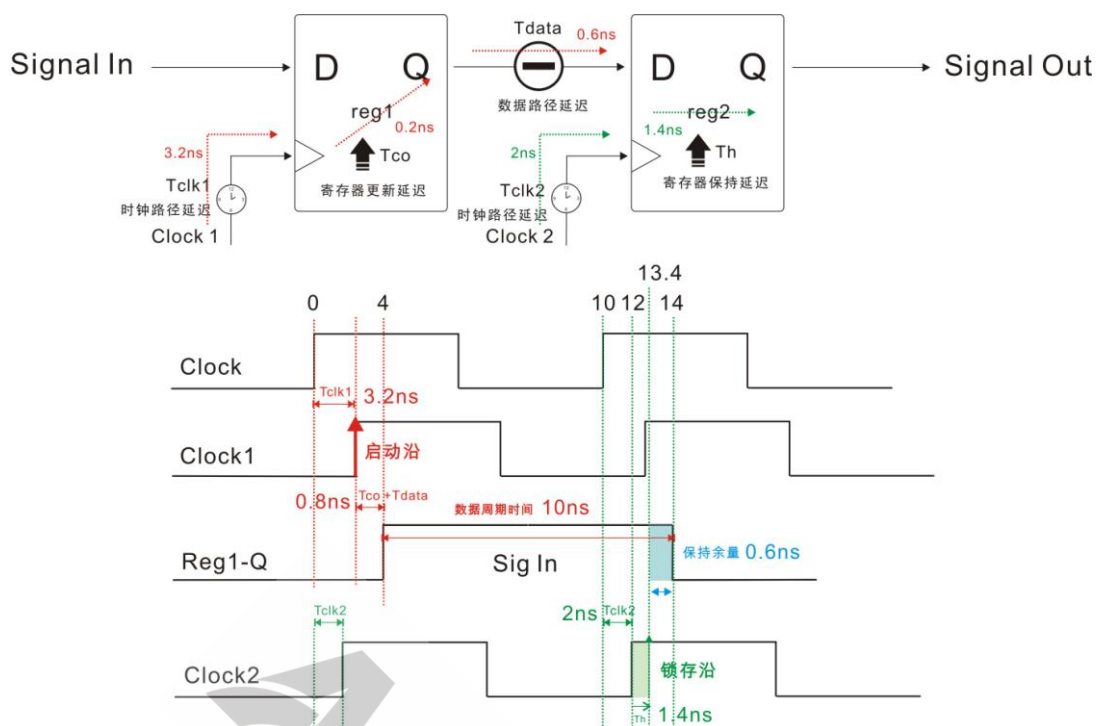


图 1.6.4 物理时序与建立余量

$$\begin{aligned}
 \text{数据保持时间 Data Hold Time} &= \text{启动沿} + T_{\text{clk1}} + T_{\text{co}} + T_{\text{data}} + \text{数据周期时间} \\
 &= 0\text{ns} + 3.2\text{ns} + 0.2\text{ns} + 0.6\text{ns} + 10\text{ns} \\
 &= 14\text{ns}
 \end{aligned}$$

$$\begin{aligned}
 \text{数据锁存 (读取|获取) 时间 Data Required Time} &= \text{锁存沿} + T_{\text{clk2}} + T_{\text{h}} \\
 &= 10\text{ns} + 2\text{ns} + 1.4\text{ns} \\
 &= 13.4\text{ns}
 \end{aligned}$$

$$\begin{aligned}
 \text{保持余量 Hold Slack} &= \text{数据保持时间} - \text{数据锁存 (获取|读取) 时间} \\
 &= 14\text{ns} - 13.4\text{ns} \\
 &= 0.6\text{ns}
 \end{aligned}$$

最后取得的保持余量是 0.6ns ... 这也意味着 保持余量是正值，所以寄存器 1 和寄存器 2 之间的保持关系是合格的。在这里，读者可能会觉得纳闷：“寄存器之间的保持关系合格化，到底和保持余量有什么关系？”呵呵呵！我们先假设寄存器 2 的 T_{h} 延迟因素可以调制的话...

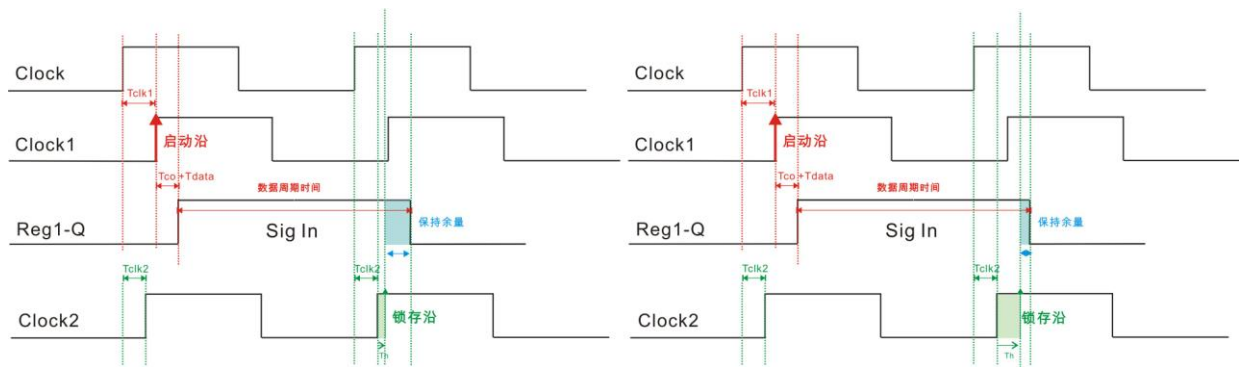


图 1.6.5 不同的 T_h 和不同的建立余量

图 1.6.5 表示了不同的 T_h 和 不同的建立余量。同学们还记得建立关系的定义吗？就是寄存器 1 发送的数据在更新（更换）之前，寄存器 2 可用最短的时间去锁存数据。在此，保持余量除是提供给 T_h 有余时间以外，它还也表示只要保持余量一天还是正值，那么寄存器 2 在寄存器更换数据之前，寄存器 2 就有多少可以优越读取数据的时间。

保持余量的故事就先告一个段落 .. 注意，到目前为止理想保持关系值还没有出场到。

=====

在这里，笔者要告诉读者们一个非常“坏”的消息 ... 一些顽皮的读者估计已经发现到，前面的内容视乎和 TimeQuest 的手册，或者官方相关内容是大为不同？甚至怀疑“笔者是不是搞错了？”然后借机寻找笔者的痛脚 ... 嘛，这一切的一切都有深长的因缘 ... 接下来慢慢听笔者解释！

早期在 TimeQuest 将近完成之际，有一位师兄说道：

师兄：没日没夜的项目终于要搞一个段落了，非常期待这款软件发出后，会不会给 fpga 界一个大惊喜呢？那些第二第三的工程师们，一定要把我当神拜。

忽然间，一位师弟插嘴道。

师弟：老大，你过来看一下 ... 求出建立余量 和 保持余量 的公式 是不是不够和谐？

师兄：怎么不和谐呢？我的设计是接近神级的 ...

师弟：有没有办法把求出建立余量和保持余量的公式凑合凑合，两者可以共用同一个公式？那么你的设计可以更神一层楼。

师兄：对呀，你怎么不早说，我的世界是没有最完美，只有更完美 ...

就这样，建立余量和保持余量的共用公式就次诞生了。在前面笔者说过“无论是理想的建立关系值或者理想的保持关系值”它们的作用出了供参考以外，还有简化 TimeQuest 模型。这一句是千真万确的，因为“保持余量”的求出过程实在太猥琐了，在此为了简化求出保持余量的过程，保持余量和建立余量共享同一个公式，那么这个“理想保持关系值”是必须的。

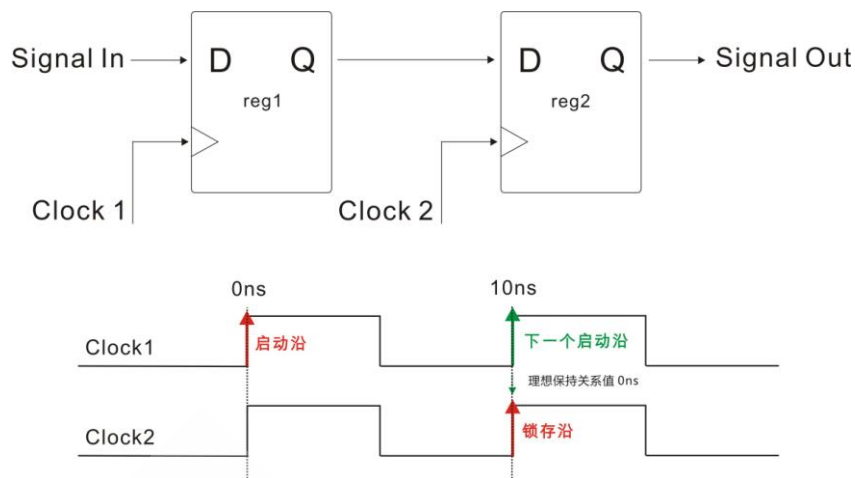
[illegible]

图 1.6.6 理想保持关系

图 1.6.6 是理想的保持关系过程，而理想的保持关系值既是 0ns。

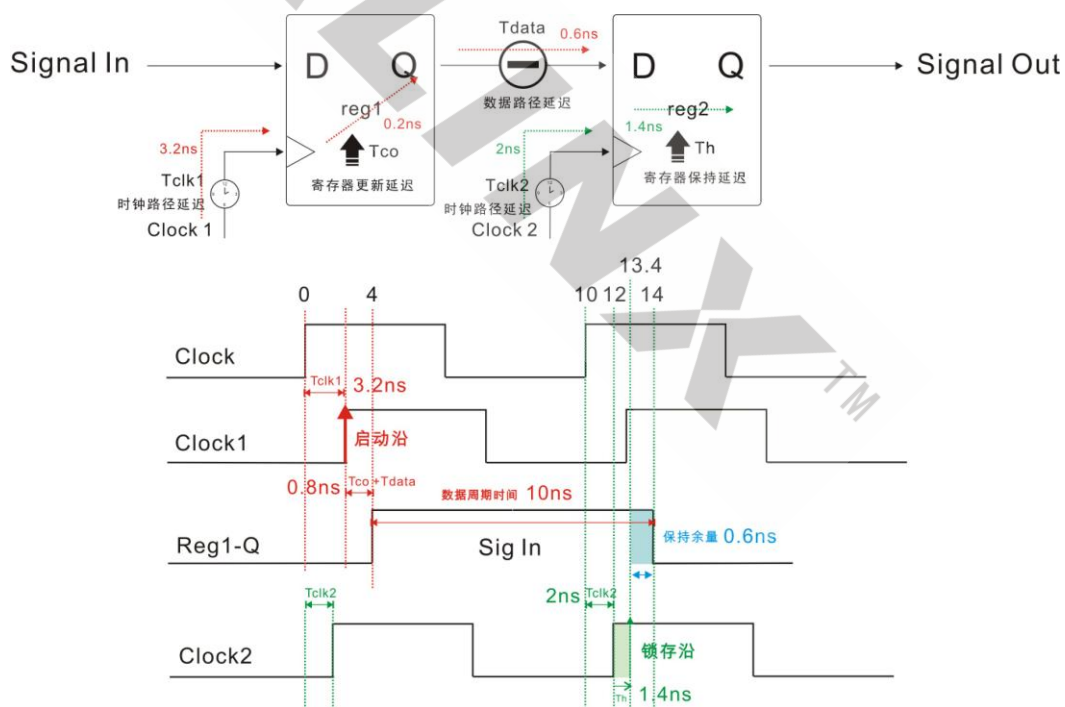


图 1.6.7 求出保持余量的物理时序图

接下来，让我们来看看建立余量和保持余量如何共享一个公式？

TimeQuest 用了一条公式评估 启动沿受 $T_{clk1} + T_{co} + T_{data}$ 延迟因数的影响：

数据抵达时间 Data Arrival Time = 启动沿 + Tclk1 + Tco + Tdata

(注意：此时不是数据保持时间 Data Hold Time，所以也没有加入数据周期时间)

TimeQuest 也用了个条式评估 锁存沿受 $T_{clk2} + T_h$ 延迟因数的影响：

$$\begin{aligned}\text{数据锁存 (获取|读取) 时间 Data Required Time} &= \text{锁存沿} + T_{clk2} + T_h \\ &= \text{理想保持关系值} + T_{clk2} + T_h\end{aligned}$$

(注意：这个评估锁存沿受延迟因素影响的公式和求得建立余量时数据锁存时间的公式非常类似)

那么求得的保持余量是：

$$\text{保持余量} = \text{数据抵达时间} - \text{数据锁存 (获取|读取) 时间}$$

好了，现在我们套用这个公式求出图 1.6.7 中的保持余量。

$$\begin{aligned}\text{数据抵达时间 Data Arrival Time} &= \text{启动沿} + T_{clk1} + T_{co} + T_{data} \\ &= 0 + 3.2 + 0.2 + 0.6 \\ &= 4\text{ns} \\ \text{数据锁存 (获取|读取) 时间 Data Required Time} &= \text{锁存沿} + T_{clk2} + T_h \\ &= \text{理想保持关系值} + T_{clk2} + T_h \\ &= 0\text{ns} + 2\text{ns} + 1.4\text{ns} \\ &= 3.4\text{ns}\end{aligned}$$

$$\begin{aligned}\text{保持余量} &= \text{数据抵达时间} - \text{数据锁存 (获取|读取) 时间} \\ &= 4\text{ns} - 3.4\text{ns} \\ &= 0.6\text{ns}\end{aligned}$$

咦咦咦？求出的结果确实是一样哦？难道是 TimeQuest 在变魔术？还是自己眼花了？
同学们我们都没有看错 ... 不错！那是 TimeQuest 模型的公式戏法。事实上不是 TimeQuest 在变魔术，而是 TimeQuest 模型懒得用头脑求得保持余量，随之就用屁股求得保持余量，这话何解呢？

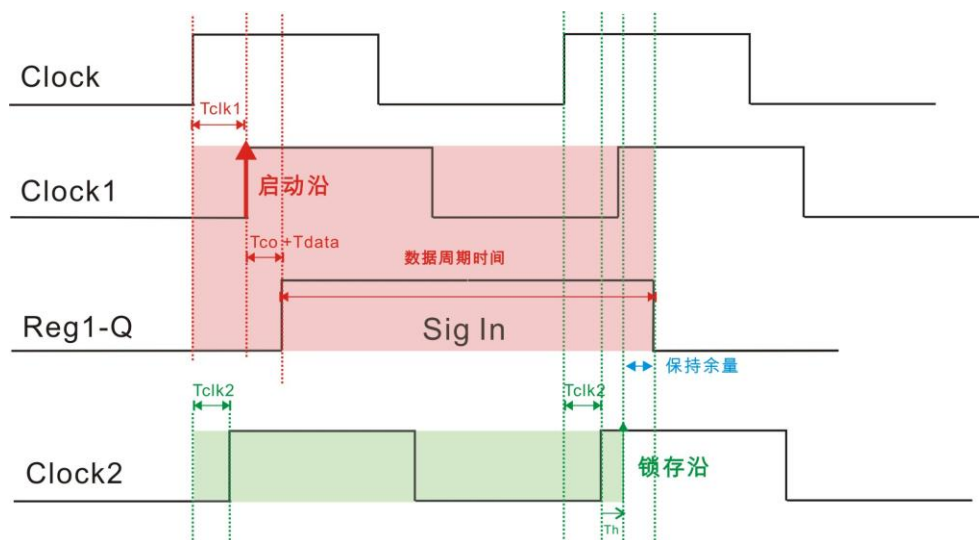


图 1.6.8 用头脑求得保持余量

图 1.6.8 是用头脑求得保持余量的方法，也就是说“红色部分的长度 - 绿色部分的长度”，亦即用数据保持时间减掉数据抵达时间。用傻瓜的话来讲，求得保持余量必须从 0ns 开始算起。

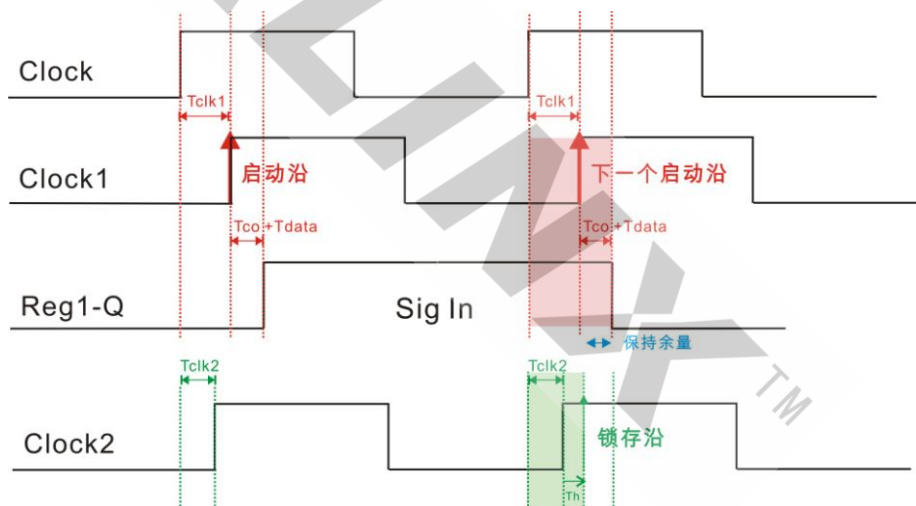


图 1.6.9 用屁股求得保持余量

图 1.6.9 是用屁股求得保持余量的方法，换句话说就是“用下一个启动沿的红色部分减掉当前锁存沿绿色的部分”，其中就要用到“理想保持关系值”。如果继续用屁股来解释的话，那么“将启动沿的总和延迟时间减掉锁存沿的总和延迟时间，然后顺便参考一下理想保持关系值”。

啊 同学们别那么惊讶，笔者也只是把事实说出来而已，师兄确实为了使建立余量和保持余量可以共用一个公式，才会开始用屁股想东西。如果同学们还是不相信的话，我们可以用几个例子来实验看看：

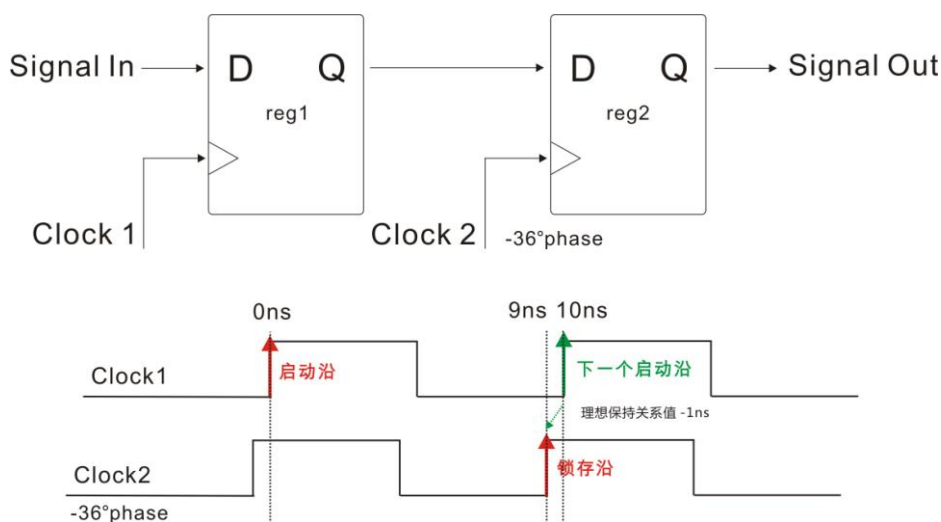


图 1.6.10 Clock2 位移 -36 度

图 1.6.10 是理想的保持关系过程，其中 Clock1 对应 Clock2 的理想保持关系值是 -1ns。而图 1.6.11 是物理的保持关系过程。

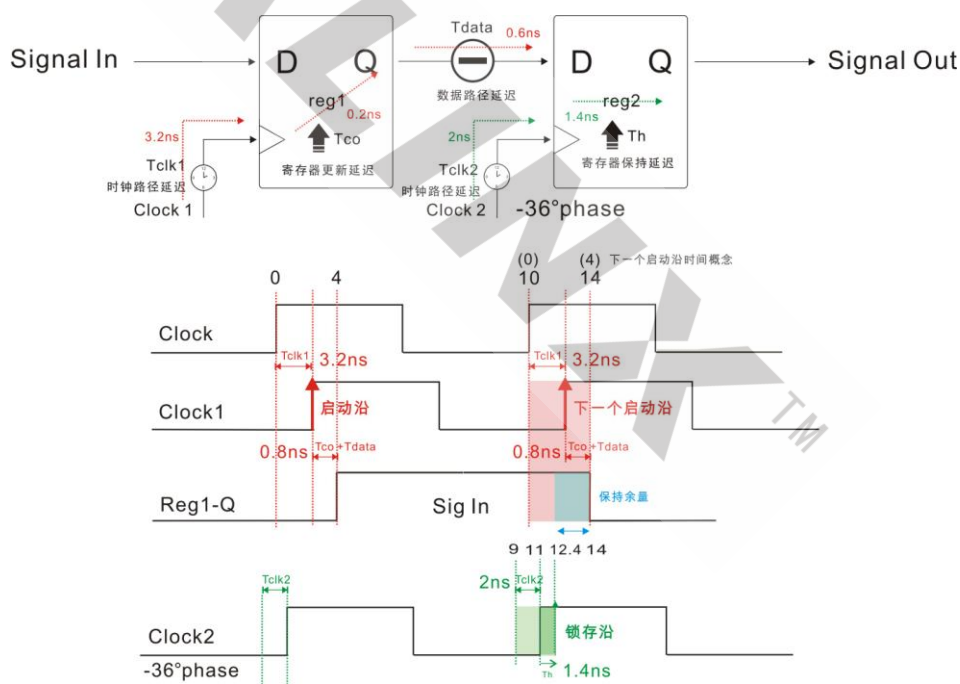


图 1.6.11 Clock2 位移-36 度的物理时序图

好了笔者先用头脑求得保持余量：

$$\begin{aligned}
 \text{保持余量} &= \text{数据保持时间} - \text{数据锁存（获取读取）时间} \\
 &= (\text{启动沿} + T_{clk1} + T_{co} + T_{data} + \text{数据时间周期}) - (\text{锁存沿} + T_{clk2} + T_h) \\
 &= (0 + 3.2 + 0.2 + 0.6 + 10) \text{ ns} - (9 + 2 + 1.4) \text{ ns} \\
 &= 14\text{ns} - 12.4\text{ns}
 \end{aligned}$$

$$= 1.6\text{ns}$$

如果用屁股求得保持余量：

$$\begin{aligned}\text{保持余量} &= \text{数据抵达时间} - \text{数据锁存（获取/读取）时间} \\ &= (\text{启动沿} + T_{\text{clk1}} + T_{\text{co}} + T_{\text{data}}) - (\text{锁存沿} + T_{\text{clk2}} + T_{\text{h}}) \\ &= (\text{启动沿} + T_{\text{clk1}} + T_{\text{co}} + T_{\text{data}}) - (\text{理想保持关系值} + T_{\text{clk2}} + T_{\text{h}}) \\ &= (0 + 3.2 + 0.2 + 0.6) \text{ ns} - (-1 + 2 + 1.4) \text{ ns} \\ &= 4\text{ns} - 2.4\text{ns} \\ &= 1.6\text{ns}\end{aligned}$$

在这里，可能有同学要问：“用屁股求得余量为什么启动沿的时间不是 10 而是 0 呢？”很简单呀，如果用屁股来求得保持余量，那么下一个启动沿的时间，被视为“重新开始”，亦即 0ns 而不是 10ns。这一点要好好的记住。



总结：

TimeQuest 说得简单点就是为懒人准备的物理时序分析工具，关于“物理时序分析”的概念就自己维基百科看看吧。其中 TimeQuest 有自己的一套“模型”用来分析物理时序从何摆脱猥琐的时序分析工作，但是 TimeQuest 的模型使用步骤还是一样猥琐不少。在宏观上 TimeQuest 的模型是基于“两个节点”和才能成立。

有关“节点”在 TimeQuest 模型里的定义有很多很多，但是为了简化笔记的内容笔者就先吧“节点”充当“寄存器”，更多有关节点的故事以后再谈。TimeQuest 模型为了确定两个寄存器（节点）之间的时序是合格的，所以用了“建立关系”和“保持关系”的概念。

在理想的状态下“建立关系”有一个理想的建立关系值，反之“保持关系”有一个理想的保持关系值，它们的作用除了提供参考以外，还可以简化 TimeQuest 模型。“保持关系”和“建立关系”会随着理想时钟源的状态也会产生不同的“理想建立关系值”和“理想保持关系值”。

那么 TimeQuest 的模型又用“什么方法来确定两个寄存器之间的建立关系和保持关系是合格的呢？”建立关系确保合格的办法就是取得正值的建立余量，同样的保持关系确保合格办法就是取得正值的保持余量。

求得建立余量的公式是：

$$\begin{aligned}\text{保持余量} &= \text{数据锁存时间} - \text{数据抵达时间} \\ \text{Setup Slack} &= \text{Data Required Time} - \text{Data Arrival Time}\end{aligned}$$

$$\text{数据抵达时间 Data Arrival Time} = \text{启动沿} + T_{clk1} + T_{co} + T_{data}$$

$$\begin{aligned}\text{数据获取时间 Data Required Time} &= \text{锁存沿} + T_{clk2} - T_{su} \\ &= \text{理想建立关系值} + T_{clk2} - T_{su}\end{aligned}$$

求得保持余量的公式是：(用头脑的话)

$$\begin{aligned}\text{保持余量} &= \text{数据保持时间} - \text{数据获取时间} \\ \text{Hold Slack} &= \text{Data Hold Time} - \text{Data Required Time}\end{aligned}$$

$$\text{数据保持时间 Data Hold Time} = \text{启动沿} + T_{clk1} + T_{co} + T_{data} + \text{数据周期时间}$$

$$\text{数据获取时间 Data Required Time} = \text{锁存沿} + T_{clk2} + T_h$$

（不要问我数据周期时间如何得到，如果稍微有 Verilog HDL 语言的时序概念的话，都

会知道如何取得。一般上数据周期时间等价于该时钟源的周期时间)

求得保持余量的公式是：(用屁股的话)

保持余量 = 数据抵达时间 - 数据获取时间
Hold Slack = Data Arrival Time - Data Required Time

数据抵达时间 Data Arrival Time = 启动沿 + Tclk1 + Tco + Tdata

数据获取时间 Data Required Time = 锁存沿 + Tclk2 + Th
= 理想保持关系值 + Tclk2 + Th

好了，笔者差不多要结束这一章节的笔记了。这一章，笔者只是单纯又简单的讲解：

“学习 TimeQuest 模型的概念时容易陷入的几个灰区而已”。

此外，笔者也大概的讲解 TimeQuest 模型中用到的公式和物理时序中出现的东东。如果按照一般方法去学习 TimeQuest 的话，很容易撞墙而已，又或者会被训练成为纯公式的使用者。TimeQuest 模型是一个很有趣的东西，如果用纯公式的方法去使用的话，就无法体验到 TimeQuest 的有趣之处以外，也不能享受学习的乐趣。

第二章：模型角色，网表概念，时序报告

2.1 TimeQuest 模型的努力

我们知道 TimeQuest 的存在是某师兄所追求的神器，目的是要给懒人们一个解决物理时序的福利，其中最耀眼的就是 TimeQuest 模型。在前面笔者也曾介绍过 TimeQuest 模型它是什么？它基于什么？它有什么误点？”。然在这一章节当中，笔者将会讨论它是用来“干什么？”。

TimeQuest 模型是用来解决两个点寄存器（节点）之间的建立关系和保持关系问题。但是在一个最简单的 HDL 设计，不可能仅是由两个寄存器就组成，里边可能会有“很多很多很多”对寄存器。结果 TimeQuest 就会用自己的时序分析模型，逐个检查“很多很多很多”对寄存器之间的“建立关系和保持关系是否合格化”。

试想看看，举例最简单的计数器设计，里边就有多少对寄存器？而且 TimeQuest 这个机器将会逐个去分析保持关系和建立关系，TimeQuest 的行为和努力真是让笔者感动到全身扑地。好了不多废话了，进入主题吧。

2.2 SDC 的网表 (Net Lists) 基本单位

网表 Net Lists 是什么？嗯 ... 笔者真的不想用太多专业的话来讲因为太无趣了，因此我们从另一个角度去看网表。笔者喜欢把 FPGA 看成一座巨大的逻辑资源库，如果读者喜欢玩具的话，也可以把它看成乐高积木库，那么 HDL 就是这个逻辑资源库的建模工具。

但是有一点比较不同的是，HDL 是一个理想的工具，HDL 的建模是软模型，然而网表就是硬模型。FPGA 综合的过程就像我们日常建立机械模型那样，我们必须先在电脑上建立一个软模型。当软模型的经过电脑模拟和测试以后，那么它会成为硬模型的理想参考物，最后模型设备会根据这个软模型，把金属生肉加工成为实物。

在 FPGA 综合的过程中，逻辑资源之间会开始“布线”，把相关的单位连线起来。设计有多大，那么连线的数量就越多，最终这庞大的连线就成为网表。

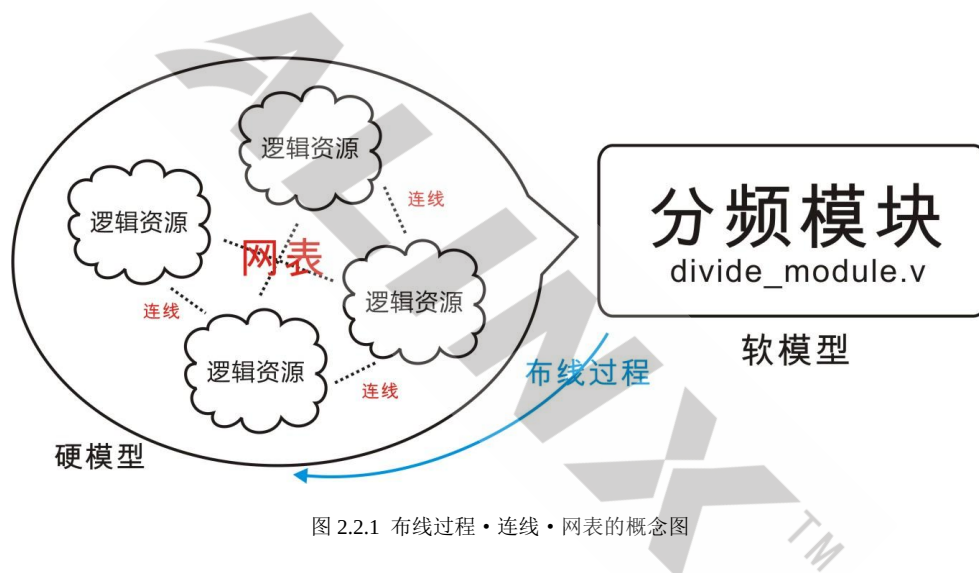


图 2.2.1 布线过程·连线·网表的概念图

图 2.2.1 是布线过程的概念图 ... 布线过程就是按软模型，在 FPGA 巨大的资源库里建立硬模型，逻辑资源之间的链接称为连线，所有连线的总和称为网表。明白了吗？对于不同的性质的网表都有不同的基本单位，对于 TimeQuest 来说它是使用 SDC 网表。有关 SDC 的背景请维基百科一下 它不在讨论范围之内。

SDC 网表有它自己的基本单位，至于是那些基本单位呢？

基本单位	说明
Cell	SDC 网表中最普遍的个体。FPGA 中可以找到的寄存器呀，片上 RAM 呀，PLL 资源呀，硬件乘法器等逻辑资源。
Pin	Cell 进出口。
Net	Pins 之间的连线。
Port	顶层模块的顶层输入输出。

Cell 从另一个角度来看，任何 FPGA 的逻辑资源都可以看成 Cell。Pin 是每一个 Cell 的进出口（软引脚）。Net 就是连线了，最后的 Port 就是顶层模块的输入输出口（硬引脚）。好了，笔者就以一个或模块 or_module.v 来解释。



图 2.2.2 或模块的建模图。

图 2.2.2 是一个软模块，亦即或模块的建模图，它有 A_Sig、B_Sig 和 CLK_Sig 输入，金额 Q_Sig 输出。当这个或模块经过综合和布线以后，它会成为 ...

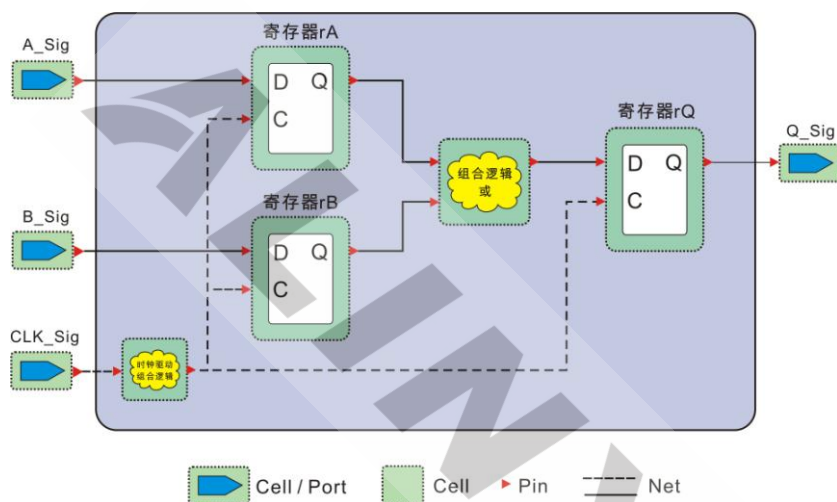


图 2.2.3 布线过后的或模块

图 2.2.3 是经过布线过后的或模块，每一个绿色框框都是 Cell，每一个红色箭头都是 Pin，每一个连线都是 Net。但是有一点比较暧昧的就是输入输出口，它既是 Cell 也是 Port，这到底是什么一回事呢？

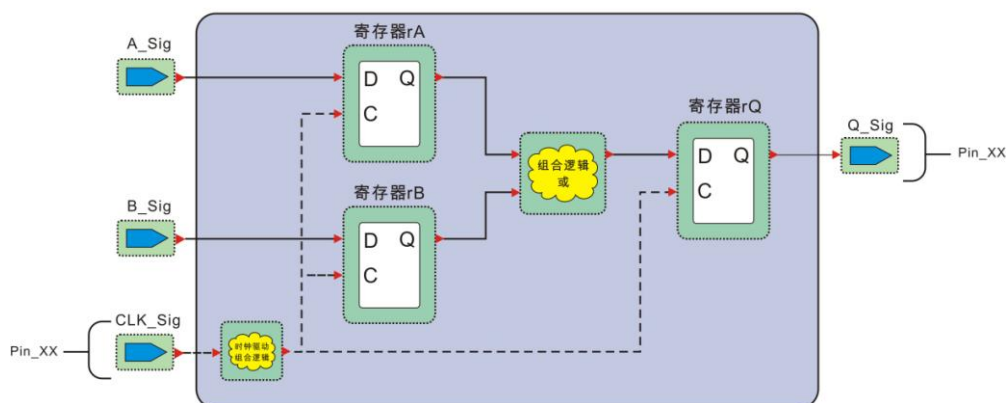


图 2.2.4 配置引脚后就成为 Port

图 2.2.4 是从图 2.2.3 的基础上，将 CLk_Sig 和 Q_Sig 配上引脚，最终造就它们都变身为 Port。SDC 网表的基本单位看看就好了，不要太计较它们是什么，因为它们的作用不大，不过它们又有什么作用呢？

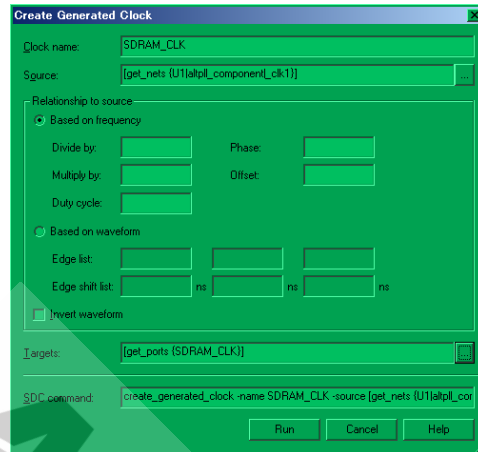


图 2.2.5 某个约束命令，基本单位的作用。

如图 2.2.5 所示，笔者简单举例一个约束命令，其功能先不管。基本上，基本单位的作用就是为约束命令设置 源对象（Source）或者目标对象（Targets）而已，如图 2.2.5 所示 ... 有一个名为 create_generated_clock 的约束命令，它约束的源对象是 pll 的输出 pin，亦即 clk1，而约束的目标对象就是名为 SDRAM_CLK 的输出 port。

2.3 TimeQuest 模型的简单实例

实验一：入门实验

想了想，笔者还是用一个简单的实例来搞清楚，大体上 TimeQuest 模型是如何工作 ... 这样的话，笔者也可以省下许多口水，读者认识也可以得更具体。



图 2.3.1 分频模块的建模图。

图 2.3.1 是久违的建模图，它是名为 divide_module.v 的分频模型，而主要功能是把 CLK_Sig 实现 4 分频，然后输出至 Q_Sig。

```
1. module C1_module
2. (
3.     input CLK_Sig,
4.     output Q_Sig
5. );
6.
7.     /*****/
8.
9.     reg [2:0]C1;
10.
11.     always @ ( posedge CLK_Sig ) C1 <= C1 + 1'b1;
12.
13.     /*****/
14.
15.     assign Q_Sig = C1[2];
16.
17.     /*****/
18.
19. endmodule
```

代码很简单而已，连小孩子都看懂了，读者不可能看不懂吧？看不懂的朋友自己仿真看看就好了。这个分频模块的内容也超级简单，就是把 C1[3]驱动 Q_Sig（第 15 行）。

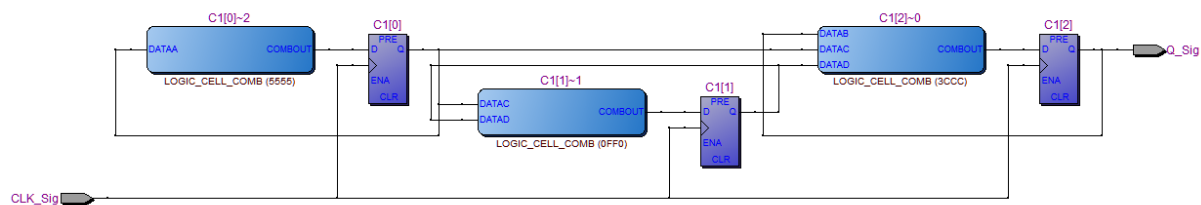


图 2.3.2 TMV 视图

图 2.3.2 是分频模块的 Technology Mapping Viewer，关于这种视图还真的是不常用，它不过是用来分析物理时序才用得到的“理想概念视图”而已。接下来我们来看看，分配模块的里边到底有多少对寄存器（节点）需要 TimeQuest 去分析他们之间的建立关系和保持关系。

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	9.004	C1[1]	C1[2]	CLK_Sig	CLK_Sig	10.000	0.000	1.036
2	9.235	C1[1]	C1[1]	CLK_Sig	CLK_Sig	10.000	0.000	0.805
3	9.235	C1[0]	C1[0]	CLK_Sig	CLK_Sig	10.000	0.000	0.805
4	9.235	C1[2]	C1[2]	CLK_Sig	CLK_Sig	10.000	0.000	0.805
5	9.422	C1[0]	C1[2]	CLK_Sig	CLK_Sig	10.000	0.000	0.618
6	9.422	C1[0]	C1[1]	CLK_Sig	CLK_Sig	10.000	0.000	0.618

图 2.3.3 六对寄存器的建立关系需要分析

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	0.499	C1[2]	C1[2]	CLK_Sig	CLK_Sig	0.000	0.000	0.805
2	0.499	C1[1]	C1[1]	CLK_Sig	CLK_Sig	0.000	0.000	0.805
3	0.499	C1[0]	C1[0]	CLK_Sig	CLK_Sig	0.000	0.000	0.805
4	0.749	C1[0]	C1[1]	CLK_Sig	CLK_Sig	0.000	0.000	1.055
5	0.750	C1[0]	C1[2]	CLK_Sig	CLK_Sig	0.000	0.000	1.056
6	1.171	C1[1]	C1[2]	CLK_Sig	CLK_Sig	0.000	0.000	1.477

图 2.3.4 六对寄存器的保持关系需要分析

同学们千万不要慌图 2.3.3~4 的结果是如何得到的 .. 在这里，笔者只是要读者弄明白 TimeQuest 模型分析物理路径的大概经过而已，同学乖 ... 暂时先忍耐。

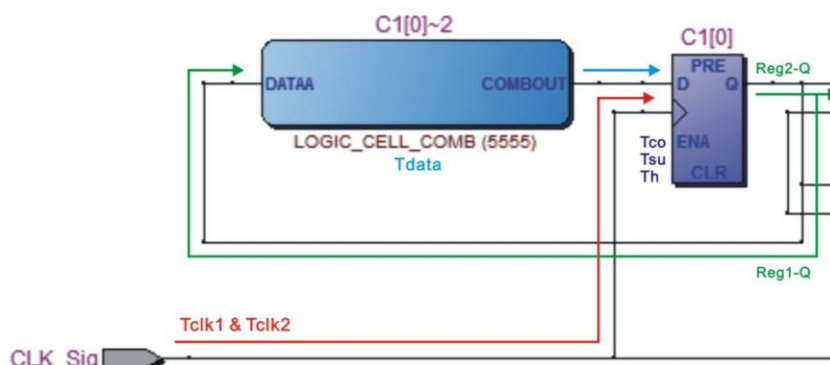


图 2.3.5 第三对 C1[0]~C1[0]寄存器

让我们先看第第 3 对，亦即 C1[0] ~ C1[0]的分析经过。在此读者可能会觉得疑惑，同样

都是 C1[0]节点，为何建立关系和保持关系的分析过程会成立呢？读者稍微注意一下，图 2.3.5 的 C1[0]的输出方向，除了 reg2-Q 以外，它还反馈给自己的 reg1-Q，节点 C1[0]~C1[0]也因此成立。

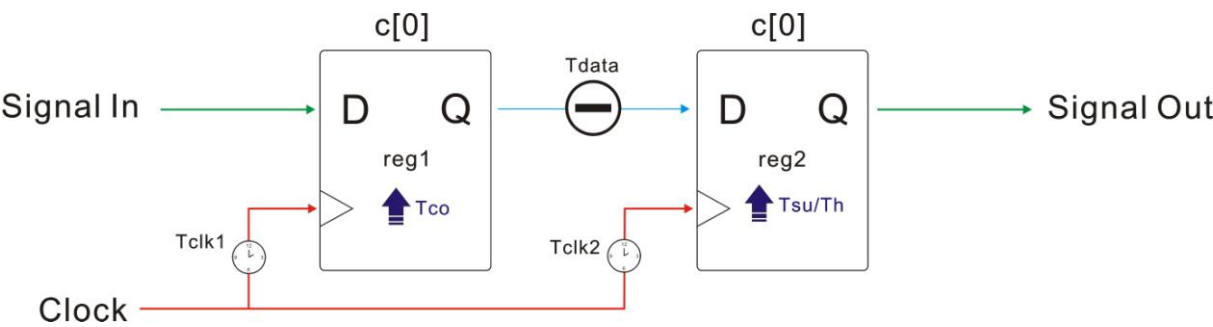


图 2.3.6 把 C1[0]拆开的概念图

图 2.3.6 是比较人性化的概念图，只要有两个寄存器（节点）那么 TimeQuest 模型就可以开始分析它们之间的建立关系和保持关系了。

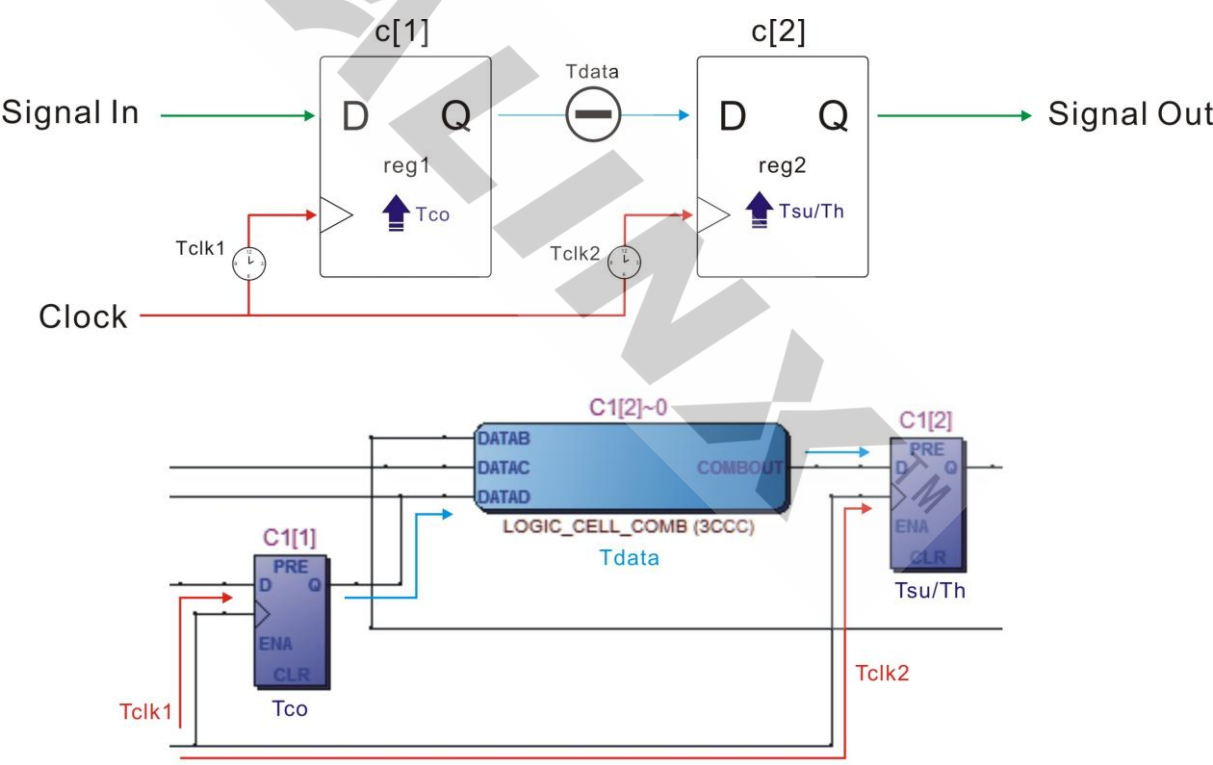


图 2.3.7 第一（六）对 C1[1]~C1[2]寄存器

相比之下，节点 C1[1]~C1[2] 比起节点 C1[0]~C1[0]更容易理解。C1[1]是寄存器 1 的第 1 位，而 C1[2]则是寄存器 2 的第 2 位，LOGIC_CELL_COMB 是 Tdata 延迟。然而其他节点也有大同小异的情形，笔者就不一一解释了，自己看着办吧。

就这样 TimeQuest 模型会一对一对的分析节点之间的建立关系和保持关系，一个设计里边有多少对节点，TimeQuest 就分析多少对。只要所有节点没有发生违规时序（建立关

系和保持关系合格，亦即建立余量和保持余量都为正值化)，那么这个设计就 okay 了。换句话说，这个分频模块的物理时序是合格的，因为图 2.3.3~4 正好反馈所有节点的建立余量和保持余量都是正直。

在此有些朋友可能着急了，问道：“图 2.3.3~4 是如何得到的？”。安啦~朋友，前面的故事还没有说完，你这只太监怎么先着急了？不想当太监就要慢慢来 ...

在这个章节当中，笔者只是想帮读者们大概搞个明白 ... TimeQuest 到底是如何使用自身的模型来进行时序分析，这些概念绝对是一个基础。有些玩 TimeQuest 的朋友，根本就没有任何概念可言，有的只是纯公式而已，当然笔者并不反对纯公式的使用，只是纯公式它太无趣了，根本上也称不上学习。

不过在这章节中，笔者太感动了，TimeQuest 真的是努力的孩子，如一个简单的分频模块，就有 6 对节点的时序需要分析，亦即 $C1[0] \sim C1[0]$ ， $C1[0] \sim C1[1]$ ， $C1[0] \sim C1[2]$ ， $C1[1] \sim C1[1]$ ， $C1[1] \sim C1[2]$ 还有 $C1[2] \sim C1[2]$ 。TimeQuest 它都会——去分析没对节点之间的时序。同样的，如果一个设计，里边有 100 对节点需要分析的话，TimeQuest 也会组个去分析 ... 读者说 TimeQuest 是不是努力的好孩子？

2.4 进一步看看 TimeQuest 模型如何分析时序

笔者需要先强调一下，目前笔者还没有准备玩弄 TimeQuest 这个工具，反之了解 TimeQuest 模型才是学习 TimeQuest 的首要功课。笔者这种醉翁之意的心情，可能会使许多心急的朋友焦虑不安 ... 不过笔者劝告道：要玩转 TimeQuest 就需要把 TimeQuest 模型每一个重要部分“窥视”清楚才行。TimeQuest 模型，绝对不是网上所说的那样，几个公式，几个图像就可以解释清楚，实际上它更加神秘许多... 更加好玩。

在 2.3 章节中，我们知道 TimeQuest 模型都会逐个分析分频模块的所有节点之间的建立关系和保持关系。在这个章节当中，我们以“实际延迟”来实验看看 TimeQuest 如何使用模型来工作。因此故事的主角继续以 2.3 章节分频模块为例。

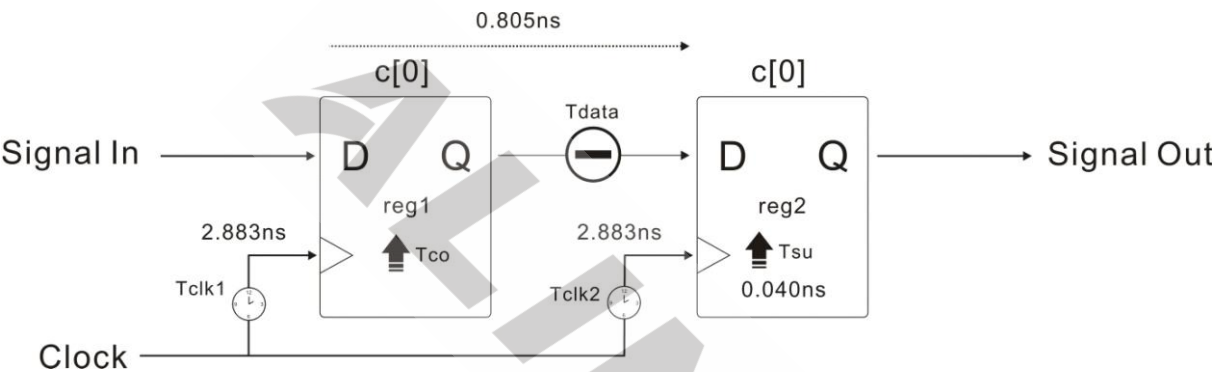


图 2.4.1 节点 C[0]~C[0]的物理状况(建立过程)。

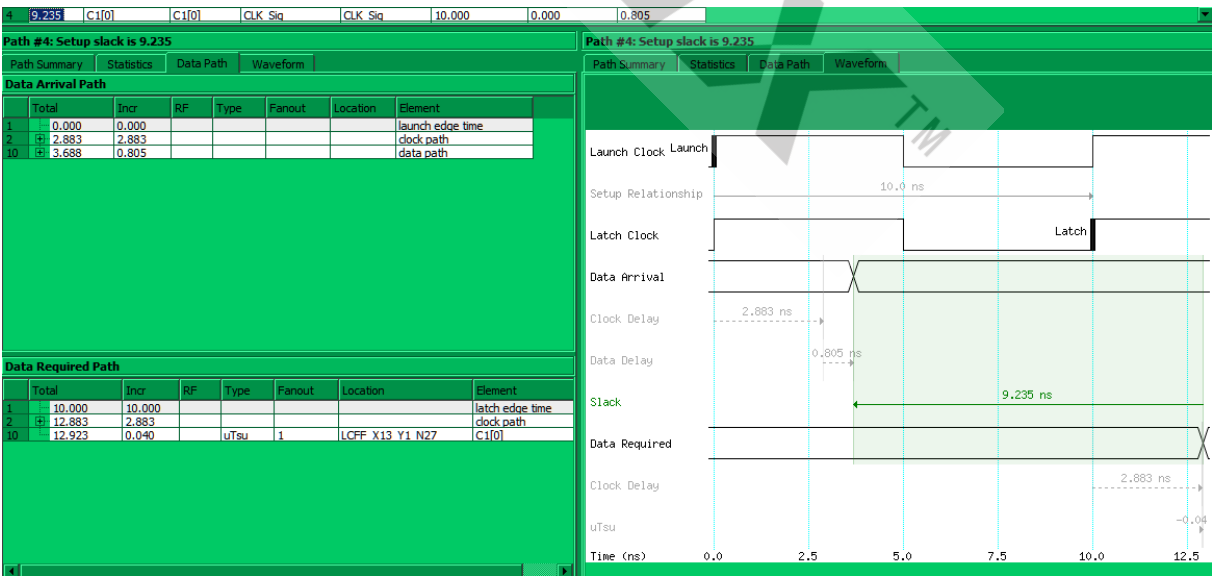


图 2.4.2 C1[0]~C1[0] 建立关系的分析过程。

图 2.4.1 是节点 C1[0]~C1[0]的物理状况，而图 2.4.1 是该节点的建立关系过程。读者暂时把焦点放在 Data Arrival Path 那边... Clock 路径亦即 Tclk1 它有 2.883ns 延迟

(TimeQuest 都是以 ns 为计算单位), Data 路径亦即 $T_{co} + T_{data}$ 它有 0.805ns 延迟。

按照公式计算的话：

$$\begin{aligned}\text{数据抵达时间 Data Arrival Time} &= \text{启动沿} + T_{clk1} + T_{co} + T_{data} \\ &= 0 + 2.883 + 0.805 \quad (T_{co} + T_{data}) \\ &= 3.688\text{ns}\end{aligned}$$

现在又把焦点放在 Data Required Path 的身上，亦即数据获取时间。锁存时间是 10ns (和理想建立关系值等价), Clock 路径亦即 T_{clk2} 它有 2.883ns 的延迟， uT_{su} 就是 T_{su} 寄存器特性，延迟结果是 0.040ns。继续按照公式来计算的话：

$$\begin{aligned}\text{数据获取时间 Data Required Time} &= \text{锁存沿 (理想建立关系值)} + T_{clk2} - T_{su} \\ &= 10\text{ns} + 2.883\text{ns} - 0.040 \\ &= 12.843\text{ns}\end{aligned}$$

erh 怎么数据获取时间 Data Required Time 的结果是 12.923ns 而不是 12.843ns，这显得 TimeQuest 把 T_{su} 加上而不是减掉？这其中发生什么事情了？是不是 TimeQuest 搞错了，还是笔者搞错了？不不不，谁都没有错，这个问题有许多因缘 ... 用傻瓜的话来讲， T_{su} 这个延迟可有可无，实际情况上它非常接近 0ns，因此 T_{su} 可以小到忽略。

笔者稍微更动一下上面的计算结果：

$$\begin{aligned}\text{数据获取时间 Data Required Time} &= \text{锁存沿 (理想建立关系值)} + T_{clk2} - T_{su} \\ &= 10\text{ns} + 2.883\text{ns} + 0.040 \\ &= 12.923\text{ns}\end{aligned}$$

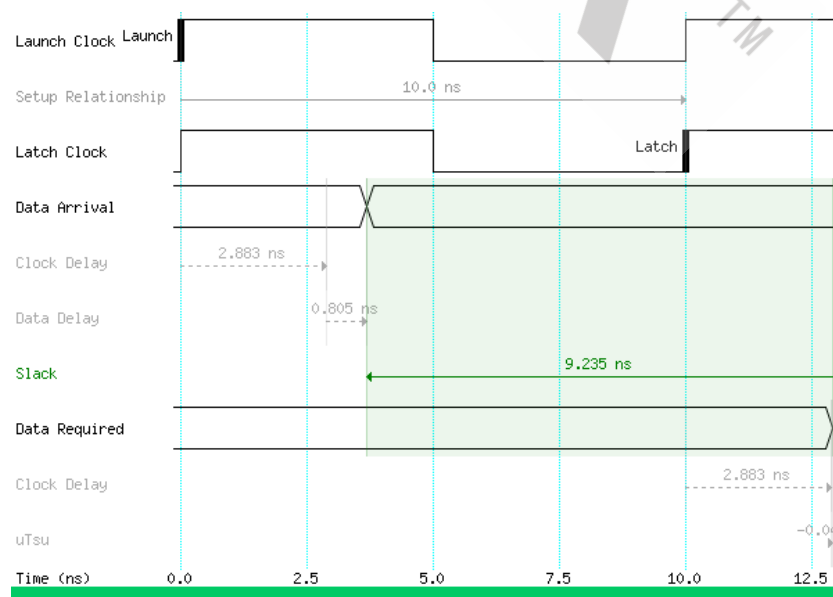


图 2.4.3 节点 c[0]~c[0] 波形图。

图 2.4.3 是 TimeQuest 为节点 c[0]~c[0]提供的波形图，它有非常直观的效果，比笔者涂鸦更规格，真的很惭愧。这张图很容易看懂而已，读者就自己看着办吧。（注意不要过度转牛角尖关于右下角的 - 0.04 ~无视无视）。

最后再计算建立余量的结果：

建立余量 Setup Slack = 数据获取时间 - 数据抵达时间

= 12.923ns - 3.688ns

= 9.235ns

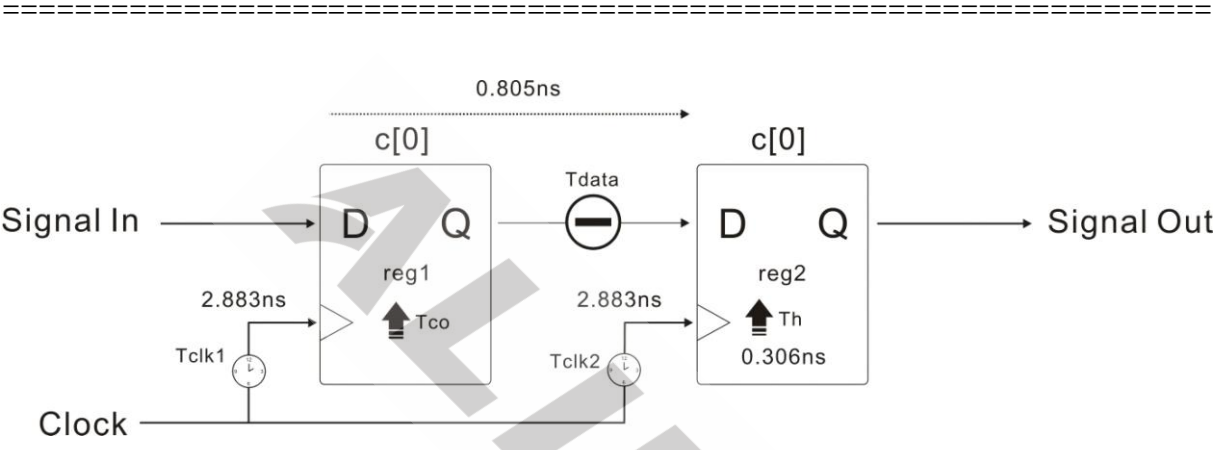


图 2.4.4 节点 C[0]~C[0]的物理状况(保持过程)。

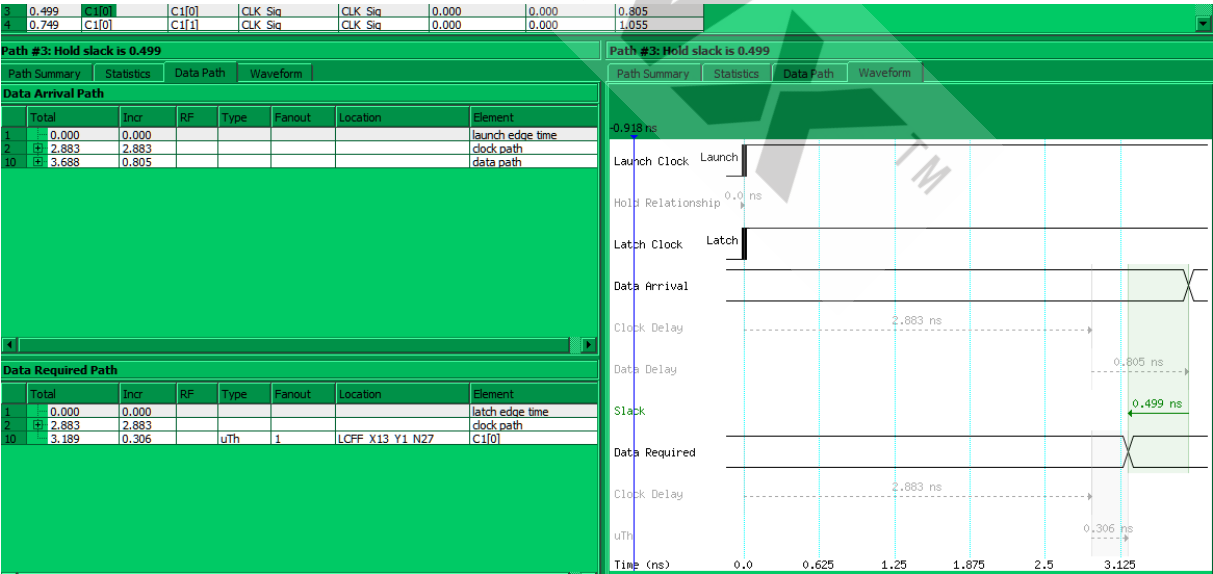


图 2.4.5 节点 C[0]~C[0]保持关系的的分析过程。

在前面笔者曾说过 TimeQuest 模型中求出保持余量的有两种方法：一种是用头脑；另一种是用屁股。很不巧 TimeQuest 确实使用屁股来计算，图 2.4.5 右边的波形图就是最好的证明。嗯 ... 读者是不是觉得很郁闷，无论怎么左看由看，感觉难看不明白？好吧，

笔者再干一次善行 ...

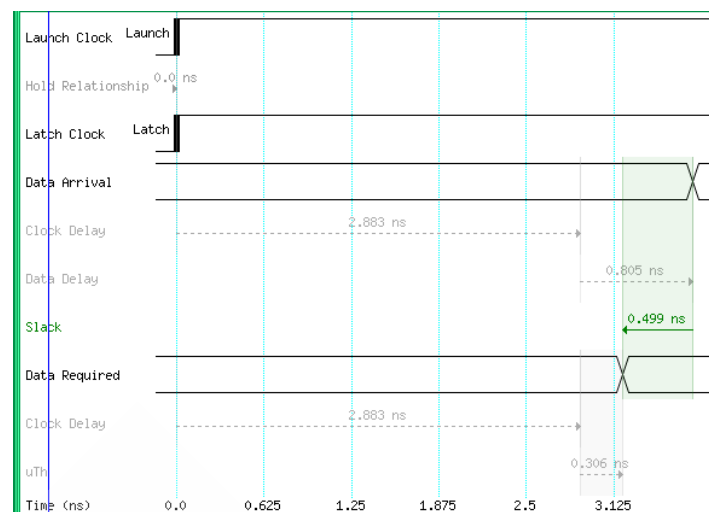


图 2.4.6 节点 C[0]~C[0]保持关系的的分析过程的波形图。

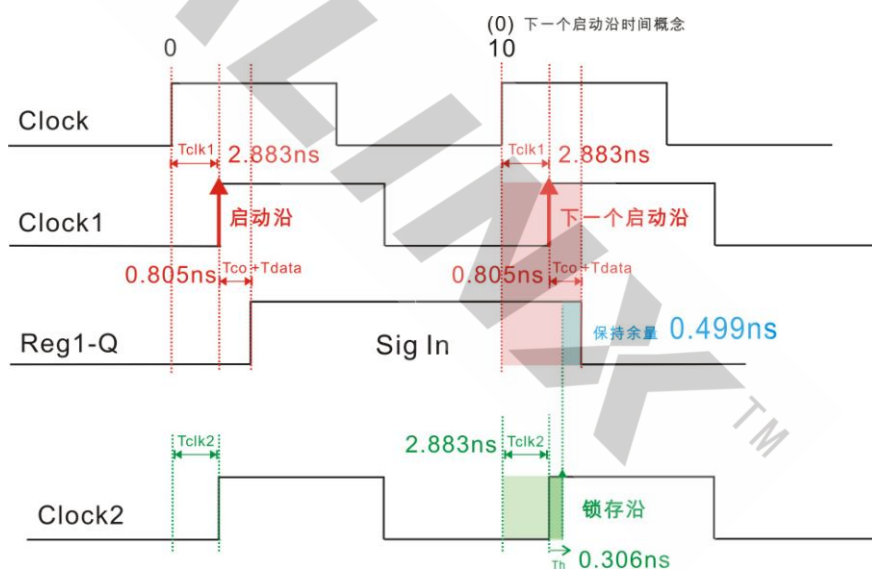


图 2.4.7 笔者为图 2.4.6 加笔的时序图。

从图 2.4.6 与图 2.4.7 的比较中，读者是否可以看出其中的跷蹊？关于这点笔者就不再解释了，自己看着办吧。好了，现在我们知道及节点 C[0]~C[0]之间的理想保持关系值是 0ns，那么保持余量是：

$$\begin{aligned}\text{数据抵达时间 Data Arrival Time} &= \text{启动沿} + T_{clk1} + T_{co} + T_{data} \\ &= 0\text{ns} + 2.883\text{ns} + 0.805\text{ns} \\ &= 3.688\text{ns}\end{aligned}$$

$$\text{数据获取时间 Data Required Time} = \text{锁存沿} + T_{clk2} + T_h$$

$$\begin{aligned} &= \text{理想保持关系值} + T_{clk2} + T_h \\ &= 0\text{ns} + 2.883\text{ns} + 0.306\text{ns} \\ &= 3.189\text{ns} \end{aligned}$$

保持余量 Hold Slack	= 数据抵达时间 - 数据获取时间
	= 3.688ns - 3.189ns
	= 0.499ns

除节点 C[0]~C[0]以外，其他 5 对节点的分析过程也是大同小异。在这里，可能有些心急的同学已经耐不了性子了说，“我要用 TimeQuest，我要搞 TimeQuest！”。安啦安啦！下一章节开始就是了！要玩 TimeQuest 之前，理解一切基本的概念是必须的，读者不要问为什么笔者如此啰嗦，因为在接下来的故事当中，读者自然而然就会明白笔者的苦心。



2.5 建立 sdc 文件与网表

有关 TimeQuest 工具的使用步骤，虽说网上许多版本，不过官方的阵营占大部分，方法都是见仁见智的东西，只要点到重点，什么方法都不重要。使用 TimeQuest 主要有两大步骤，一是配置，二是分析。

配置是指编译器的优化配置，因为不同的配置选项都会产生不同的时序结果。而 TimeQuest 分析，顾名思义就是使用 TimeQuest 分析某个设计。好了！废话还是少说，让我们开始期待已久的 TimeQuest 初次之旅。

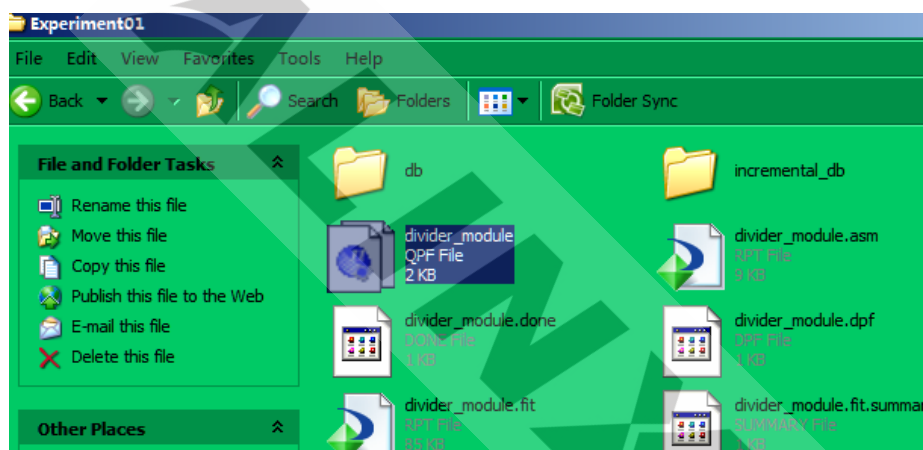


图 2.5.1 打开实验一项目

首先先打开 divider_module 实验一的工程，如图 2.5.1。该设计笔者已经编译过了，为了安全起见，笔者建议再编译一次。

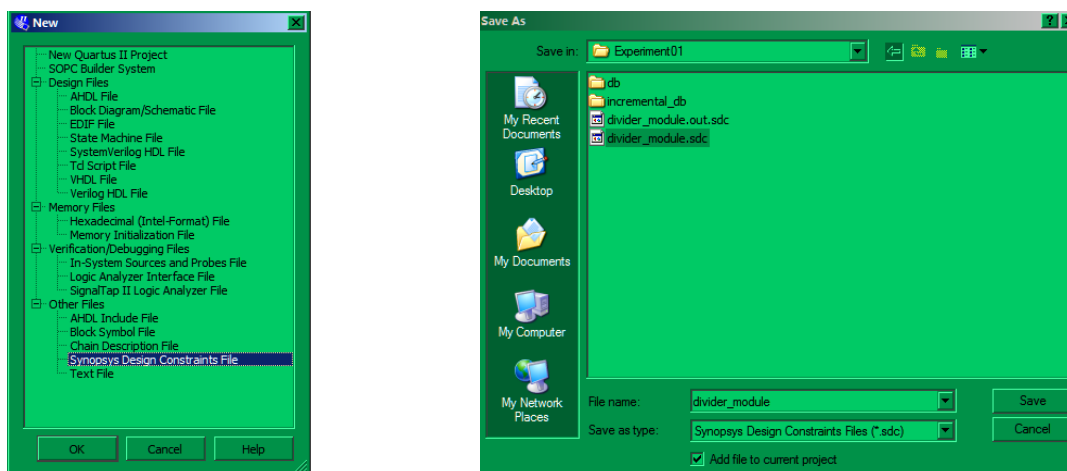
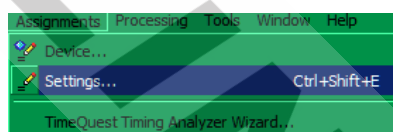


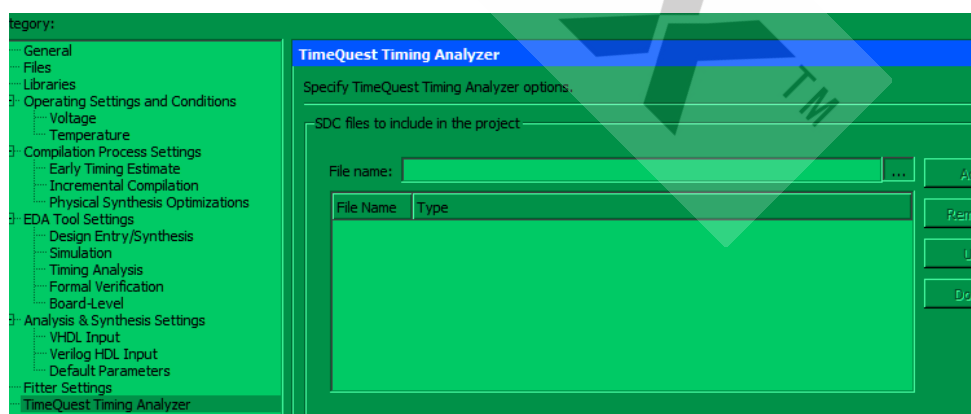
图 2.5.2 建立同名 sdc 文件。

然后新建一个同名的 SDC 文件在实验一的目录下，该文件取名和实验一的项目名一样，亦即 divider_module.sdc，如图 2.5.2 所示。估计实验一的目录下已经有笔者原先建立好的 .sdc 文件，读者们可以删掉再建立，还是直接覆盖都行。

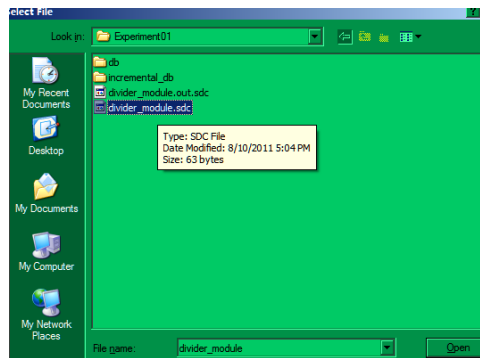
接下来将 divider_module.sdc 文件设置为该实验的默认 sdc 文件，过程如图 2.5.3 所示：



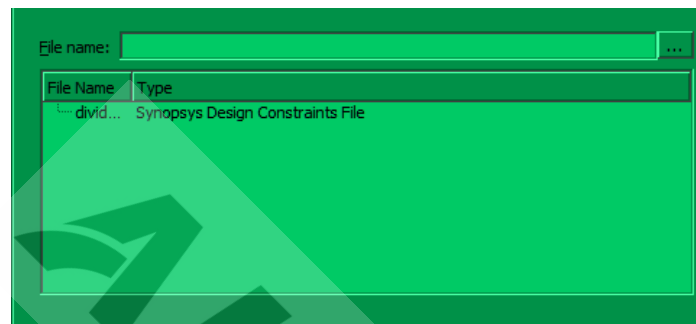
打开设定



选择“TimeQuest Timing Analyzer”，然后点击“...”。



将 divider_module.sdc 装载进来。



点击 Add 把它添加进来。

图 2.5.3 设置实验的默认 sdc 文件。

在这里，读者稍微注意一下 ... Quartus II 版本 10 之前，Classic Timing Analyzer 会作为默认的静态时序分析工具。反之 10 和以后的版本，TimeQuest 作为默认时序分析工具。所以呀，不同的版本之间可能会有所出入，如果读者使用的 Quartus II 的版本是 10 之前的话必须将静态时序分析设置为 TimeQuest。当读者把 divider_module.sdc 文件添加进来以后，为以防万一再编译一次。



图 2.5.4 打开 TimeQuest

当编译完成以后，点击图 2.5.4 显示的 “” 图标，然后就会正式进入 TimeQuest。

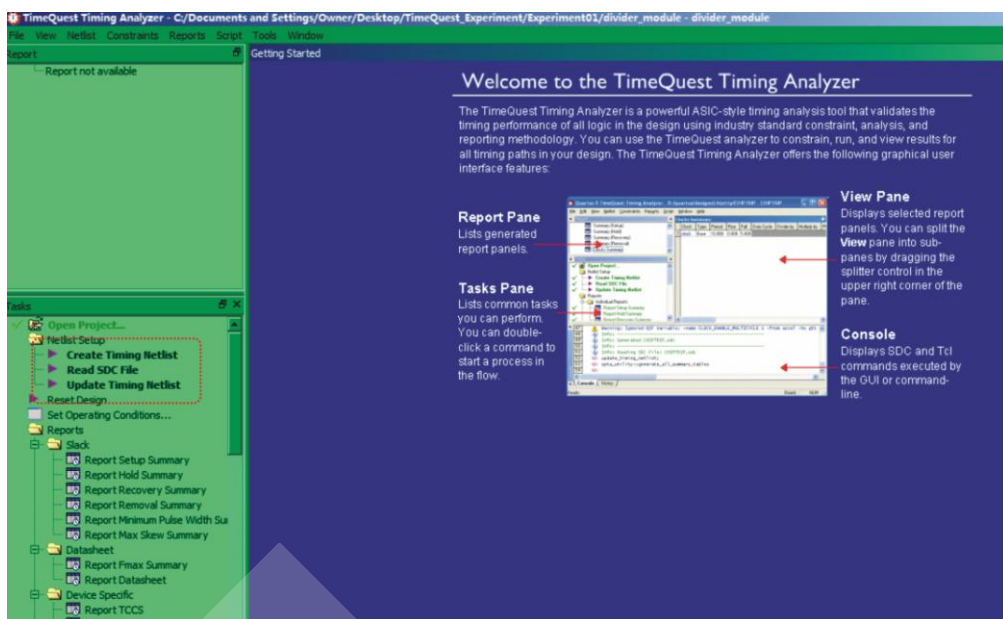


图 2.5.5 TimeQuest 界面

图 2.5.5 是 TimeQuest 的界面，注意左边的红色框图吗？它是这一章节的主角了。

- (一) Create Timing Netlist 说简单点就是建立网表，如果用笔者的话来说，就是软模型 divider_module.v 开始布线，然后建立硬模型。
- (二) Read SDC File 就是读取该项目中 .sdc 文件中“约束配置”。
- (三) Update Timing Netlist 就是更新网表。每当我们更动过“旧网表的约束配置”，然后我们必须再一次更新它。

关于“约束”这一词英文是 Constraint，从字面它是“什么被什么强迫或者限制”的意思。事实上 Constraint 这一词用在 TimeQuest 上，没有那么黄和暴力，而且范围也很广。TimeQuest 是一个工具也是一个笨蛋，宏观上“Constraint”的用意就是“告诉 TimeQuest 要怎么怎么干，要注意什么什么”。

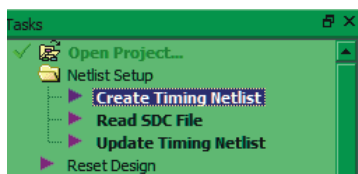


图 2.5.6 自动建立网表。

好了，我们要正式开始建立网表了。如图 2.5.6 所示，只要我们“双击”Create Timing Netlist，那么 TimeQuest 就会自动建立默认网表。但是不是所有同学都那么懒惰的 除了自动建立以外，我们也可以手动建立网表。

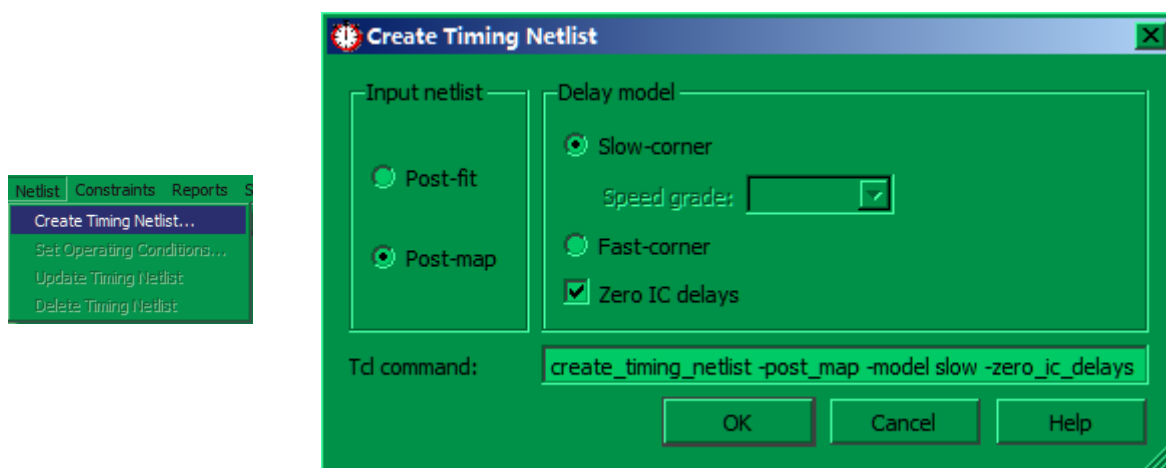


图 2.5.7 手动建立网表。

图 2.5.7 是手动建立网表的经过, 先是从 Netlist 菜单下点击 点击“Create Timing Netlist”, 如图 2.5.7 左图所示, 然后就会自动弹出右图。右图的 Input Netlist 有 Post-fit 和 Post-map 选项, 而且 Delay model 又有 Slow-corner, Speed grade, Fast-corner 和 Zero IC delays 等选项。那么多选项, 想必读者又要抓狂了, 呜哇哇!

同学们千万不要慌, 右图的各种选项设置会造就不同质量的网表, 我们先粗略了解下各种选项字面上的意思 ...

Input netlist 字面上的意思是 “输入网表”。

Post-fit 官方的定义是：

“The TimeQuest analyzer performs timing analysis on a timing netlist that contains physical synthesis optimizations and fitting”。

用傻瓜的话来讲, Post-fit 网表会在建立之际会参考编译器的优化设置结果 (亦即使用 TimeQuest 的第一步)。

Post-map 官方的定义是：

“The TimeQuest analyzer performs timing analysis on a timing netlist that contains complete synthesis information”

用傻瓜的话来讲, Post-map 网表会参照综合结果而建立。结果来说, Post-map 网表更接近原型 (没有优化), 但是 post-fit 更接近物理结果 (有优化)。

Delay model 的意思如同字面上所示, 亦即网表的延迟模型, 用傻瓜的话来讲就是 “网

表的环境”。 “笔者你在说什么，网表的延迟模型和网表的环境到底有什么关系？” 汗！真是一群心急的孩子 ...

Slow-corner 的官方定义是：

“Uses the **worst-case** (slowest) timing model to compute delays depending on the speed grade of the device specified in the Speed grade list.”

worst-case 字面上是指最差环境，也指极端情况。

Fast-corner 的官方定义是：

“Uses the **best-case** (fastest) timing model to compute delays.”

Best-case 字面上是指最好环境，也指舒适情况。

根据 worst-case 与 best-case 的选择也会造就不同的质量的硬模型(网表)，在此读者只要简单记住，worst-case 硬模型有高保险余量高性能限制，而 best-case 硬模型有第保险余量低性能限制即可。如果用傻瓜的话来讲，best-case 比 worst-case 的硬模型更接近实际的网表。

所谓的 zero_ic_delay 就是会无视一些网表基本单位的延迟。

根据上述各种设置以后，我们会得到不同质量的网表 ... 而“不同质量的网表”到底是什么一会事呢？笔者以数学的“ π ”为例，它的常量是 3.142，但是随着精度的不同，它的常量可以是 3.14159265358979，也可以是 3.1415926535897932384。

不过“网表质量”的具体用意到底是什么呢？尝试想象看看 3.142 的 π ，它可以是 3.1415，它也可以是 3.14159；换之 3.141592 的 π ，它不可以是 3.142 也不可以 3.1415。这就是“保险余量”的用意。嘛 ... 有关网表质量就先解释道这里吧，往后读者有一定的基础以后笔者会再详细讨论。

=====

笔者再次强调一下，以上的内容纯属笔者自己的看法而已，至于读者信不信，就自己看着办吧。最后，至于什么样配置的网表才是最正规呢？这个是没有答案的问题！什么样的设计，就需要什么样的网表。不过对于一般实验而言，它就足够的说：

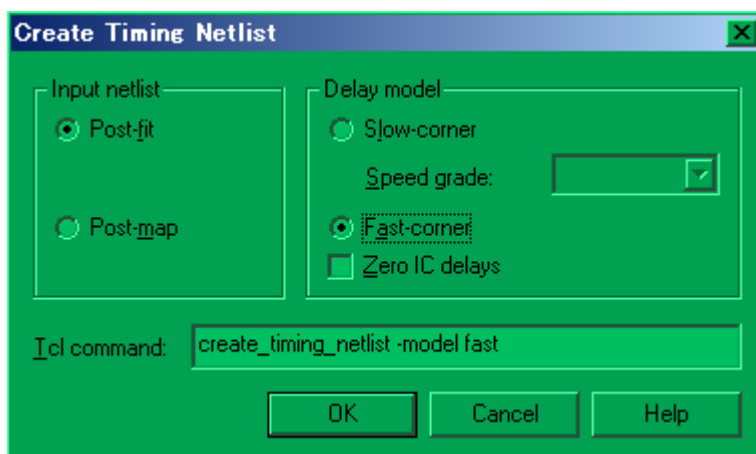


图 2.5.8 够用的网表质量。

图 2.5.8 是够用的网表质量，这种 Create Timing Netlist 的选项也是最接近 fpga 的性能。

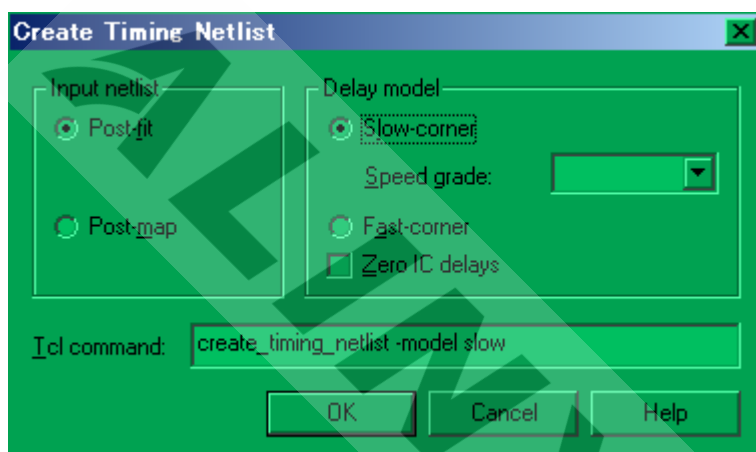


图 2.5.9 默认的网表质量。

然而 TimeQuest 在默认下，亦即快捷双击“Create Timing Netlist”，TimeQuest 会产生如图 2.5.9 所示的网表质量，又称默认网表。

好了，我们先闲聊一会儿让大脑放松放松 ... 在此有读者可能会问：“笔者对 TimeQuest 正体的约束过程，有什么样的概念呢？”嗯，虽然这是一个好问题 ... 让笔者慢慢解释吧。

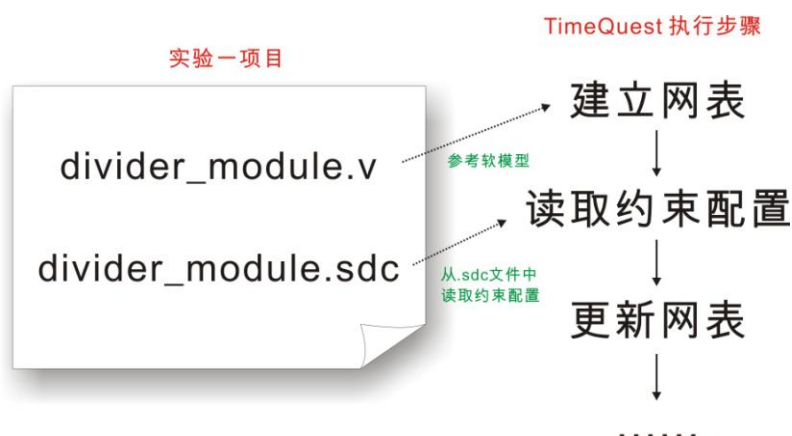


图 2.5.10 TimeQuest 正体的约束过程。

从图 2.5.10 中我们可以知道，当我们要为某一个模块（实验）执行静态时序分析（物理时序分析）的时候，我们必须先建立网表。然而在建立网表的过程，TimeQuest 必须参考软模型，亦即 divider_module.v。但是网表的质量可以根据需要自行调整。接下来 TimeQuest 必须从 divider_module.sdc 中读取“任何的约束配置”，而“更新网表”是把“任何的约束配置”在网表生效。

在 TimeQuest 的一系列执行活动中，如建立网表，读取约束配置，更新网表 等等“这一切都是 TimeQuest 的活动”，所谓 TimeQuest 的活动，它需要从项目中参考相关的内容，如图 2.6.11 所示。假设实验一通过时序分析，然后我们要把实验一的内容下载到开发板中 ... 这个时候我们所需要的东西也只有 .v 文件生成的硬模型而已。

用简单的话来说，当某个实验下载到开发板的时候，它有属于自己的“物理网表（硬模型）”。为了测试这个“物理网表”到底合不合格，所以我们需要用 TimeQuest 这个工具“仿真”这个“物理网表”，而 TimeQuest 不会直接仿真“物理网表”，而是仿真不同质量的网表，又或者仿真不同质量的硬模型。

假设这个设计是发生在极端的环境里，这时候我们就要选择 worst-case 质量的网表，然后透过约束命令将各种延迟信息，时钟信息告诉 TimeQuest。接着 TimeQuest 就会在这个模拟的环境中测试该硬模型，如果时序合格的话，那么该设计即使在极端的环境里也能正常发挥。

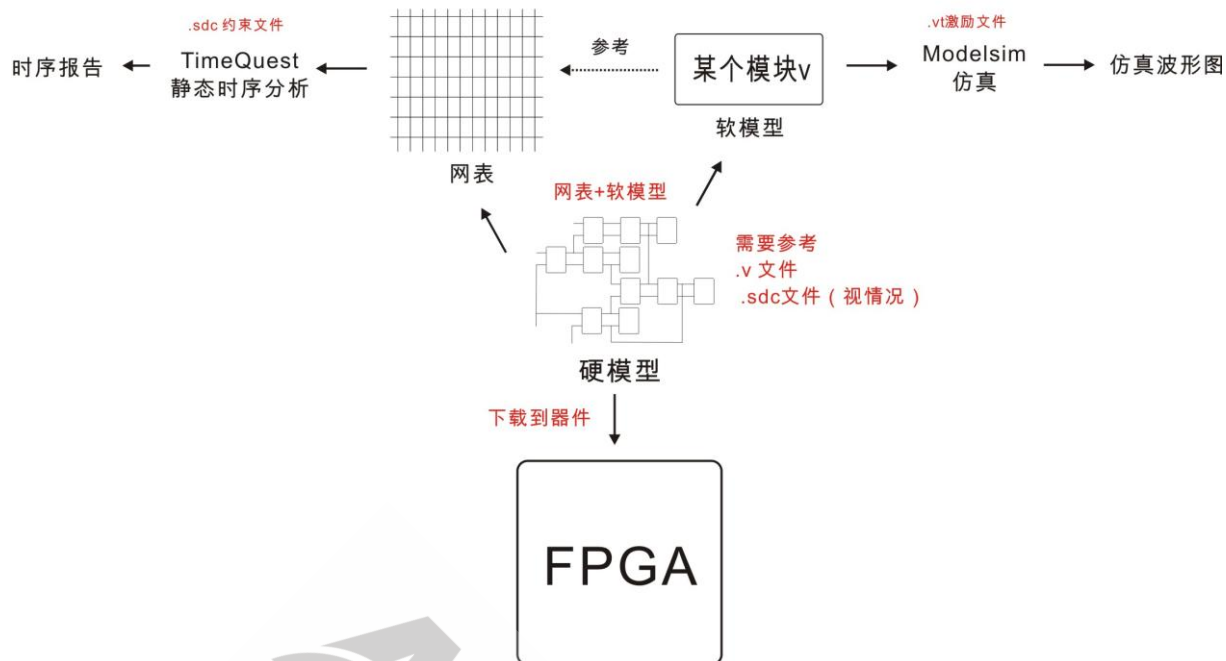


图 2.5.11 笔者眼中的 TimeQuest 概念。

图 2.5.11 是笔者眼中的概念图，硬模型也就是某个.v 文件经过编译和综合的结果。然而一个硬模型可以分成软模型和网表。软模型可以用 Modelsim 仿真模块的功能（行为），其中需要 .vt 文件作为激励过程；网表建立的过程中需要参考软模型；网表可以经 TimeQuest 计算时序，看是否合格？其中可以用 .sdc 文件来“约束”网表。

在早期的实验当中，笔者通通都无视物理时序，换句话说就不把 .sdc 文件看在眼里。当 Quartus II 发现某个设计如果没有被指定 .sdc 文件，那么 Quartus II 就会用默认的“Classic Timing Analysis”去分析网表了（Classic Timing Analysis 似乎只是在早期的 Quartus II 出现--版本 10.0 以前）。如果时钟频率不高的话，以黑金开发板为例 -- 20Mhz，50ns 的周期时间，不管有没有静态时序工具分析网表，硬模型产生的结果都“够用”了。

那些熟读过笔者笔记的家伙，间接上会明白笔者的思维和思路是非常逻辑，所使用的方法当然也是非常逻辑。在这里笔者所强调的是，既然笔者是如此逻辑那么执行 TimeQuest 的步骤（过程）也要逻辑。读者竟可以的话，把 .sdc 文件看成是硬模型的激励文本，或者 TimeQuest 的参考物。

2.6 约束时钟

对 TimeQuest 而言，约束的大概意思是“**注意什么，或者干什么**”，接近的意思是“**利用命令再现分析环境**”有关这一点它与 Modelsim 非常类似，因为 Modelsim 是利用描述语言（其中包括验证与综合）再现仿真环境。所谓“约束时钟”就是告诉 TimeQuest，某个设计“**到底有什么样的时钟**？”。好了，让我们开始 2.5 章节的实验一吧。

实验一的网表配置如下所示，这种作为没有什么特殊的意思。

```
create_timing_netlist -post_map -model slow -zero_ic_delays
```

实验一所使用的 CLK_Sig 是 100Mhz，50%占空比的典型时钟，我们有责任把这个信息告诉 TimeQuest，该过程就是俗称的“约束时钟”。



图 2.6.1 打开同名 sdc 文件。

首先先确定在实验一的项目中 TimeQuest 已经被打开，然后在 QuartusII 中打开和实验一同名的 .sdc 文件，如图 2.6.1 所示。

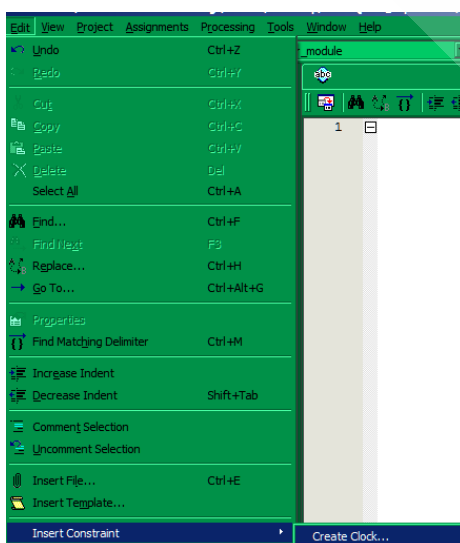


图 2.6.2 调出 create clock 界面。

接着点击 Edit => Insert Constraint => Create Clock ... 如图 2.6.2.所示。

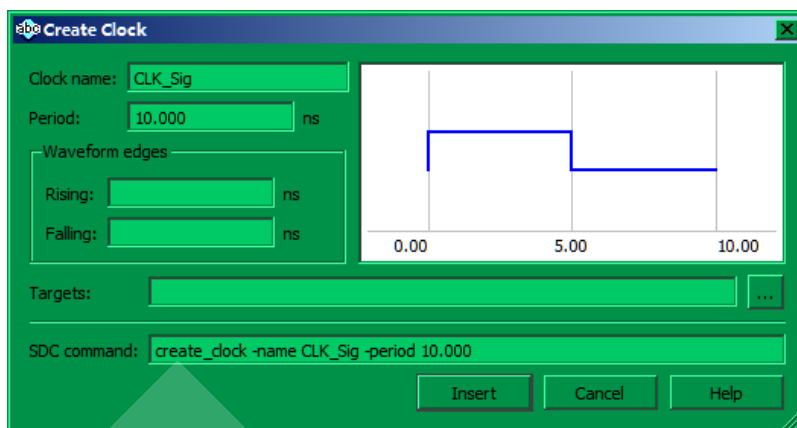


图 2.6.3 Create Clock 界面（时钟声明）。

然后会弹出一个 Create Clock 的窗口，在 Clock name 随便输入一个名字，在这里笔者就输入 CLK_Sig，然后 Period 是 10.00ns 亦即 100Mhz 频率的意思。至于 Rising 和 Falling 是设置占空比，放空下是 50%的占空比。完后点击 < ... > 取得 CLK_Sig 对应的输入口。

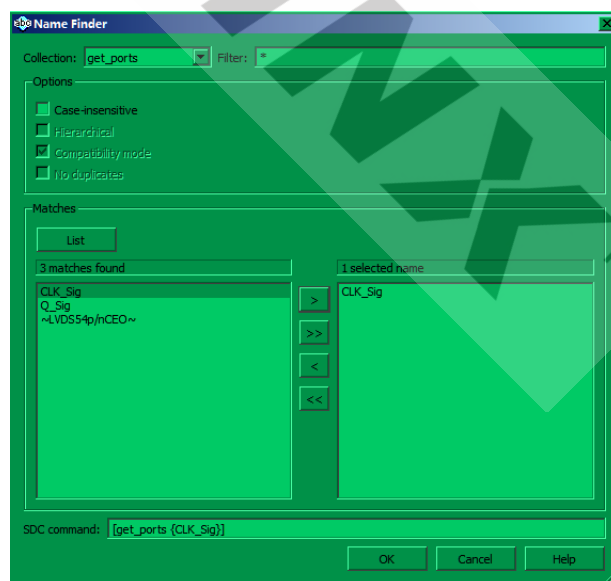


图 2.6.4 寻找对应的基本单位。

图 2.6.4 是经图 2.6.3 点击 < ... > 以后弹出的窗口。接下来我们要干的工作就是告诉 TimeQuest，我们这个 CLK_Sig 的目标对象是哪一个基本单位？

Collection 方面设置 get_ports 亦即选择 port 基本单位。Filter < * > 表示没有过滤，然后点击 < List > 就会列出频模块的所有 port 基本单位。选择 CLK_Sig 然后，再点击

">" 把 CLK_Sig 从左边加入右边，最后点击 <OK> 就行了。

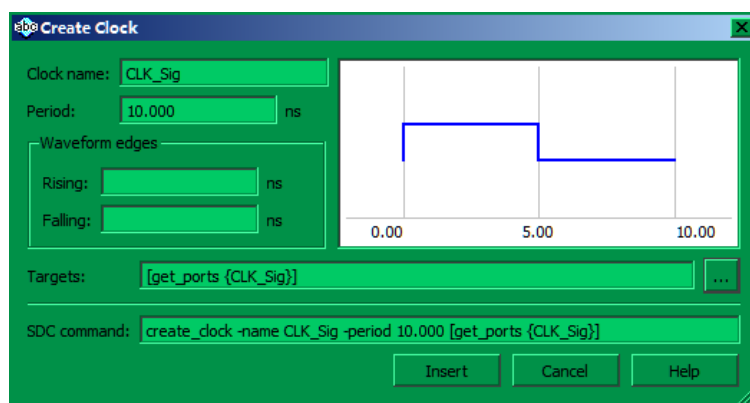


图 2.6.5 对应目标为 port 基本单位的 CLK_Sig。

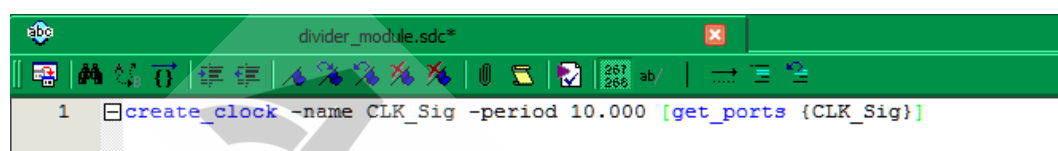


图 2.6.6 生成约束时钟的约束命令（约束代码）。

图 2.6.3 变成图 2.6.5 以后，再最后点击 < Insert > ，然后 create_clock -name CLK_Sig -period [get_ports{CLK_Sig}] 这条 SDC 命令会自动添加并且显示在 Quartus II 中的 .sdc 文件里，结果如图 2.6.6 所示。

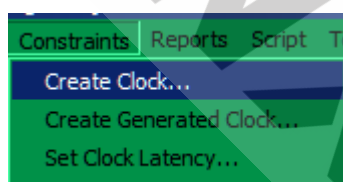


图 2.6.7 在 TimeQuest 中约束时钟。

图 2.6.1~6 是笔者常用的方法，不过笔者也喜欢直接在 TimeQuest 中添加时钟约束，如图 2.6.7 所示那样，基本上过程都是一致的。



图 2.6.8 在 TimeQuest 中保存 sdc 文件。

最后点击保存，这样 .sdc 文件就更新完毕，如图 2.6.8 所示。

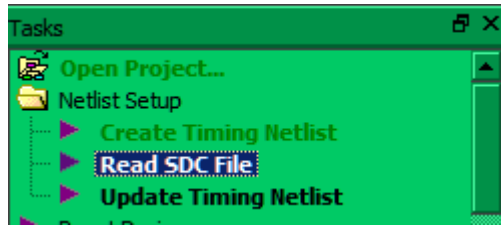


图 2.6.9 读取 sdc 文件。

接下来我们把换面切换到 TimeQuest，然后双击 Tasks 任务窗口中的 Read SDC File，那么 TimeQuest 就会从实验一中的 divider_module.sdc 文件中读取约束配置。到目前为止我们的 divider_module.sdc 仅包含如下这一个命令而已。

```
create_clock -name CLK_Sig -period 10.00 [ get_ports {CLK_Sig} ]
```

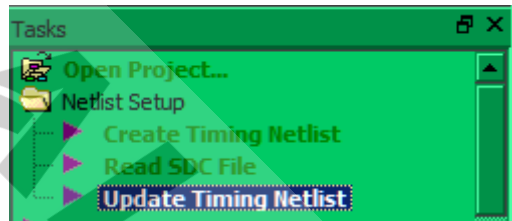


图 2.6.10 更新网表。

当双击 Read SDC File 以后，那么 TimeQuest 就会知道“分频模块所使用的时钟信号名为 CLK_Sig 周期是 10.00ns，而且目标对象是 CLK_Sig 这个 port 基本单位”。最后双击 Update Timing Netlist 如图 2.6.10 所示，那么“约束时钟”的动作就会更新完毕。

2.7 时序报告

在前一章节当中笔者为实验一约束了“100Mhz- 10ns 50%”的 CLK_Sig。用另一句话来说，笔者“恐吓” TimeQuest 说道：“喂！实验一的分频模块是用 10ns 50% 的 CLK_Sig，而且 CLK_Sig 的输入经由 CLK_Sig 输入端口”。在 2.6 章节的最后 我们双点击“Update Timing Netlist” 过后就进入闲聊了 ...

在这一章节当中，我们的工作是要列出时序分析的结果，亦即时序报告。关于时须报告的“列出过程”都是因人而异的，这现象也常常让许多同学法问：“有没有固定的步骤？”好了，在这里就按照笔者的习惯来列出时序报告。

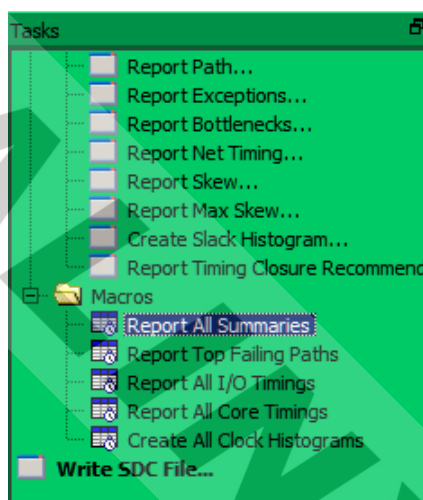


图 2.7.1 列出时序简报。

首先需要列出所有的“时序简介”，在 Task 界面里双点击 < Report All Summaries >，如图 2.7.1 所示，Report All Summaries 会列出所有简单的约束报告。

Clocks Summary						
	Clock Name	Type	Period	Frequency	Rise	Fall
1	CLK_Sig	Base	10.000	100.0 MHz	0.000	5.000

图 2.7.2 时钟信号信息。

到目前为止，我们只是约束一个 CLK_Sig 而已，如图 2.7.2 所示。我们所约束的 CLK_Sig 是 10ns 周期，50% 的占空比（ Rise - 0.0 Fall - 5.0 ）。因此 TimeQuest 已经知道该实验有一个名为 CLK_Sig 周期是 10ns 的时钟信号。

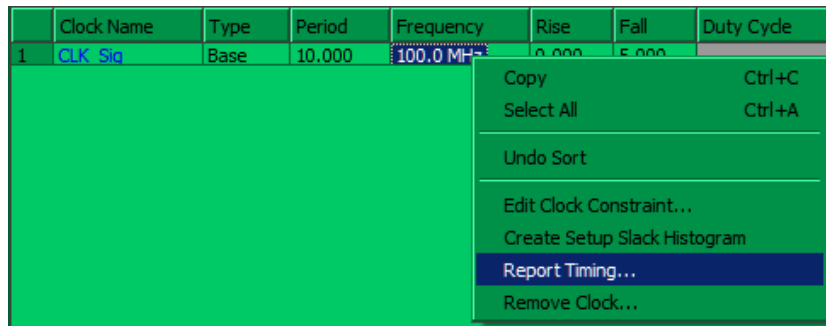


图 2.7.3 调出 Report Timing 界面。

接下来我们要报告出，在实验一的网表中，所有和 CLK_Sig 有关的网表基本单位。如图 2.7.3 所示，在 CLK_Sig 的简介中，“右键点击”然后会弹出一个窗口，最后点击 < Report Timing ... > 就可以了。

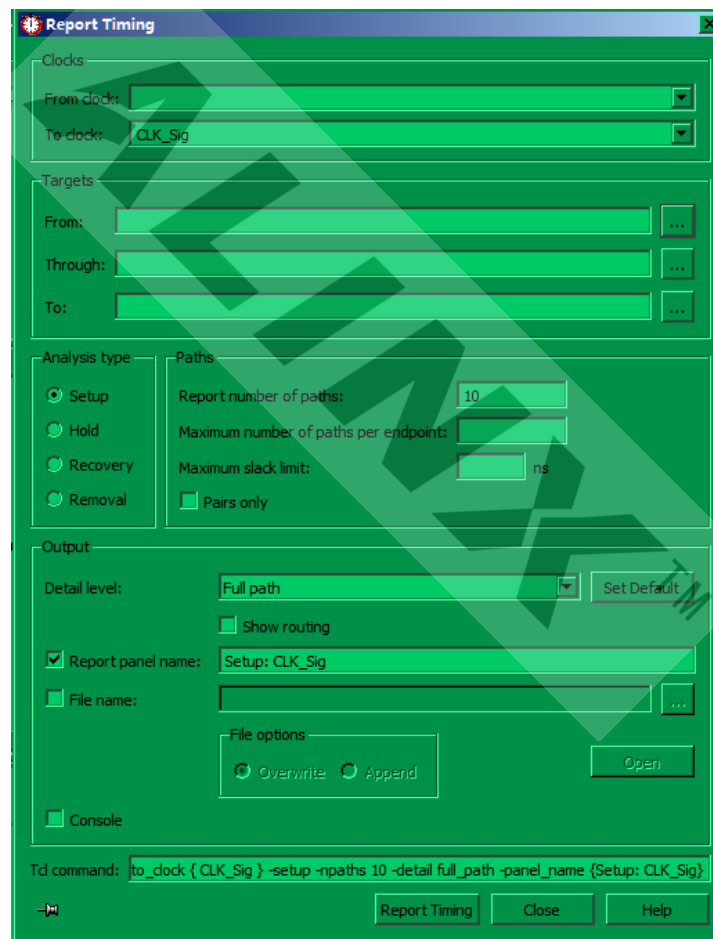


图 2.7.4 Report Timing 的基本界面。

图 2.7.4 是 Report Timing 的基本界面，对于第一次遇见它的朋友来说，可能会吓了一跳 ... Report Timing 几乎是最常用的工具之一，它主要分析 setup, hold, recovery 还有 removal 等时序结果（暂时先不管 recovery 和 removal）。

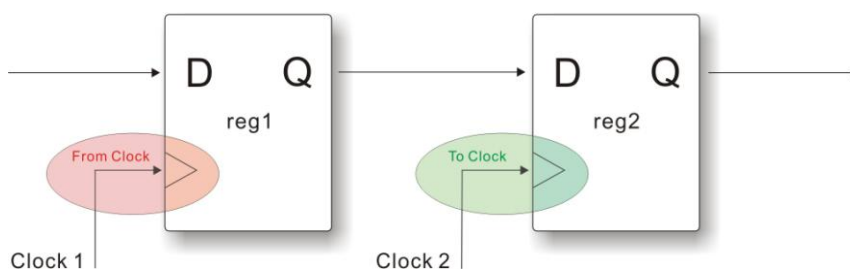


图 2.7.5 From 和 To 与基本单位。

我们知道 TimeQuest 的模型只有在“一对节点”的情况下才能工作。From Clock 在某种意义上是指 reg1 的提供时钟，亦即源时钟；反之 To Clock 在某种意义上是指 reg2 的提供时钟，亦即目的时钟，如图 2.7.5 所示。

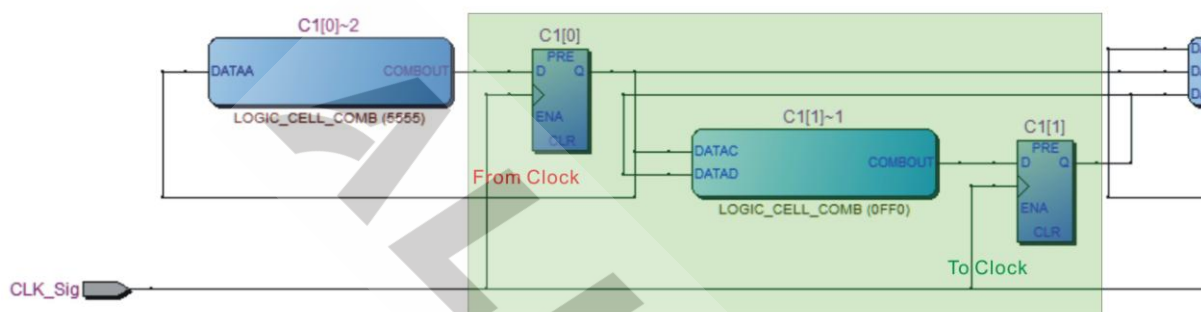


图 2.7.6 节点 C1[0]~C1[1]的 from clock 与 to clock。

如果以实验 1 为例，在 TMV 视图当中，矩形中有一对节点，亦即 C1[0]~C1[1]（图 2.7.6）。其中 C1[0]是 Reg1，反之 C1[1]是 Reg2，Reg1 和 Reg2 它们同是使用 CLK_Sig。在这里我们需要知道一个 TimeQuest 的硬道理“TimeQuest 模型需要一对节点才能成立”。假设笔者在 From Clocks 中输入 CLK_Sig，那么这一对节点 C1[0]~C1[1] 就会被牵扯进来，同样的道理，当笔者在 To Clock 中输入 CLK_Sig，那么 C1[0]~C1[1]也会被牵扯进来。



图 2.7.8 target 的对应选项。

图 2.7.8 中的 Targets 有 3 个选项，亦即 From，Through 和 To，选项的功能如字面上的意思。Targets 是某个节点的基本单位，以实验一为例，如果笔者在 From 中输入 C1[0]如图 2.7.8 所示，该动作就是把 C1[0]有关联的节点都列出来（也可以看成是把 Reg1 有关的“节点”都关联进来）。

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	9.004	C1[1]	C1[2]	CLK Sig	CLK Sig	10.000	0.000	1.036
2	9.235	C1[1]	C1[1]	CLK Sig	CLK Sig	10.000	0.000	0.805
3	9.235	C1[0]	C1[0]	CLK Sig	CLK Sig	10.000	0.000	0.805
4	9.235	C1[2]	C1[2]	CLK Sig	CLK Sig	10.000	0.000	0.805
5	9.422	C1[0]	C1[2]	CLK Sig	CLK Sig	10.000	0.000	0.618
6	9.422	C1[0]	C1[1]	CLK Sig	CLK Sig	10.000	0.000	0.618

图 2.7.9 实验的所有节点。

Command Info	Summary of Paths							
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	9.091	C1[0]	C1[0]	CLK Sig	CLK Sig	10.000	0.000	0.949
2	9.258	C1[0]	C1[2]	CLK Sig	CLK Sig	10.000	0.000	0.782
3	9.258	C1[0]	C1[1]	CLK Sig	CLK Sig	10.000	0.000	0.782

图 2.7.10 实验一部分节点（from 对应 c1[0]）。

我们知道实验一中的分频模块，主要由 6 对节点组成，如图 2.7.9 所示。那么和 C1[0] 有关的节点如图 2.7.10 所示。

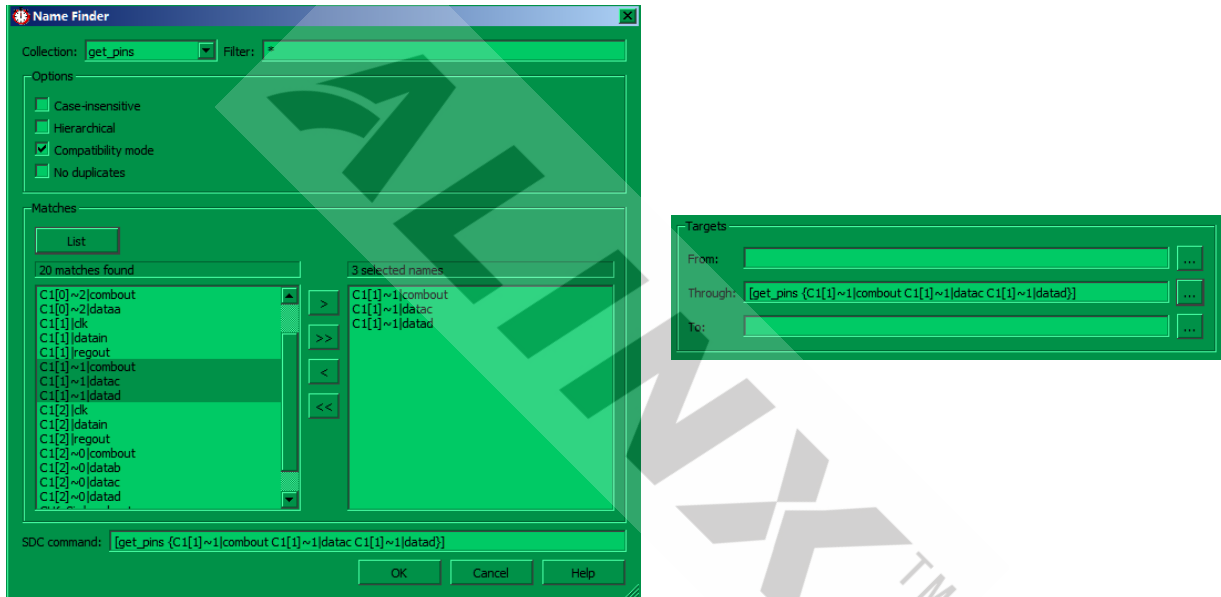


图 2.7.11 Through 的对应选线。

其外，Through 的意思如字面上一般——经由，简单说就是“经过什么”。如图 2.7.11 所示，C1[0]~C1[1]节点之间就有“C1[1]~1”这个组合逻辑。假设笔者在 Through 中输入“C1[1]~1|combout, C1[1]~1|dataa, C1[1]~1|datad”，救过就是把 C1[1]~1 组合逻辑有关的节点关联进来。

Command Info	Summary of Paths							
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	9.258	C1[0]	C1[1]	CLK Sig	CLK Sig	10.000	0.000	0.782
2	9.422	C1[1]	C1[1]	CLK Sig	CLK Sig	10.000	0.000	0.618

图 2.7.12 与 C1[1]~1|combout, C1[1]~1|dataa, C1[1]~1|datad 有关的节点。

结果会如图 2.7.12 所示，与 C1[1]~1 组合逻辑有关联的节点只有 C1[0]~C1[1]和 C1[1]~C1[1]而已。至于 To 是 From 的反面的意思，笔者就不详细介绍了，读者可以把

它看成“把 Reg2 有关的节点都关联进来”。

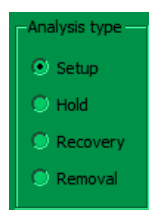


图 2.7.13 时序分析选项。

图 2.7.13 是 Analysis type 的界面。TimeQuest 的模型除了用来分析 Setup 建立关系，Hold 保持关系以外，还有 Recovery 和 Removal。Recovery /Removal 和 Setup /Hold 的差别就在于异步与同步之分，我们暂时不谈它们。假设笔者要列出 (Setup) 建立关系的分析结果，那么只要使能 Setup 就行。

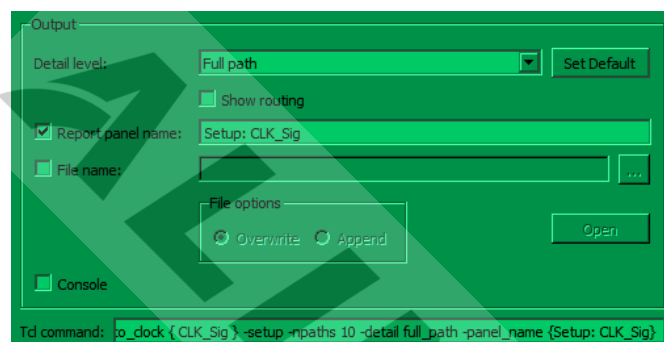


图 2.7.14 output 选项。

图 2.7.14 是 Output 的界面，基本上没有好介绍的，默认下就足够了。最下面的是 Tcl command 命令，看看就好了不要太认真。

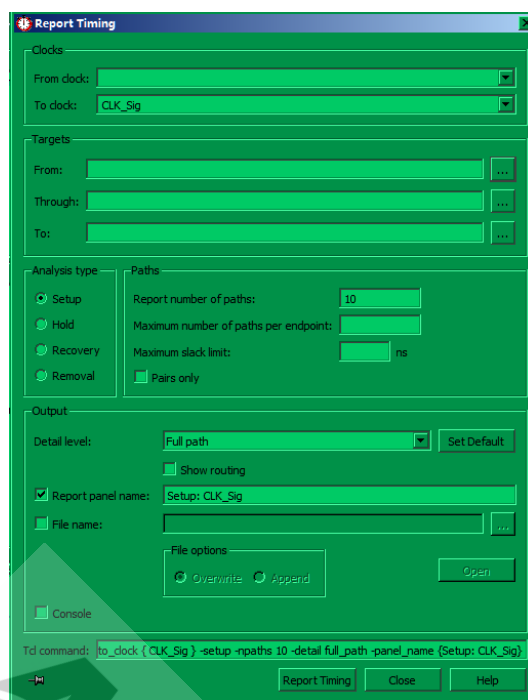


图 2.7.15 Report Timing 界面。

再一次，笔者简单介绍一下图 2.7.15 当前的配置：

在 Clocks 界面中，To Clock 输入了 CLK_Sig，根据实分频模块，其中的 6 对节点都是使用同样的 To Clock，换句话说就是把实验一的全部“6 对节点”都关联进来，而 Targets 空空如也，默认下这也表示把“所有节点”都关联进来的意思：在 Analysis type 中，Setup 被使能，换句话说就是分析建立关系。好了，关于 Report Timing 的界面，笔者介绍得差不多了，笔者是时候点击 <Report Timing>。

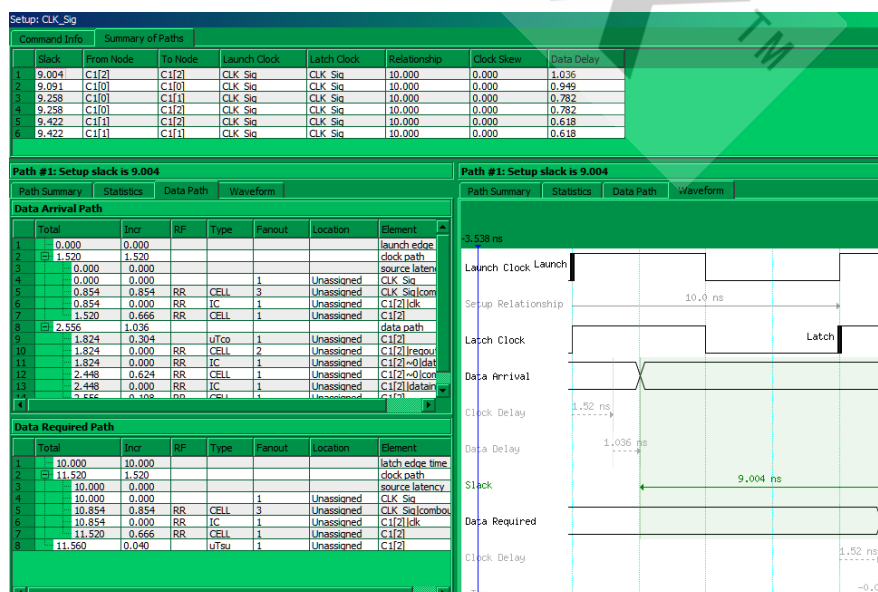


图 2.7.16 详细的 setup 时序报告。

最后就会出现如图 2.7.16 所示的详细 setup 时序报告。

=====

好了,先给自己一段消化的小时间吧 ... 基本上要列出时序报告是没有什么固定的步骤,但是对于一些懒人来说(笔者)最喜欢就是先列出“全部约束—Report All Summary”,然后再“Report Timing”直接把 CLK_Sig 有关的节点都列出来。就这样,要报告出建立关系的结果,就在 Analysis Types 中使能 setup;反之,要报告出保持关系的结果,就在 Analysis Types 中使能 hold 即可。

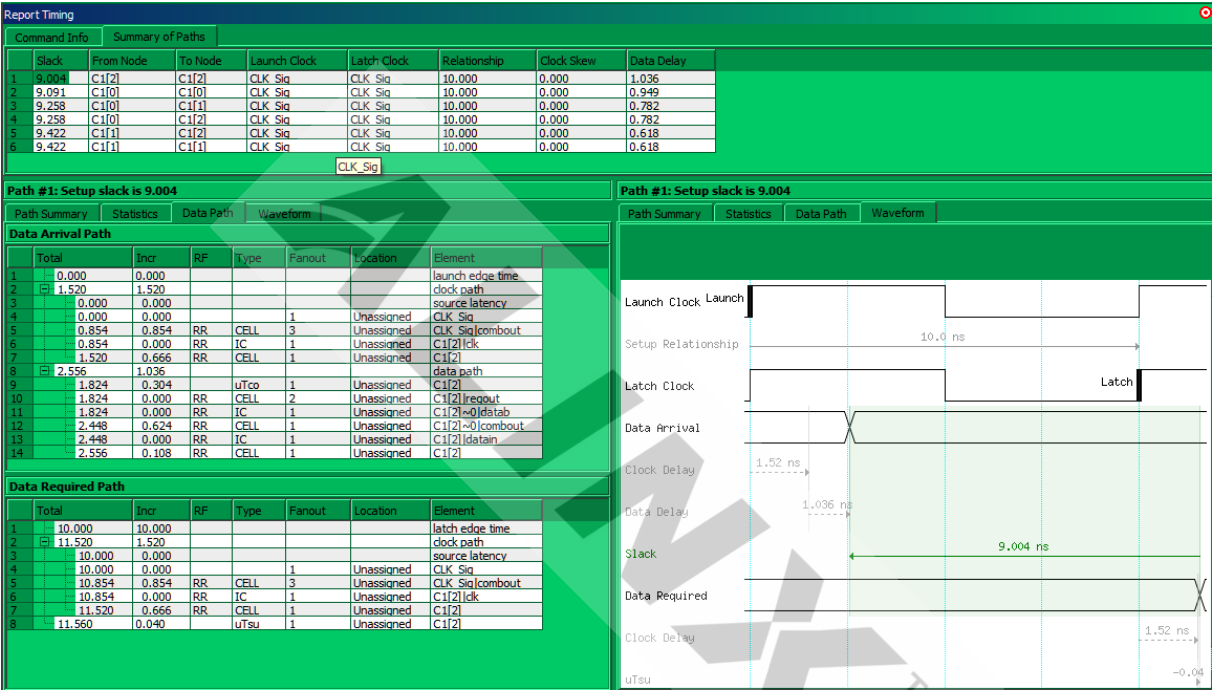


图 2.7.17 详细的 setup 时序报告。

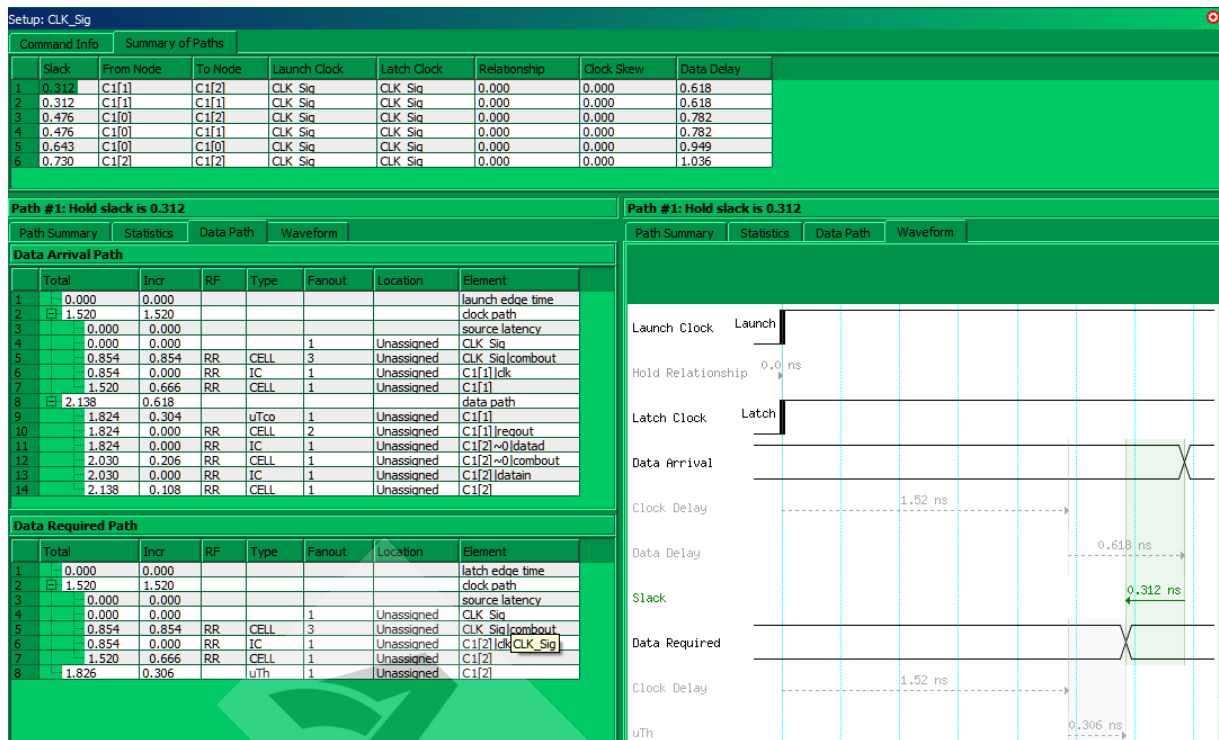


图 2.7.18 详细的 hold 时序报告。

最后我们来看看实验一中的分频模块的建立关系和保持关系的时序报告 如图 2.7.17~18 所示，建立余量和保持余量均为正值，换句话说实验一的时序已经合格。

总结：

在这一章节当中笔者简单并且详细的讨论关于 “TimeQuest 如何使用自身的模型去分析时序是否合格”，“网表”，“约束”，“TimeQuest 的使用步骤” 和 “时序报告” 等概念。第二章的一切一切，笔者还是在讨论一些基本的概念而已。

在笔者的眼中，要入门 TimeQuest 明白一些最基本的内容是关键的部分。TimeQuest 虽说它是一个笨蛋的工具，但是设计者是用什么思维去设计它，这是学习 TimeQuest，也是掌握 TimeQuest 的重点。以前，笔者初次接触 TimeQuest 的时候，一直在 “如何使用 TimeQuest” 这个大门外打转，结果笔者打圈圈 3/4 年。

那时候的笔者虽然知道 “保持余量” 的概念，但是却对求出 “保持余量” 的公式充满着问题 因为笔者还不知道如何使用屁股去分析保持余量的公式。此外，笔者也对 “TimeQuest 如何用自身的模型” 去分析一个设计，充满着许多问号。最后还有一个难题就是 “笔者没有网表的大致定义 ”

“网表的定义” 是把所有概念联系起来的重要 “中间人”，只有知道网表的位置和定义，那么所有东西都可以串联起来了，这种感觉如同工藤新一的头部穿过电流那样。

好了，笔者不废话了，最后来个简单的总结吧：

1. 一个硬模型基本上是由网表和软模型经过编译组成的。
2. TimeQuest 是一个分析网表 “是否合格” 的工具（读者可以看成是网表的仿真工具）。
3. 其中网表的生成需要参考软模型。
4. 网表有质量之分。
5. “约束” 的定义，在这个章节当中是 “告诉 TimeQuest 干什么，或者注意什么”。
6. TimeQuest 是用模型逐个去分析所有节点之间的保持建立关系。
7. Technology Map Viewer 和 TimeQuest 有关系。
8. 可以把 sdc 看成是硬模型的激励文本。
9. TimeQuest 没有固定的使用步骤。
10. “Report Timing” 是一个很好用的工具。

嗯，就是这些而已。更多有趣的内容和学习，我们下一章再见。

第三章：TimeQuest 扫盲文

3.1 余量的概念

笔者常常在思考，虽说 TimeQuest 是一个分析物理时序是否合格的数学工具，但是 TimeQuest 真正意义上是在针对什么？这是一个很有趣的问题，而 TimeQuest 有一个关键字，就是“约束”英文是“Constraint”。

“约束”如果用傻瓜的话来解释，就是告诉 TimeQuest “应该干甚么或者注意甚么”，然而其中的“甚么”又是“甚么”？这绝不是在鸡蛋里挑骨头的问题。如果不把“这个什么”搞清楚，我们就不能完全理解 TimeQuest 到底为了什么目的而问世？根据笔者的理解“这个什么”就是延迟因数，亦即“TimeQuest 是一个针对各种延迟因数作出约束的静态时序分析工具”。

有一个问题笔者一直无法释怀，当 Modelsim 仿真 HDL 模块之际，如果考虑延迟信息，时序就会出现物理延迟，换之在 Modelsim 界面上出现的波形，事实上也是静态时序的一种，不过 Modelsim 只能看个大概而已，而不能做出实际的分析。所以笔者一直认为，Modelsim 既然要仿真就仿真理想时序而不是物理时序。所以说，物理时序分析的工作我们就交给 TimeQuest 去干。

理想时序是不存在延迟因数的时序，而物理时序是充满延迟因数的时序；理想时序是 HDL 产生的行为，而物理时序硬模型产生的行为。理想时序可以用 Modelsim 来仿真，而物理时序可以用 TimeQuest 来分析。

好了！终于搓到重点了，笔者曾讲过：“Verilog HDL 是关心理想时序，而 TimeQuest 是关心物理时序”。《整合篇》未完成之前，笔者认为道：“理想时序和物理时序是两个世界的家伙”。换之，《整合篇》完成之后，笔者认为道：“理想时序和物理时序可以用黏糊概念联系起来”。

让我们先来聊聊黏糊概念吧：

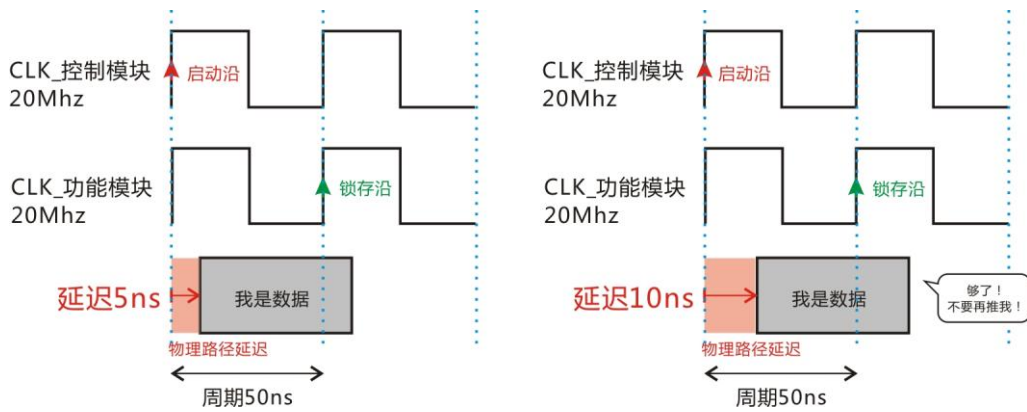


图 3.1.1 物理时序的推向概念。

图 3.1.1 所示是物理时序的推向概念，数据被 5ns/10ns 延迟向右推挤以后就失去对齐性，而推向的距离就是延迟值。

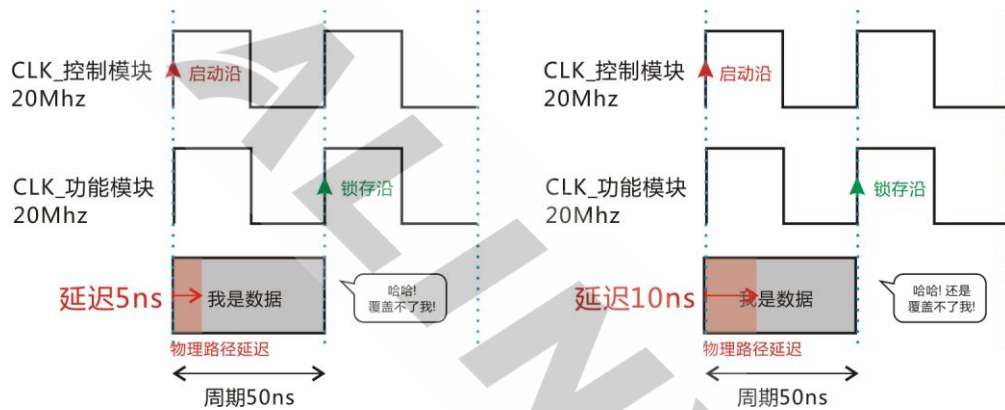


图 3.1.2 理想时序的覆盖概念。

图 3.1.2 所示是理想时序的覆盖概念，数据和时间沿之间没有失去对齐性，然而延迟实数会根据数值覆盖数据。黏糊概念有着一个逻辑，假设同学带足 1 万块出远门，只要 1 万块消费不到零值，这趟远门是开心的。这种比喻就好比一块有 10ns 周期的数据，只要 10ns 的容量不完全被延迟覆盖，这块数据就不会失去有效性。

黏糊概念和保险余量，建立余量还有保持余量有许多相似的地方，不过黏糊概念是用图形表达，而保险余量，建立余量还有保持余量是用数值来表达。笔者曾一度认为，TimeQuest 的延迟取值是按实际的物理延迟，不过事实恰恰相反，TimeQuest 对延迟取值是夸张的表现，这话何解呢？

假设有一条路径的物理延迟是 10ns，如果是 worst-case 网表 TimeQuest 就会报 18ns，如果是 best-case 网表 TimeQuest 就会报 12ns。很明显，18ns 还有 12ns 的报值比起实际的 10ns 多出 8ns 还有 2ns，其中这 8ns 和 2ns 的差别就是保险余量。

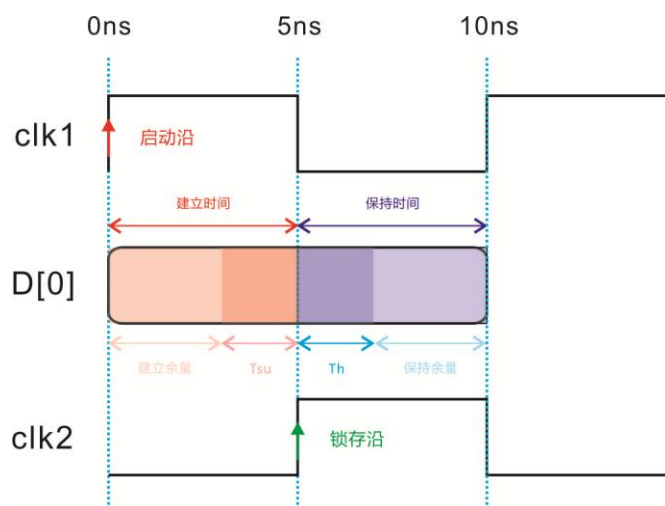


图 3.1.3 一块数据的有效性。

假设一块数据有 10ns 的周期，而锁存沿正好在 5ns 的位置锁存数据。根据理论而言，一块数据的有效性主要分为建立时间与保持时间。Tsu 是寄存器在锁存数据之前需要的最小建立时间（热生时间），换之 Th 是寄存器在锁存数据之后需要的最小保持时间（确保时间）。然而建立时间减掉 Tsu 就是建立余量，而保持时间减掉 Th 就是保持余量。（如图 3.1.3 所示）。

建立余量是用来评估数据有效的建立时间，只要建立余量一天不被吃光，这块数据的建立是有效的。保持余量是用来评估数据有效的保持时间，只要保持余量一天不被吃光，这块数据的保持是有效的。

所以说，保险余量，建立余量还有保持余量越多，设计就越安全。这感觉就好比读者出远门有 1 万块的金钱余量。不过保险余量与建立余量还有保持余量是不同层次的概念余量。保险余量的感觉就好比读者向妈妈要钱买玩具。

妈妈问：“玩具要多少钱？”，假设玩具的成本是 10 元，不过根据不同的卖家价钱也有可能是 18 元或者 20 元 ... 于是，贪心的读者便狮子夸大口说：“玩具要 50 元”。只要读者向妈妈要钱越多，妈妈对读者的信任就会越来越少。换一个角度来思考的话，10 元的成本就是实际的延迟，不同卖家的价钱就是根据环境因数造成的预测延迟，而读者开口的 50 元就是保险余量，至于妈妈对读者的信任就是性能限制。

50 元就像 worst-case 网表（硬模型），30 元就好比 best-case 网表（硬模型）。有 50 元虽然买玩具更安全，但是妈妈越不信任读者，换之 30 元比起 50 元虽然损失一些安全性，但是妈妈更加信任读者。以此类推，假设玩具店 A 售价是 32 元，只有 30 元保险余量而已的 best-case 当然买不起的玩具店 A 的玩具。换之 50 源的 worst-case 却买得起。

再假设，玩具店 B 售价是 15 元，30 元的 best-case 网表或者 50 元的 worst-case 网表必定买得起玩具店 B 的玩具。不过读者千万不要太贪心骗妈妈成本只有 10 元的玩具售价是 100 元啊，不然的话妈妈会永远对你失去信心。

网表质量中也有一个关键的概念，亦即性能限制 ... 假设路径 A 有 10ns 的延迟，而这个 10ns 的延迟又叫成本延迟，是实际的路径延迟取值。不过 TimeQuest 声明的网表往往都会高出成本延迟，如 worts-case 网表会使路径 A 取值为 50ns，又或者 best-case 网表会使路径 A 取值为 20ns。

TimeQuest 有一个评估值，称为 Fmax。Fmax 是用来评估节点的最高频率。每当网表质量越低，节点的延迟取值就会越高，Fmax 也随之会越低。如果 Fmax 越低 fpga 就不能发挥实际的速度和性能，因此我们可以这样说：网表质量越低，保险余量越大，Fmax 评估越低，性能限制也越高。



3.2 延迟因数：内部延迟因数

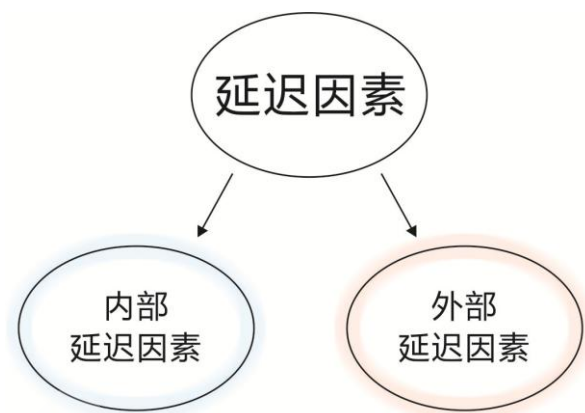


图 3.2.1 延迟因数。

延迟因数一般上可以分为两种，亦即内部延迟因数和外部延迟因数，如图 3.2.1 所示。



图 3.2.2 内部延迟因素。

不过 TimeQuest 比较在意的内部延迟因数有图 3.2.2 所示，其中是 pll，内部路径，组合逻辑还有寄存器特性。不管哪一种内部延迟因数，它们都有一个共同点，亦即 TimeQuest 会自动知晓它们的延迟取值。此外，根据不同质量网表，延迟取值也会有所不同 worst-case 网表的延迟取值比 best-case 网表的延迟取值当然多出不少。

PLL 事实上就是一种不开又不关的内部延迟因数，虽然 TimeQuest 会认识它，但是 TimeQuest 不会完全公开它；内部路径可说是最普遍的内部延迟因数，其中 data path 还有 clock path 都会出现它的身影；组合逻辑可说是内部延迟因数中最臃肿的家伙；寄存器特性算是不怎么起眼的内部延迟因数。

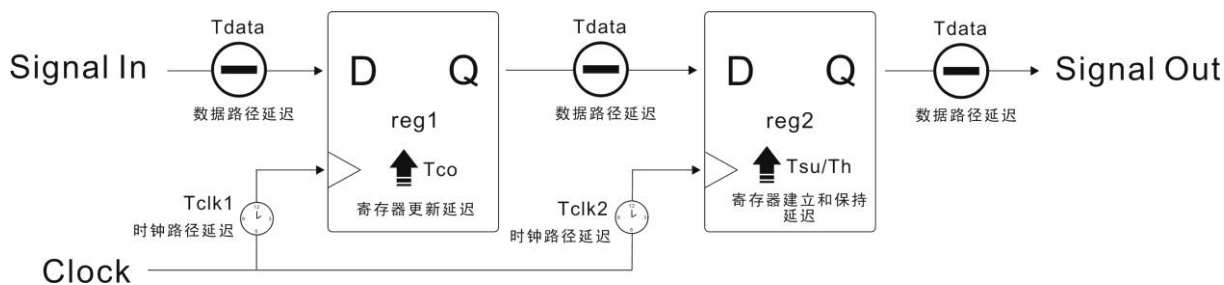


图 3.2.3 某对节点的内部延迟。

图 3.2.3 是某对节点的物理情形，其中充满许多内部延迟因数， T_{data} 可以是路径延迟或者组合逻辑； T_{clk1} 与 T_{clk2} 就是路径延迟；而 $T_{co}/T_{su}/T_{th}$ 就是内部延迟因数的寄存特性。不过很遗憾的是图 3.2.3 没有 pll 的声影，没关系 ... 笔者会另行举例的。

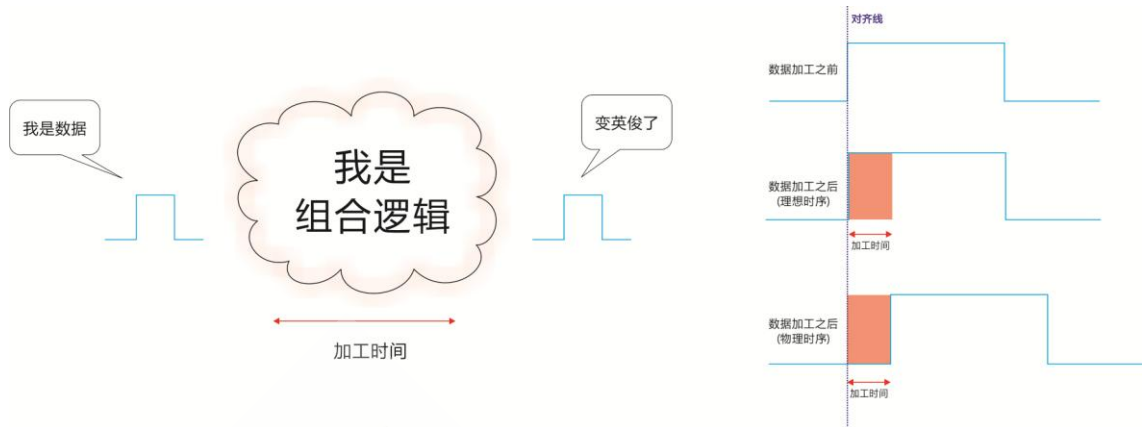


图 3.2.4 组合逻辑加工数据信号。

要明白 pll 之前，读者必须组合罗加工数据的基本概念，如图 3.2.4 所示，有一块数据经过组合逻辑以后就变英俊了，而数据便英俊的过程如右图所示。在理想时序下，加工时间以及组合逻辑的延迟会将部分数据覆盖掉，换之 ... 物理时序的情况下，加工时间会向右推挤数据。

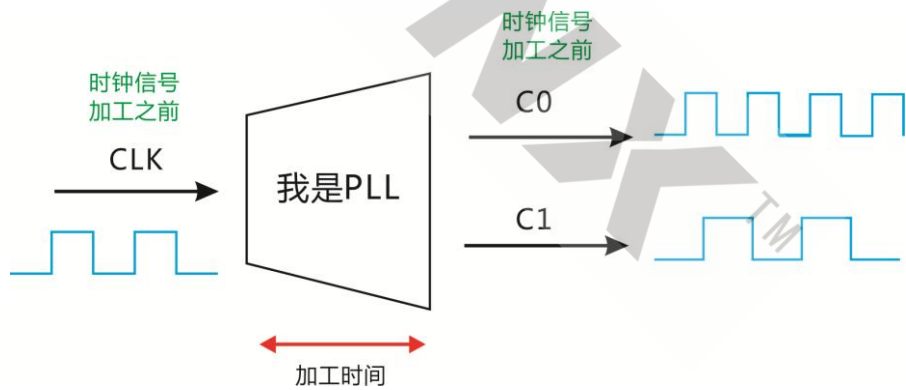


图 3.2.5 PLL 加工时钟信号。

pll 与一般的组合逻辑区别的地方就在于，组合逻辑加工数据信号，而 pll 则加工时钟信号。如图 3.2.5 所示，clk 是加工之前的时钟信号，而 c0 与 c1 是加工之后的时钟信号，其中 pll 也有加工时间亦即 pll 延迟。

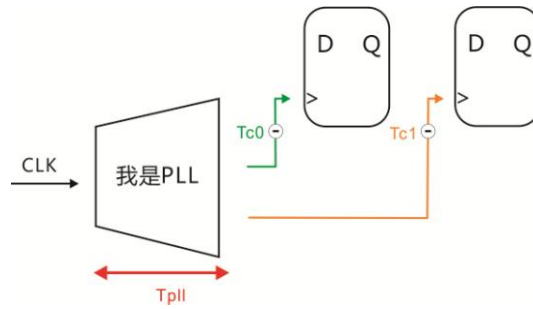


图 3.2.6 PLL 给两个寄存器各别提供 C0 和 C1 的时钟信号。

我们假设 C0 和 C1 都有供源对象，其中时钟信号 C0 和 C1 经过的路径多少也有延迟，其中又名为 Tc0 和 Tc1，如图 3.2.6 所示。

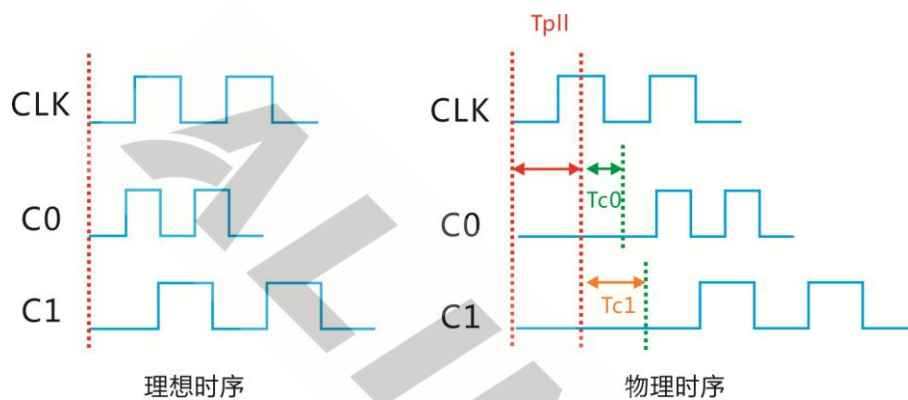


图 3.2.7 PLL 的理想时序和物理时序。

仿真过 pll 的同学当然知道，如果仿真没有加入延迟信息，pll 就会产生左图的理想时序 ... 换之，如果仿真加入延迟因数就会产生右图的物理时序。其中 Tpll 作为时钟信号 c0 和 c1 的起跑线。TimeQuest 在分析 pll 的时候会特别关心 Tc0 和 Tc1 时钟路径。让笔者说说一些题外话，左图的 pll 会产生的理想时序，期间 pll 和其他组合逻辑一样，亦即触发即时事件。

求得建立余量的公式是：

$$\begin{aligned} \text{保持余量} &= \text{数据锁存时间} - \text{数据抵达时间} \\ \text{Setup Slack} &= \text{Data Required Time} - \text{Data Arrival Time} \end{aligned}$$

$$\text{数据抵达时间 Data Arrival Time} = \text{启动沿} + T_{clk1} + T_{co} + T_{data}$$

$$\begin{aligned} \text{数据获取时间 Data Required Time} &= \text{锁存沿} + T_{clk2} - T_{su} \\ &= \text{理想建立关系值} + T_{clk2} - T_{su} \end{aligned}$$

求得保持余量的公式是：(用头脑的话)

保持余量 = 数据保持时间 - 数据获取时间
 $\text{Hold Slack} = \text{Data Hold Time} - \text{Data Required Time}$

数据保持时间 $\text{Data Hold Time} = \text{启动沿} + T_{\text{clk1}} + T_{\text{co}} + T_{\text{data}} + \text{数据周期时间}$

数据获取时间 $\text{Data Required Time} = \text{锁存沿} + T_{\text{clk2}} + T_{\text{h}}$

求得保持余量的公式是：(用屁股的话)

保持余量 = 数据抵达时间 - 数据获取时间
 $\text{Hold Slack} = \text{Data Arrival Time} - \text{Data Required Time}$

数据抵达时间 $\text{Data Arrival Time} = \text{启动沿} + T_{\text{clk1}} + T_{\text{co}} + T_{\text{data}}$

数据获取时间 $\text{Data Required Time} = \text{锁存沿} + T_{\text{clk2}} + T_{\text{h}}$
 $= \text{理想保持关系值} + T_{\text{clk2}} + T_{\text{h}}$

上面有三段公式，笔者想必读者不会觉得陌生吧？其中在公式出现的 $T_{\text{co}}/T_{\text{h}}/T_{\text{su}}$ 就是内部的寄存器特性。嘛 ... 有关内部延迟因数，寄存器特性的作用极微甚小，不足影响保持余量和建立余量，而寄存器特性在物理时序中它们都会完全忽略掉。

3.3 延迟因数：外部延迟因数

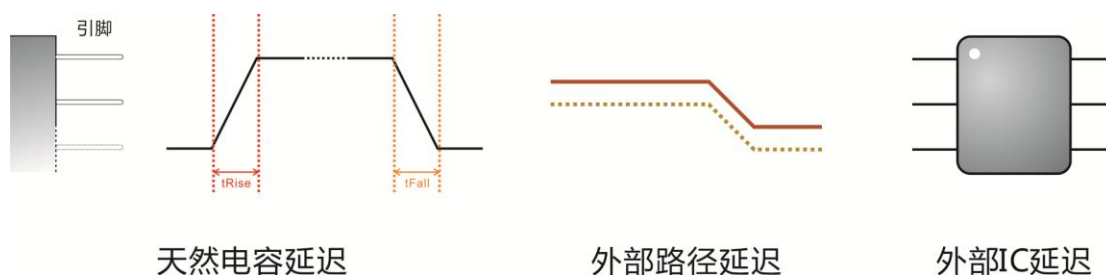


图 3.3.1 各种外部延迟因数。

图 3.3.1 是各外部延迟因数，其中引脚的电容性会引起 t_{Rise} 与 t_{Fall} 延迟，而外部路径是最妙的延迟因数，最后还有外部 IC 的寄存器特性。

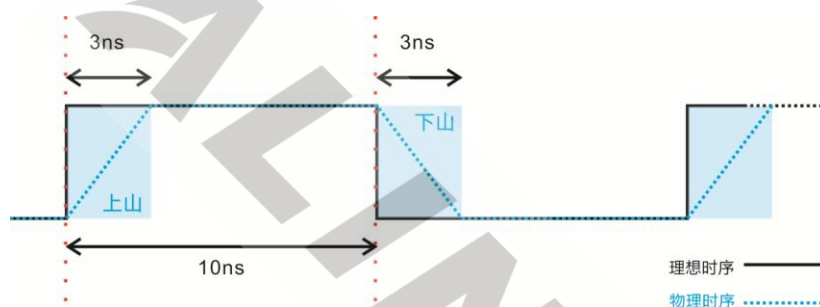


图 3.3.2 理想时序与物理时序。

器件引脚存有电容性是小孩都知道的常识，而引脚的电容性必定会产生 t_{Rise} 与 t_{Fall} 延迟，不过不是每本一本器件手册都会举例 t_{Rise} 与 t_{Fall} 的具体时间。在器件书册没有举例的情况下，我们还必须手动将电容性转换 t_{Rise} 还有 t_{Fall} 的具体时间。

如图 3.3.2 所示，在理想时序下是不存在什么 t_{Rise} 或者 t_{Fall} ，反之物理时序就会出现 t_{Rise} 与 t_{Fall} 延迟。根据黏糊概念而言， t_{Rise} 和 t_{Fall} 好比寄生虫，寄生在数据的身体里边。同学们很可能觉得笔者这样形容 t_{Rise} 和 t_{Fall} 有点不正确 ... 不相信笔者也不打紧，让我们再看其他例子。

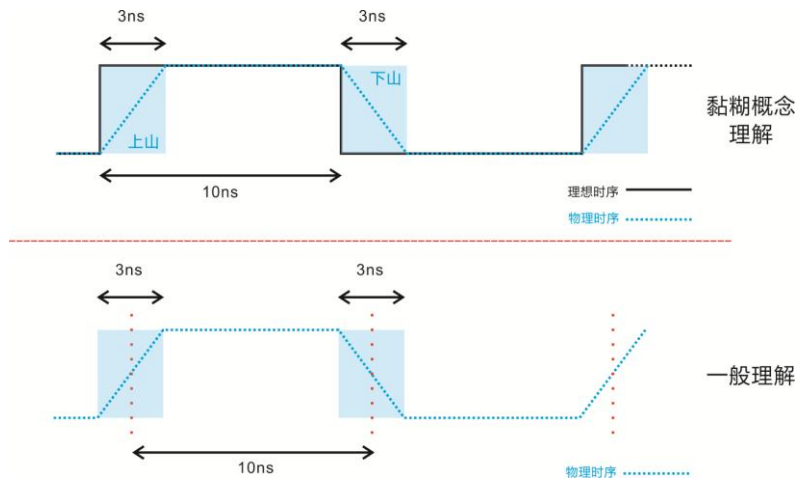


图 3.3.3 黏糊概念的理解和一般理解的比较。

图 3.3.3 的上图是黏糊概念如何理解 t_{Rise} 与 t_{Fall} ，而下图是一般概念如何理解 t_{Rise} 与 t_{Fall} 。事实上，上图和下图有着等价的关系，不过在理解和编辑上 ... 黏糊概念比起一般理解有更多优势。

t_{Rise} 与 t_{Fall} ，原本不是 TimeQuest 的正规问题，这不过是笔者犯贱而已，才会兴致勃勃去研究它。事实上，只要用黏糊概念再加上相关的约束命令， t_{Rise} 与 t_{Fall} 也可以约束的。

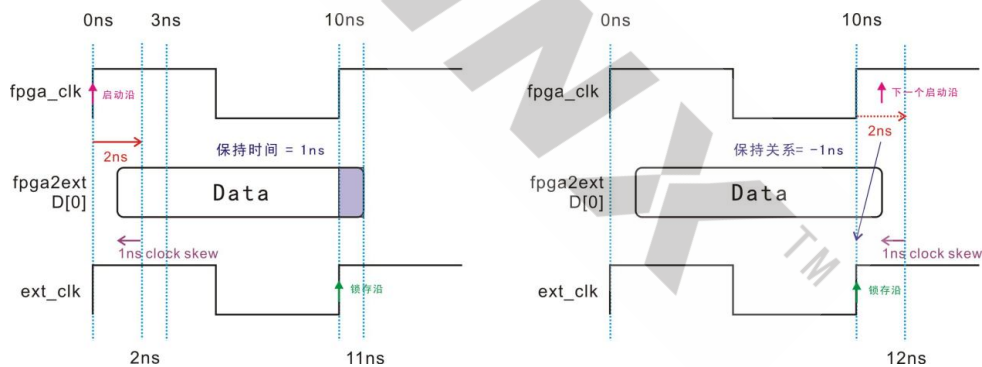


图 3.3.4 单位宽数据的例子。

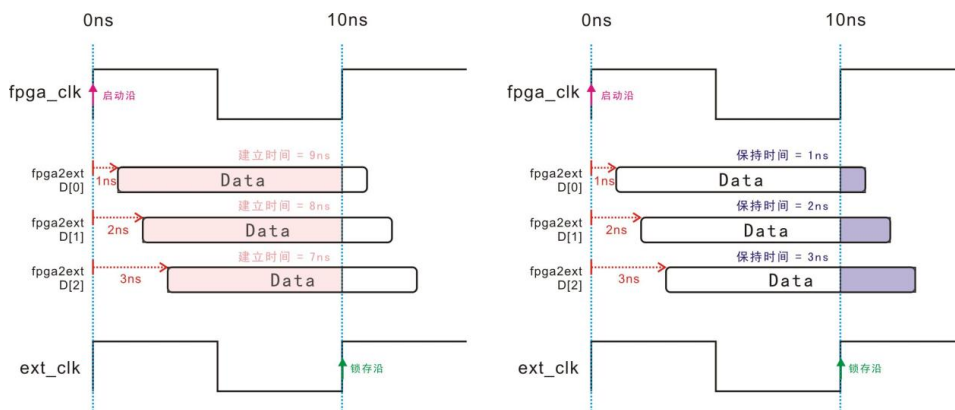


图 3.3.5 多位宽数据的例子。

外部路径算是学习 TimeQuest 最关键的部分。TimeQuest 在认识外部路径的时候，会区分数据路径和时钟路劲。TimeQuest 在认识数据路径的时候，又会根据数据的位宽进一步区分认识，如图 3.3.4 和图 3.3.5 所示。为什么 TimeQuest 要那么麻烦进一步区分数据的单位位宽和多位宽？事实上这是与约束命令的行为有关。

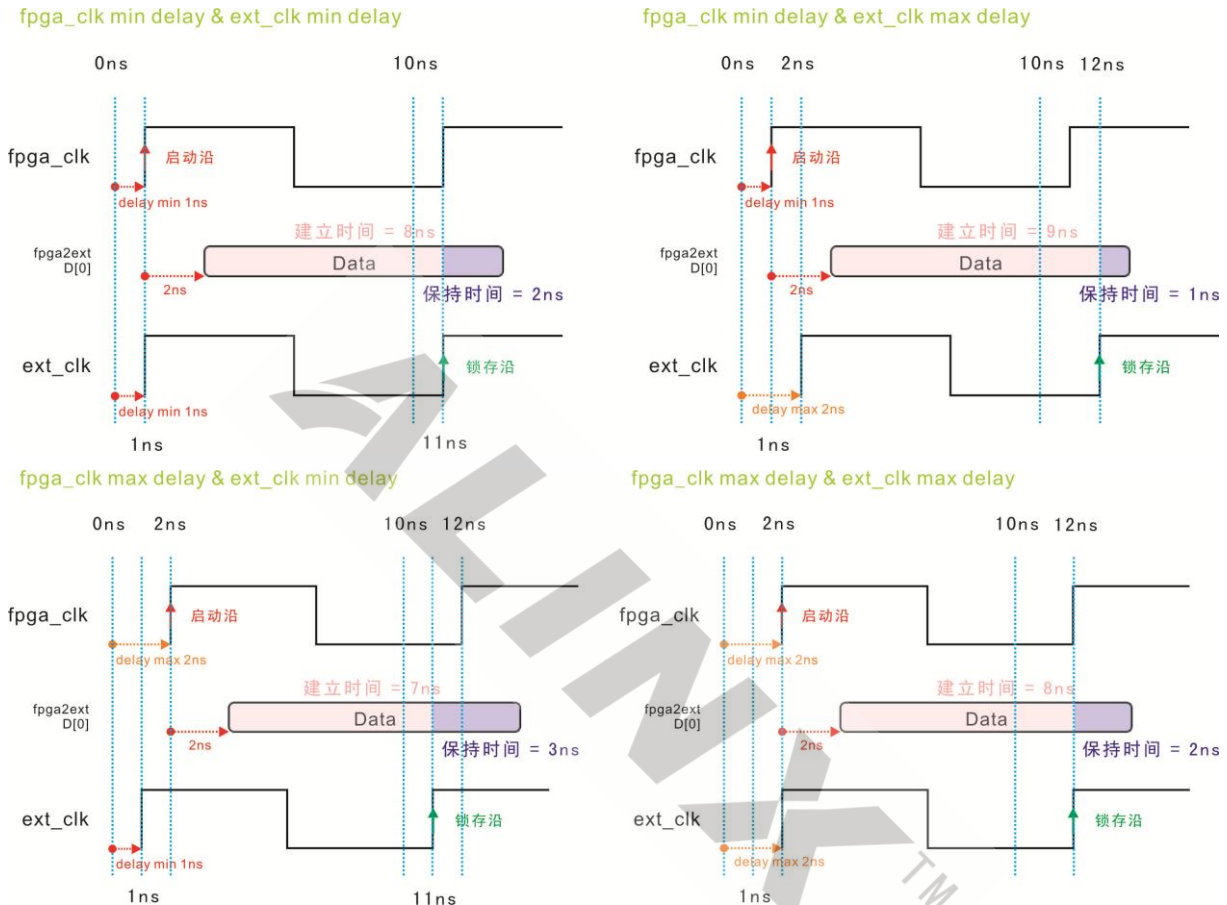


图 3.3.6 时间抖动产生不同可能性的时序。

至于外部时钟路径更妙了 ... 根据理论而言，外部的时钟源比起内部的时钟源更容易产生时间抖动，而时间抖动会给外部时钟路径产生不同延迟，也因此产生多可能性的时序图，如图 3.3.6 所示。笔者在研究时钟抖动的时候吃过不少苦头，而 TimeQuest 也用了一大半的约束命令针对外部时钟路径。从这观点来看，TimeQuest 有多重视外部时钟路径。

此外，外部路径还有一个非常重要的参数，亦即介电参数 ... 不同原材料的 PCB 版都有不同程度的介电参数。介电参数就像是高速公路的原材料，如黑油（沥青路），沙地，石砖等。介电参数越高就表示该高速公路的表面越不平坦，信号在上面行走时，就会产生越多的延迟。

而如何将路径的长度乘上介电参数再换算延迟时间，笔者就不打算介绍了 ... 换之有现成的转换软件可以为我们偷懒，呵呵呵！

```
assign Q = rQ;
```

上面一段代码是 HDL 经典的描述行为，亦即用寄存器 rQ 驱动 Q 输出口。如果以此类推，外部 IC 的输出口，或者输入口一定也是由寄存器驱动。换言之，外部 IC 也定存在寄存器特性，不是吗？IC 的寄存器特性可以在相关的器件手册上查到。



3.4 第一层与第二层时间要求

第一层时间要求和第二层时间是整合时序衍下的概念，这概念的产生主要是为 HDL 可以“照顾”物理时序，单用文字来描述会非常抽象 ... 就让笔者用例子来讲解吧。

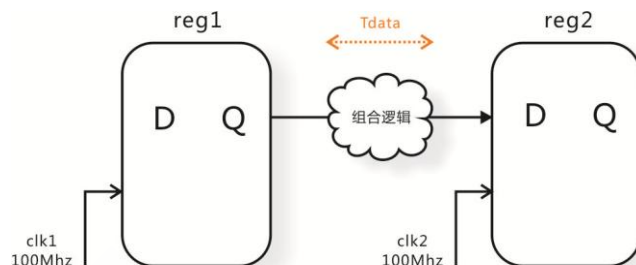


图 3.4.1 两个寄存器夹着一个组合逻辑。

图 3.4.1 是一个典型的 TimeQuest 模型 亦即两个寄存器夹着一个组合逻辑 ... reg1 和 reg2 同样使用的 100Mhz 的时钟信号，而它们之间夹着的组合逻辑，有 Tdata 别名的延迟压力，我们也可以看成数据加工时间。首先我们必须知道，对于 100Mhz 的时钟信号而言，1 个时钟的周期时间是 10ns，如果 HDL 设计基于这个 10ns 周期的时钟信号，那么这个 10ns 是 HDL 最小的时间。此外我们也必须知道，HDL 只有能力产生理想时序。

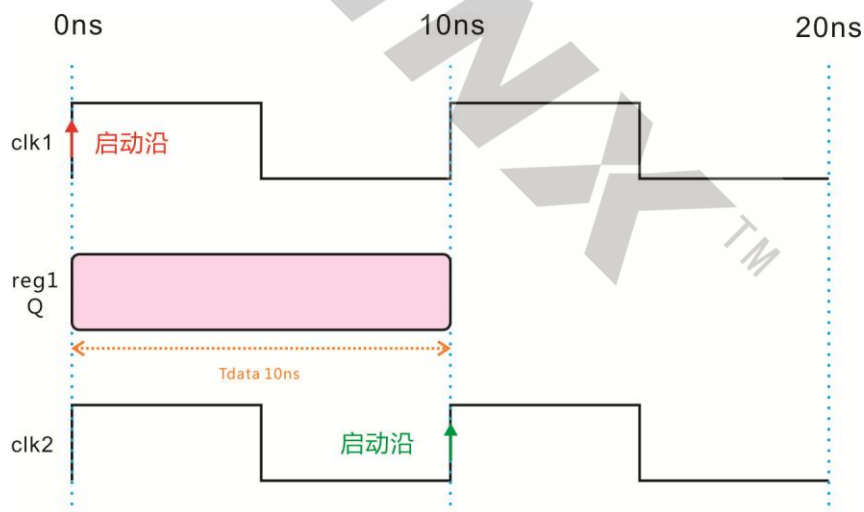


图 3.4.2 HDL 可以照顾又可以涉及的第一层时间要求。

举个例子，假设读者想要生产面包，生成一块面包需要的最小时间也称为空时间或者加工时间；换之，制作面包的原材料又称供源。这个比喻好似图 3.4.2 的情况，如图 3.4.2 所示 ... 我们先假设 Tdata 有 10ns 延迟压力，亦即浅红色的矩形。而启动沿的数量就是空时钟，在这里有 1 个启动沿，亦即给足 1 个空时钟或者 10ns 的加工时间。

然而 reg1-Q 的数据长度就是供源，亦即空白矩形，换言之 reg1-Q 给足 1 个时钟，或者

10ns 的供源。好了，重点来了 ... 上述说过 HDL 的最小时间是 10ns，而 Tdata 的延迟压力恰好是 10ns。简单说，reg1 只要驱动组合逻辑 1 个时钟，期间什么也不做，而 HDL 可以这样表示：

```
case(i)

    0: //r1Q 是驱动组合逻辑的寄存器
    begin r1Q <= 1'b1; i <= i + 1'b1; end
    ...

endcase
```

为了稳定驱动，r1Q 可以多供源一个时钟：

```
case(i)

    0, 1: // 第一层时间要求多供源一个时钟
    begin r1Q <= 1'b1; i <= i + 1'b1; end
    ...

endcase
```

上述代码中的步骤 0，r1Q 赋值 1'b1 一个时钟。在此我们可以这样说，HDL 不仅可以照顾到 Tdata 的 10ns 时间要求，也可以设计到它。因此 Tdata 可以断定为第一层时间要求。

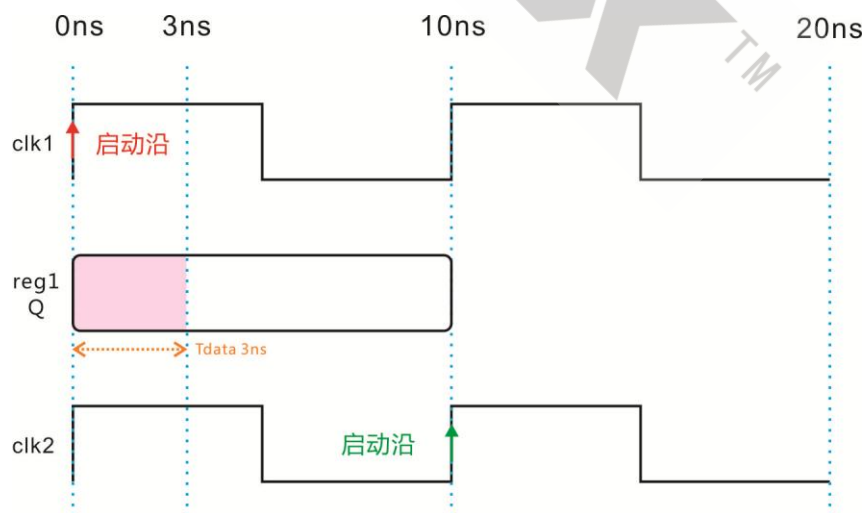


图 3.4.3 HDL 可以照顾不过不可以涉及到的第二层时间要求。

我们再假设 tData 有 3ns 的延迟压力，根据黏糊概念这个 3ns 的延迟压力会覆盖掉数据的一部分，如图 3.4.3 所示 ... 启动沿的数量只有一个，亦即给足 1 个空时钟或者 10ns

的加工时间。此外，空白矩形有 10ns 的长度。基本上，HDL 对 3ns 的物理延迟实在无能为力，余下 HDL 能尽力的事情就是确保给足 1 个空时钟，此外 reg-Q 也必须给足供源 1 个时钟，HDL 可以这样表示：

```
case(i)

    0: //r1Q 是驱动组合逻辑的寄存器
    begin r1Q <= 1'b1; i <= i + 1'b1; end
    ...

endcase
```

上述代码中的步骤 0 给足 1 个空时钟至于，r1Q 赋值 1'b1 也长达一个时钟。因此我们可以这样说，HDL 只可以照顾到 Tdata 的 3ns 时间要求，可是却涉及不了，因此 Tdata 可以断定为第二层时间。

```
case(i)

    0, 1: // 第二层时间要求多供源一个时钟
    begin r1Q <= 1'b1; i <= i + 1'b1; end
    ...

endcase
```

虽然 rQ1 多供源一个时钟可以解决第二层时间要求，如上所示。话虽那么说，一般拥有第二层要求的时序都是条件都非常苛刻，随便多供源一个时间可能会使延迟传输，又或者造就信号不同步。除此之外，第二层时间要求往往又多又小，如果为所有第二层时间多供源一个时钟，即不是要耗费打量宝贵的时间与资源呢？

如果 HDL 遇到第二层时间，它该怎么办？很简单，HDL 只要尽量产生理想时序即可，用黏糊概念来说，只要延迟压力不要覆盖全部数据就行了。余下的工作就交给 TimeQuest 等静态时序分析工具分析看看，第二层时间要求到底会不会破坏数据的有效性，又或者检测供源是否足够？

如何明白第一层或者第二层时间要求的关键地方 ... 读者必需从 HDL 的角度去思考各种时间要求。假设我们要用 HDL 产生 1Mhz 的 SPI 时序，这时候如果 fpga 有 100Mhz 的时钟频率 100Mhz 的频率使 HDL 有 10ns 最小时间，换之要用 10ns 产生 1Mhz 的 SPI 时序是非常有余，许多小地方都可以涉及到。

当 HDL 完成软模型的设计，而且软模型也总和成为硬模型，想必硬模型的内部一定有许多 10ns 最小时间无法涉及时间要求，这时候我们就要使用 TimeQuest 等工具去分析他们。除了硬模型里边的时间要求以外，一些高速器件的驱动如 SDRAM，第二层时间要

求也会出现外边。

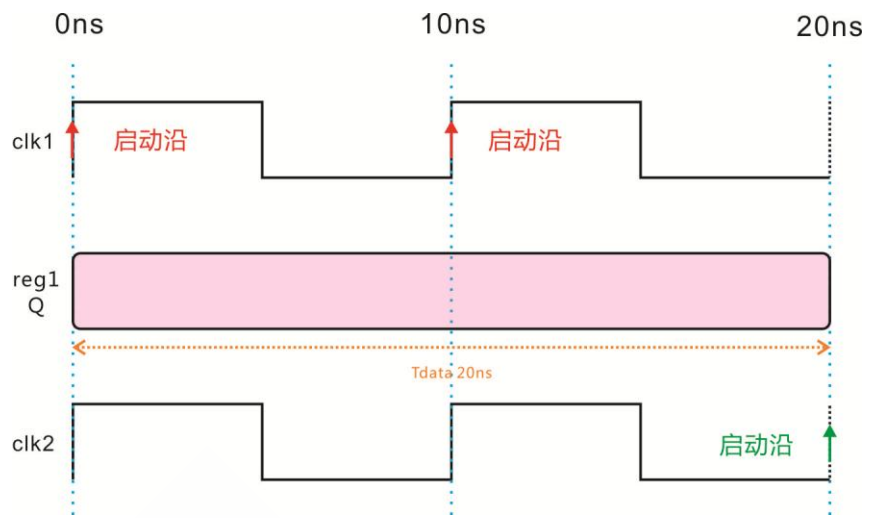


图 3.4.4 HDL 可以照顾又可以涉及的第一层时间要求（多时钟）。

除了单时钟的第一层与第二层时间要求以外，同样也有多时钟的第一层与第二层时间要求。如图 3.4.4 所示，有 2 个启动沿亦即给足 2 个空时钟，reg1-Q 的空白矩形又 20ns 的长度，亦即供源有 20ns，然而 2 个空时钟和 20ns 的供源不仅照顾到 20ns 的 Tdata，而且又涉及其中，因此我们可以断定 20ns 延迟的 Tdata 视为第一层时间。

HDL 可以这样表示：

```
case(i)

    0, 1, 2: // 第一层时间要求多供源一个时钟
    begin r1Q <= 1'b1; i <= i + 1'b1; end
    ...

endcase
```

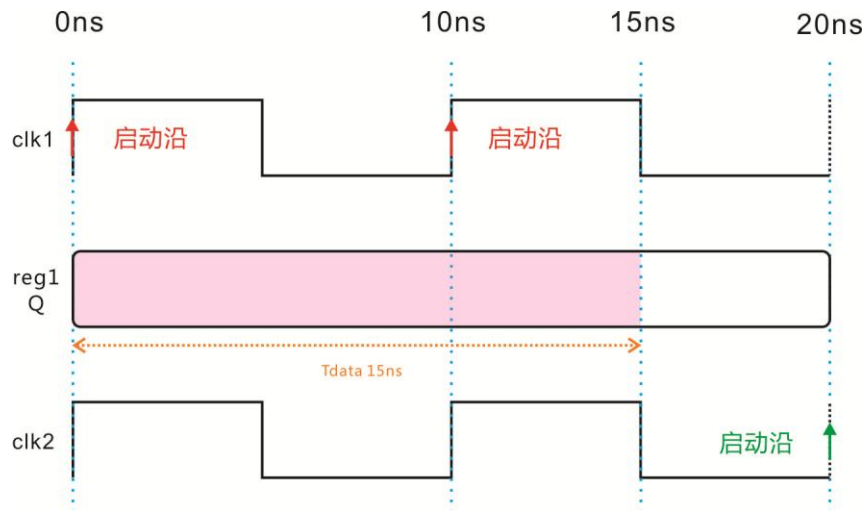


图 3.4.5 HDL 可以照顾不过不可以涉及的第二层时间要求（多时钟）。

图 3.4.5 是 HDL 可以照顾但是却涉及不了的第二层时间要求，其中有 2 个启动沿亦即有 2 个空时钟，而 reg1-Q 也有 20ns 的长度，不过 Tdata 却只覆盖 15ns 的部分而已。这种情况下我们可以说，2 个空时钟和 20ns 的供源虽然已经给足，但是却涉及不了，因此断定 15ns 延迟的 Tdata 为第二层时间要求。

HDL 可以这样表示：

```
case(i)

    0, 1: // 第二层时间要求
    begin r1Q <= 1'b1; i <= i + 1'b1; end
    ...
endcase
```

如果多时钟的第一层或者第二层时间要求发生在[时序产生或者驱动器件](#)的话，我们可以使用黏糊概念将就而过。相反的，如果多时钟的第二层时间要求[发生在硬模型里边](#)的话，故事的说法就不一样了...

我们知道 TimeQuest 是一个笨蛋，在默认的情况下 TimeQuest 模型都是以[单时钟作为基准](#)去分析节点是否合格。有时我们会遇见如图 3.4.5 的情形，这时候我们就要使用相关的约束命令告诉 TimeQuest [“某对节点是用 N 个时钟工作”](#)。如此一来，TimeQuest 就会知道某对节点是使用 N 个时间工作。

不过，到底是什么情形会使第二层时间要求发生在硬模型的里面？答案很简单，那就是组合逻辑的应用。《整合篇》曾讨论过即时建模，那是建立大组合逻辑的方法，大组合逻辑所需的加工时间往往超过 HDL 本身的最小时间。

最后我们可以这样总结：

HDL 涉及不了的时间要求交给 TimeQuest 去做 ,不过在此之前 HDL 必须尽量做好 “理想时序” 的部分，最后 “物理时序” 的部分则交给 TimeQuest 去分析和涉及。



3.5 山寨 TimeQuest 模型

在第一章笔者曾说过，TimeQuest 模型的基本结构是由一对节点组成，不过节点是没有个性的东西，如果节点放在 fpga 内部，它就会成为寄存器，又称 TimeQuest 内部模型。换之如果节点放在 fpga 外部，它就成为 fpga 和外部器件 (ic)，又称 TimeQuest 外部模型。

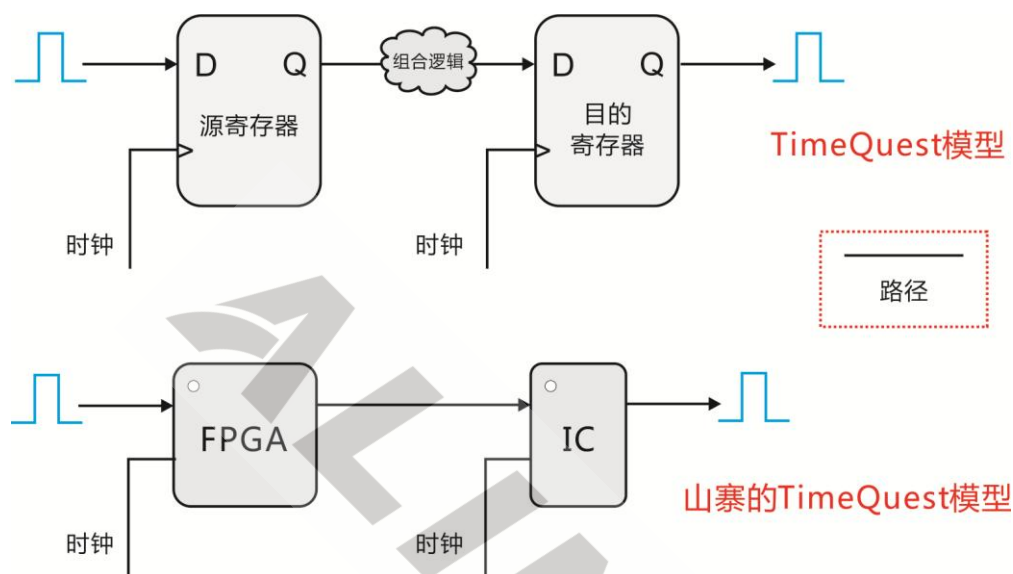


图 3.5.1 TimeQuest 模型（内部模型），山寨 TimeQuest 模型（外部模型）。

所谓山寨 TimeQuest 模型是指，TimeQuest 模型是否可以在外部成立。换句话说，外部 fpga 和外部器件又如何成立一对寄存器，而寄存器之间的延迟因数又是什么？如图 3.5.1 所示，默认的 TimeQuest 又指内部模型，而山寨的 TimeQuest 模型又指外部模型。内部模型的节点变成源寄存器和目的寄存器，而外部模型的节点变成 fpga 和 ic。

笔者曾思考过，TimeQuest 模型虽有内外之分，可是却有相同概念？原因很简单，TimeQuest 的设计者和笔者一样都是懒人 ... 呵呵！开玩笑而已。他们这样做的目的是“同用思路”，换言之：一套可以应用在内部模型的分析方法，同样也可以应用在外部的模型。

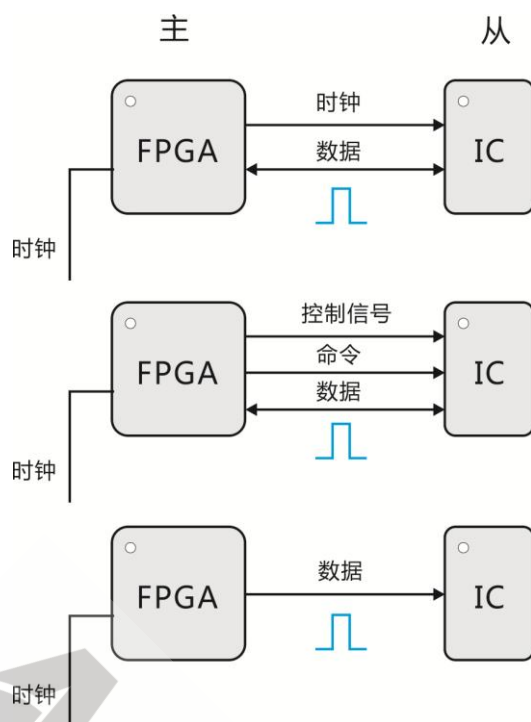


图 3.5.2 无法山寨 TimeQuest 模型的情况。

不过外部模型不一定是山寨成功，如图 3.5.2 所示，如果主机 fpga 给从机 ic 驱动时钟信号和数据。结果而言 TimeQuest 模型一定会山寨失败，这种情形有 spi, iic, ps/2 时序等。此外，没有时钟信号，而有控制信号或者控制命令之间的数据传输也会让 TimeQuest 模型上寨失败，这种情形有静态 ram，传统 lcd 驱动等。还有一种情形就是单纯的数据传输，也会无法山寨成功 TimeQuest 模型，这种情形有 uart 和 vga 时序等。

读者可能会问：“如果外部模型山寨失败的话，那么外部延迟因数又如何约束呢？”这是一个多疑的问题，TimeQuest 只是一匹只懂在模型里打滚的笨蛋而已，如果模型无法成立 TimeQuest 就会无用武之地。此时，我们也只能做好内部模型的分析即可 ... 外边就交个人品和运气。

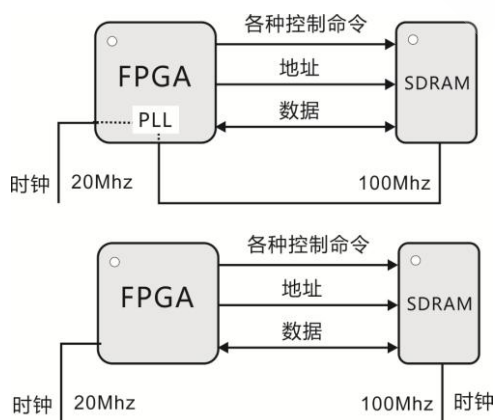


图 3.5.3 FPGA 和 SDRAM 的外部模型。

外部模型可以成功山寨的经典例子就是 sdram，如图 3.5.3 所示。fpga 与 sdram 之所以可以成功山寨 TimeQuest 模型是因为 fpga 与 sdram 都独自的时钟。上图的 20Mhz 时钟信号经过 fpga 内部的 pll 以后，成为 100Mhz 再供源 sdram ... 这种情形的 sdram 时钟不是图 3.5.2 所示的驱动时钟，而是供源时钟。

上图和下图有着等价的关系。在实际的环境里，为了运动 sdram 而外加载 100Mhz 的时钟信号是不环保的行为，为了地球 sdram 必须向 fpga 借助时钟信号，这个借助的时钟信号也称为供源时钟。对供源时钟而言 TimeQuest 非常重视它，此外也有相关的约束命令分别针对供源时钟。供源时钟有 3 种类，直接供源，寄存器供源，还有 pll 供源，而图 3.5.3 的上图就是 pll 供源。

同学们知道内部模型与外部模型的具体区别吗？内部模型与外部模型的区别就在于知道与不知道，自动与手动。所谓知道与不知道是指，有关内部模型的各种延迟信息 TimeQuest 可以通过综合器取得。换之，外部模型的延迟信息必须人为劳动，透过相关的约束命令告诉 TimeQuest 才行，这就是所谓的自动和手动。

3.6 约束命令和约束行为

学习约束命令是学习 TimeQuest 的关键，然而如何理解约束命令的约束行为才能掌握约束命令。约束命令顾名思义就是命令的存在，主要是人类和 TimeQuest 沟通的工具。而约束行为是约束命令的执行过程。

笔者曾说过 TimeQuest 就好比网表（硬模型）的仿真软件，约束命令也有各种各样的行为，如第二章所学过的 `create_clock` 就是建立时钟刺激的约束命令。很遗憾的是，比起约束行为的解释官方更加重视约束命令的使用方法。在研究各种命令的约束行为之际，笔者可吃过不少苦头，呵呵。

此外，笔者也浏览过许多相关的 TimeQuest 解释，说实在笔者只能用一句“残念”来感叹。先不管约束命令的使用方法对错与否？说实在，使用者到底明不明白自己在使用什么才是笔者最关心的地方。这种情形就好比小孩子玩枪械却不懂枪械的危险。

笔者有一个想法必须交代清楚，据笔者所知 ... 笔者认为约束命令的功用，就是**再现分析环境而已，而不是干扰硬模型的实际内容**。笔者相信，支持这种解释和反对这种解释的朋友有不相参半 ... 约束命令究竟能不能干扰硬模型的内容？这种问题，只能见仁见智，因为没有绝对的答案。但是换作笔者，笔者宁愿相信 TimeQuest 只是硬模型的“模拟器”而已，就像 Modelsim 角色一样。

sdc 文件有如 vt 文件，约束命令好比验证语言 ... 不过 Modelsim 仿真对象是软模型，而 TimeQuest 的分析对象是硬模型。Modelsim 只用产生理想时序，看看模块的行为是否达到预期的效果。换之 TimeQuest 只用分析物理时序，看看综合结果是否造就时序违规，或者硬模型能不能在特定的环境下很好运行。

一言盖过，如何使用约束命令不是笔者在乎的地方，而约束命令的行为才是笔者感兴趣的地方。因为笔者相信，通过了解约束行为更能了解 TimeQuest 和掌握它。

总结：

第三章只是扫盲文而已，不过同学们不要小看扫盲的作用 ... 扫盲文可以给个大体的概念，让读者在接下来的学习用不着像无头苍蝇无头乱飞之余；又可以让读者可以知晓学习的方向。虽然扫盲文的没有仔细的精度，不过扫盲文的作用几乎举足轻重，至于读者信不信就见仁见智了。

从第三章的一开始我们先讨论各种余量的概念。建立余量和保持余量是和 TimeQuest 模型有关，而保险余量是和网表质量有关。不过，保险余量只是笔者自取的名字而已，与官方无关。黏糊概念是图形表达余量，而建立余量与保持余量是数值表达余量，而保险余量只是概念表达余量。

造成时序违规的凶手就是“延迟因数”，延迟因数有可以分为内部延迟因数与外部延迟因数。常见的内部延迟因数有：pll，路径（数据路径与时钟路径），还有寄存器特性；而常见的外部延迟因数有：引脚电容性（tRise 与 tFall），外部走线，还有 ic 的寄存器特性。

内部延迟因数几乎可以被 TimeQuest 认知，所以也没有什么好谈的；换之，外部延迟因数 TimeQuest 却完全不认识。TimeQuest 模型也有内部与外部之分，内部模型充满内部延迟因数而外部模型充满外部延迟因数。

就是因为内部延迟因数 TimeQuest 它什么都知道的关系，所以 TimeQuest 会自动认识，也用不着认为告诉它相关的延迟信息。反之外部延迟因数就是因为 TimeQuest 它什么都不知道，结果我们必须手动透过相关的约束命令，再告诉它相关的外部延迟信息。

虽然第三章的内容还插上不相关的第一层和第二层的时间要求概念，不过笔者认为有必要认识一下，如果读者是纯 TimeQuest 用者，笔者就不勉强明白。反之如果想把 HDL 与 TimeQuest 联系起来，明白它们是关键的地方。第一层与第二层时间要求是整合思想下的产物，它们也是黏糊概念的兄弟。

一般上，如果 HDL 最小时间可以照顾和涉及某个时间要求，称为第一层。换之，如果 HDL 最小时间只能照顾却无法涉及那个时间要求，就称为第二层。HDL 所谓的照顾就是产生理想的时序，而所谓的涉及就是空时间和供源完全相符时间要求。可以照顾却涉及不了的时间要求，只能交给 TimeQuest 去分析。此外时间要求也分为单个时钟和多个时钟。

最后我们也小谈一下 TimeQuest 模型的山寨行为，实际上不是要 TimeQuest 模型去模仿什么 ... 而是分析 TimeQuest 模型到底能不能在外部成立，典型的外部模型就是 fpga 与 sdram。此外我们也谈到约束命令与约束行为的相关问题，充其量这也是笔者的多疑而已，谈不上什么问题 ... 读者能不能接受就见仁见智了。

第四章：内部延迟与其他

第三章我们知道为何物理时序会发生违规，原来都是“[延迟因数惹的祸](#)”。延迟因数即有分内外，在接下来的章节，我们会逐步探索“[那个约束命令针对那个延迟因数](#)”。好！事不宜迟，就先拿 PLL 来开刀吧。

4.1 PLL 与约束命令

PLL 是什么？专业的全名是很长很臭的 Phase Lock Loop，至于内部的执行原理更长更臭。不过傻瓜点说，PLL 在 FPGA 里边就是某个“[针对时钟信号加工的组逻辑](#)”，负责的工作有：稳定，增频，减频，位移时钟信号等。

虽说 PLL 类似加工时钟信号的组逻辑，不过 PLL 不是由逻辑资源东拼西凑而成的山寨版组逻辑，它是基于硬件的组逻辑。PLL 的葫芦里装有什么药，除了 FPGA 的厂商以外，外人都不得而知。此外 FPGA 厂商又得想办法“约束”一下 PLL，就是这种半遮半开的情况下，唯有 PLL 拥有自己专属的约束命令。这种矛盾的情况不知是祸是福？

常用的 PLL 约束命令如下：

```
derive_pll_clocks
derive_clock_uncertainty
```

derive_pll_clocks

读者是否还记得曾在第二章出现的 `create_clock` 这个约束命令吗？不错，那是[告诉 TimeQuest 某设计到底使用什么时钟](#)，粗略一点也可看成是仿真中的“时钟刺激”。那么 `create_clock` 又和 `derive_pll_clocks` 有什么关系？

实际上 `derive_pll_clocks` 是一个偷懒用的约束命令。一般 PLL 都会声明 N 个时钟，如果换做 `create_clock` 我们要 N 个时钟都添加 `create_clock` 命令去约束。如果 PLL 声明 50 个输出，既不是要断掉十指逐个逐个去约束这 50 个时钟信号吗？因为如此，`derive_pll_clocks` 这个约束命令就诞生了，明白吗懒人们？

一概论过，无论我们用 PLL 声明多少时钟输出，只要填上 `derive_pll_clocks` 这个犯懒的命令，TimeQuest 就会识作了。TimeQuest 会自行建立相关的时钟约束，然后做出报告。

derive_clock_uncertainty

在这里，笔者先简单讲解一下关于 clock uncertainty，clock uncertainty 是指时钟信号的不确定因数。这个命令用来告诉 TimeQuest 有关时钟的不确定性。严格来讲，这个约束命令不仅针对 pll 更是针对所有与时钟信号 不过根据手册规定 凡是建造工艺 65nm 或者以上的 FPGA，都必须加上这条约束命令。然而建造工艺是 65nm 以下的 FPGA 加上也无妨，TimeQuest 会自动无视。



4.2 约束 PLL 时钟

在这里笔者想要讲解的不是针对单个时钟信号的约束，而是静态时序分析的第一步，亦即时钟约束的步骤。

	From	To	Assignment Name	Value	Enabled	
1		HSYNC	Location	PIN_146	Yes	
2		RGB_Sig[2]	Location	PIN_142	Yes	
3		RGB_Sig[1]	Location	PIN_143	Yes	
4		RGB_Sig[0]	Location	PIN_144	Yes	
5		CLK	Location	PIN_28	Yes	
6		RSTn	Location	PIN_23	Yes	

图 4.2.1 整合篇——实验十引脚配置，其中从外部借用一个名为 CLK 的 20Mhz 时钟信号。

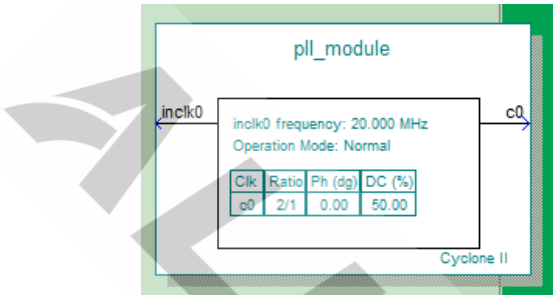


图 4.2.2 整合篇——实验十有关 PLL 的声明，其中 c0 是倍频为 40Mhz 的时钟信号。

假设笔者想要分析一下《整合篇》——实验十的物理时序是否合格，首先我们知道实验十只向外部借用一个名为 CLK 的 20Mhz 时钟信号（图 4.2.1 所示）。然而 20Mhz 不足以驱动 vga，所以借用 PLL 的力量将原有 20Mhz 的 CLK 倍频两倍为 40Mhz 的时钟信号，结果名为 c0（图 4.2.2 所示）。为此，这个实验的第一步约束步骤就是针对以上几个时钟信号做出约束的动作。

实验二 vga 模块的物理时序分析

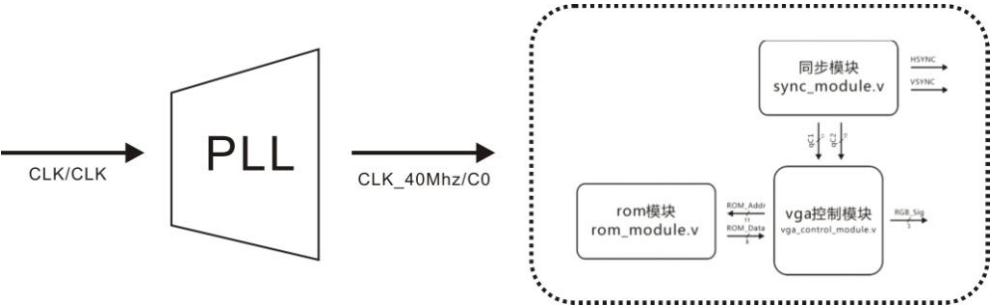


图 4.2.3 整合篇——实验十建模图。

图 4.2.3 是本实验的建模图，我们不管右边组合模块的内容。从图 4.2.3 中我们可以知道一个事实，亦即 CLK/CLK 经过 PLL 以后成为 CLK_40Mhz/c0，然后 CLK_40Mhz/C0

作为组合模块的主时钟源。简单点说，我们所要分析的物理时序是与 CLK_40Mhz/c0 时钟信号所扯上的各种资源。

不多话，为该实验建立一个名为 vga_module.sdc 的文件，然后将下面的内容拷贝进去：

```
create_clock -name {CLK} -period 50.000 -waveform { 0.000 25.000 } [get_ports {CLK}]

derive_pll_clocks
derive_clock_uncertainty
```

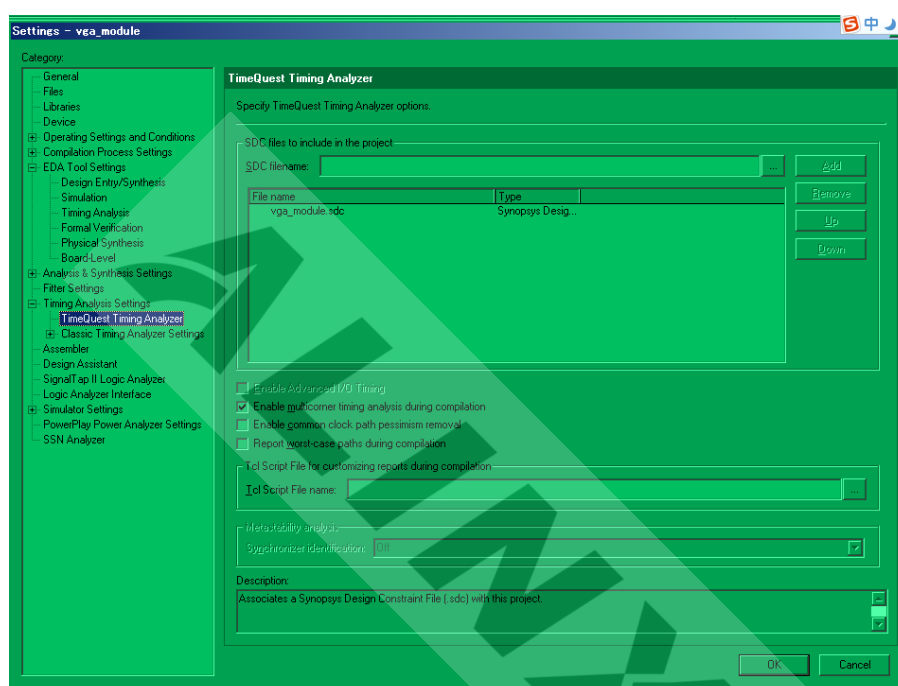


图 4.2.4 设定实验二的 sdc 文件。

再者将该 sdc 文件设定本实验的约束文本，如图 4.2.4 所示，忘记编译的同学先编译一下。接下来再按下这个可爱的图标  大约几秒钟的功夫就会跳出 TimeQuest 的界面。这回笔者彻底犯懒了，笔者就用质量最差的网表（默认网表）。

```
create_timing_netlist -model slow
```

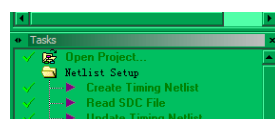


图 4.2.5 快捷生成默认网表

如果同学们也懒惰输入这段 tcl 命令，可以双击 Create Timing Netlist，TimeQuest 就会自动生成，最后顺便按次序双击 Read SDC File 和 Update Timing Netlist（如图 4.2.5 所示）。

我们先暂停一下吧，笔者怕有些同学不仅会更不上也会充满疑问“笔者，用那么差质量的网表真的行吗？”其实这没有什么行不行的问题，以 EP2C8Q208C8 的建造工艺为例，40Mhz 的时钟频率用默认网表的质量已经足够了。

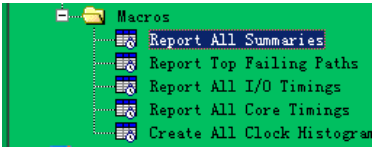


图 4.2.6 生成时序的简报。

Clocks Summary											
	Clock Name	Type	Period	Frequency	Rise	Fall	Duty Cycle	Divide by	Multiply by	Master	Source
1	CLK	Base	50.000	20.0 MHz	0.000	25.000					{ CLK }
2	U0 altpll_component pll clk[0]	Generated	25.000	40.0 MHz	0.000	12.500	50.00	1	2	CLK	U0 altpll_component pll inclk[0] { U0 altpll_component pll clk[0] }

图 4.2.7 时间（时序）简报。

再来双击 Report All Summaries （如图 4.2.6 所示）就会跳出图 4.2.7 的时间（时序）简报。根据图 4.2.7 我们可以读取几个信息 ...

其一 CLK 时钟信号的频率是 20Mhz，周期为 50ns，半周期 25ns，其功用除了 PLL 的输入以外几乎残废。其二 CLK_40Mhz 时钟周期为 40Mhz，周期是 25ns，半周期是 12.5ns，50% 占比率，输入为 CLK 输出为 ...PLL{Clk[0]}，简称为 CLK_40Mhz，该时钟作为本实验的时钟源。

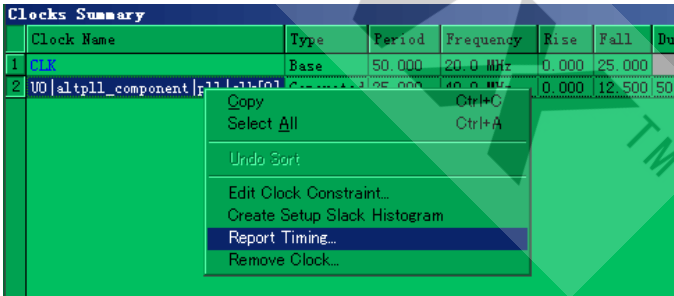


图 4.2.8 调出 CLK_40Mhz 的 Report Timing 界面。

接下来就要右键 CLK_40Mhz 然后调出 Report Timing 的窗口，如图 4.2.8 所示。

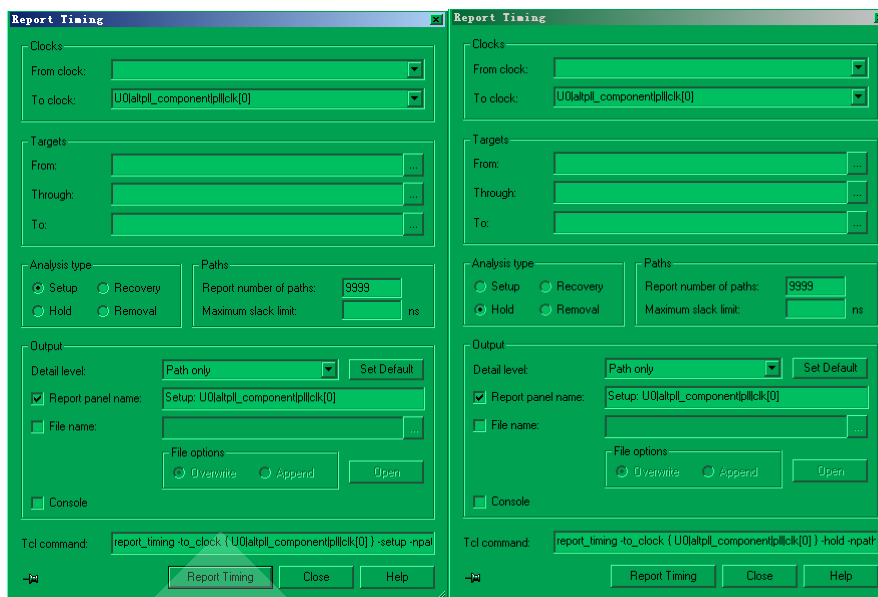


图 4.2.9 执行 CLK_40Mhz 关联的时序报告（建立 setup 和保持 hold）。

然后相续执行相关的 Setup 和 Hold 时序报告，如图 4.2.9 所示。在这里，笔者先插上若干无关的提示 ... 有心的同学可能都会觉疑 **From clock 为什么会留空**？笔者就好心为失忆的同学复习一下第二章的内容！

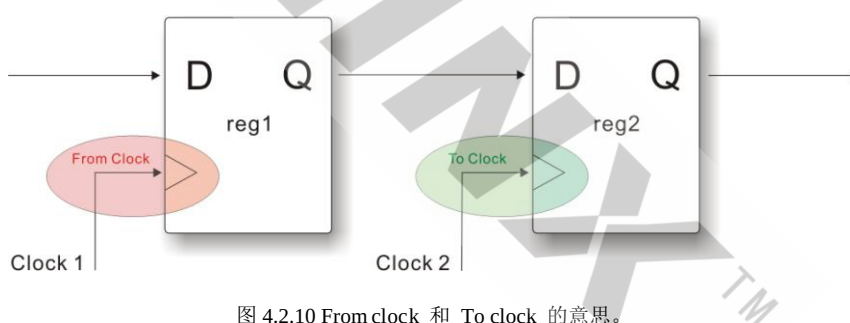
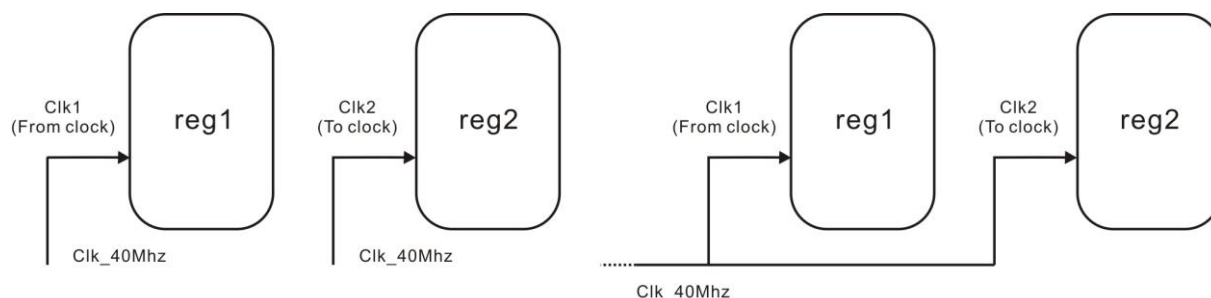


图 4.2.10 From clock 和 To clock 的意思。

根据 Time Quest 的模型 From clock 就是源基础器（reg1）的时钟；To clock 就是目的寄存器（reg2）的时钟。不过根据实验二，20Mhz 的 CLK 时钟源除了作为 PLL 的输入以外，它几乎接近残废。换之 40Mhz 的 CLK_40Mhz 作为 vga_module 的时钟源。



观点一（TimeQuest 角度）

观点二（实验二 角度）

图 4.2.11 两种角度观看 From 和 To Clock。

如果进一步讲解的话，可以将 From clock 和 To clock 分为两个角度。观点一是从 TimeQuest 的视角来观看 From clock 和 To clock，TimeQuest 是一个笨蛋它觉得两个 Clock 理应分开的。换之，观点二是从我们的视角去思考，我们知道 CLK 时钟型号已经残废，换做 Clk_40Mhz 取代它成为实验 vga_module 的主时钟源。同时我们也知道，整个模块的寄存器都使用相同的主时钟源。

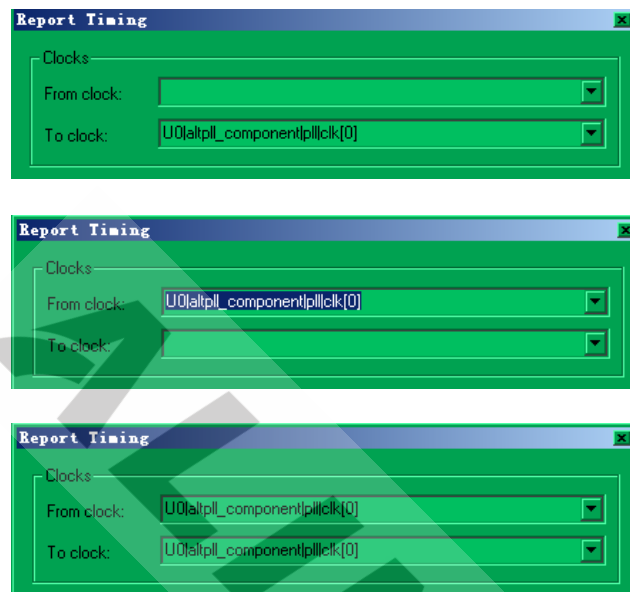


图 4.2.12 三种等价的 From clock 和 To clock 选项。

结果说，无论是哪一种选项对据实验二的 vga_module 来说都是等价的（如图 4.2.12 所示）。换句话说，只要和 CLK_40Mhz 有关联的寄存器都会一一列出。

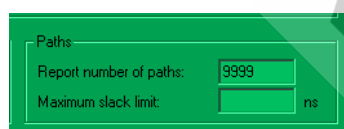


图 4.2.13 path 的深度。

“笔者 ... 请问那个四条九是什么意思？（图 4.2.13）”呵呵，那是时序报告的深度。TimeQuest 实际上是不折不扣的笨蛋既懒惰虫，默认下它只会分析 10 条和 from clock 或者 to clock 相关的时序报告。根据 vga_module 模块 from clock 或者 to clock 有关的分析节点大约有 1100 个左右，所以将 path 深度填上 9999 为了一举列出所有时序信息，然后再逐个击破不合规的时序结果。当然 9999 不是固定的参数，随着建模的增加这个参数也会跟着提升，同学们应该注意下。

Setup: W0 altpll_component pll clk[0]									
Command Info		Summary of Paths							
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay	
1	15.113	...rta_address_reg0	...odule:U3 rRGB[0]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
2	15.113	...rta_address_reg0	...odule:U3 rRGB[2]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
3	15.113	...rta_address_reg1	...odule:U3 rRGB[0]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
4	15.113	...rta_address_reg2	...odule:U3 rRGB[0]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
5	15.113	...rta_address_reg3	...odule:U3 rRGB[0]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
6	15.113	...rta_address_reg4	...odule:U3 rRGB[0]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
7	15.113	...rta_address_reg5	...odule:U3 rRGB[0]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
8	15.113	...rta_address_reg6	...odule:U3 rRGB[0]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
9	15.113	...rta_address_reg7	...odule:U3 rRGB[0]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
10	15.113	...rta_address_reg8	...odule:U3 rRGB[0]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
11	15.113	...rta_address_reg9	...odule:U3 rRGB[0]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
12	15.113	...rta_address_reg10	...odule:U3 rRGB[0]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
13	15.113	...rta_address_reg1	...odule:U3 rRGB[2]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
14	15.113	...rta_address_reg2	...odule:U3 rRGB[2]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
15	15.113	...rta_address_reg3	...odule:U3 rRGB[2]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
16	15.113	...rta_address_reg4	...odule:U3 rRGB[2]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
17	15.113	...rta_address_reg5	...odule:U3 rRGB[2]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
18	15.113	...rta_address_reg6	...odule:U3 rRGB[2]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
19	15.113	...rta_address_reg7	...odule:U3 rRGB[2]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
20	15.113	...rta_address_reg8	...odule:U3 rRGB[2]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
21	15.113	...rta_address_reg9	...odule:U3 rRGB[2]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
22	15.113	...rta_address_reg10	...odule:U3 rRGB[2]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.832	
23	15.682	...rta_address_reg0	...odule:U3 rRGB[1]	...onent pll clk[0]	...onent pll clk[0]	25.000	-0.095	9.263	

图 4.2.14 部分 setup 时序报告。

Setup: W0 altpll_component pll clk[0]									
Command Info		Summary of Paths							
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay	
1	0.499	sync_module:V1 rH	sync_module:V1 rH	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	0.805	
2	0.499	sync_module:V1 rV	sync_module:V1 rV	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	0.805	
3	0.743	sync_module:V1 rV	rBy1[0]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.049	
4	0.744	rBy2[0]	rBy3[0]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.050	
5	0.746	rBy1[1]	rBy2[1]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.052	
6	0.746	sync_module:V1 rH	rBy1[1]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.052	
7	0.747	rBy2[1]	rBy3[1]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.053	
8	0.877	rBy1[0]	rBy2[0]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.183	
9	0.937	...module:V1 C2[1]	...odule:U3 rAddr[5]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.243	
10	1.172	...module:V1 C2[6]	...module:V1 C2[6]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.478	
11	1.177	...module:V1 C1[2]	...module:V1 C1[2]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.483	
12	1.178	...module:V1 C1[0]	...module:V1 C1[0]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.484	
13	1.181	...module:V1 C2[8]	...module:V1 C2[8]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.487	
14	1.181	...module:V1 C2[4]	...module:V1 C2[4]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.487	
15	1.184	...module:U3 r2[2]	...odule:U3 rRGB[1]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.490	
16	1.185	...module:V1 C2[1]	...module:V1 C2[1]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.491	
17	1.201	...module:V1 C1[9]	...module:V1 C1[9]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.507	
18	1.207	...module:V1 C1[7]	...module:V1 C1[7]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.513	
19	1.221	...module:V1 C2[3]	...module:V1 C2[3]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.527	
20	1.225	...module:V1 C1[6]	...module:V1 C1[6]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.531	
21	1.225	...module:V1 C2[5]	...module:V1 C2[5]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.531	
22	1.225	...module:V1 C2[7]	...module:V1 C2[7]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.531	
23	1.225	...module:V1 C1[8]	...module:V1 C1[8]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.531	
24	1.228	...module:V1 C2[9]	...module:V1 C2[9]	...onent pll clk[0]	...onent pll clk[0]	0.000	0.000	1.532	

图 4.2.15 部分 hold 时序报告。

其他的细节基本不重要,点击 Report Timing(图 4.2.9),相续会出现深度为 1100 的 setup 和 hold 报告,其下无论哪个余量 slack 都是正直,结果表示 vga_module 的时序是合格的。如图 4.2.14 的部分 setup 时序报告和图 4.2.15 的部分 hold 时序报告(由于时序报告的数量实在是太多了,笔者仅载录其中一部分作为演示而已)。

此外还有一个偷懒的 tcl 指令,也可以一句列出所有相关的时序报告,亦即:

```
qsta_utility::generate_all_core_timing_reports "Report Timing (Core)" 1000
```

或者双击 Macros 目录下的 Report All Core Timings (如图 4.2.16 所示)。

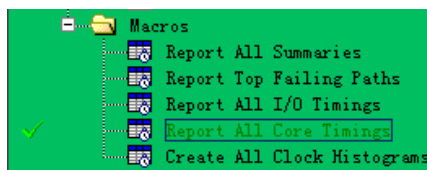


图 4.2.16 快捷一举列出相关的时序报告。

默认下 Report All Core Timings 只是列出 setup 时序报告 1000 分而已，如果同学们是执行如图 4.2.16 所示的动作，我们是无法扩大时序分析的深度。唯有手动输入 tcl 命令（如上），将 1000 换为 9999 即可取得等价的效果。这个 tcl 命令到底有多大的用处就见仁见智了，笔者只是好意提醒而已。

根据 setup 时序报告（图 4.2.14）和 hold 时序报告（图 4.2.15）各 1100，我们没有看见“红色”的字样出现，换句话说 vga_module 这个模块的物理时序是合格的（在某种程度上），因此我们可以总结说“《整合篇》——实验十的 vga_module 没问题！”

结论：

实验二除了举例有关 PLL 的命令以外，我们也了解静态时序分析的第一步骤，亦即时钟方面的约束。此外，也稍微对 Report Timing 逐步深入了解，何为一举列出多有相关的时序报告等。



4.3 延迟怪兽①

师兄曾在第三章说过，TimeQuest 是离不开 TimeQuest 模型的数学工具，一旦 TimeQuest 模型无法被应用在分析上，TimeQuest 这个工具就会略显残废。TimeQuest 模型最简单的概念就是两个节点，具体点说就是两个寄存器夹着一个组合逻辑，直接点说就是两个寄存器之间存在延迟。

初学 TimeQuest 的时候，笔者曾产生这样的疑问：“**无论是什么样的设计，Quartus 的编译器是不是有足够的智能保证 FPGA 内部各个节点的时序都处于合格的状态？**”之所以会产生这样的想法，是因为“即时建模”的出现。“即时”是什么？根据理想时序的思想，即时的定义是“**无视时钟取得即时结果**”，“即时建模”就基于即时操作的建模技巧。

“即时建模”有一个变态的特性，无论那个模块有多么长的执行步骤，都可以转换为“即时操作”，只要一个时钟就可以取得总结果。说简单点“即时建模”就是将**某个模块转换成组合逻辑**。

“即时建模”执行“即时操作”，“即时操作”又是组合逻辑的事件触发，粗略点说“即时建模”拥有组合逻辑的特性，但是“即时建模”又和组合逻辑有明显的区别。教科书上所谓的组合逻辑是几个逻辑门组合而成的东东，笔者称为“**小组合逻辑**”。然而“即时建模”是“**大组合逻辑**”，因为只要条件允许“即时建模”是没有结局的，它可以无限扩展。

在《整合篇》里笔者也说过 ... 组合逻辑都带有延迟，延迟的真凶有如第三章讨论过的各种延迟因数。“即时建模”不停扩展，亦即组合逻辑的延迟压力也会不停增加。所以说“即时建模”有个一致命的缺点，亦即“**庞大的延迟压力**”会随着不断扩展而提升。

最终，这种“延迟压力”一定会超过 FPGA 可承受的极限，成为巨大的延迟怪兽。笔者的问题很简单：**Quartus 的编译器是不是智能到会自动优化这头怪兽呢？如果是又如何优化？如果不是 TimeQuest 又如何约束？用什么命令约束？**

顺便补上，所谓的“极限”是指 FPGA 的制作工艺，还有时钟周期 ... 越精致的制作工艺（65nm，45nm 等）内部的各种延迟因数也会随之缩小；越长的时钟周期，组合逻辑既有更多足够的时间加工数据。

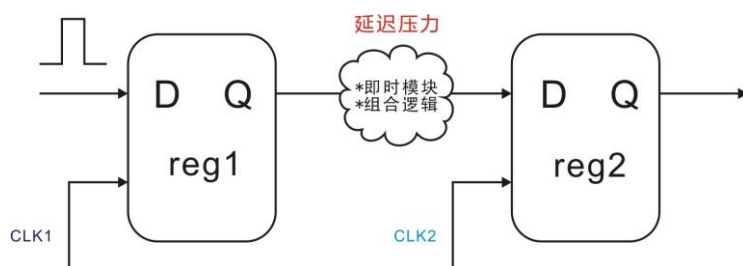


图 4.3.1 假设图。

因此，我们可以为上述的种种内容绘出一个假设图，如图 4.3.1 所示它是一个最基本的 TimeQuest 模型，亦即两个寄存器夹着一个组合逻辑。CLK1 是源寄存器的时钟信号，CLK2 是目的寄存器的时钟信号，假设它们同为一个时钟。此外，被夹在中间的组合逻辑是由即时模块建立而成的。左边的信号表示，数据从 reg1 移向 reg2 然后经过即时模块。

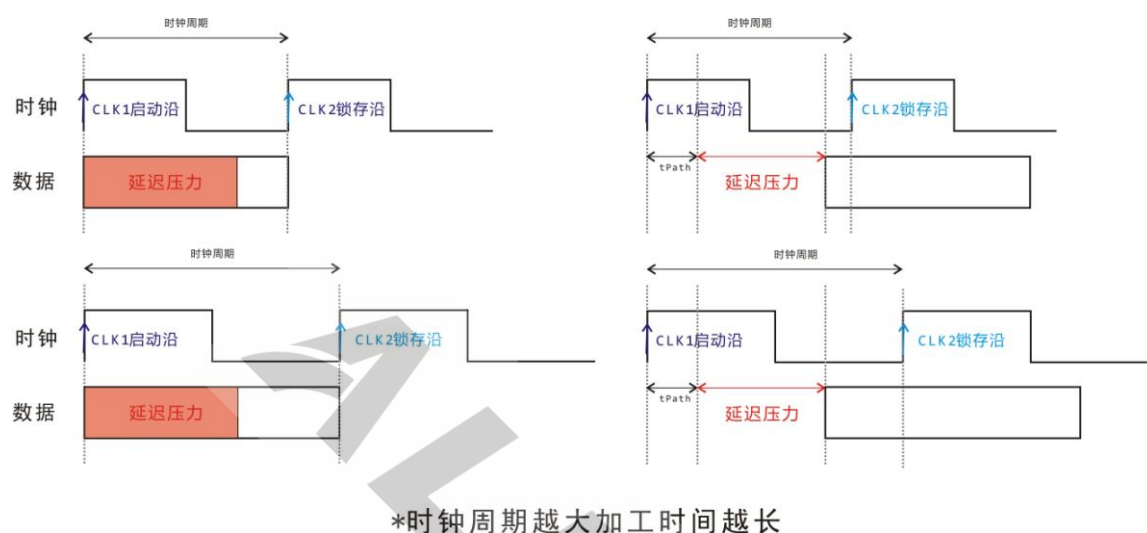
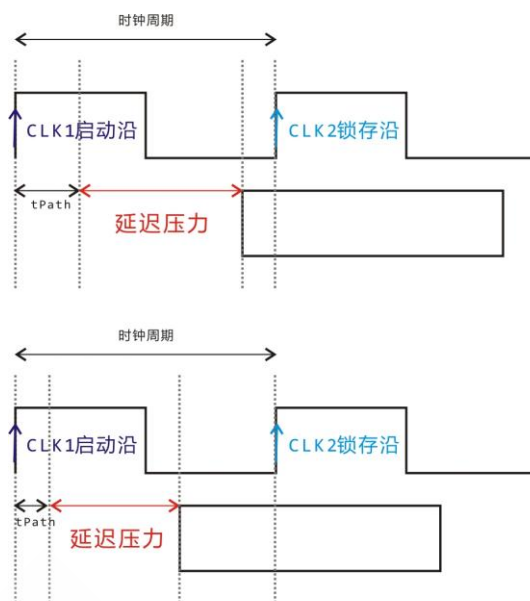


图 4.3.2 时钟周期加长对延迟压力的影响。

图 4.3.2 是加长时钟对延迟压力影响的假想图，左边是理想时序另一边则是物理时序。理想时序图与物理时序图之间的差别就是不存在任何物理特性，不过时钟周期的加长对延迟压力的影响却有相同的结果。如左图所示，增长时钟周期则表示组合逻辑拥有更多的时间加工数据，也亦即延迟压力的覆盖率会跟着减少。

右图的变化上虽然没有左图明显，但是却可以看到 CLK2 的锁存沿向右偏移些许，不管怎么说 CLK2 锁存沿越是接近数据的中心，时序就会越稳定。加大时钟周期的方法有多种，不过最为常见如图 4.3.2 所示，亦即**减慢时钟频率**。



*制作工艺越精致延迟因数越小

图 4.3.3 制作工艺精密对延迟压力的影响。

FPGA 的制作工艺是老百姓无法控制的参数，这要根据 FPGA 厂商的天气和心情。2012 年的今天，45nm 几乎成为主流了，随着科技的发展比 45nm 更精致的制作工艺也会随之问世。不过我们还是稍微了解一下吧。FPGA 的制作工艺会直接影响 FPGA 的体积，内部的各种延迟因数，无论是路径还是逻辑门也会缩小。如图 4.3.3 所示，随着制作工艺的提升 t_{Path} 和 延迟压力也会跟着缩小，而 t_{Path} 的延迟因数是路径。

好了，问题开始扯远了，接下来笔者用实验来验证一下笔者问题：[Quartus 的编译器](#)有没有足够的智能保证各个节点的时序合格呢？哪怕是巨无霸的延迟怪兽？

实验三 延迟怪兽的暴动

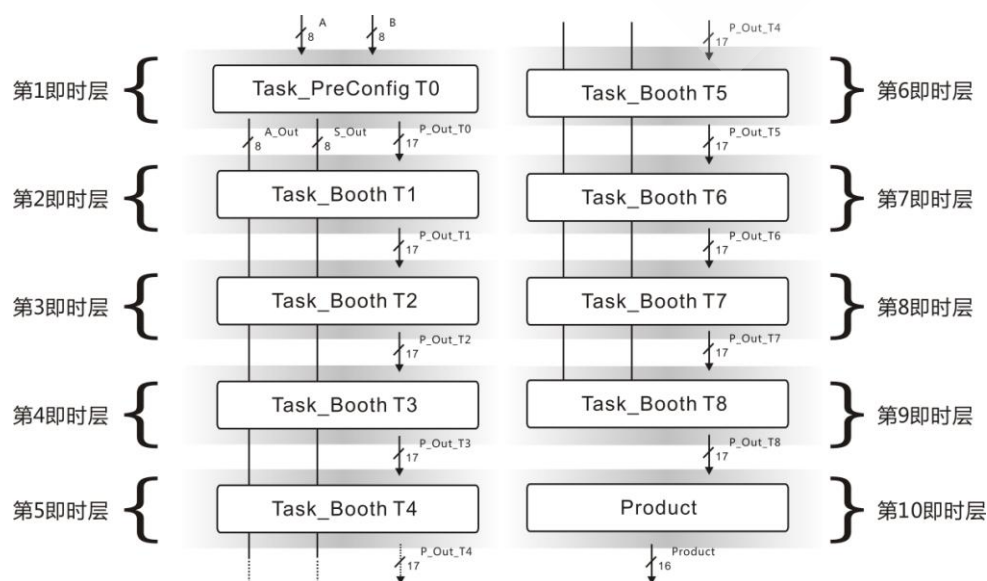


图 4.3.4 即时式 Booth 乘法器的建模图。

实验三测试的对象是抽取《整合篇》——实验二十六的即时式 booth 乘法器，图 4.3.4 则是建模图。哇！那是什么，好可怕啊 ... 这是《整合篇》中提及的即时建模，想了解的朋友可以复习《整合篇》，不了解的朋友就无视吧 ... 懂不懂原理对实验三来说都不重要，同学们仅有把它看成**拥有乘法功能的组合逻辑**即可。



图 4.3.5 组合逻辑的即时模块。

实际上，即时式 Booth 乘法器目前只是组合逻辑而已，它的前后没有被寄存器夹着，我们知道 TimeQuest 模型要是山寨失败，TimeQuest 就无法进行物理时序的分析。结果说，我们需要帮忙为该乘法器的前后加上寄存器。

Delay_monster.v

```

1.  module Delay_monster
2.  (
3.      input CLK,
4.      input RSTn,
5.      input [7:0]A,
6.      input [7:0]B,
7.      output [15:0]Result
8.  );
9.
10.  /*****/
11.
12.  wire [15:0]Result_U1;
13.  immediate_booth_multiplier_module U1( rA, rB, Result_U1 );
14.
15.  /*****/
16.
17.  reg [7:0]rA,rB;
18.  reg [15:0]rProduct;
19.  reg [3:0]i;
20.
21.  always @ ( posedge CLK or negedge RSTn )
22.      if( !RSTn )

```

```

23.         begin
24.             rA <= 8'd0;
25.             rB <= 8'd0;
26.             i <= 4'd0;
27.             rProduct <= 16'd0;
28.         end
29.     else
30.         case( i )
31.
32.             0:
33.                 begin rA <= A; rB <= B; rProduct <= 16'd0; i <= i + 1'b1; end
34.
35.             1:
36.                 begin rProduct <= Result_U1; i <= 4'd0; end
37.
38.         endcase
39.
40.
41.     /*****/
42.
43.     assign Result = rProduct;
44.
45.     /*****/
46.
47. endmodule

```

上面一段非常简单的代码，第 1~8 行声明该模块的出入口；第 13 行是即时式 Booth 乘法器的实例化——名为 U1，它分别由 rA 和 rB 驱动，输出致连线 Result_U1。第 21~38 是核心功能，第 17~19 行声明相关的寄存器，亦即 rA，rB 和 rProduct；第 24~27 行是复位操作。

第 32~33 的步骤 0 先将 A 和 B 的值暂存在 rA 和 rB，随之 rProduct 清零，然后步骤 i 递增以示下一个步骤。在步骤 i 进入下一个步骤之际，U1 立刻触发即时事件。结果第 35~36 行，U1 会输出结果 Result_U1，然后将其暂存在 rProduct 寄存器。最后再由 rProduct 驱动 Result（第 43 行）。



图 4.3.6 即时模块 U1 Make Up 完毕。

如果不管核心功能等细节问题，粗略点 Delay_monster 模块的模样如图 4.3.6 所示。分别 rA 和 rB 驱动即时式 Booth 乘法器的输入，输入则由 rProduct 寄存器暂存。这样一来，U1 就是上寨成功，结果 U1 可以用 TimeQuest 来分析。

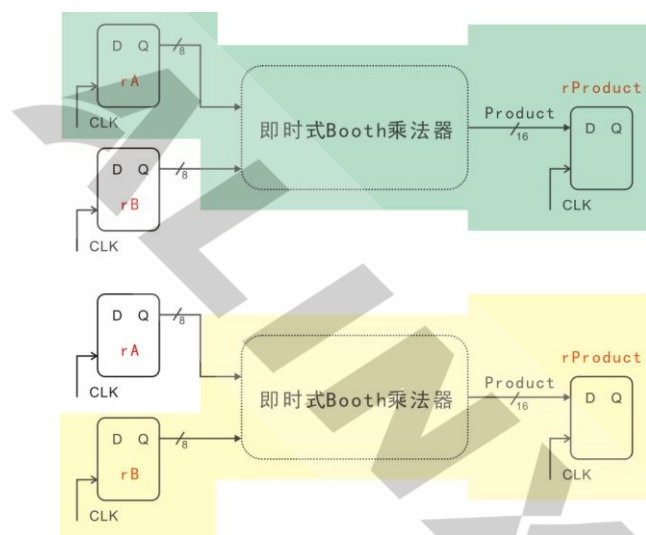


图 4.3.7 成功上寨 2 个 TimeQuest 模型。

同样，如果不计较一些细节问题的话 ... 结果如图 4.3.7 所示，我们已经成功上寨 2 个 TimeQuest 模型在 U1 的身上，一个是 rA 经 U1 到 rProduct；另一个则是 rB 经 U2 到 rProduct。如果同学蛋疼的话，笔者不介意打开 Technology Map 视图来看看，同学一定会被细节打败，咔咔咔！笔者用狗头保证！

先编译一下模块 Delay_monster，然后建立一个名为 Delay_monster.sdc 的文件，随之填下：

```
create_clock -period 50 -name CLK [get_ports {CLK}]
```

保存以后就完工了，执行的次序和实验二一样。打开 TimeQuest，建立默认网表（双击 Create Timing Netlist）

```
create_timing_netlist -model slow
```


然后双击 Read SDC File 和 Update Timing Netlist ,载入 Delay_monster.sdc 之余顺便更新网表。接下来双击 Report All Summaries ,最终会出现如图 4.3.8 的时间简报。笔者稍微罗嗦一下,这个实验的时钟信号只有 CLK 而已,因此时间简报只会呈现 CLK 而不是其他。也就说 CLK 信号贯穿整个 Delay_monster 模块。

Clocks Summary						
	Clock Name	Type	Period	Frequency	Rise	Fall
1	CLK	Base	50.000	20.0 MHz	0.000	25.000

图 4.3.8 时钟简报。

好的,关键的时刻到了!事实上 Delay_monster 拥有 9999 以上的节点也分析,如果我们像实验二那样一举全部和 CLK 有关的路径或者节点的话,那是一场等待的恶梦(不过同学们可以试试,不过搞当电脑后果要自负呀)。不过,懒惰的同学就会问:“有没有快捷的方法一举列出违规的节点?”



图 4.3.9 一举列出所有违规的节点。

如下的 tcl 命令就是一举列出所有违规的节点(路径),其中参数 200 是默认节点数量,参数可以自由修改。同样也可以双击 Report Top Failing Paths 如图 4.3.9 所示,不过无法修改参数。

```
qsta_utility::generate_top_failures_per_clock "Top Failing Paths" 200
```

双击 Report Top Failing Paths 看看 ... 会发生什么? TimeQuest 反馈如下信息 ... 呵呵! 没有任何违规的节点。

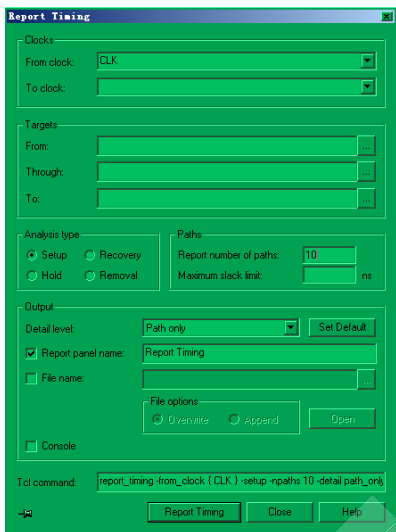
```
No failing paths found.
```

嘛,笔者还是简单举例 10 个路径(节点),看看[延迟怪兽如何影响各个节点的时序合格化](#);又或者看看[延迟怪兽如何影响 Setup 和 Hold 时序分析](#)。



图 4.3.10 快捷调出 Report Timing 界面。

双击 Report Timing 快捷调出 Report Timing 界面(如图 4.3.10 所示),



Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
12.622	rA[0]	rProduct[13]	CLK	CLK	50.000	0.021	37.439
12.624	rA[0]	rProduct[13]	CLK	CLK	50.000	0.021	37.437
12.642	rA[1]	rProduct[13]	CLK	CLK	50.000	0.021	37.419
12.644	rA[1]	rProduct[13]	CLK	CLK	50.000	0.021	37.417
12.657	rA[0]	rProduct[13]	CLK	CLK	50.000	0.021	37.404
12.659	rA[0]	rProduct[13]	CLK	CLK	50.000	0.021	37.402
12.666	rA[0]	rProduct[13]	CLK	CLK	50.000	0.021	37.395
12.677	rA[1]	rProduct[13]	CLK	CLK	50.000	0.021	37.384
12.679	rA[1]	rProduct[13]	CLK	CLK	50.000	0.021	37.382
12.686	rA[1]	rProduct[13]	CLK	CLK	50.000	0.021	37.375

图 4.3.11 随便列举 10 个节点，读出 setup 报告。

根据 Delay_monster 模块，From clock 或者 To clock 任何选项都会得到等价的结果。分析属性为 Setup 路径深度（节点数量）10，报告纤细 Path only，选项准备完毕以后点击 Report Timing 就会跳出右图——随便列举 10 个节点有关 Setup 的余量分析。咋看 ... Data Delay 比往常几个实验来得还要“凶”，可是却没有造成时序违规 ... 为什么？

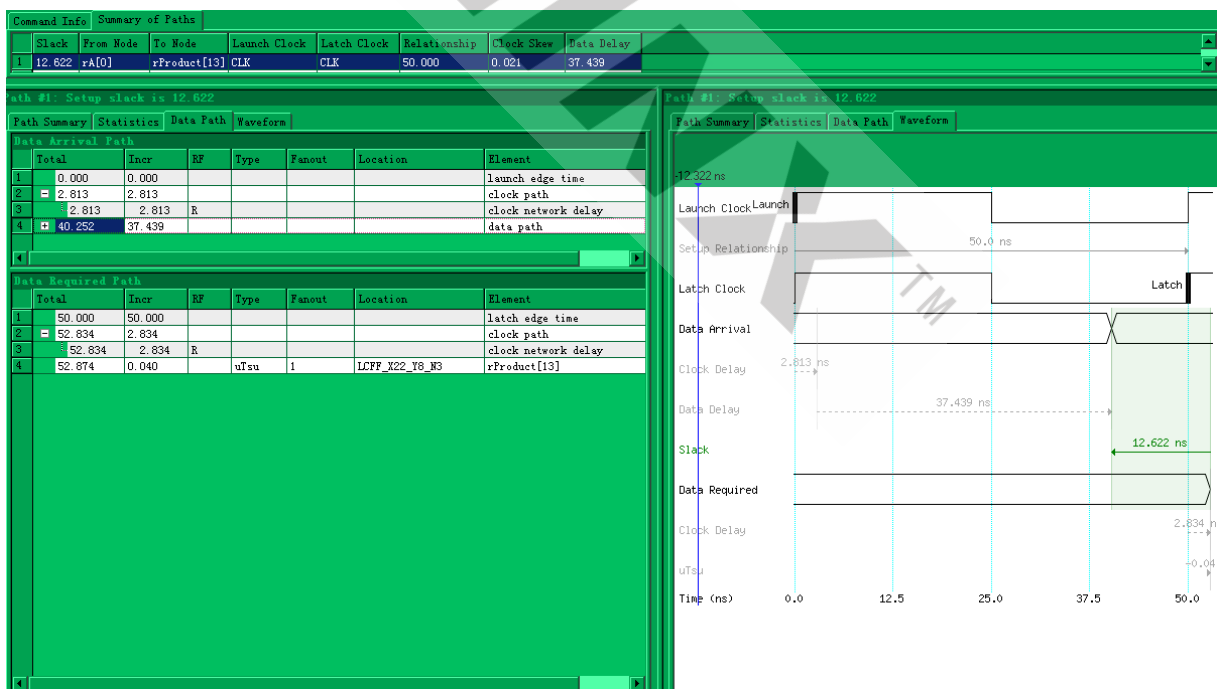


图 4.3.12 节点 1 的细节。

打开节点 1 的细节观察，如图 4.3.12 所示 ... 会造成 data delay 那么凶的原因是 Data Arrival Path（数据抵达路径）的 Data path 出现怪兽，啊 ... 我的菩萨！这只怪兽就是在 Delay_monster 模块中饲养的 即时式 Booth 乘法器，只要同学将其列出 ... 就会看到

怪兽的五脏六腑，亦即各种和即时式 Booth 乘法器有关的组合逻辑。

幸好怪兽没有造成时序违规 不过奇怪的是，怪兽只是骚扰 Data Arrival Path 大道而已，又不见怪兽骚扰 Data Required Path 大道（数据读取/锁存路径）。不过，不知道怪兽是否也会捣乱 Hold 分析？事不宜迟，让我们来瞧瞧。

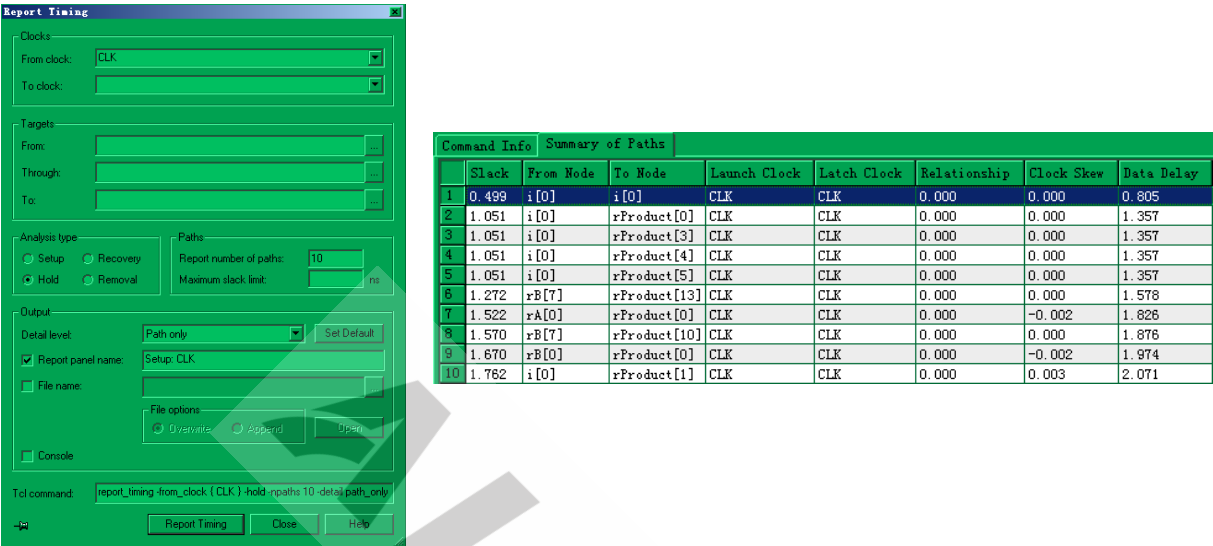


图 4.31.3 随便列举 10 个节点，读出 hold 报告。

setup 余量有关公式：
数据抵达时间 Time Arrival Time = 启动沿时间 + Tclk1 + Tco + Tdata

hold 余量有关公式：
数据保持时间 Data Hold Time = 启动沿 + Tclk1 + Tco+ Tdata + 数据周期时间

根据 Setup 余量和 Hold 余量的求得公式 Tdata 就是组合逻辑的象征，亦即组合逻辑越肥大，Tdata 也会一直线上升。结果而言，我们可以肯定延迟怪兽也会捣蛋 Hold 时序的分析。如图 4.3.13 所示，笔者随便列举 10 个节点（路径）然后列出时序结果，奇怪的事情发生了！注意最右边 Data Delay 的数值，既然那么“渺小” ... 有没有搞错？

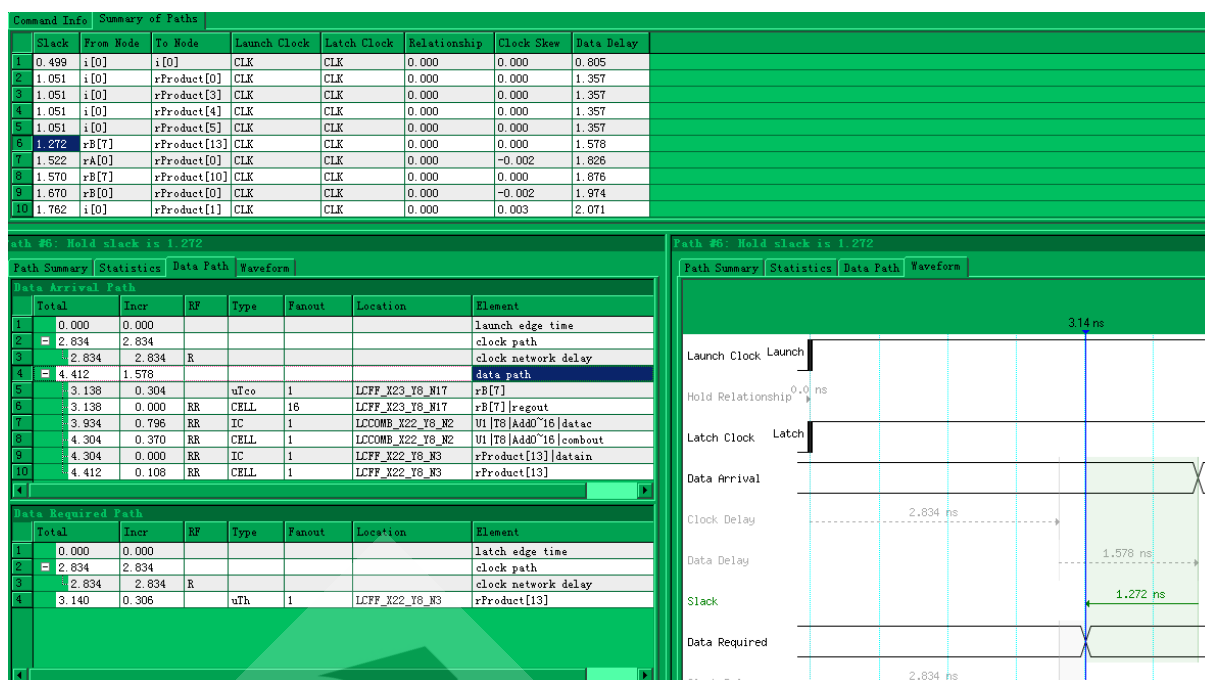


图 4.3.14 节点 6 的细节。

绝对没有搞错，注意 Data Path 的展开信息，那是延迟怪兽的“五脏六腑”，无疑延迟怪兽确实已经影响 Hold 时序分析，不过效果不像 setup 分析那么强烈而已。

笔者曾说过，时钟周期也是其中一个影响组合逻辑的因数，为了让延迟怪兽的骚动更加疯狂，我们尝试缩短时间周期从 20Mhz(50ns) 至 100Mhz(10ns)，亦即更改 CLK 的约束参数 ... 如下所示：

```
create_clock -period 10 -name CLK [get_ports {CLK}]
```

然后重复上述动作直到双击 Report All Summaries 为止。

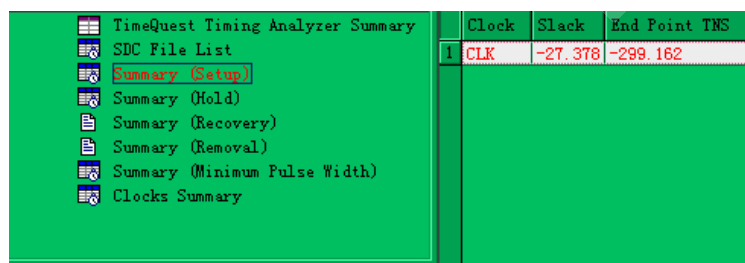


图 4.3.15 亮红灯的 Setup 报告。

结果如图 4.3.15 所示，setup 报告既然亮红灯了 ... 我们得赶紧一举列出违规的节点(双击 Report Top Failing Paths)。

Setup: CLK								
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	-27.378	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.439
2	-27.376	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.437
3	-27.358	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.419
4	-27.356	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.417
5	-27.343	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.404
6	-27.341	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.402
7	-27.334	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.395
8	-27.323	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.384
9	-27.321	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.382
10	-27.314	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.375
11	-27.300	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.361
12	-27.299	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.360
13	-27.280	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.341
14	-27.279	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.340
15	-27.152	rA[0]	rProduct[7]	CLK	CLK	10.000	0.032	37.224
16	-27.150	rA[0]	rProduct[7]	CLK	CLK	10.000	0.032	37.222
17	-27.141	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.202
18	-27.139	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.200
19	-27.132	rA[1]	rProduct[7]	CLK	CLK	10.000	0.032	37.204
20	-27.130	rA[1]	rProduct[7]	CLK	CLK	10.000	0.032	37.202
21	-27.122	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.183
22	-27.121	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.182
23	-27.120	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.181
24	-27.119	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.180
25	-27.117	rA[0]	rProduct[7]	CLK	CLK	10.000	0.032	37.189
26	-27.115	rA[0]	rProduct[7]	CLK	CLK	10.000	0.032	37.187
27	-27.108	rA[0]	rProduct[7]	CLK	CLK	10.000	0.032	37.180
28	-27.102	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.163
29	-27.100	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.161
30	-27.097	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.158
31	-27.097	rA[1]	rProduct[7]	CLK	CLK	10.000	0.032	37.169

图 4.3.16 亮红灯的 setup 时序结果。

红啊！满江红啊！怪兽开始杀人啦！同学不要忘了，默认下 Report Top Failing Paths 只是随意举例 200 个有问题的节点而已，实际上有问题的节点大概是超过 10000 个以上，到底有多少“人”已经遭殃？完了 ... 彻底玩完了，终于玩火自焚刺激怪兽了 ... 延迟怪兽已经造成一发不可收拾的后果。我们该怎么办 ... 呜呜呜 (To be continue)

4.4 延迟怪兽②

前情提要：

一只名为 即时式 booth 乘法器的延迟怪兽养活在 Delay_monster.v 的体内，当时钟频率为 20Mhz 的时候，怪兽虽然有对 Setup 和 Hold 时序结果造成影响，不过不至于使结果违规。可是一个好玩，将时钟周期从 50ns 缩短至 10ns，最终造成怪兽暴动，疯狂杀人 ... Setup 时序报告出现血淋淋的满江红！

要解决这个难题，我们有几个选择：

- (一) 消灭怪兽（放弃使用即时式 Booth 乘法器）。
- (二) 增宽时钟周期。
- (三) 使用约束命令。

选择一是最直接的方法，亦即消灭怪兽放弃使用该乘法器，说白了那只是潇洒不潇洒的问题而已。选择二是温柔的方法，亦即加宽时钟周期以致抚摸怪兽的情绪，让它不在“杀人”，setup 时序报告也不会满江红。选择三是最无情的方法，既利用怪兽也无视怪兽的情绪，然而使用强制的约束手段驾驭怪兽 ...

好啦 ... 别把话说到那么难听啦！选择三本来就是学习 TimeQuest 的目的 ... 章节 4.3 笔者曾提问过“Quartus 的编译器有没有足够的智能保证，内部的物理时序不会出现违规呢？如果是 Quartus 如何优化？如果不是 TimeQuest 如何约束？用什么命令”，然而答案也非常明显的。接下来我们又该用什么约束命令来确保适时序合格呢？

在此之前，同学们需要回忆一下第三章中曾出现的“约束行为”。在约束行为中“加”的概念比起“减”的概念更为重要，也就是说在接下会使用的约束命令，将演示如何实现“加”的概念。

实验四 约束延迟怪兽

实验四和实验三的内容基本上不变，除了时钟约束从 20Mhz 变成 100Mhz，亦即：

```
create_clock -name {CLK} -period 10.000 -waveform { 0.000 5.000 } [get_ports {CLK}]
```

之所以延迟怪兽会到处杀人，是因为我们没有给足组合逻辑加工数据的时间，会有同学突发奇想道：“能不能设法减少硬件上给组合逻辑带来的延迟？”，这位同学非常遗憾，除了减少即时建模的即时层，或者提高 FPGA 的制作工艺以外，余下就只有提高网表的质量而已，不过目前还不是时候谈论网表。在此我们需要用到 set_multicycle_path 约束命令。

严格来说 set_multicycle_path 的约束行为是告诉 TimeQuest 更动某对节点的启动沿和锁存沿 ... 想必读者听不明白是吧？说傻瓜点就是，告诉 TimeQuest 某对节点很可能不是用 1 个时钟周期工作。

Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	-21.193	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.017	31.216
2	-21.120	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.017	31.143
3	-21.109	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.017	31.132
4	-21.108	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.017	31.131
5	-21.103	rA[1]	rProduct[15]	CLK	CLK	10.000	-0.017	31.126
6	-21.093	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.017	31.116
7	-21.090	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.017	31.113
8	-21.082	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.017	31.105
9	-21.079	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.017	31.102
10	-21.075	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.017	31.098

图 4.4.1 随意举例 10 个节点的 setup 报告。(约束之前——频率 100Mhz 时钟周期 10ns)

Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	0.499	i[0]	i[0]	CLK	CLK	0.000	0.000	0.805
2	1.057	i[0]	rProduct[0]	CLK	CLK	0.000	0.000	1.363
3	1.057	i[0]	rProduct[3]	CLK	CLK	0.000	0.000	1.363
4	1.232	rB[7]	rProduct[7]	CLK	CLK	0.000	0.000	1.538
5	1.479	rB[1]	rProduct[1]	CLK	CLK	0.000	-0.001	1.784
6	1.559	i[0]	rB[0]	CLK	CLK	0.000	-0.001	1.864
7	1.559	i[0]	rA[0]	CLK	CLK	0.000	-0.001	1.864
8	1.559	i[0]	rA[1]	CLK	CLK	0.000	-0.001	1.864
9	1.559	i[0]	rA[2]	CLK	CLK	0.000	-0.001	1.864
10	1.559	i[0]	rA[3]	CLK	CLK	0.000	-0.001	1.864

图 4.4.2 随意举例 10 个节点的 hold 报告。(约束之前——频率 100Mhz 时钟周期 10ns)

图 4.4.1 是随意举例 10 个节点的 setup 报告，则图 4.4.2 则是 hold 报告 ... 继续看图 4.4.3 由于时钟周期不足的关系导致 setup 时序报告跟着亮红灯。因为 100Mhz 仅有的 10ns 时钟周期不足以给足 31ns 左右的薪水 (Data Delay)。为此我们需要将时钟周期加宽到 40ns 左右。

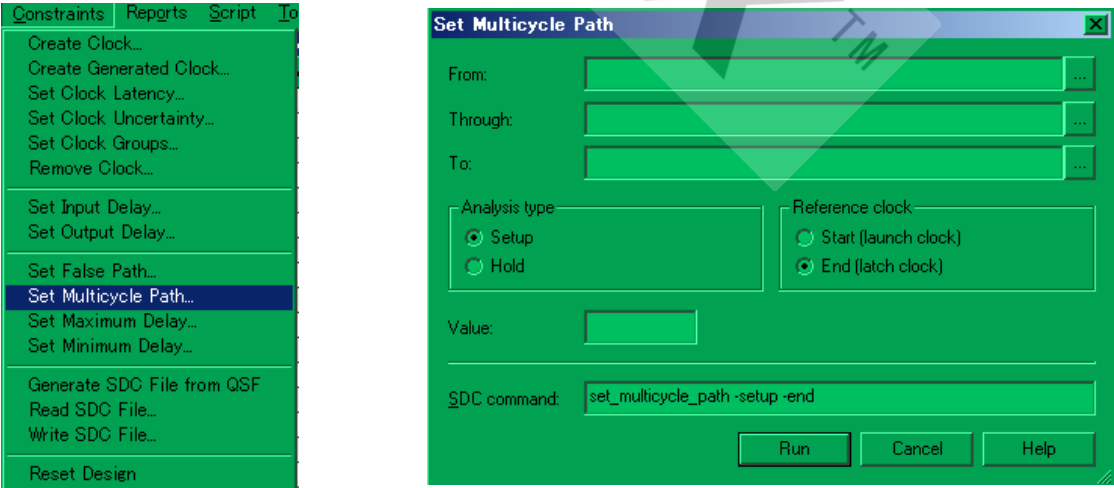


图 4.4.3 打开 Set Multicycle Path 界面。

我们先打开 Set Multicycle Path，如图 4.4.3 所示，该界面在 Constraints 菜单下调出。

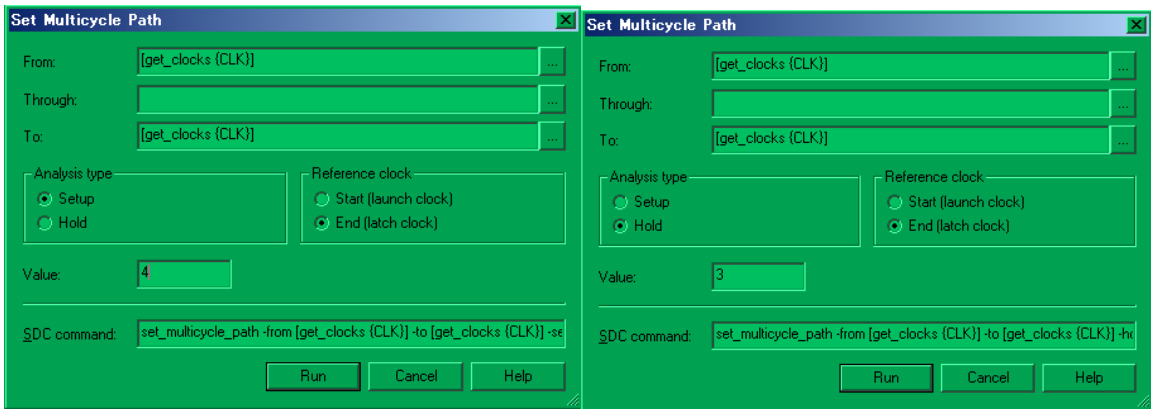


图 4.4.4 全局节点的节点减频 4 倍。

接下来我们要将 From 和 To 均设为主时钟-CLK, 然后双双为 setup 和 hold 的 Value 填入 4 与 3, Reference clock 皆为 End, 然后点击 Run (如图 4.4.4 所示)。

```
set_multicycle_path -from [get_clocks {CLK}] -to [get_clocks {CLK}] -setup -end 4
set_multicycle_path -from [get_clocks {CLK}] -to [get_clocks {CLK}] -hold -end 3
```

然后两条约束命令会自动添加到 sdc 文件下。以上的设置是告诉 TimeQuest 所有与 CLK 有关的节点都是用 4 个时钟周期干活。再双击 Update Timing 该约束命令就会生效。

Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	8.807	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.216
2	8.880	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.143
3	8.891	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.132
4	8.892	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.131
5	8.897	rA[1]	rProduct[15]	CLK	CLK	40.000	-0.017	31.126
6	8.907	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.116
7	8.910	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.113
8	8.918	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.105
9	8.921	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.102
10	8.925	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.098

图 4.4.5 随意举例 10 个节点的 setup 报告。(约束之后——频率 100Mhz 时钟周期 10ns)

Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	0.499	i[0]	i[0]	CLK	CLK	0.000	0.000	0.805
2	1.057	i[0]	rProduct[0]	CLK	CLK	0.000	0.000	1.363
3	1.057	i[0]	rProduct[3]	CLK	CLK	0.000	0.000	1.363
4	1.232	rB[7]	rProduct[7]	CLK	CLK	0.000	0.000	1.538
5	1.479	rB[1]	rProduct[1]	CLK	CLK	0.000	-0.001	1.784
6	1.559	i[0]	rB[0]	CLK	CLK	0.000	-0.001	1.864
7	1.559	i[0]	rA[0]	CLK	CLK	0.000	-0.001	1.864
8	1.559	i[0]	rA[1]	CLK	CLK	0.000	-0.001	1.864
9	1.559	i[0]	rA[2]	CLK	CLK	0.000	-0.001	1.864
10	1.559	i[0]	rA[3]	CLK	CLK	0.000	-0.001	1.864

图 4.4.6 随意举例 10 个节点的 hold 报告。(约束之后——频率 100Mhz 时钟周期 10ns)

然后再随意举例 10 个节点的 Setup 和 hold 报告看看, 不堪入目的满江红基本上也消失的无影无踪。读者稍微注意一下图 4.4.5 的 setup relationship, 不是原先的 10ns 而是 40ns。

接着笔者再重新打开 Delay_monster ，然而 sdc 文件中的时钟约束从 100Mhz (10ns) 改为 25Mhz (40ns) 看看 ... 代码更动如下所示：

```
create_clock -name {CLK} -period 10.000 -waveform { 0.000 5.000 } [get_ports {CLK}]
改为
create_clock -name {CLK} -period 40.000 -waveform { 0.000 20.000 } [get_ports {CLK}]
```

然后再随意举例 10 个 setup 和 hold 的时序报告 ...

Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	8.807	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.216
2	8.880	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.143
3	8.891	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.132
4	8.892	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.131
5	8.897	rA[1]	rProduct[15]	CLK	CLK	40.000	-0.017	31.126
6	8.907	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.116
7	8.910	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.113
8	8.918	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.105
9	8.921	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.102
10	8.925	rA[0]	rProduct[15]	CLK	CLK	40.000	-0.017	31.098

图 4.4.7 随意举例 10 个节点的 setup 报告。(未约束——频率 25Mhz 时钟周期 40ns)

Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	0.499	i[0]	i[0]	CLK	CLK	0.000	0.000	0.805
2	1.057	i[0]	rProduct[0]	CLK	CLK	0.000	0.000	1.363
3	1.057	i[0]	rProduct[3]	CLK	CLK	0.000	0.000	1.363
4	1.232	rB[7]	rProduct[7]	CLK	CLK	0.000	0.000	1.538
5	1.479	rB[1]	rProduct[1]	CLK	CLK	0.000	-0.001	1.784
6	1.559	i[0]	rB[0]	CLK	CLK	0.000	-0.001	1.864
7	1.559	i[0]	rA[0]	CLK	CLK	0.000	-0.001	1.864
8	1.559	i[0]	rA[1]	CLK	CLK	0.000	-0.001	1.864
9	1.559	i[0]	rA[2]	CLK	CLK	0.000	-0.001	1.864
10	1.559	i[0]	rA[3]	CLK	CLK	0.000	-0.001	1.864

图 4.4.8 随意举例 10 个节点的 end 报告。(未约束——频率 25Mhz 时钟周期 40ns)

同学自己比较一下图 4.4.5~6 与 图 4.4.7~8 结果是不是一样呢？它们之间的区别就在于：一个是频率为 100Mhz 加上约束；另一个频率为 25Mhz 未约束。

“akuei2 你到底要讲什么 ,我越来越糊涂了 ... 拜托了 ,别再玩弄我的猪脑了 !” —— 想必有写同学已经不耐烦了。笔者不是有意捣乱 ,在此笔者只是想证明一个事情而已 ,Set Multicycle Path 的功是告诉 TimeQuest “某对节点用 N 个时钟工作”。不过比较特别的是 ,它不仅可以针对 “所有节点” ,同样它也可以针对 “单对节点”。

此外 ,笔者也想提醒一件事 ... Set Multicycle Path 充其量是告诉 “某对节点用 N 个时钟工作” 或者告诉 TimeQuest “某对节点的启动沿和锁存沿有所更动”。实际上该约束命令没有任何能力干涉硬模型的内容。好了 ! 前面只是热身而已 ,接下来我们会慢慢解剖 Set Multicycle Path 这个命令。

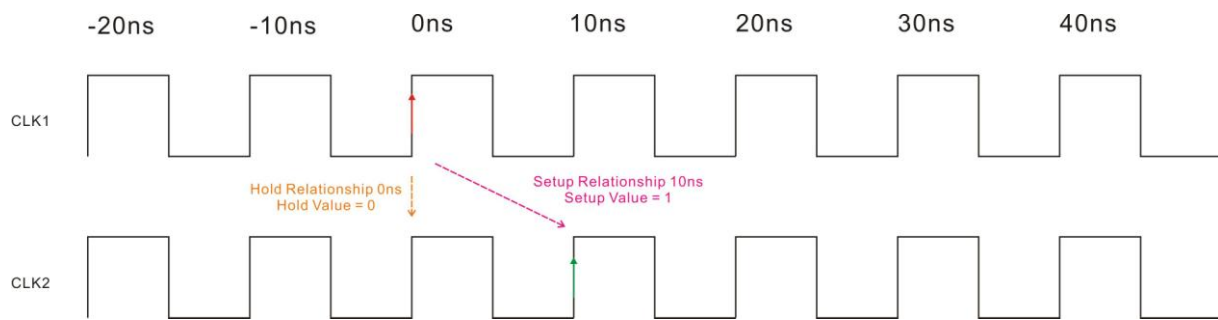


图 4.4.9 Setup Relationship 和 Hold Relationship 的理想关系值。

让我们先回忆第一章曾出现过的 Setup Relationship 和 Hold Relationship 亦即建立和保持理想关系值的取值过程，如图 4.4.9 所示在默认即理想的情况下 Setup Relationship 必定等价与时钟周期亦即 10ns，然而 Hold Relationship 必定 0ns。不过对于 Set Multicycle Path 来说还有 Setup Value 和 Hold Value 之分，默认情况下 Setup Value 为 1，Hold Value 为 0。

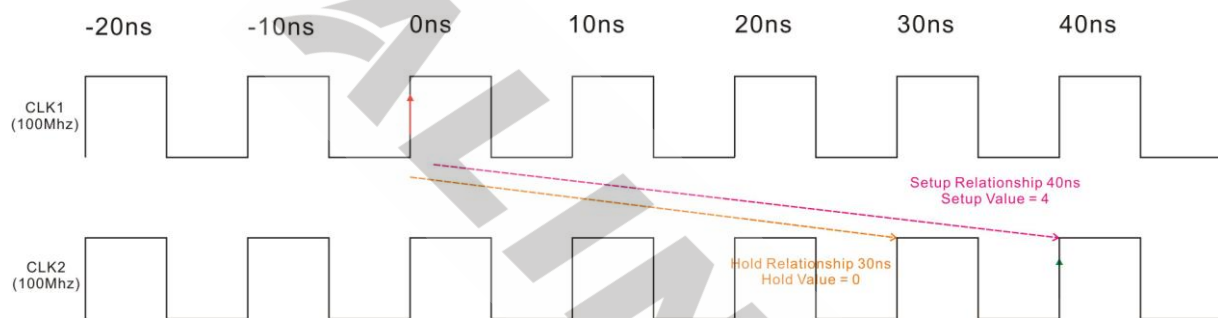


图 4.4.10 Setup Value = 4, Hold Value = 0。

实际上，我们知道 Setup Relationship 仅有 10ns 是不足以应付延迟怪兽的胃口，因此将设置 Set Multicycle Path 中的 Setup Value 为 4 ... 如图 4.4.10 所示，Setup 的虚线箭头向右移至 CLK2 的第 4 个时钟，亦即 40ns 处，因此 Setup Relationship 增加为 40ns。不过糟糕的是，当 Setup Value 更动 Hold 的虚线箭头也跟着移动，指向 30ns 的地方。

为什么 Setup 虚线箭头移动，Hold 虚线箭头也会跟着移动呢？鬼晓得，这是 TimeQuest 的游戏规则，我们也能遵守而已。随着 Setup Value 更动，Hold 虚线箭头跟着更动，结果 Hold Relationship 也随之增加至 30ns。我们知道 0ns 是 Hold Relationship 的理想值，该值增加只会坏掉节点的时序而已 ... 不相信吗？我们来计算一下 ...

求得保持余量的公式是：(用屁股的话)

保持余量 = 数据抵达时间 - 数据获取时间

Hold Slack = Data Arrival Time - Data Required Time

数据抵达时间 Data Arrival Time = 启动沿 + Tclk1 + Tco + Tdata

$$\begin{aligned}\text{数据获取时间 Data Required Time} &= \text{锁存沿} + T_{clk2} + T_h \\ &= \text{理想保持关系值} + T_{clk2} + T_h\end{aligned}$$

上述是用屁股求得 Hold Slack 的公式，其中 Data Arrived Time 和 Data Required Time 左右该余量是否违规。接下来我们把焦点转向 Data Required Time 的公式，一段抢眼的红字，亦即“理想保持关系值”就是所谓的 Hold Relationship，只要该值增加 Data Required Time 也会增加。该事实也告诉我们，只要 Data Required Time 越接近 Data Arrived Time，Hold 余量就越难合格。

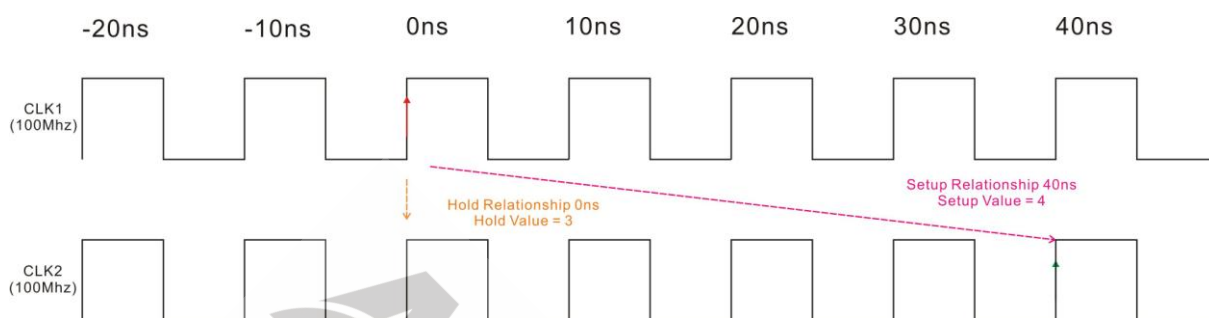


图 4.4.11 Setup Value = 4, Hold Value = 3。

为此，我们必须设法降低 Hold Relationship 的值，如图 4.4.11 所示 ... 原先的 Hold Value 从 0 改为 3，结果 Hold 虚线箭头向左移动 3 个时钟，最终落在 0ns 的地方，因此 Hold Relationship 也减至 0ns。最后我们得到 Setup Value 为 4，Hold Value 为 3，双双对应 Setup Relationship 为 40ns，Hold Relationship 为 0ns 的结果。

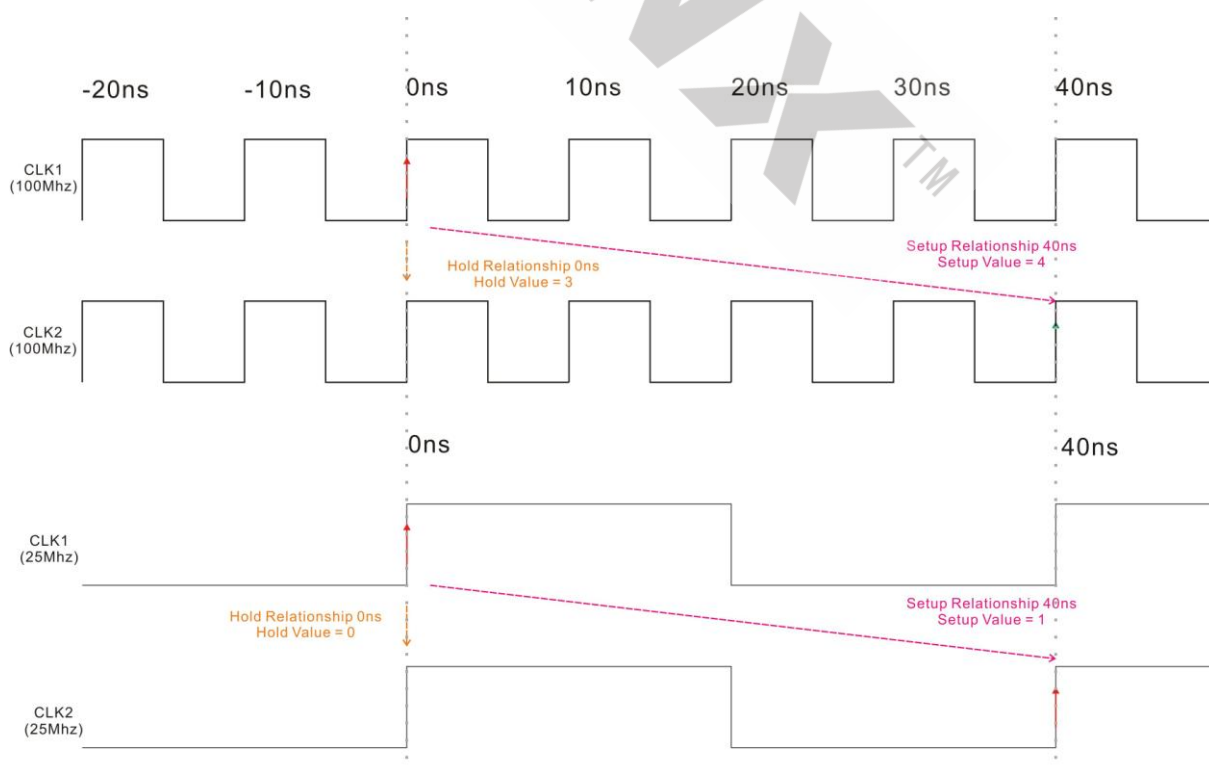


图 4.4.12 100Mhz 经 Set Multicycle Path 约束与 25Mhz 未被约束，之间的等价关系。

我们稍微换个角度来思考，如图 4.4.12 所示一个是经 Set Multicycle Path 约束的 100Mhz，另一个则是未经约束的 25Mhz。无论是上看还是下看，它们之间的关系是完全等价的。

“某对节点用 N 个时钟工作”只是 set multicycle path 比较粗略的功能说明而已，实际上 Set Multicycle Path 就是告诉 TimeQuest “指向 Setup/Hold 虚线箭头有所更动”。该约束命令不仅针对所有节点，而且也可针对以单对局部，让我们马上试试看 ...

Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	-20.941	rA[0]	rProduct[15]	CLK	CLK	10.000	0.016	30.997
2	-20.905	rA[0]	rProduct[15]	CLK	CLK	10.000	0.016	30.961
3	-20.898	rA[0]	rProduct[15]	CLK	CLK	10.000	0.016	30.954
4	-20.824	rA[0]	rProduct[15]	CLK	CLK	10.000	0.016	30.880
5	-20.822	rA[0]	rProduct[15]	CLK	CLK	10.000	0.016	30.878
6	-20.788	rA[0]	rProduct[15]	CLK	CLK	10.000	0.016	30.844
7	-20.786	rA[0]	rProduct[15]	CLK	CLK	10.000	0.016	30.842
8	-20.779	rA[0]	rProduct[15]	CLK	CLK	10.000	0.016	30.835
9	-20.777	rA[0]	rProduct[15]	CLK	CLK	10.000	0.016	30.833
10	-20.773	rA[0]	rProduct[15]	CLK	CLK	10.000	0.016	30.829

图 4.4.13 随意举例 10 个节点的 setup 报告。

笔者重新打开频率为 100Mhz 而且还未约束的 Delay_monster 模块，然后随便举例 10 个节点的 Setup 报告，如图 4.4.13 所示 ... 均为满江红。在此，我们只要针对 rA[0] ~ rProduct[15] 进行约束 ...

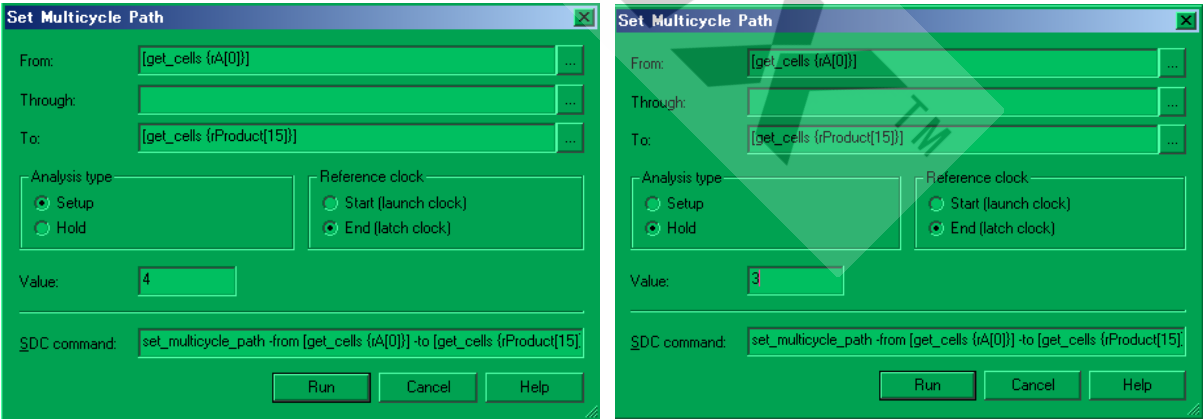


图 4.4.14 针对节点 rA[0]~rProduct[15] 执行 Set Multicycle Path 约束。

如图 4.4.14 所示 ,From 选项只要针对 rA[0] ,To 选项则只要针对 rProduct[15] 然后 设置 Setup Value 为 4 , Hold Value 为 3 即可，然后点击 Run，下面两段代码就会自动添加在 SDC 文件下。

```
set_multicycle_path -from [get_cells {rA[0]}] -to [get_cells {rProduct[15]}] -setup -end 4
set_multicycle_path -from [get_cells {rA[0]}] -to [get_cells {rProduct[15]}] -hold -end 3
```

接着双击 Update Timing Netlist 更新网表 ,然后在随意举例 10 个节点看看 ... 不过不同的是 rA[0]~rProduct[15] 有关的违规报告已经从此消失在这个世界上。





图 4.4.15 延迟怪兽有关的节点。

```

32.          0:
33.          begin rA <= A; rB <= B; rProduct <= 16'd0; i <= i + 1'b1; end
34.
35.          1:
36.          begin rProduct <= Result_U1; i <= 4'd0; end

```

如果一点一点约束的话，最终会耗死自己而已，所以我们必须寻找捷径 ... 图 4.4.15 是由 Delay_monster.v 在代码行 32~36 想象出来的关系图，亦即 rA, rB 经即时式 Booth 乘法器，致 rProduct 有关的所有节点。因此，我们只要按照图 4.4.15 的信息来执行约束即可。

From 选项方面选择 rA[0]~rA[7], rB[0]~rB[7] (无视 feeder); To 选线方面选择 rProduct[0]~[15]即可。

From :

```
[get_cells {rA[0] rA[1] rA[2] rA[3] rA[4] rA[5] rA[6] rA[7] rB[0] rB[1] rB[2] rB[3] rB[4] rB[5] rB[6] rB[7]}]
```

To :

```
[get_cells {rProduct[0] rProduct[1] rProduct[2] rProduct[3] rProduct[4] rProduct[5] rProduct[6] rProduct[7]
rProduct[8] rProduct[9] rProduct[10] rProduct[11] rProduct[12] rProduct[13] rProduct[14] rProduct[15]}]
```

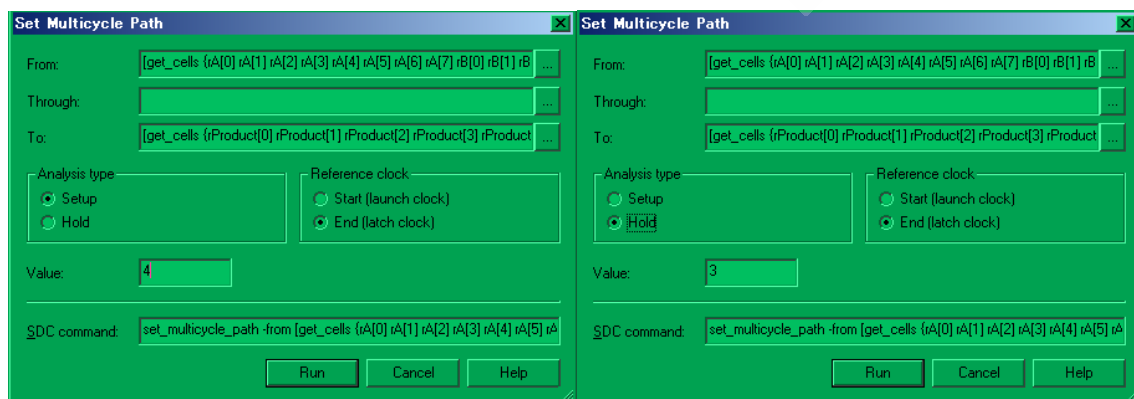


图 4.4.16 From 选线 rA[0]~rA[7], rB[0]~rB[7], To 选线 rProduct[0]~[15]。

再来重复与图 4.4.14 的操作，结果如图 4.4.16 所示，然后点击 Run，相关的 sdc 指令就会添加进入 sdc 文件内（这回代码因为太过冗长所以被阉掉了）。接着双击 Update Timing Netlist 更新网表，等待一段时间以后双击 Report Top Failing Path 看看，同学们会发现什么也没有，亦即“**满江红已经消失了 ... 怪兽不再杀人了 ...**”。

很好！我们已经成功使用 Set Multicycle Path 压制怪兽暴动之余，也不改动 100Mhz 主时钟频率，真是可喜可贺 ... 不过同学们别天真了，放心还是太早，虽然我们已经解决延迟怪兽的暴动事件，但是更可怕的连锁反应却已经在后头发生了 ... 这段话究竟是怎么回事呢？（To Be Continued ...）



4.5 Set Multicycle Path 的噩梦

严格来说这章节所要讨论的充其量只是笔者个人的担忧和多心而已，笔者宁愿自己是想多了，不过执着的念头始终让笔者方不小心来，至于为何呢？让我们先返回《时序篇》的年头开始说起 ...

笔者曾说过每个模块消耗几个时钟完成一次性的操作可有不定也有固定，不过更多时候模块都有固定的时钟消耗数来完成一个周期的操作，为了容易记忆笔者称呼这种操作周期为“模块周期”。一个健全的模块周期不仅拥有固定的时钟消耗数，而且什么时候执行甚麽 —— 这些事实是绝对不可改变。错了会打乱整个设计的执行节奏。

继续使用 delay_monster.v 作为例子：

代码 4.5.1 delay_monster 的相关操作。

```
32.          0:
33.              begin rA <= A; rB <= B; rProduct <= 16'd0; i <= i + 1'b1; end
34.
35.          1:
36.              begin rProduct <= Result_U1; i <= 4'd0; end
```

上面一段代码告诉我们，T0 的时候将 A 和 B 载入相对应的 rA 与 rB 里。在 T1 的时候，即时式 Booth 乘法器吐出结果，然后将其寄存到 rProduct 里，然后操作结束。换言之，delay_monster 模块一个模块周期仅消耗 2 个时钟而已。根据理想时序而言，即时式 Booth 乘法器是触发即时事件的组合逻辑而已，因此一个时钟已经足够。结果说，从 Verilog 的角度上来看 delay_monster.v 这个软模型是正确无误的。

不过，一旦这个软模型准备转换为硬模型，噩梦就开始了。默认网表下，即时式 Booth 乘法器的延迟压力大约是 30ns 左右，很明显频率 100Mhz 的主时钟，delay_monster 模块仅用 2 个时钟，亦即 20ns 是绝对支付不了该组合逻辑的延迟压力 ... 换之频率为 25Mhz 作为 delay_monster 模块的主时钟，它就拥有 80ns 的有余时间，应付 30ns 的延迟压力却却有余。

嘛 ... 这是理所当然的事实吗，不过上述的事实，Set Multicycle Path 约束命令和笔者的忧虑又有什么关系呢？让笔者还是用图来说话吧 ...



图 4.5.1 建模图。

图 4.5.1 是根据上述代码所创建的建模图，虽然与图 4.4.15 有点相识，不过为了更能详细说明一切，笔者已经变更一些。如果忽略位宽，基本图 4.5.1 有两个节点，亦即 rA 经即时式 Booth 乘法器 致 rProduct；又 rB 经 即时式 Booth 乘法器 致 rProduct。然后，经过[默认网表的评估](#)，即时式 Booth 乘法器大延迟压力大约为 31ns，为方便绘图在此取值为 30ns。

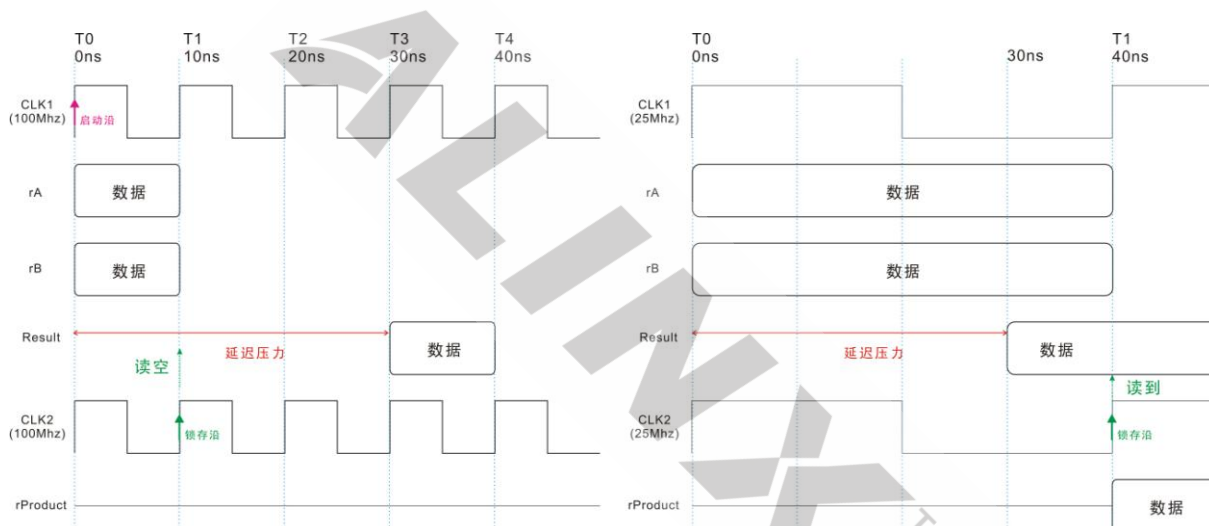


图 4.5.2 代码 4.5.1 在频率为 100Mhz（左图）和频率（25Mhz）的理想时序活动记录。

图 4.5.2 是代码 4.5.1 在两个不同频率下，亦即 100Mhz 和 25Mhz 的时序活动记录。

我们先将焦点转向右图，右图是时钟频率为 25Mhz 亦即 1 个时钟周期为 40ns。根据代码 4.5.1 中的步骤 0（32~33 行）T0 的时候 rA 和 rB 被赋值，同时间即时式 Booth 乘法器也开始工作。即时式 Booth 乘法器的加工时间大约是 30ns，换之 T1 之前亦即 30ns 的地方，加工结果 Result 就已经被吐出，然后在 T1 之际 rProduct 就会轻松的读入 Result。

接下来，我们把焦点转向图 4.5.2 的左图 ... 那是主时钟为 100Mhz 的情况下，在 T0 的时候 rA 和 rB 被赋值，不过即时式 Booth 乘法器需要 30ns 加工时间，因此在 T1 之际数据还没加工完成，结果 rProduct 读空。Result 在 T3 的时候才输出，最快也要在 T4 的时候才能读取。

在实验四中，虽然我们为 delay_monster 加入 multicycle path 的约束以致时序合格。事实上那只是 TimeQuest 这个数学工具在数字上的计算成功而已，实际中我们还没有配合模块周期来考虑 multicycle path 为模块周期带来的“噩梦”。

再来我们先用硬件的角度去观察 multicycle path 的行为 ...

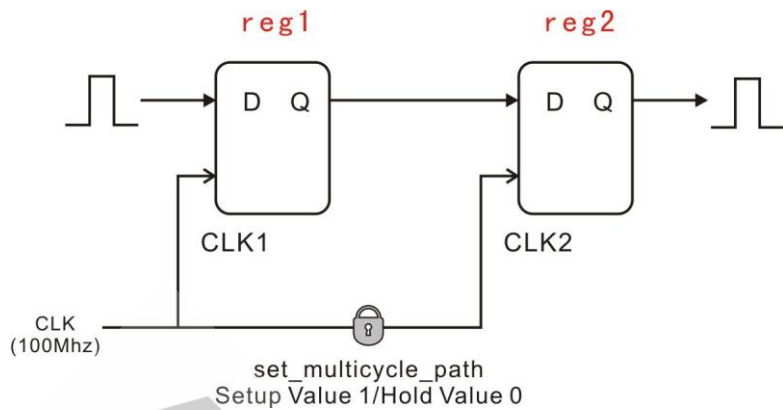


图 4.5.3 某节点被 multicycle path 约束为 Setup Value 1/Hold Value 0。

图 4.5.3 是某个节点被 multicycle path 约束为 Setup Value1/ Hold Value 0 ,multicycle path 在假想里就好比在该节点的 CLK2 路径中添加一个枷锁，不过 setup value = 1 与 hold value = 0 这种取值有约束和没约束没什么两样，不管怎么样我们还是来观察它的时序活动。（读者稍微注意一下，这个枷锁只是想象上的东西，实际上不存在硬模型的身上）

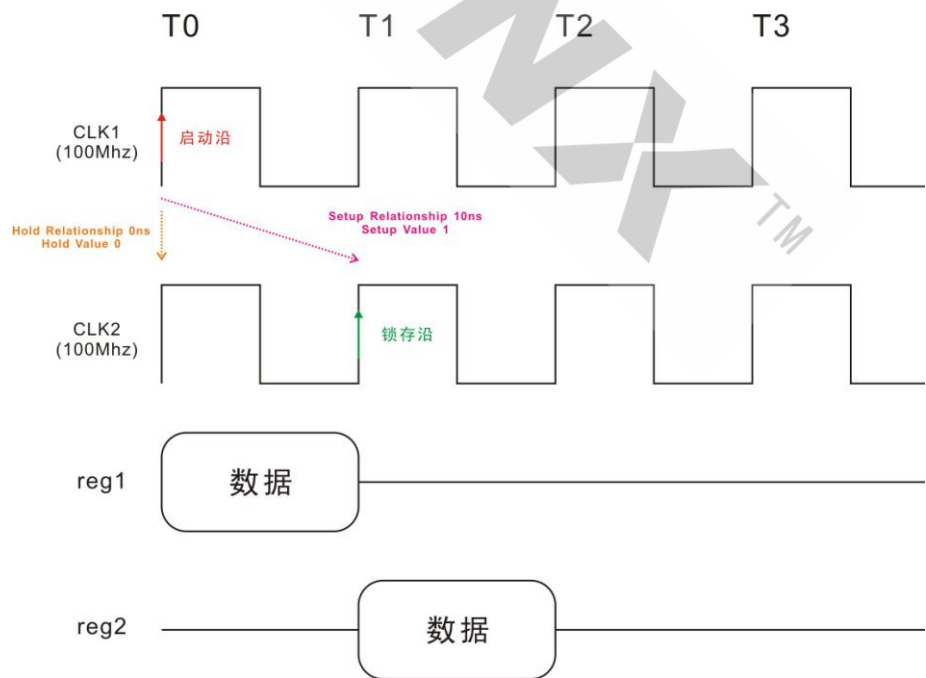


图 4.5.4 某节点 Setup Value 1/Hold Value 0 的时序活动。

我们假设 reg1 在 T0 发送仅有 1 个时钟周期的数据，在 T1 之际 reg2 会轻松的读取从 reg1 发送过来的数据，结果如图 4.5.4 所示。

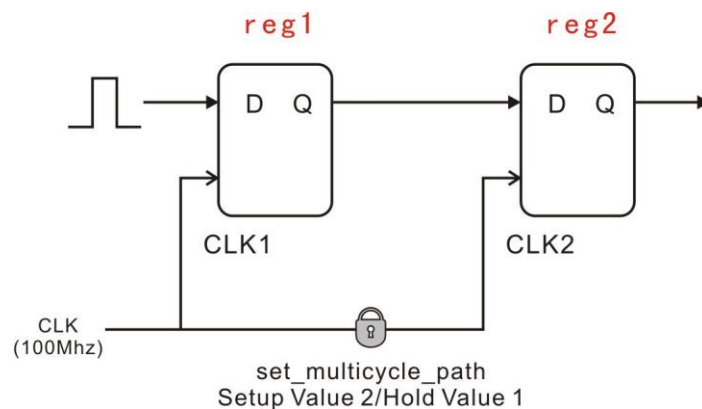


图 4.5.5 某节点被 multicycle path 约束为 Setup Value 2/Hold Value 1。

图 4.5.5 是某节点 multicycle path 的约束结果 Setup Value 为 2 和 Hold Value 为 1。如果仔细观察，我们会发掘到图 4.5.5 与图 4.5.4 有明显的不同，亦即 reg2 没有吐出任何数据，为什么呢？让我们来观察该节点的时序活动 ...（读者再稍微注意一下，这个枷锁只是 TimeQuest 的想象而已，实际与 HDL 的理想时序，或者硬模型本身无关）

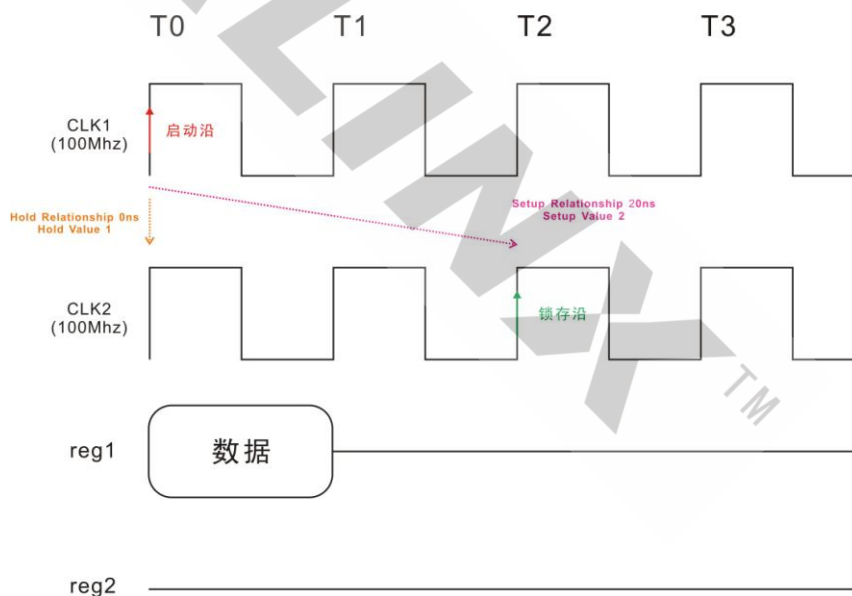


图 4.5.6 某节点 Setup Value 2/Hold Value 1 的时序活动。

我们依然假设 reg1 在 T0 的时候发送仅有 1 个时钟周期的数据给 reg2，不过有点遗憾的是 ... 由于该节点受到约束，亦即 Setup Value 为 2 和 Hold Value 为 1，因此 CLK2 的锁存沿也随着移动，结果 reg2 在 T1 的时候却发呆；在 T2 之际，reg2 准备读取数据的时候，可怜的它只能读到“空气”。

set multicycle path 该约束命令实际上它会更改启动沿或者锁存沿的触发时候。为了让同学们更有感的了解，我们再来观察图 4.5.6 节点的不同时序活动 ...

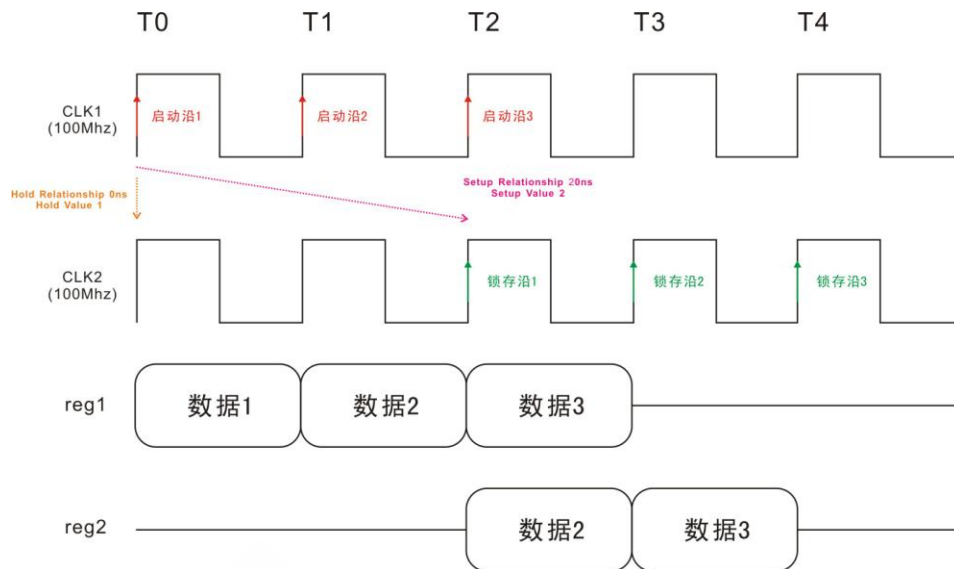


图 4.5.7 某节点 Setup Value 2/Hold Value 另一个时序活动。

图 4.5.7 是某节点的不同时序活动，在 T0~T2 之际 reg1 连续发送数据 1~3，至此之后一直保持沉默。reg2 在 T1 的时候由于受约束的影响结果在原地发呆。在 T2 之际，亦即锁存沿 1，对应启动沿 1，该读的数据也是数据 1 才对，可是 reg2 不小心读到了数据 2。在 T3 之际，亦即锁存沿 2，对应启动沿 2，该读的对象也是数据 2 才对，却又不小心读到了数据 3。在 T4 之际，亦即锁存沿 3，正 reg2 要发愤图强的时候，才发现头上已经没有数据可读了。

当同学们看到这里，隐约中是否已经察觉到笔者的忧虑呢？笔者的忧虑就是，**当我们用 set multicycle path 告诉 TimeQuest 某对节点用 N 个时钟工作，可是充其量这也是 TimeQuest 的妄想而已，因为实际的 HDL 行为或者硬模型没有任何改变。**

代码 4.5.1 delay_monster 的相关操作。

```

32.          0:
33.          begin rA <= A; rB <= B; rProduct <= 16'd0; i <= i + 1'b1; end
34.
35.          1:
36.          begin rProduct <= Result_U1; i <= 4'd0; end

```

让我们再仔细分析代码 4.5.1 的一切 ... 在步骤 0 的时候 rA 与 rB 以后，注 rA 和 rB 自步骤 0 之后一直驱动即时式 Booth 乘法器，然后 i 递增以示下一个步骤。在步骤 1 的时候，rProduct 读取该乘法器的输出亦即 Result_U1，此时 i 不在递增以示永远停留在步骤 1。简单说就是在**步骤 0 给即时式 booth 乘法器给源，然后在步骤 1 读取结果。**

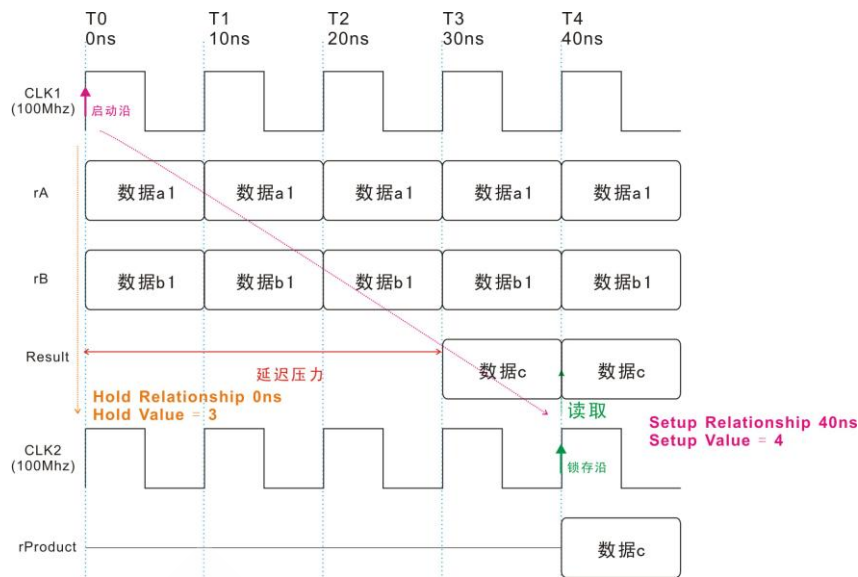


图 4.5.8 代码 4.5.1 在频率为 100Mhz 加约束 Setup Value 4/Hold Value 3 的详细时序活动。

图 4.5.8 我们可以看到，经过 $\text{setup value} = 4$ 和 $\text{hold value} = 3$ 的 set multicycle path 约束以后 TimeQuest 的妄想是：T0 之际 delay monster 模块在步骤 0 给即时式 booth 乘法器给衰，然后 delay monster 给空 3 个时钟，亦即 T1~T3 在打盹（3 个空步骤），然后在 T4 的时候，也是 delay monster 在步骤 4 读取结果。

笔者需要强调，在 TimeQuest 的妄想中 delay monster 用 5 个时钟工作，或者说 delay monster 有 5 个时钟的模块周期。不过 delay monster 的实际 HDL 行为，只用 2 个时钟工作，或者说实际的 delay monster 有 2 个时钟的模块周期。

很明显，delay monster 实际的模块周期与 TimeQuest 妄想中的 delay monster 模块周期有很大的出入，前者用 2 个时钟工作，而 TimeQuest 受 set multicycle path 冲昏头以后却妄想 delay monster 用 5 个时钟工作。这就是笔者所谓的忧虑。

根据代码 4.5.1 我们知道该 delay_monster 模块周期总共消耗 2 个时钟，可是根据图 4.5.8 的结果 TimeQuest 妄想该模块一共用了 5 个时钟周期。这种马头不对狗嘴的情形当然不被我们允许啦 ... 在此我们应该更动一下 delay_monster 的 hdl 内容才行。

4.6 不可理喻的 Set False Path

笔者曾在实验四里做过一个非常有趣的举动，亦即有关 delay_monster 模块：其中一种是将 100Mhz 的主时钟都约束为 Multicycle Path (Setup Value =4, Hold Value =3)；另一种则是直接使用 25Mhz 的主时钟。然而不同的操作却得出一样的结果 ... 借此，奇怪的念头在笔者的猪脑袋里开始萌生 ...

造就时序违规的凶手就是 rA[0]~[7], rB[0]~[7]和 rProduct[0]~[15]有关的节点而已，其犯罪动机就是在肚子里养了一只延迟怪兽，造成时钟周期不足够喂养它（时钟周期小于延迟压力）。解决的方法也挺简单，只要给足怪兽食物事情就完事休矣。不过根据实验四的结果，由于 Multicycle Path 约束的原因，却不小心打乱模块周期，造成半得倍失的局面。

未解决这个问题，我们必寻找考其他的出路 ... 为此，我们可以接签一下“25Mhz 主时钟”的经验。默认网表下，即时式 Booth 乘法器的延迟压力大约 30ns，结果 40ns 的时钟周期足够应付它。如果我们换另一个角度来思考，只要借助一下“黏糊关系”的力量，即时是 100Mhz 的时钟周期，我们只要给足时钟不是万事 Okay 吗？

代码 4.6.1 “黏糊概念”的给足时钟。

```
32.          0:
33.          begin rA <= A; rB <= B; rProduct <= 16'd0; i <= i + 1'b1; end
34.
35.          1,2,3:
36.          i <= i + 1'b1;
37.
38.          4:
39.          begin rProduct <= Result_U1; i <= 4'd0; end
```

具体一点让我们悄悄代码行 33~34 是关键 ... 章节 4.5 笔者也举例出实验四的 2 个关键条件，其中 rA 和 rB 供源 3 个时钟以上是最重要的。换句话说，即使我们为即时式 Booth 乘法器提供足够的时钟，但是驱动源 rA 和 rB 不足也会换取失败的结果。第 33 行代码执行着为 rA 和 rB 供源的角色，然而 34~35 行则停留 3 个时钟。

简单点说，33 行的操作加上 35~36 的 3 个时钟给空，供源一共保持 4 个时钟 ... 如果主时钟为 100Mhz，4 个时钟亦即 40ns。在此，延迟怪兽已经有足够的食物和时间 ... 随之在步骤 4 中（第 38~39 行）rProduct 就可以轻松的读取结果。

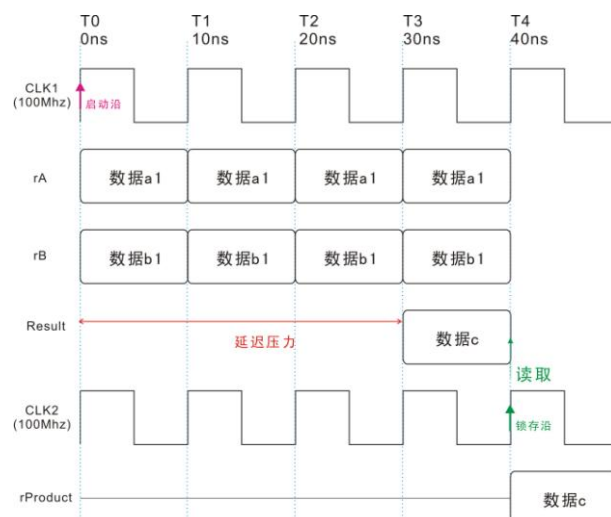


图 4.6.1 代码 4.6.1 的时序活动。

图 4.6.1 是代码 4.6.1 的时序活动 ... 然而时序活动完全按着步骤 i 的指向活动着，如 T0~T3 的时候 rA 和 rB 持续为即时式 Booth 乘法器供源之际，有给足加工时间；在 T4 的时候，rProduct 如步骤 i 所愿般读取输出 Result。结果而言，TimeQuest 的妄想中的 delay monster 就完全符合该模块实际的 HDL 行为了。

实验五 延迟怪兽受死吧！

Setup: CLK									
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data ...	v
1	-19.157	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.179	
2	-19.128	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.150	
3	-19.127	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.149	
4	-19.107	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.129	
5	-19.071	rA[0]	rProduct[13]	CLK	CLK	10.000	-0.018	29.093	
6	-19.069	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.091	
7	-19.066	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.088	
8	-19.061	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.083	
9	-19.057	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.079	
10	-19.056	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.078	
11	-19.047	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.069	
12	-19.042	rA[0]	rProduct[13]	CLK	CLK	10.000	-0.018	29.064	
13	-19.041	rA[0]	rProduct[13]	CLK	CLK	10.000	-0.018	29.063	
14	-19.033	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.055	
15	-19.021	rA[1]	rProduct[15]	CLK	CLK	10.000	-0.018	29.043	
16	-19.021	rA[0]	rProduct[13]	CLK	CLK	10.000	-0.018	29.043	
17	-19.018	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.038	
18	-19.015	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.037	
19	-18.992	rA[1]	rProduct[15]	CLK	CLK	10.000	-0.018	29.014	
20	-18.992	rA[0]	rProduct[15]	CLK	CLK	10.000	-0.018	29.014	

图 4.6.2 部分与 rA, rB 和 rProduct 有关节点的 Setup 报告。

双击 Report Top Failing Path 随便举例 200 个节点的 Setup 报告，在此我们依然看见一片满江红 ... 这是什么一回事？我们不是按照理论给予 rA, rB 和 rProduct 有关节点足够的供源和时间吗？为何会这样？原因很简单，到目前位置我们还没有告诉 TimeQuest “节点 rA~rProduct 或者节点 rB~rProduct 是用 4 个时钟工作”。不过，实验五笔者不像使用 set multicycle path 这个命令而是其他。

让我们回忆第二章的内容，我们知道 TimeQuest 既是笨蛋又是勤劳的数学工具，它会一点一点检查所有模块的节点，然而 TimeQuest 只会用“1 个时钟的情况”去检查节点（Set Multicycle Path 约束例外）。

为此 HDL 行为的“给足时钟”是人为，如果不使用类似 set multicycle path 的约束命令，TimeQuest 是不晓得对节点用 N 个时钟工作，可是实验五笔者又不想用它。首先，让我们再一次观察图 4.6.2 右边的 Data Delay，平均值为 30ns，而 delay monster 的 HDL 内容已经给足 4 个时钟的供源和空时间。

虽然我们在心里上十稳九拿该模块不会有问题，不过汪海一片红始终碍眼和碍事。因此，我们要使用另一个约束命令，亦即 Set False Path 要 TimeQuest 无视相关的节点分析。Set False Path 的用法很简单，之下输入相关的节点，无视操作就会被执行。

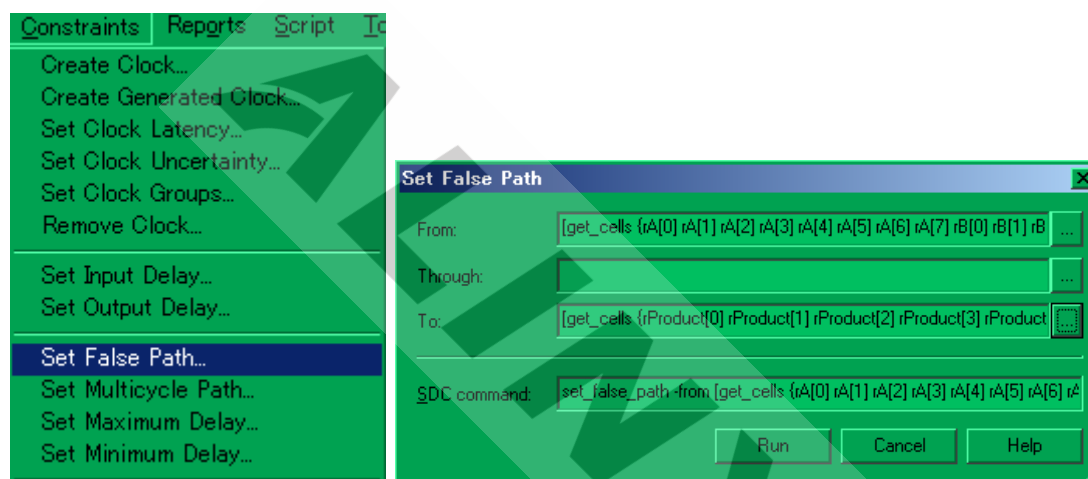


图 4.6.3 调出 Set False Path 界面。

先从 Constraints 菜单中调出 Set False Path 界面，然后将 From 选项填入 rA[0]~[7] 和 rB[0]~[7]，至于 To 选项填入 rProduct[0]~[15]，接着点击 Run 即可。

```
set_false_path -from [get_cells {rA[0] rA[1] rA[2] rA[3] rA[4] rA[5] rA[6] rA[7] rB[0] rB[1] rB[2] rB[3] rB[4] rB[5] rB[6] rB[7]}] -to [get_cells {rProduct[0] rProduct[1] rProduct[2] rProduct[3] rProduct[4] rProduct[5] rProduct[6] rProduct[7] rProduct[8] rProduct[9] rProduct[10] rProduct[11] rProduct[12] rProduct[13] rProduct[14] rProduct[15]}]
```

上述代码就会自动加入 sdc 文件中，然后双击 Update Timing Netlist，最后双击 Report Top Failing Path。同学们会发觉到 TimeQuest 不再反馈任何时序的违规信息。

Set False Path 约束命令式一个不可理喻的东西，它会要求 TimeQuest 在分析时无视相关的节点 ... 同时 Set False Path 也是一个危险的约束命令，如果同学们没有足够的信心，千万不要随意使用。

经历许久人与怪兽的战斗终于可以划上句号，为了解决延迟怪兽的暴动事件，无论遇上多少失败，身为勇士的同学即屹立不倒，无数次掉落的圣剑也不停举起 ... 如此坚持仅为了我们背后的家园还有家人

嘛 ... 开个玩笑而已，放松点少年。最后让我们来总结一下在实验三提出的问题，即是：“Quartus 的编译器有没有足够的智能保证任何节点都合格？如果是又如何优化？如果不是又怎么约束？用什么约束命令？”在此我们可以这样总结 ...

事实上，编译器它不会聪明到可以自动优化延迟怪兽给时序带来的违规，因此我们需要约束相关的节点。用到的命令有 Set Multicycle path ，该功能宛如告诉 TimeQuest 某对节点用 N 个时钟工作，实际上是告诉 TimeQuest 某对节点的启动沿或者锁存沿的触发行为有所更改。虽然使用该约束命令可以使得 TimeQuest 在分析上足以解决问题，可是却导致 TimeQuest 的妄想和实际的 HDL 行为产生出入。所以要小心慎用。

此外，Set False Path 是不可理喻的约束命令，它会要求 TimeQuest 无视相关的节点。使用 Set False Path 之前，最好确认自己是否有足够的理论自己的做法是对的。如果实验五的 delay monster 恰恰好仅用 2 个时钟工作，使用该命令反而会弄巧成拙。

总结：

这一章我们讨论了有关内部延迟因数的种种，借此进一步掌握 TimeQuest 的各个操作还有各种微妙的特性，当然，重点部分还是约束命令的了解上。结束之前让我们简要的回顾一下各个约束命令的内容：

```
derive_pll_clocks  
derive_clock_uncertainty
```

是 pll 专用的约束命令，其中 `derive_pll_clocks` 是专门为 pll 打造的懒人版 `create clock` 约束命令。至于后者就意义不明了 ... 好像是和制作工艺和时钟抖动有关系，嗯 ... 往后有机会再接触也不迟。

```
set_multicycle_path
```

这个约束命令粗略点就是告诉 TimeQuest 某对节点用 N 个时钟工作，实际上它是告诉 TimeQuest 某对节点的启动沿和锁存沿的触发行为有所更改。笔者想定读者一定会对 `reference clock` 的 `Start` 选项感到好奇，因为在实验中笔者只顾着用 `End`，结果没机会讲解。

嘛 ... 同学们就暂时忘了它吧，以目前的实验实在做找不到任何机会，如果同学们有兴趣可以自行查看官方的手册，好像叫做 `multicycle path exception`，里边有介绍原理和用法。笔者个人更注重实践，往后有机会再做打算。

```
set_false_path
```

这个无赖即不可理喻的约束命令基本上没什么好说，该功能是命令 TimeQuest 无视某个节点。嘛 .. 只要用得好好它还是大家的好朋友，相反的，如果用不好就要遭殃了。

基本上就是这些而已，同学们可能会为笔者没有提及寄存器相关的延迟因数而感到窝囊，对吧？这个嘛 ... 下一章一定会有啦，目前本章的目的是让鸭仔适水和作热身而已。学习应该慢慢来，是不是有道理呢？

最后让我们稍微反思一下，TimeQuest 如果没有结合 Verilog 一起探讨的话，基本上 TimeQuest 宛如硬邦邦的数学工具而已。相反，一旦结合 Verilog 各种特质来探讨，意外的我们会发现许多有趣的认识。果然相交一年，完成《整合篇》之后再看回 TimeQuest，让笔者有全新的认识，之前的辛劳都是有价值的。请问同学们是否也有同样的感受呢？

第五章：网表质量与外部模型

5.1 网表质量的概念

笔者在第二章稍微谈过网表 ... 不过现在又为何会无缘无故提及它呢？笔者以前曾以为 TimeQuest 等静态时序工具是按照实际的物理情况来评估物理时序，不过事实却不是如此。即使静态时序工具再怎么神，身为数学工具的它只能模拟出“近似”的情况而已，其中一定留有余量。

同样，笔者也在第四章不停重复“默认网表下 ... 默认网表下” ... 难道读者不觉得有伏笔吗？笔者不是在卖关子，因为根据笔者的脚步而言，要理解接下来的内容，必须对 TimeQuest 做好足够的认识。为此，笔者特地绕了一个大圈，为了就是让同学们做好准备。

TimeQuest 作为数学工具必须根据各种 CASE 作出分析，CASE 简称点就是环境或者状况，然而 TimeQuest 却有两种 CASE，其一是 Worst Case，其二是 Best Case，前者对应 slow model，后者对应 fast model。

表 5.1 Worst Case 和 Best Case 的对应因数。

Case	模型	环境	温度	性能限制
Worst Case	slow	极端	低温/高温	高
Best Case	fast	最佳	室温	低

如表 5.1 所示，除了 slow model 和 fast model 以外，Worst Case 分别对应的其他因数有，极端环境，还有低温/高温的影响。换之 Best Case 对应应有最佳环境和室温。

所谓 Worst Case 是指极端的环境，也指主供源的电压不稳定以外，还有各种内部延迟因数都推向极端，结果拖累整个硬模型的速度——Fmax，最终成为 slow model，如此反思于是越坏的情况，衍生的保险余量就越多，因此性能限制也很大。

所谓 Best Case 是指最佳或者理想环境，也指不仅足够而且还稳定，此外各种内部延迟因数越近似实际，也愈小，结果而言整个硬模型的速度——Fmax 也会跟着提高，最终成为 fast model。如此反思，越是理想的环境，一旦遇上麻烦可衍生的保险余量也有限，因此性能限制也很小。

笔者这样一讲，可能会让同学们一时之间不知所措 ... 不过我们可以这样认为。保险余量就好比一位生存者，常常会思考最坏的情况，假设情况 A。换言之，只要一天不发生比情况 A 更糟的事情，这位生存者都可以有余过活。相反的，那些乐天的屁民们已经习惯过着舒适的生活，假设为情况 B。换句话说，只要稍微发生比情况 B 哪怕糟一点点的事情，屁民们就会被吓到半死。

TimeQuest所谓的默认 CASE —— slow model也指最极端的情况 结果而言 Time Quest 就像为世界末日作好准备的生存者，往往评估和分析极端环境的各种状况。粗略一点说，凡是通过 slow model 分析的硬模型，fast model 理应也合格。反过来说，凡是在 fast model 合格（屁民）的硬模型，不一定通过 slow model 的分析。

继续深入之前，让我们回顾一下什么是 post-fit 什么是 post-map。虽然官方的手册上有定义，用俗话说的话，post-fit 网表会参考近似物理的参数（优化之后），而 post-map 只会参考原生的编译结果（优化之前）。论质量而言，post-fit 比起 post-map 更适合。

所谓的默认网表，是指 post-fit model -slow，也是双击 Create Netlist 快捷选项会产生的结果，对应的 tcl 命令式 create_timing_netlist -model slow。结果而言 TimeQuest 是不折不扣的生存主义者，打从一开始就已经评估任何最坏的情况。当然读者也可以回忆一下在第三章中所描述的“玩具与金钱还有妈妈”的比喻。

这个章节的重点，我们仅要了解各种 case 对内部延迟因数造成的影响，其中也包括寄存器的特性，如 tsu，tco 和 th 等。笔者继续以实验三的 delay_monster 为例子，其中除了 100Mhz 的 create clock 声明以外，一切约束命令通通都拿掉。

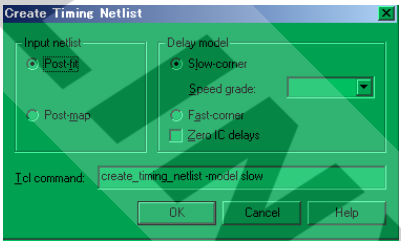


图 5.1.1 默认网表 worst case / slow model

首先我们先示范默认网表，亦即 worst case 或者 slow model，读者可双击 create timing netlist 选项，也可以调出 Create Timing Netlist 窗口，然后手动建立（如图 5.1.1 所示）

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	-27.378	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.439
2	-27.378	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.437
3	-27.358	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.419
4	-27.356	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.417
5	-27.343	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.404
6	-27.341	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.402
7	-27.334	rA[0]	rProduct[13]	CLK	CLK	10.000	0.021	37.395
8	-27.323	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.384
9	-27.321	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.382
10	-27.314	rA[1]	rProduct[13]	CLK	CLK	10.000	0.021	37.375

图 5.1.2 随意举例 10 个节点的 setup 报告。

让后随意列出 10 个节点的 setup 报告，目前该时序违规与否，不是我们要关心的问题。然而重点就在于 slack 和 Data Delay 值，以节点 1 为例 -27.378ns 和 37.439ns。

Path Summary Statistics Data Path Waveform							
Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					launch edge time
2	2.813	2.813					clock path
3	2.813	2.813	R				clock network delay
4	40.252	37.439					data path
5	3.117	0.304		uTco	1	LCFF_X21_Y9_N1	rA[0]
6	3.117	0.000	RR	CELL	16	LCFF_X21_Y9_N1	rA[0] reset
Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	10.000	10.000					latch edge time
2	12.834	2.834					clock path
3	12.834	2.834	R				clock network delay
4	12.874	0.040		uTsu	1	LCFF_X22_Y8_N3	rProduct[13]

图 5.1.3 节点 1 的详细信息 (Setup)。

然后打开节点 1 的详细信息，在此观测 clock path—2.813ns 还有 uTco—0.304ns 和 uTsu—0.040ns。

Command Info Summary of Paths								
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	5.952	rA[0]	rProduct[13]	CLK	CLK	0.000	0.021	6.279
2	7.182	rA[0]	rProduct[13]	CLK	CLK	0.000	0.021	7.509
3	7.325	rA[0]	rProduct[13]	CLK	CLK	0.000	0.021	7.652
4	7.581	rA[0]	rProduct[13]	CLK	CLK	0.000	0.021	7.908
5	7.616	rA[0]	rProduct[13]	CLK	CLK	0.000	0.021	7.943
6	8.241	rA[0]	rProduct[13]	CLK	CLK	0.000	0.021	8.568
7	8.269	rA[0]	rProduct[13]	CLK	CLK	0.000	0.021	8.596
8	8.343	rA[0]	rProduct[13]	CLK	CLK	0.000	0.021	8.670
9	8.402	rA[0]	rProduct[13]	CLK	CLK	0.000	0.021	8.729
10	8.414	rA[0]	rProduct[13]	CLK	CLK	0.000	0.021	8.741

图 5.1.4 随意举例 10 个节点的 hold 报告。

再来随意举例 10 个节点的 hold 报告，以节点 1 为例，slack 为 5.952ns，而 Data Delay 为 6.279ns。

Path Summary Statistics Data Path Waveform							
Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					launch edge time
2	2.813	2.813					clock path
3	2.813	2.813	R				clock network delay
4	9.092	6.279					data path
5	3.117	0.304		uTco	1	LCFF_X21_Y9_N1	rA[0]
6	3.117	0.000	RR	CELL	16	LCFF_X21_Y9_N1	rA[0] reset
Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					latch edge time
2	2.834	2.834					clock path
3	2.834	2.834	R				clock network delay
4	3.140	0.306		uTh	1	LCFF_X22_Y8_N3	rProduct[13]

图 5.1.5 节点 1 的详细信息 (Hold)。

再来展开节点 1 的详细信息和观察 uTh—0.306ns。

Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	26.75 MHz	26.75 MHz	CLK	

图 5.1.6 Fmax 结果。

接着双击 Report Fmax Summary 调出 Fmax 简报，结果如图 5.1.6 所示—26.75Mhz。

=====

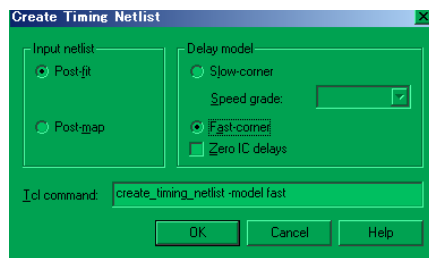


图 5.1.7 自定义网表 best case / fast model

在此让我们建立 best case 网表,亦即 fast model。这个必须手动建立,如图 5.1.7 所示。

Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	-1.194	rA[0]	rProduct[13]	CLK	CLK	10.000	0.022	11.248
2	-1.191	rA[0]	rProduct[13]	CLK	CLK	10.000	0.022	11.245
3	-1.182	rA[0]	rProduct[13]	CLK	CLK	10.000	0.022	11.236
4	-1.181	rA[1]	rProduct[13]	CLK	CLK	10.000	0.022	11.235
5	-1.179	rA[0]	rProduct[13]	CLK	CLK	10.000	0.022	11.233
6	-1.178	rA[1]	rProduct[13]	CLK	CLK	10.000	0.022	11.232
7	-1.173	rA[0]	rProduct[13]	CLK	CLK	10.000	0.022	11.227
8	-1.170	rA[0]	rProduct[13]	CLK	CLK	10.000	0.022	11.224
9	-1.169	rA[1]	rProduct[13]	CLK	CLK	10.000	0.022	11.223
10	-1.166	rA[1]	rProduct[13]	CLK	CLK	10.000	0.022	11.220

图 5.1.8 随意举例 10 个节点的 setup 报告。

接着随意列举 10 个节点的 setup 报告,同样 ... 此时时序违规与否都不是重点,让我们注意节点 1 的 slack—1.194ns 还有 Data Delay—11.248ns

Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					launch edge time
2	1.469	1.469					clock path
3	1.469	1.469	R				clock network delay
4	12.717	11.248					data path
5	1.610	0.141		uTco	1	LCFF_X21_Y9_N1	rA[0]

Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	10.000	10.000					latch edge time
2	11.491	1.491					clock path
3	11.491	1.491	R				clock network delay
4	11.523	0.032		uTsu	1	LCFF_X22_Y8_N3	rProduct[13]

图 5.1.9 节点 1 的详细信息 (Setup)。

再来展开节点 1 的详细信息,关注 clock path—1.469ns,uTco—0.141ns 和 uTsu—0.032。

Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	1.845	rA[0]	rProduct[13]	CLK	CLK	0.000	0.022	2.019
2	2.198	rA[0]	rProduct[13]	CLK	CLK	0.000	0.022	2.372
3	2.208	rA[0]	rProduct[13]	CLK	CLK	0.000	0.022	2.382
4	2.267	rA[0]	rProduct[13]	CLK	CLK	0.000	0.022	2.441
5	2.270	rA[0]	rProduct[13]	CLK	CLK	0.000	0.022	2.444
6	2.471	rA[0]	rProduct[13]	CLK	CLK	0.000	0.022	2.645
7	2.507	rA[0]	rProduct[13]	CLK	CLK	0.000	0.022	2.681
8	2.526	rA[0]	rProduct[13]	CLK	CLK	0.000	0.022	2.700
9	2.548	rA[0]	rProduct[13]	CLK	CLK	0.000	0.022	2.722
10	2.557	rA[0]	rProduct[13]	CLK	CLK	0.000	0.022	2.731

图 5.1.10 随意举例 10 个节点的 hold 报告。

同样随意举例 10 个节点的 hold 时序报告,关注节点 1 的 slack—1.845ns。

Path Summary Statistics Data Path Waveform							
Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					launch edge time
2	1.469	1.469					clock path
3	1.469	1.469	R				clock network delay
4	3.488	2.019					data path
5	1.610	0.141		uTco	1	LCFF_X21_Y9_M1	rA[0]
6	1.610	0.000	FF		16	LCFF_X21_Y9_M1	rA[0]
Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					latch edge time
2	1.491	1.491					clock path
3	1.491	1.491	R				clock network delay
4	1.643	0.152		uTh	1	LCFF_X22_Y8_M3	rProduct[13]

图 5.1.11 节点 1 的详细信息 (Hold)。

接下来展开节点 1 的详细信息，关注 uTh—0.152ns。

Fmax Summary			
	Fmax	Restricted Fmax	Clock Name
1	89.33 MHz	89.33 MHz	CLK

图 5.1.12 Fmax 结果。

最后取得 Fmax 简报 —— 89.33Mhz

根据上述的内容，笔者以节点 1 为例 (Setup 和 Hold 各不同)，然后根据不同的网表状况，再取各种参数做出表 5.1。

表 5.1 节点 rA[0]~rProduct[13] 革命剧 Worst Case 和 Best Case 的相关参数比较。

rA[0]~rProduct[13]	worst case/ slow model	best case/ fast model
Setup Slack	-27.378ns	-1.194ns
Data Delay	37.439ns	11.248ns
Clock Path	2.813ns	1.469ns
uTco	0.304ns	0.141ns
uTsu	0.040ns	0.032ns
Hold Slack	5.952ns	1.845ns
uTh	0.306ns	0.152ns
Fmax (约束前)	26.75Mhz	89.33Mhz
Fmax (约束后)	107.01 MHz	357.27 MHz

实际上表 5.1 不用看都知道那是一面倒的情形，亦即节点 rA[0]~rProduct[13] 的 worst case 相较 best case 之下都呈现各种极端的参数结果。由于 worst case 把所有内部延迟因数都推向极端，结果导致 Fmax 的速度下降 (无论是约束前或者约束后)。不过表 5.1 还有一个特殊的情形让同学们在不知觉中产生疑问 ...

“worst case 的 hold slack 不应该比 best case 的 hold slack 更小才对吗？”，让我们回忆一下用屁股求出 hold 余量的公式：

保持余量 = 数据抵达时间 - 数据获取时间
Hold Slack = Data Arrival Time - Data Required Time

数据抵达时间 Data Arrival Time = 启动沿 + Tclk1 + Tco + Tdata

数据获取时间 Data Required Time = 锁存沿 + Tclk2 + Th
= 理想保持关系值 + Tclk2 + Th

Hold Slack 是 Data Arrival Time 相减 Data Required Time 以后的结果，其中最为关键的是 Tdata (Data Delay)，worst case 与 best case 相比之下 worst case 的 Data Delay 还有大过 best case 的 Data Delay。结果也因此造成 worst case 的 hold 余量大于 best case 的 hold 余量。

=====

笔者曾经这样思考过：“创建网表会不会也是某种约束呢？”例如创建默认网表 A，sdc 文件下就会添加相关的约束，然后编译器会依照这个约束命令来创建网表 A 的硬模型；如果是网表 B，就会创建网表 B 的硬模型等 ... 这是学习 TimeQuest 的思路陷阱。

事实上，网表的概念非常直接 ... 首先把 TimeQuest 想象成硬模型的模拟器，我们可以假设 best case 硬模型是最接近物理硬模型，不过相较实际的硬模型 best case 的硬模型还保留 10~20% 的余量。假设物理硬模型的某个延迟值是 10ns，那么 best case 硬模型对于 10ns 采取保留的 12ns。

相反之下 worst case 硬模型是最保险的硬模型，比起 best case 硬模型 worst case 硬模型相较物理硬模型有 80~90% 的余量。再假设物理硬模型的某个延迟值是 10ns，那么 worst case 硬模型会采取保险的 19ns。

这个作法有一个好处 ... 就算 best case 硬模型遇见状况，如果不超过 10~20% 的余量范围，结果还是健康。相反对于 worst case 硬模型而言，余量范围比起 best case 硬模型还要更大。举例某个状况是发生 35% 的范围，那么 best case 硬模型就会出现问題，换之 worst case 硬模型还是稳如泰山。

让我们继续思考 ... 有关延迟怪兽实验的重点就是“即时式 Booth 乘法器”。31ns 的延迟压力是不是实值呢？31ns 延迟压力是 worst case 硬模型的保险估算，最接近是实值是 best case 硬模型，延迟压力大约是 10ns 左右。即时式 Booth 乘法器的压力取值实际上比 best case 硬模型还要低，不过具体的数值是多少呢？这点我们就知道了。

实验六的 delay_master 笔者以 worst case 硬模型评估即时式 booth 乘法器的延迟压力，大约是 30ns 左右，然后供源和加工时间都给予 40ns 的。

总体而言，默认下的 TimeQuest 都是采取保险措施，亦即默认下网表是 slow model，默

认硬模型是 worst case。不过，这不是强制性的 ... 同学们还有各种选项自由选择，如 post-fit 和 post-map，或者 speed grade，还是 zero ic delay 等。

不过根据一般使用习惯，有时候需要比 worst case 更极端的环境 ... 假设同学预测到在环境 A 会使时钟频率更动 10%，除了选择 worst case 硬模型以外，同学还可能不放心，所以在 create clock（声明主时钟）的时候，会故意为主时钟留下 10~20% 的余量 ... 如 100Mhz 的频率，会用 80~90Mhz 或者 100~120 Mhz 的频率分别测试 worst case 硬模型。

另外还有一种说法说，best case 网表 setup 分析不容易合格，worst case 网表 hold 分析不容易合格。结果会分别用 best case 针对 setup 分析，用 worst case 针对 hold 分析，如果 best case 下 setup 分析有余量 & worst case 下 hold 分析有余量，那么这个模块就是没问题。

此外也有这样的说法，在室温（常温）以外的极端温度：如在低温状态，所有资源会有所迟钝；高温状态会提高资源的阻值。所以 worst case 有时候也用来测试模块在某种恶劣的温度下是否能正常工作。

嘛 ... 网表的故事是千奇百怪的，对于普通的使用而言：worst case 最保险但是性能限制大，best case 保险有限但是性能限制小。说白了，假设 FPGA 的最高频率是 300Mhz，如果使用 best case，最高频率使用允许是 280Mhz 左右；如果使用 worst case，最高频率的使用允许是 60Mhz 左右。

总结说“牛排到底要几成熟最后还是要根据客官当时的口味”，笔者是非常怕死，所以吃牛排都是全熟，亦即 worst case 最适合笔者 ... 结果同学又如何呢？最后补上不相关的话题 ... 当 TimeQuest 为某个设计建立网表（硬模型）的时候，而这个网表只是分析对象却没有环境因数，或者缺少分析环境 ... 因为如此，我们需要透过各种约束命令为分析对象制造刺激，以致达到接近的现实的环境因数，这也是俗称的网表约束。

这个环境因数就是所谓的内部延迟因数，还有外部延迟因数。内部延迟因数 TimeQuest 可以自动晓得，反之外部延迟因数却必须人为告知。假设某个设计的硬模型，它的外部路径有 3ns 的延迟 ... 当我们用 TimeQuest 这个硬模型的时候，我们有必要将 3ns 延迟的外部路径告诉 TimeQuest，好让它可以分析 3ns 的延迟是否对这个硬模型的时序作出违规。

5.2 Fmax 评估值

fmax 是什么？ Fmax 是一个用处不大的评估值，亦即**某节点的最高频率（也可以理解成是组合逻辑的数量）**，同时也是网表性能限制的评估值。当我们双击 Report Fmax Summary，TimeQuest 会将某个节点的 Fmax 提出；如果某个设计有时序违规，TimeQuest 又会将那个有问题的 Fmax 提出。

Fmax 之所以暧昧是因为它仅评估某个节点 ... 不，更贴切说是部分节点的最高频率。在一群小朋友之中有几位是特别调皮，结果我们却不能说所有小朋友都一样调皮，Fmax 的情形就好比这个比喻。在前面我们了解过 worst case 是某个极端的状况，亦即余量范围的保留很大，内部延迟压力的保险取值也随之很高，结果却导致 Fmax 的评估降低。换句话说，best case 的 Fmax 评估比起 worst case 有更好的表现（也更接近实体的 Fmax 评估）。

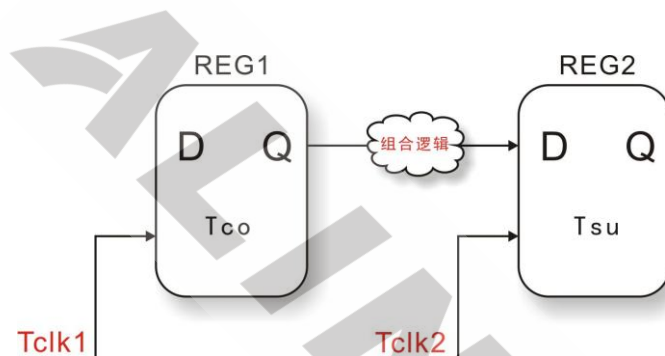


图 5.2.1 Fmax 评估模型。

$$fMAX = 1 / (<register to register delay> - <clock skew delay> + <micro setup delay> + <micro clock to output delay>)$$

$$\begin{aligned} \text{clock skew delay} &= <\text{clock to destination register delay}> - <\text{clock to source register delay}> \\ &= Tclk2 - Tclk1 \end{aligned}$$

图 5.2.1 是 Fmax 评估模型（基本上也是基于 TimeQuest 分析模型）而 Fmax 求出的公式如上述所示，包含：reg2reg delay 就是两者寄存器之间的总和延迟，亦包括组合逻辑延迟，路径延迟等；clock skew delay，中文叫时钟差也是 Tclk2 相减 Tclk1 的结果；micro setup delay 的符号是 Tsu，也是目的寄存器最小的建立时间；micro clock to output delay 的符号是 Tco，也是源寄存器输出最小延迟时间。

从一个简单的公式中我们可以理解到，Fmax 事实上是**单个节点的最高频率评估值**，其中最左右 Fmax 的凶手就是 reg2reg delay ... 粗略点说，**Fmax 高低是否，是由寄存器之间夹着的组合逻辑的延迟压力来决定**。Fmax 之所以暧昧是因为 TimeQuest 每当给出 Fmax，实际上是针对“**最差劲节点**”的评估值。

换句话说，**如果某个设计出现时序违规，TimeQuest 就会给出那个“最差劲节点”的 Fmax，**

而不是其他 ... 那怕其它节点也是违规，只要不是最差劲就不会被 TimeQuest 提名。

“如何提升 Fmax” 算是学习 TimeQuest 的迷信吧 ... 有许多同学认为只要 Fmax 比系统时钟（主时钟）的频率还要高时序越 Okay。话说这种迷信也不无道理，因为 Fmax 是针对“最差劲节点” ... 换言之，如果该“最差劲节点”得到好成绩，那些不是最差劲的节点的成绩当然比“最差劲节点”好。这回就让笔者暂时入乡随俗和大伙一起迷信，一同寻找“提升 Fmax”的升仙之路。

Fmax 另一种既是“组合逻辑的数量”，如何减少组合逻辑的数量，直接上与 HDL 的建模有十足的关系，不过我们先抛开 HDL 不说 ... 如此一来，网表的质量就是下一个因数，同样我们也抛开网表不说 ... 这个抛开 那个抛开 余下还有什么因素可以影响 Fmax 呢？啊 ... 就只剩下编译器了。

我们知道软模型要变成硬模型就需要经过编译器的综合 ... 话是这么说，不过我们又要如何设置编译器呢？回去 Quartus，Assignment 菜单进入 Setting：

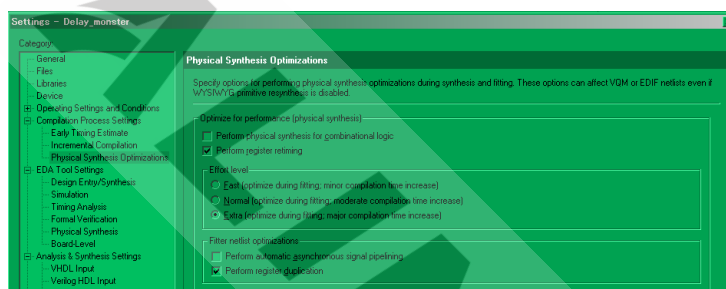


图 5.2.2 Physical Synthesis Optimizations 优化选项。

如图 5.2.2 所示，我们可以打开相关的选项如 Perform register retiming；Effort Level - Extra；Perform register duplication。

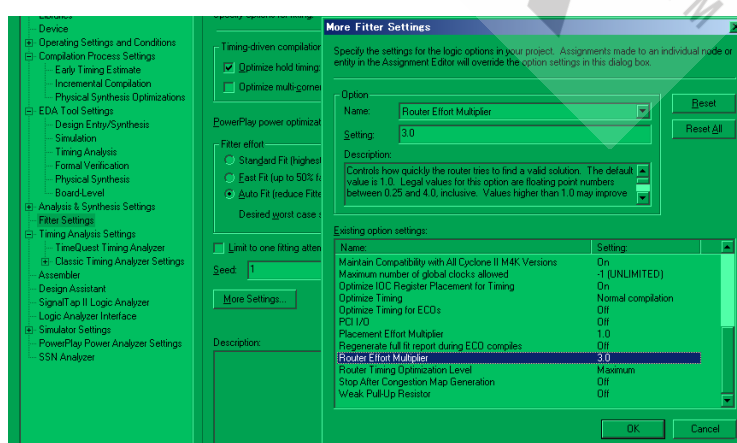


图 5.2.3 Filter Settings 优化选项。

或者进入 Filter Setting 打开 More Settings 然后将 Router Effort Multiplier 设置 3.0 还有 Router Timing Optimization level 设置 Maximum。基本上设置上述优化选项以后，

Fmax 可以提高 10~15%，不过要增加编译的时间。（至于这些优化选项有什么具体的意义？请高抬贵手放过笔者，自己回去翻翻 Quartus 手册就可以知道了）

好了，让我们把主题切回 Verilog ... 虽然说减少组合逻辑就等于提升 Fmax，不过在建模的时候，使用组合逻辑是不可避免的，而且组合逻辑的使用数量我们也是模糊。举个例子来说：

```
if( C1 == 10 ) begin ... end
```

当我们声明 if，不知使用了 N 个组合逻辑 ... 然后声明 == 有不知道使用了 N 个组合逻辑。说实在，如果不使用 if 或者 == 等这些关键字 HDL 就不是 HDL 了。既然如此 ... HDL 与 Fmax 之间还存在什么“优化空间”呢？我们又该如何下手呢？同学们还记得《整合篇》里出现过的“[操作模式](#)”吗？事实上，我们可以从操作模式上下手，不过这话又如何说呢？

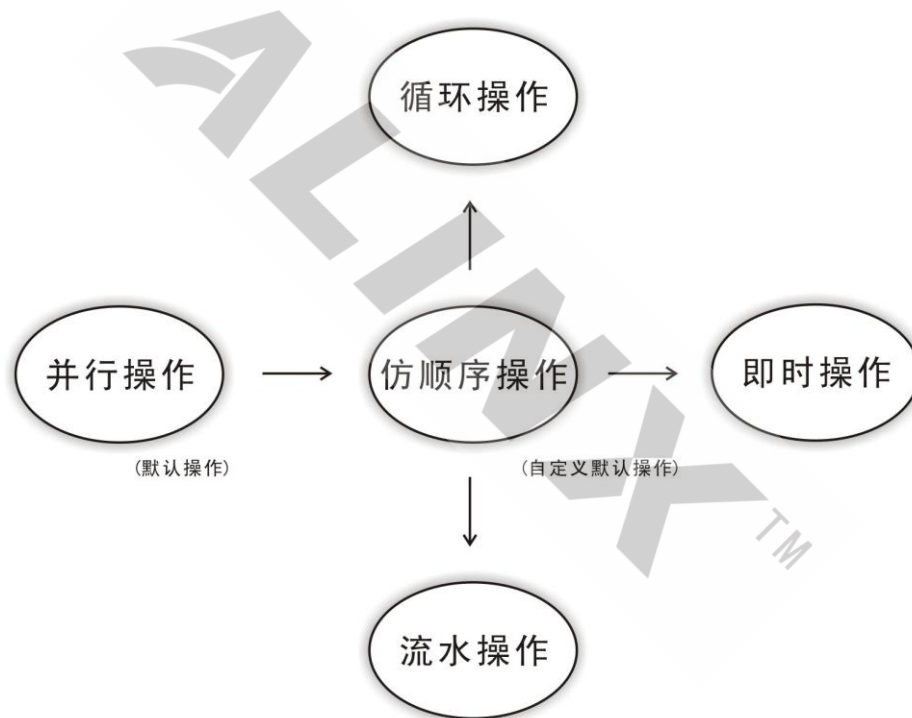


图 5.2.4 笔者所知的各种操作模式。

HDL 有并行操作的本质，并行操作自然而然也是 HDL 的默认操作，可是这种默认操作就像没有容器的水源，不好控制。结果笔者就自定义属于自己的默认操作，亦即仿顺序操作了 ... 并行操作如果要模仿顺序操作，根本上就要付出最基本数量的组合逻辑，建立可以支持顺序操作的结构。

据笔者所知，仿顺序操作可以转换成为其他操作模式 ... 如循环操作，它本质上还是顺序性的操作模式，不过它重复利用同一个资源，结果而言会比仿顺序操作少用一点资源；即时操作就不用说了，本身就是一个大组合逻辑，也是 Fmax 最大的敌人；至于流水操

作是最接近并行操作的操作模式，它不像仿顺序操作那样用而外的资源建立支持顺序的结构，结果而言它拥有的 F_{max} 比其它操作模式高。

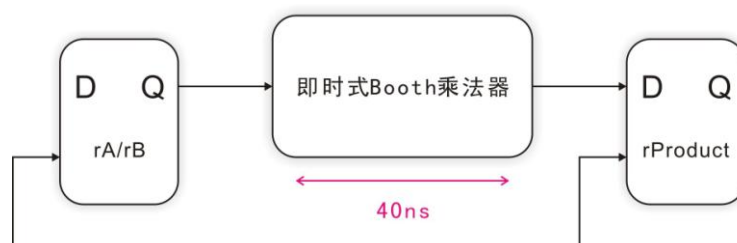


图 5.2.4 即时操作。

如果不苟其他小节，让笔者举个简单的例子，如图 5.2.4 所示，夹在两个寄存器之间是即时式 Booth 乘法器。换言之两个寄存器之前存有一个 40ns 延迟压力的组合逻辑，不过过大的延迟压力会降低 F_{max} ，因此我们必须更改 Booth 乘法器的操作模式，其中流水操作是最为理想。

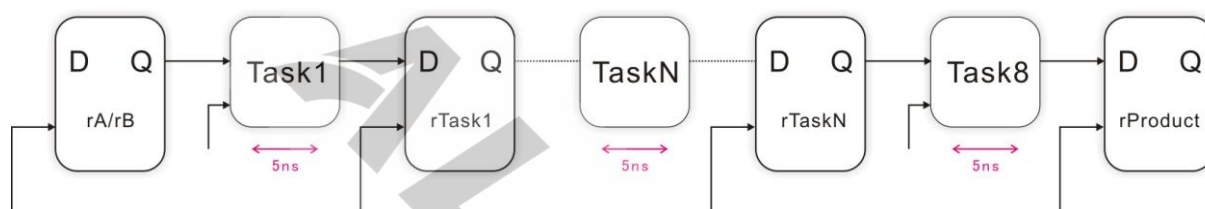


图 5.2.5 流水操作。

同样继续不苟其他小节的话 ... 笔者将即时式 Booth 乘法器随便分尸成 8 个部分，因此 40ns 的延迟压力就会被品均化为 5ns，延迟压力的降低随之也会提升 F_{max} 。上述只是粗略的例子而已，虽说流水操作在 F_{max} 的表现上拥有最好的成绩，不过流水操作不是什么场合也适合，如“**步骤性强的操作**”它就显得英雄无用武之地了。

HDL 除了操作模式以外，还有 HDL 的使用习惯 ... 如：嵌套 If 的层次又或者 $\leq$ 和 $=$ 的使用，让笔者举个简单的例子。

```
if( C1 == 1 ) //嵌套 if
    if( C1 == 2 )
        if( C1 == 3 )

begin rA = 1 + Z; rB <= rA; end // 应用即时事件
```

嵌套 if 按原理来说会建立更深的逻辑层次 ... 虽然这样的做法没有错，不过从代码的解读来讲是非常不推荐。应用即时事件是笔者的坏习惯，好处上可以偷吃时钟，坏处上增加该节点的延迟压力。总结来说 ... HDL 如何优化 F_{max} 归根就底还是需要根据当时的情况而定，所以是没有绝对的规则。

好了，有关 F_{max} 的故事就讲到这里吧 ... 同学们好好记住， F_{max} 不过是一个用处不大的评估值而已，虽说 F_{max} 越高越好，不过不是每个设计都需要那么高的 F_{max} 。此

外 $F_{\max}$ 也是性能限制的另一种评估值。



5.3 外部模型①

如果笔者说：“大部分的约束命令都是为外部延迟因数而存在 ...”，同学们又相信吗？事实上，这句话一点也不夸张 ... 在第三章中笔者曾解释说，外部和内部之间的区别就是 TimeQuest 知道与不知道，自动与手动。直接点说，内部各种延迟因数就好比 TimeQuest 身上的一块肉，它当然知道这块肉是什么样子，既然它知道这块肉是什么样子，TimeQuest 自然会晓得这块肉的相关信息。这就是所谓的 知道和自动。

外部延迟因数就好比别人身上的一块肉，TimeQuest 自然而然不知道这块肉的具体形状，结果那个人就无比告诉 TimeQuest 自己身上的那块肉是在哪里，是什么形状等相关信息。者就是所谓的不知道与手动。因此如此，TimeQuest 有大部分的约束命令就是用来告诉 TimeQuest 相关的“外部信息”。

在此之前，先让我们回忆一下之前学过的内容 ... TimeQuest 事实上只会在模型里打滚的猪头，亦即只要它离开模型，它就不晓得如何打滚了，这个模型就是我们熟悉的 TimeQuest 模型。所以说，TimeQuest 能不能分析外边的时序状况之前，TimeQuest 模型能不能山寨？如何山寨？才是头号任务。

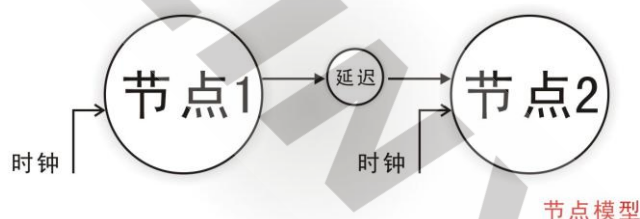


图 5.3.1 节点概念的 TimeQuest 模型

图 5.3.1 是最没有个性的 TimeQuest 模型 ... 换句话说，节点模型是概念上的 TimeQuest 模型，不过除了时钟固定以外，节点 1，节点 2 和延迟都会随着山寨的目标不同也跟着改变。

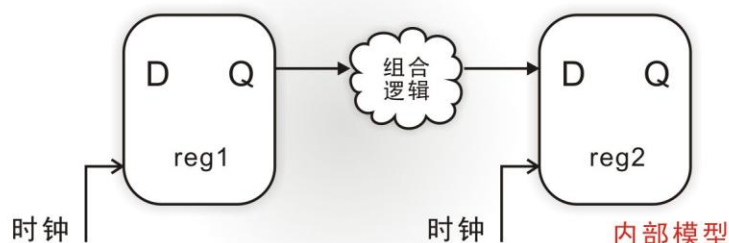


图 5.3.2 内部的 TimeQuest 模型。

图 5.3.2 是大伙再熟悉不过的内部 TimeQuest 模型，同时间也是默认下的 TimeQuest 模

型，亦即除了时钟信号不变以外，就是两个寄存器夹着一个组合逻辑。根据 FPGA 的基本资源而言，除了寄存器以外的一切，如乘法器，PLL 等都可以看成组合逻辑。

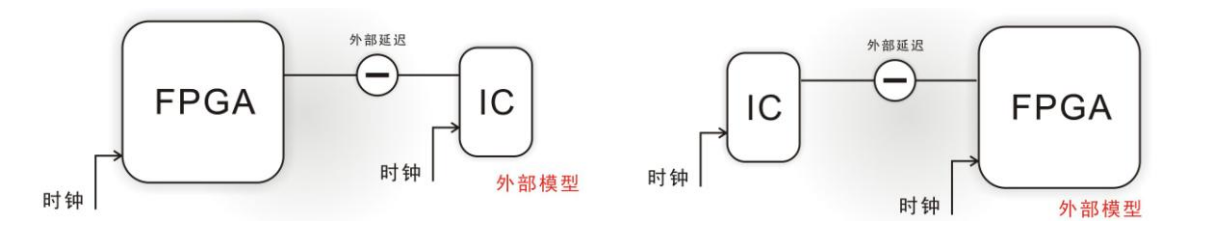


图 5.3.3 外部的 TimeQuest 模型。

图 5.3.3 是外部的 TimeQuest 模型（简称外部模型）也就是接下来我们要学习的内容。事实上外部模型是最麻烦，也是上镜率最低的 TimeQuest 模型。它之所以麻烦是因为它有 fpga2ic 或者 ic2fpga 之分，fpga2ic 也是 fpga 输出致 ic，又说 ic 读入 fpga；换之 ic2fpga 是 ic 输出致 ic，又说 fpga 读入 ic。至于外部延迟有第三章所说的种种。

之所以说外部模型上镜率最低是因为外部模型的山寨成功率是非常低，除了特定的 ic 芯片如 sdram 或者高速 dac/adc 等以外，外部模型几乎不能成立。话虽这么说，不过大部分约束命令却因它而生，可见它在学习中有多重的分量。

丑话说在前头，在后来的实验里，笔者也只能用 sdram 作为学习对象，那些不熟悉的同学应该好好补补一下与 sdram 有关的知识 ... 老王卖瓜一定会自卖自夸，笔者建议《整合篇》的第三章，嘻嘻。那么，让我们开始吧。

笔者在前面说过，外部模型和内部模型之间的区分就在于知道与不知道，自动与手动。至于是什么样的“不知道”的外部因数，必须通过“手动”告诉 TimeQuest 呢？

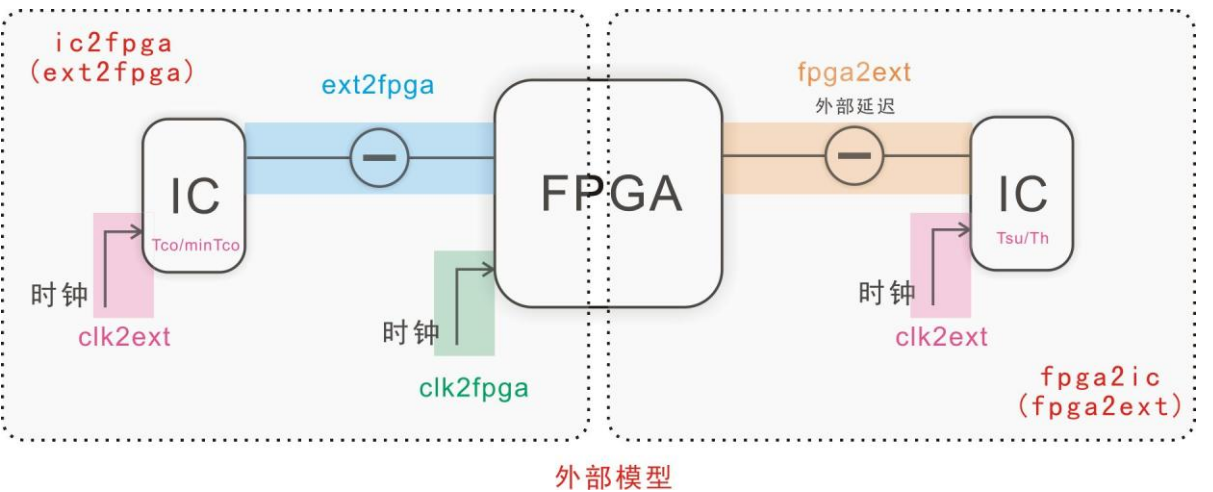


图 5.3.4 各种需要告诉 TimeQuest 的延迟因数。

基本上要告诉 TimeQuest 的外部延迟因数不算多，在此有可以划分 2 个情况：

fpga2ic :

fpga2ext 是 fpga 致 ic 信号的走线延迟；
clk2fpga 是时钟信号致 fpga 的走线延迟；
clk2ext 是时钟信号致 fpga 的走线延迟；
Tsu/Th 外部器件的寄存器特性；
(Tsu 信号建立时间；Th 信号保持时间)

ic2fpga :

ext2fpga 是 ic 致 fpga 信号的走线延迟；
clk2fpga 是时钟信号致 fpga 的走线延迟；
clk2ext 是时钟信号致 fpga 的走线延迟；
Tco/minTco 外部器件的寄存器特性；
(Tco 信号输出时间；minTco 最小信号输出时间)

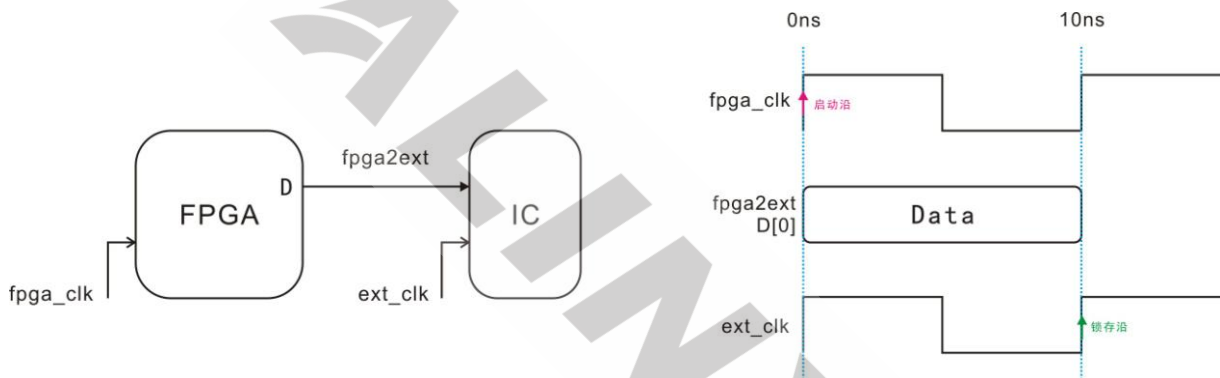


图 5.3.5 (左) fpga2ic 简单例子 (右) 理想时序图。

假设有一个简单的 fpga2ic 例子，如 5.3.5 的左图所示。fpga 向 ic 发送一个时钟周期的 D[0]，如 5.3.5 的右图所示，此时时序图是理想状态。

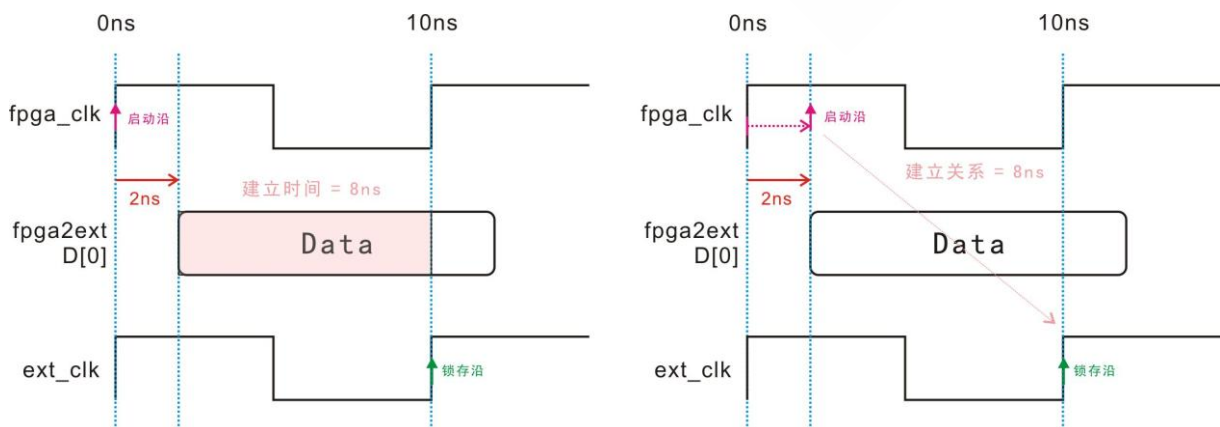


图 5.3.5 建立时间与建立关系的等价图。

图 5.3.5 左边是信号 D[0]有 2ns 的延迟，随之导致 Data 向右移动 2ns，最后 Data 的建立时间剩下 8ns。图 5.3.5 右边是等价图，这个等价图基本上是无视 Data。换句话说，2ns 的延迟，让启动沿向右移动 2ns，结果使得建立关系成为 8ns。图 5.3.5 有一个重点，亦即左图是数据与时钟相互作用，取得数据的建立时间；相反，右图里的时钟和数据相互作用不大，而是用启动沿和锁存沿取建立关系。

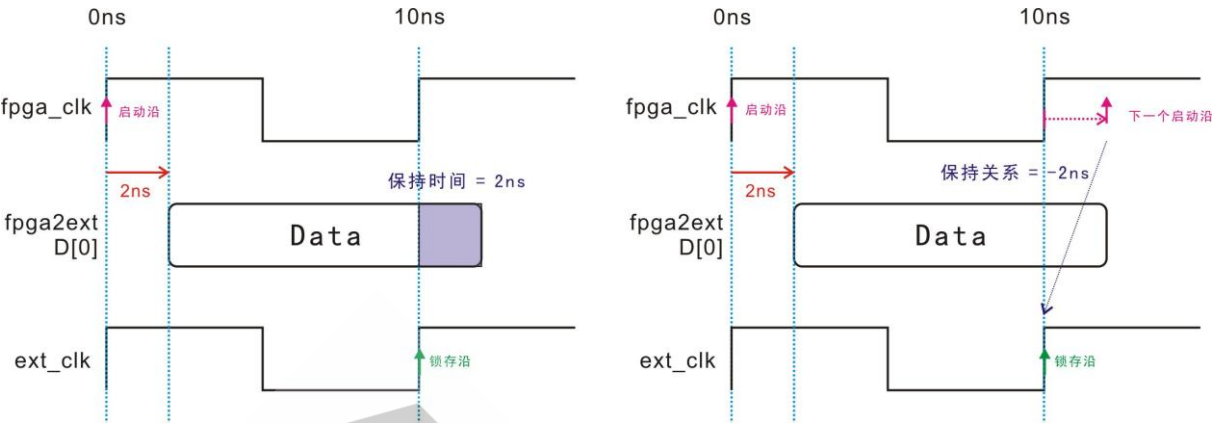


图 5.3.6 保持时间与保持关系的等价图。

如果建立时间和建立关系有等价图，同样保持时间与保持关系也有等价图，如 5.3.6 所示。左图是 D[0] 受 2ns 延迟，接着数据向右推向 2ns，最后取得 2ns 的保持时间。换之，右图是建立时间的等价图，亦即 2ns 延迟影响下一个启动沿向右移动 2ns，结果导致建立关系取得-2ns。

建立时间或者保持时间是分析一般时序图时用来评价数据的有效性，换之建立关系和保持关系是 TimeQuest 应用在模型或者公式的参考值。不过不管怎么样，理想时序 ... 更贴切说是在理想状态下，建立时间或者建立关系等于时钟周期，保持时间或者保持关系相当于零值。在此，我们先理解这些，接着再继续往后看 ...

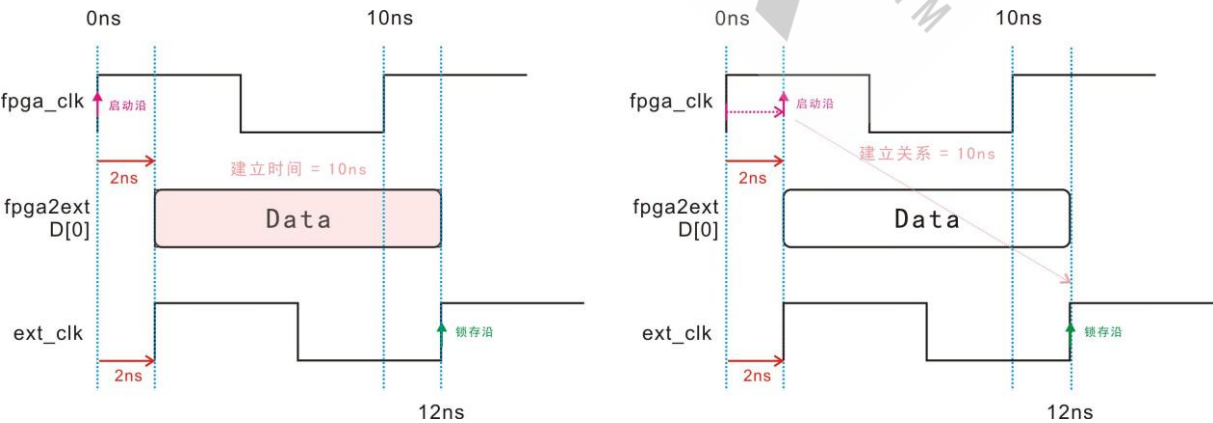


图 5.3.7 ext_clk 右移，建立时间与建立关系的等价图。

除了数据信号会受延迟影响以外，同样时钟信号也会受延迟影响，图 5.3.7 是 ext_clk 信号受 2ns 延迟的结果 ... 左图中，ext_clk 受延迟影响而向右移动 2ns，结果建立时间为 10ns；换之，右图是建立关系的等价图，亦即由于 ext_clk 受 2ns 延迟的影响，锁存

沿也从原本 10ns 的位置，移至 12ns，结果造成建立关系为 10ns。

这种看法是不伤大雅的碎事，不过对于 TimeQuest 而言，尤其是计算上 ... 我们知道 TimeQuest 是利用 Setup Relationship (建立关系) 与 Hold Relationship (保持关系) 作为计算的参考和补助 (或者简化)，单是数据信号跑来跑去 TimeQuest 已经够烦了，如果时钟再参一脚也跑来跑去的话，结果会造成 TimeQuest 会抓狂的,为此 ...

=====
师弟：我说，师兄呀 ... TimeQuest 在计算的时候，有没有利用“手段”，**不让时钟跑来跑去，而只让数据跑来跑去**？

师兄：什么跑来跑去？你的意思是不是说“**让时钟信号的延迟变化影响在数据身上**”，亦即 TimeQuest 计算时，**时钟信号位置基本不变，不过时钟信号受延迟影响的结果直接叠加在数据信号身上**？

师弟：真不愧是我的偶像，那么快就悟出本意 ...

师兄：少拍马屁了 ... 不过，TimeQuest 确实有用“手段”！

师弟：手段？什么样的手段？我很好奇！

(师弟的眼睛睁得大大的 ...)

师兄：在 Fmax 的章节中，有没有了解过 Clock Skew？

师弟：有是有 ... clock skew 就是目的寄存器 clk 相减源寄存器 clk 的时钟差，对否？

师兄：正解！

师弟：不过，师兄啊 ... 我虽然知道 clock skew 的定义而已，但不晓得实际用途却。

师兄：fmax 与 clock skew 放在一起谈的确意义不大，毕竟影响 fmax 的凶手还是组合逻辑的延迟压力。

师弟：那么，clock skew 的实际用途是甚麽？

师兄：嘛，别急师弟，让我们看 akuei2 如何解释 ...

(师兄话玩用手指着电脑屏幕 ...)

=====
clock skew 的定义正如师弟所言“的寄存器 clk 相减源寄存器 clk 的时钟差”，为方便笔者就称呼“时钟差”，然而 clock skew 的公式如下所示。clock skew 结果最终**会直接作用在数据的延迟上**。

clock skew = < destination reg clock delay > - < source reg clock delay >
时钟差 = <源寄存器时钟源延迟> - < 目的寄存器时钟源延迟 >

对 fpga2ic 的外模型而言，fpga_clk 是源寄存器时钟然而 ext_clk 是目的寄存器时钟。从图 5.3.7 中我们可以知道 fpga_clk 的延迟是 0ns，而 ext_clk 的延迟是 2ns，clock skew 取值自然是 2ns。

clock skew = < destination reg clock delay > - < source reg clock delay >

```

clock skew    = <ext_clk delay> - <fpga_clk delay>
               = (2 - 0)ns
               = 2ns

```

```

data delay    = 2ns
data delay'   = <data delay> - <clock skew>
               = 2ns - 2ns
               = 0ns

```

data delay 数据延迟

data delay' 时钟差作用后的数据延迟

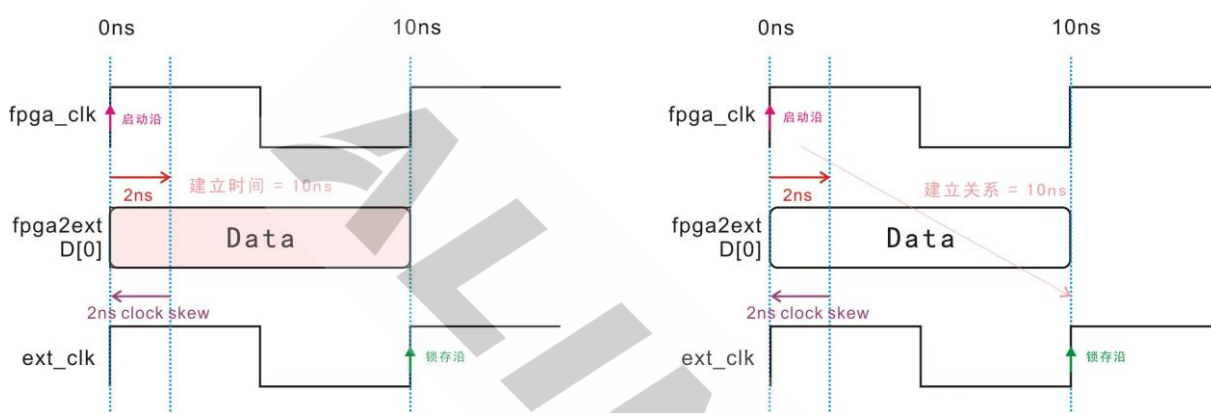


图 5.3.8 clock skew，建立时间与建立关系的等价图。

图 5.3.8 是图 5.3.7 受 clock skew 作用以后，建立时间与建立关系的等价图。从中我们可以知道 fpga_clk 还有 ext_clk 不再跑来跑去，换之原本向右移动 2ns 的 D[0]，受时钟差 2ns 作用以后的位置是 0ns。结果造成建立时间为 10ns，至于右图的启动沿也受到时钟差的作用，从原本的 2ns 返回 0ns，最终建立关系取得 10ns。

图 5.3.8 的建立时间与建立关系结果，基本上和图 5.3.7 的建立时间与建立关系一模一样。这下 clock skew 真的神了 ... “欸，笔者！clock skew 真的那么神吗？还是你老子在故弄玄虚？”爷爷我是不骗人的，不信吗？好，让我们再实验看看 ..

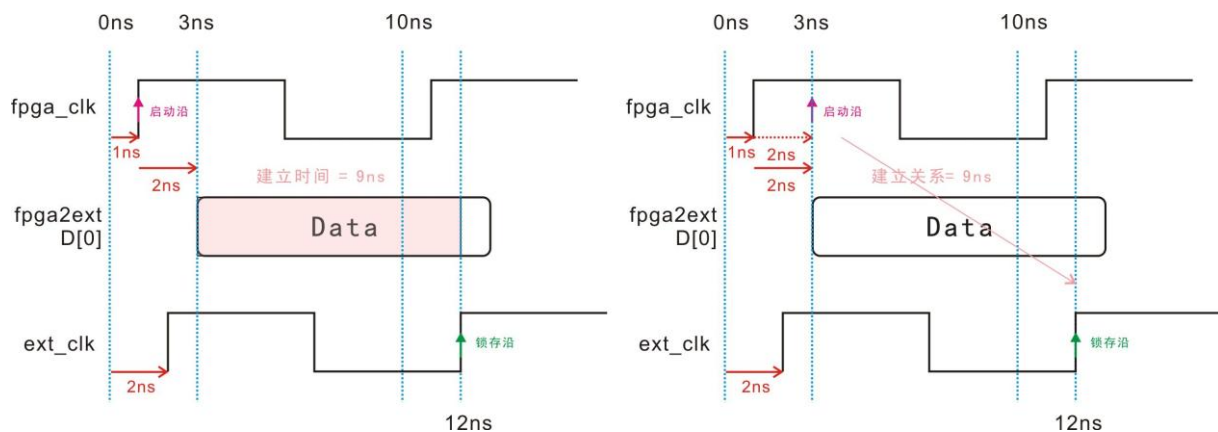


图 5.3.9 fpga_clk 与 ext_clk 右移，建立时间与建立关系的等价图。

图 5.3.9 左图中的 fpga_clk 延迟 1ns；ext_clk 延迟 2ns；D[0]的延迟还是之间的 2ns。结果而言，Data 的建立时间是 9ns。然而右图中，启动沿不仅受 D[0]的 2ns 延迟，也受 fpga_clk 的 1ns 延迟影响，结果启动沿从原本的 0ns 变成 3ns。不过，锁存沿本身也受 ext_clk 的 2ns 延迟影响，最终取得的建立关系是 9ns。然后让我们来计算一下 clock skew 与 data delay'

```

clock skew    = < destination reg clock delay > - < source reg clock delay >
clock skew    = < ext_clk delay > - < fpga_clk delay >
               = (2 - 1)ns
               = 1ns

data delay    = 2ns
data delay'   = < data delay > - < clock skew >
               = 2ns - 1ns
               = 1ns
  
```

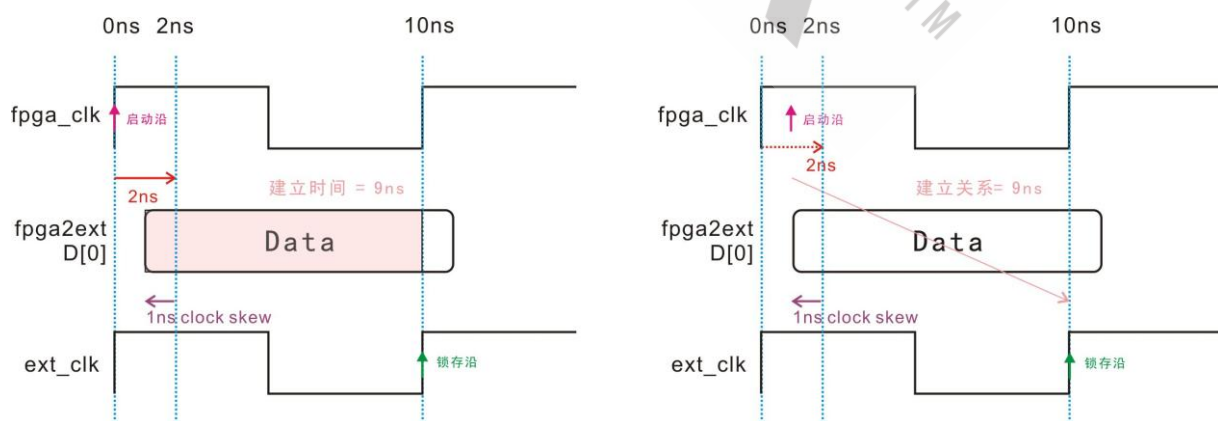


图 5.3.10 clock skew，建立时间与建立关系的等价图。

图 5.3.10 是图 5.3.9 受时钟差作用下的结果，左图中的 Data 从原本的 2ns 延迟，由于受到时钟差影响，结果延迟变成 1ns，最终建立时间取得 9ns。换之，右图中的启动沿原本位置是 2ns，不过受到时钟差的影响，结果退回 1ns，建立关系从而得到 9ns。好了，让

我们在一次比较图 5.3.10 和图 5.3.9 的建立时间与建立关系，答案是不是一样呢？这回信服了吧，哇咔咔！哇咔咔！

同学们现在还不能放心，目前我们只是了解到 clock skew 如何影响数据的延迟，从而使得图 5.3.10 是图 5.3.9 的建立时间和建立关系取得同样结果 ... 如果换个角度来想 clock skew 作用现象也是不是会发生在保持时间与保持关系的身上呢？本当に興味深い。

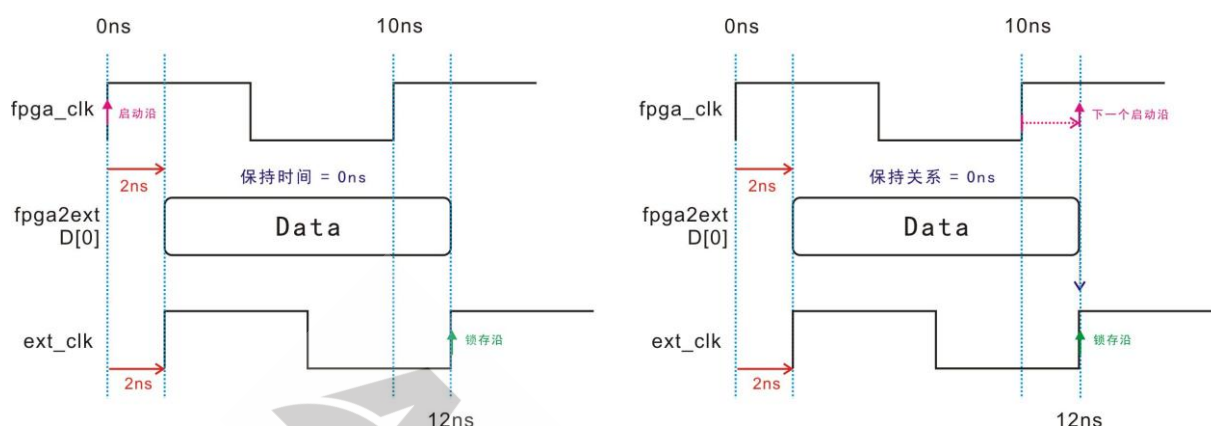


图 5.3.11 ext_clk 右移，保持时间与保持关系的等价图。

图 5.3.11 左图是 ext_clk 同 D[0]延迟 2ns，结果造成保持时间为 0ns。然而右图中的下一个时间沿虽然受到 2ns 的延迟，从原本的 10ns 变成 12ns，不过由于 ext_clk 也有 2ns 的延迟，保持关系最后取得 0ns。接下来，让我们计算一下时间差和受作用以后的 data delay:

```
clock skew    = < destination reg clock delay > - < source reg clock delay >
clock skew    = < ext_clk delay > - < fpga_clk delay >
               = (2 - 0)ns
               = 2ns

data delay    = 2ns
data delay'   = < data delay > - < clock skew >
               = 2ns - 2ns
               = 0ns
```

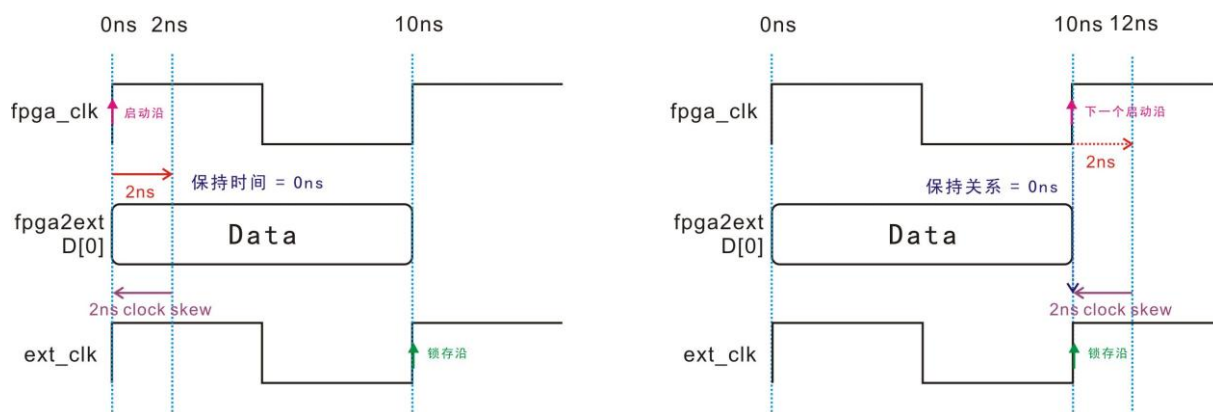



图 5.3.12 clock skew，保持时间与保持关系的等价图。

图 5.3.12 与 图 5.3.11 的保持时间和保持关系对于之下，同样的答案，同样的结果，不同的只是发生经过。图 5.3.12 的左图，数据原本受 2ns 的延迟以后应该在 2ns 的位置，可是被 2ns 的时钟差绊了一道以后变回到 0ns 的位置，从而取得 0ns 的保持时间。换之，右图中的下一个启动沿原本受 2ns 的延迟以后，位置应该是 12ns 才是，可是被 2ns 的时钟差便跌回 10ns 的位置，结果页造成 0ns 的保持关系。

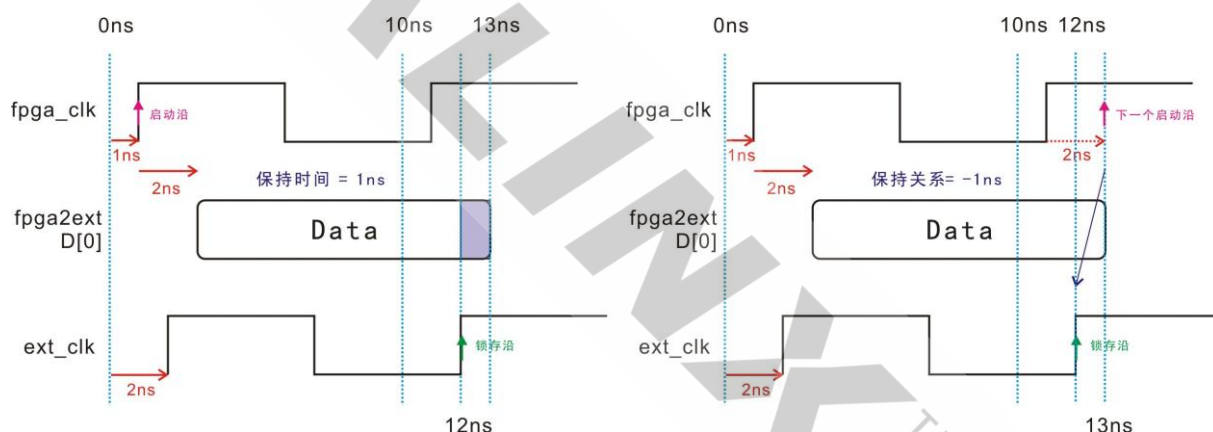


图 5.3.13 fpga_clk 和 ext_clk 右移，保持时间与保持关系的等价图。

图 5.3.13 左图中由于 fpga_clk 受延迟 1ns，ext_clk 受延迟 2ns，结果造成保持时间为 1ns。换之，由图中的下一个启动沿由于受到 fpga_clk 的延迟，再加上 D[0] 的延迟，最终停留在 13ns 的位置，不过锁存沿也受到 ext_clk 延迟 2ns 的影响，从而保持关系成为 -1ns。还是先计算时钟差和受作用以后的 data_delay' ...

```

clock skew    = < destination reg clock delay > - < source reg clock delay >
clock skew    = < ext_clk delay > - < fpga_clk delay >
               = (2 - 1)ns
               = 1ns

data delay    = 2ns
data delay'   = < data delay > - < clock skew >
               = 2ns - 1ns

```

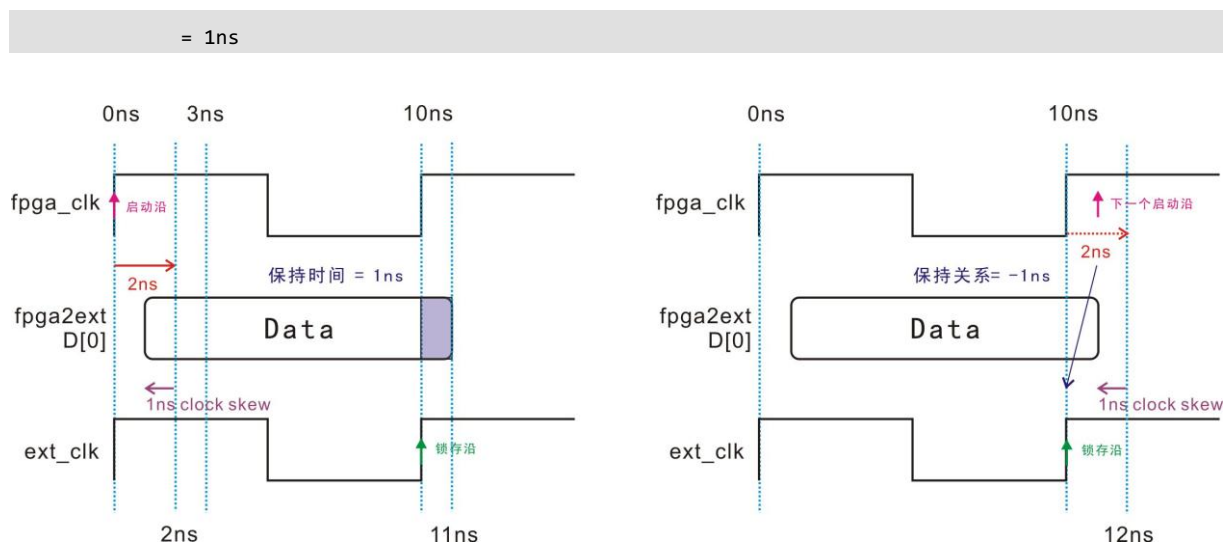


图 5.3.14 clock skew，保持时间与保持关系的等价图。

图 5.3.14 与图 5.3.13 相较之下，同样的保持时间，同样的保持关系，不同的只是发生过 ... 左图中的 D[0] 受 2ns 的延迟，原先位置应该在 2ns 才对，不过由于受到时间差 1ns 的作用，结果退回到 1ns 的地方，最终取得 1ns 的保持时间。换之，右图的下一个启动沿，原本受 2ns 的延迟，位置应该在 12ns 才是，不过它不小心撞到 1ns 的时间差，就这样跌在 11ns 的位置，最后也不小心取得 -1ns 的保持关系。

嘻嘻，经过那么多实验 ... 同学们也该相信笔者不在故弄玄虚吧！？笔者想必许多同学一定很好奇，这章节我们不是应该在谈有关外部模型和外部延迟有关的种种吗？为什么无缘无故又扯上那么多稀奇古怪的内容？嘛 ... 别着急，这些都是最基本知识，习得它以后同学们就可以飞天遁地了。

5.4 外部模型②

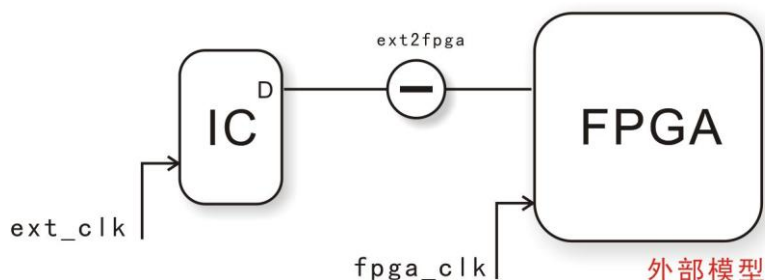


图 5.4.1 ic2fpga (ext2fpga) 的外部模型。

在章节 5.3 的时候，我们已经谈过 fpga2ic 的外部模型，还有建立时间，建立关系，保持时间，保持关系，或者 clock skew 作用等内容。然而这章节将要讨论的是 ic2fpga 外部模型，至于内容基本上与 fpga2ic 大同小异而已，fpga2ic 与 ic2fpga 之间的差别就在 fpga 与 ic 的角色扮演换了位置而已：fpga2ic 是 fpga 输出数据 ic 读；ic2fpga 是 ic 输出数据 fpga 读。

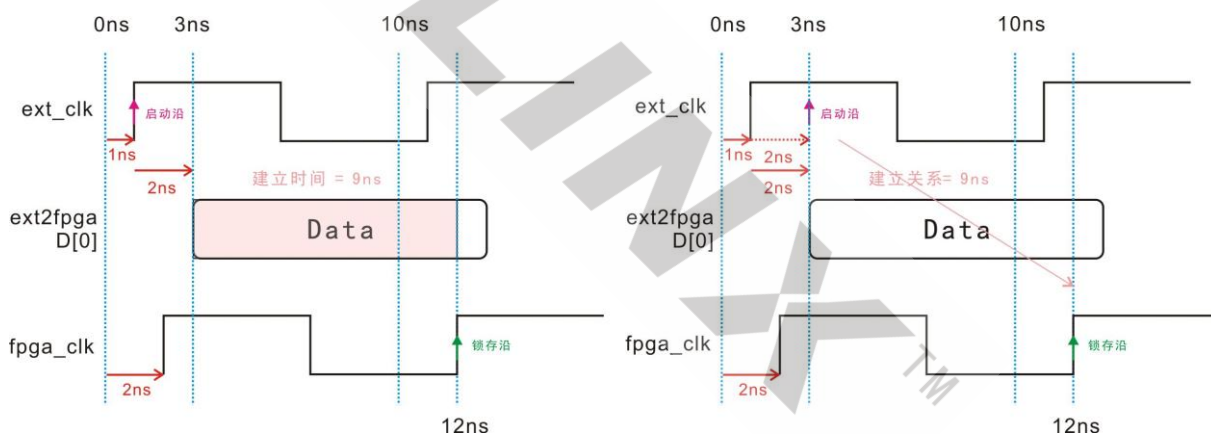


图 5.4.2 ext_clk 和 fpga_clk 右移，建立时间与建立关系的等价图。

图 5.4.2 是不是眼熟？那当然，笔者只是替图 5.3.9 换张皮而已，不过习惯上还是随便解释一下。左图中的 ext_clk 受 1ns 延迟，而 fpga_clk 受 2ns 延迟，再加上 D[0] 的 2ns 延迟，最终导致建立时间为 9ns。换之右图中的启动沿，受 D[0] 延迟 2ns 再加上 ext_clk 延迟 1ns 的影响，结果位于 3ns ... 不过 fpga_clk 也受 2ns 的延迟，最后建立关系取得 9ns 的成绩。

```
clock skew    = < destination reg clock delay > - < source reg clock delay >
clock skew    = < fpga_clk delay> - < ext_clk delay>
              = (2 - 1)ns
              = 1ns
```

```

data delay      = 2ns
data delay'     = <data delay> - <clock skew>
                = 2ns - 1ns
                = 1ns

```

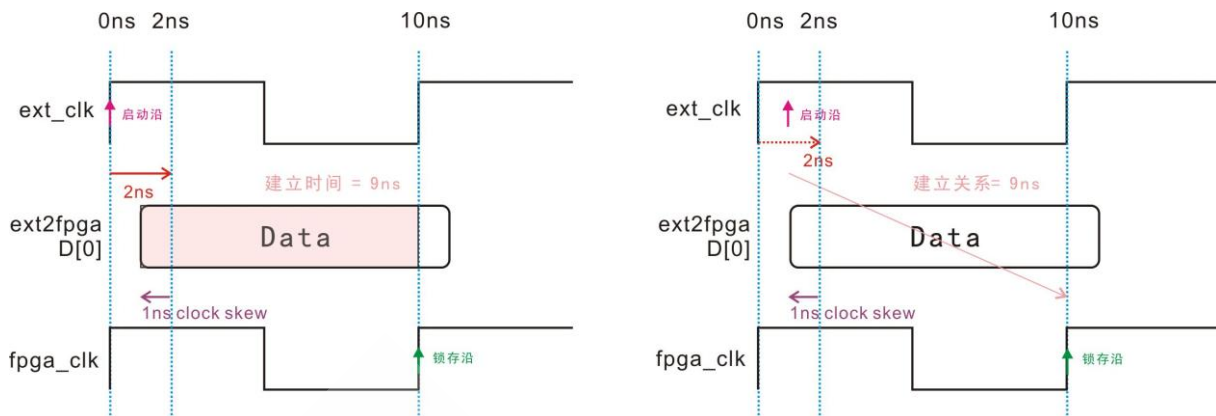


图 5.4.3 clock skew, 建立时间与建立关系的等价图。

图 5.4.3 是图 5.4.2 受 clock skew 影响的时序图。ext_clk 与 fpga_clk 时钟差是 1ns，这个时钟左右了原先位置应该在 2ns 的 D[0]，不小心倒回 1ns 的位置，最后取得 9ns 的建立时间。换之，右图中的启动沿原先受 D[0] 的 2ns 延迟，位置应该落在 2ns 才是，不过被 1ns 的时间差绊了一脚，然后跌在 1ns 的位置，因此取得 9ns 的建立关系。

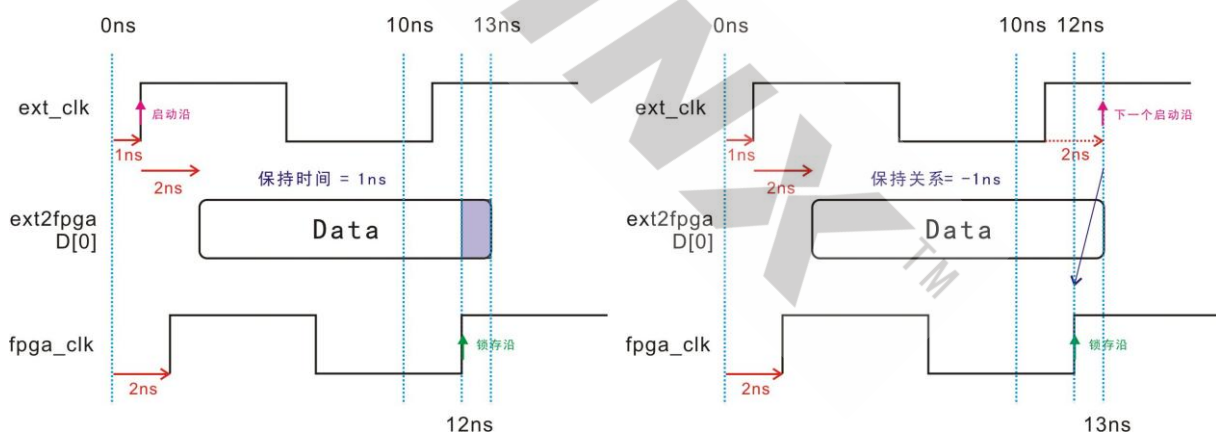


图 5.4.4 ext_clk 与 fpga_clk 右移，保持时间与保持关系的等价图。

不要怀疑,图 5.4.4 同是图 5.3.13 的换皮结果 ... 左图中,由于 D[0] 受到自身 2ns 的延迟,还有 ext_clk 延迟的影响,导致起跑线在 3ns,而 fpga_clk 也受 2ns 的延迟,最后导致保持时间为 1ns;换之,右图中的下一个启动沿由于受到 1ns 的 ext_clk 延迟,再加上 D[0] 本身的 2ns 延迟,结果下一个启动沿的位置从 13ns 开始,不过由于 fpga_clk 也受到 2ns 延迟的影响,最终保持关系取得-1ns。

```

clock skew      = < destination reg clock delay > - < source reg clock delay >
clock skew      = < fpga_clk delay> - <ext_clk delay>
                = (2 - 1)ns

```

```

= 1ns

data delay    = 2ns
data delay'   = <data delay> - <clock skew>
              = 2ns - 1ns
              = 1ns

```

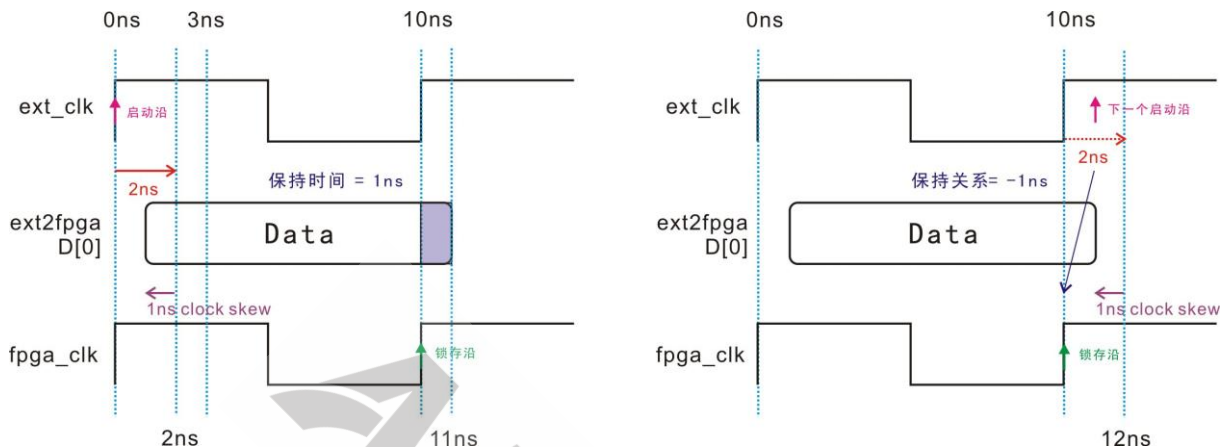


图 5.4.5 clock skew，保持时间与保持关系的等价图。

图 5.4.5 是图 5.4.4 受 clock skew 作用以后的时序图。左图中的 D[0]原本应该从 2ns 的位置起跑，可是不小心踩到 1ns 的时间差，然后跌倒在 1ns 的位置，从而造就 1ns 的保持时间。换之，由图中的下一个启动沿，原受到 2ns 的延迟而在 12ns 的位置，结果还是一样衰，不小心踩到 1ns 的时间差，然后跌在 11ns 的位置，最后导致保持关系为-1ns。

到目前为止，我们只是在了解单位宽信号（数据）的传输而已 ... 更多时候，信号位宽都是 N*字节为单位，因此我们有必要好好学习一下 TimeQuest 是如何“应付”这些多位宽的数据呢？

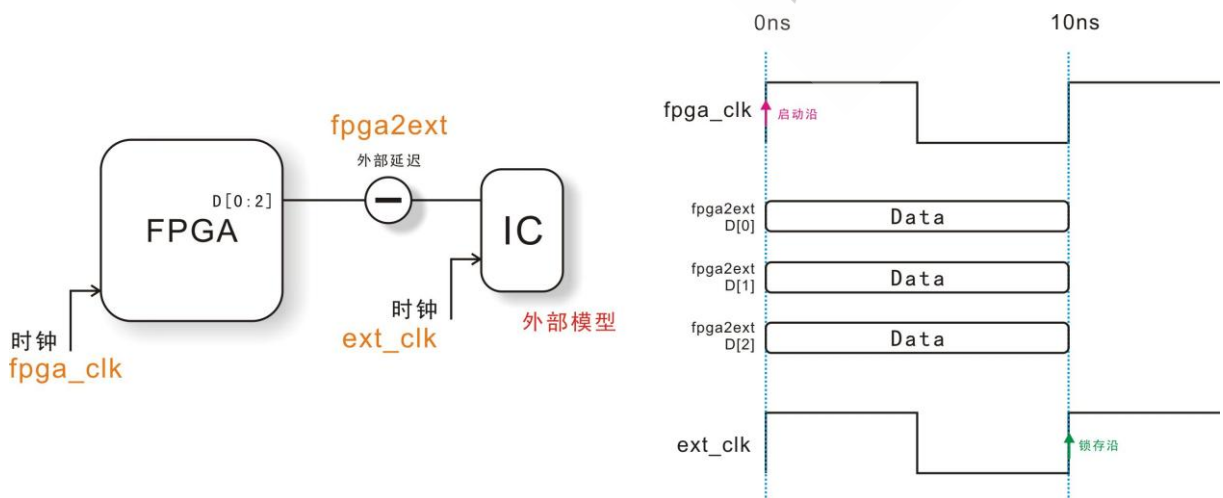


图 5.4.6 fpga2ic 外部模型的多位宽数据传输。

图 5.4.6 左图相较上一章节的 fpga2ic 的外部模型不是 D[0]而是 D[0:2] ,也就是多位宽数据传输 ,然而右图是多位宽传输的理想时序图 (fpga 向 ic 发送 3 位 10ns 周期的数据/信号)。

TimeQuest 认识单位宽传输与多位宽传输的方法基本上是大同小异 ,换句话说 ... 某个单位宽重复 n 次后就会成为多位宽。笔者知道自己在说废话 ,不过也就是这句废话让笔者领悟到 ,TimeQuest 不会那么努力一个一个去识别单位宽 ,而且我们也不可能逐个逐个将单位宽告诉 TimeQuest ... 不然我们和 TimeQuest 都会抓狂。因此 TimeQuest 在应付多位宽的时候偷偷用 “手段” ? 这个 “手段” 又是什么呢 ?

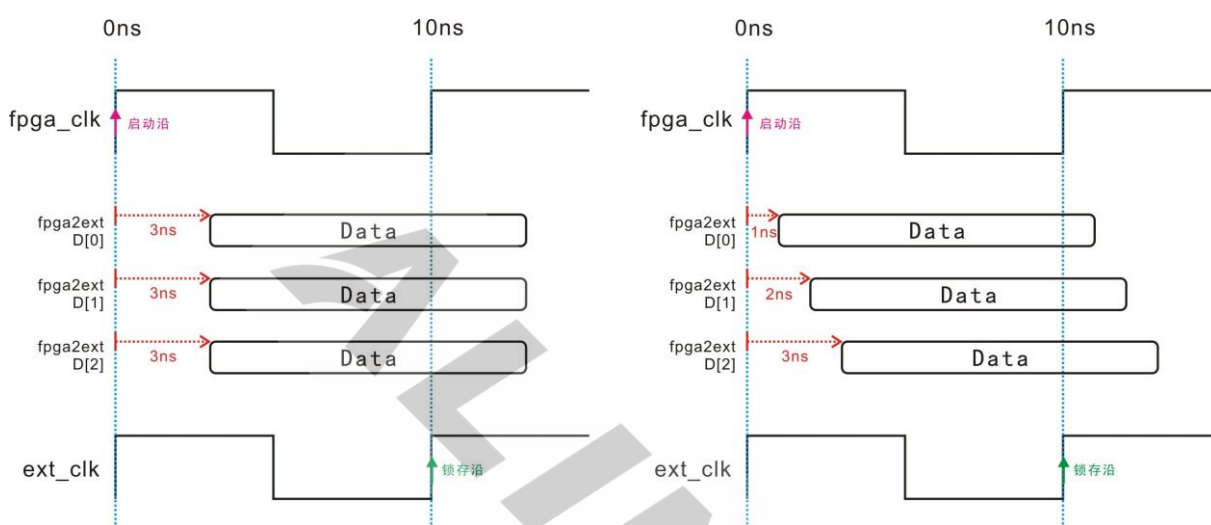


图 5.4.7 均为与不均匀的延迟压力。

假设 fpga2ext 的路上存有延迟 ,结果造成数据 D 向右移动 ,这时候我们会遇见延迟压力均匀与不均匀的问题 ,延迟压力均匀如图 5.4.7 左图所示 ,换之延迟压力不均匀如图 5.4.7 右图所示。延迟压力如果均匀的话 ,不仅在审美上 ,绘制时序图上 ,还是 JDL 在时序产生上 ,都非常便利。相反的 ,如果延迟压力不均匀的话 ,在许多方面上会麻烦一点。

笔者在《整合篇》中曾经举例过 ,自定义的乘法器比起传统的乘法器更容易控制延迟压力均匀与否。当然所谓的延迟压力均匀不是图 5.4.7 的左图中出现一样 ... 那么完美的数值 ,实际上会是 2.8ns , 2.9ns , 3.1ns 等接近 3ns 的数值。事实上 ,延迟压力均不均匀只是完美主义的追求而已 ,对实际的影响不大 ,话虽如此 ... 不过可以的话笔者也会努力去均匀一点各个延迟压力 ,毕竟这样看会比较顺眼一点~笑。

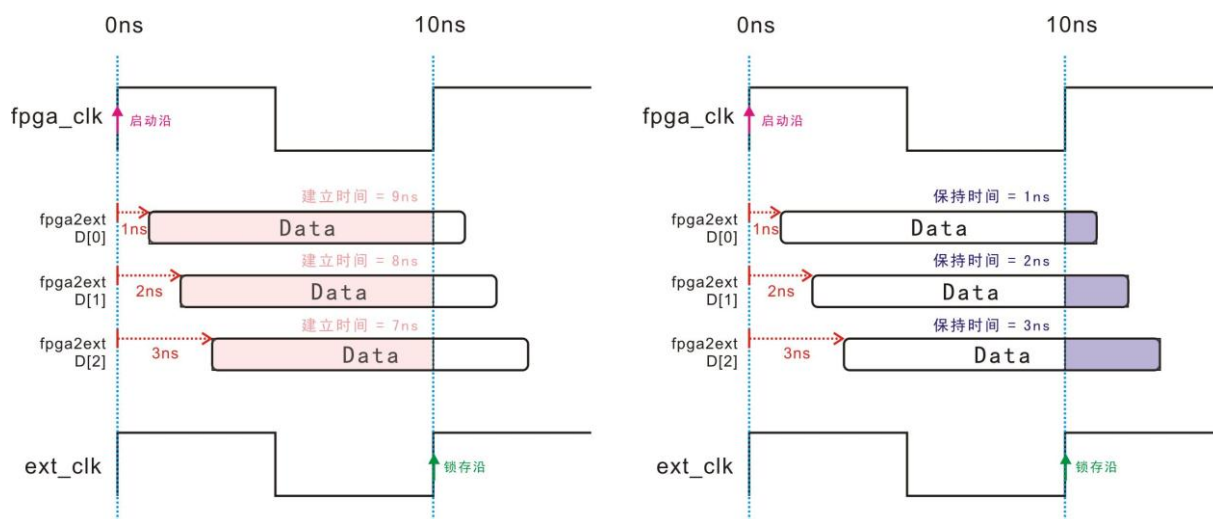


图 5.4.8 延迟压力不均匀与建立时间和保持时间的关系。

让我们切回主题 ... 延迟压力不均匀是时序的家常便饭,图 5.4.7 的右图告诉我们最大的延迟是 D[2] 的 3ns, 最小的延迟是 D[1] 的 1ns。因此, 我们可以这样说道:

```
D[0..2] delay max = 3ns;
D[0..2] delay min = 1ns;
```

图 5.4.8 是延迟压力不均匀给建立时间和保持时间造成的影响。让我们来思考吧 ... 图 5.4.8 的左图中, 有 3 个信号 (数据), D[0]的建立时间为 9ns, D[1]的建立时间为 8ns, 还有 D[2]的建立时间为 7ns。“请问哪一个数据的建立时间最危险的?”, 在物理时序角度上, 亦即建立时间越少越危险, 我们当然将矛头指向 D[2]。

图 5.4.8 的右图中, 同样也是 3 个信号, D[0]的保持时间为 1ns, D[2]的保持时间为 2ns, D[3]的保持时间为 3ns。“请问哪一个数据的保持时间是最危险的?”, 在物理时序角度上思考, 亦即保持时间越少越危险, 结果最危险的保持时间自然是 D[0]。因此我们可以这样结论:在 D[0]~D[2]三个位宽中, delay max 影响建立时间(建立关系), delay min 影响保持时间(保持关系)。

“欸 ... 笔者怎么搞的? 我越来越搞糊涂了” 这个问题一定会由心而生吧? 呵呵 ... 笔者没有胡来, 笔者也没有胡搞, 同学们稍微回忆一下 “fmax 最差劲节点” 的故事。事实上, 上述的结论与 “最差劲节点” 都有相同的原理 ...

TimeQuest 在分析 D[0..2]建立时间的时候, 它只要注意 “最危险的建立时间” 即可, 按逻辑而言, 没有什么比 “最危险” 更危险。同样思路, TimeQuest 在分析 D[0..2]保持时间的时候, 它只要注意 “最危险的保持时间” 即可, 因为没有什么比 “最危险” 更危险。

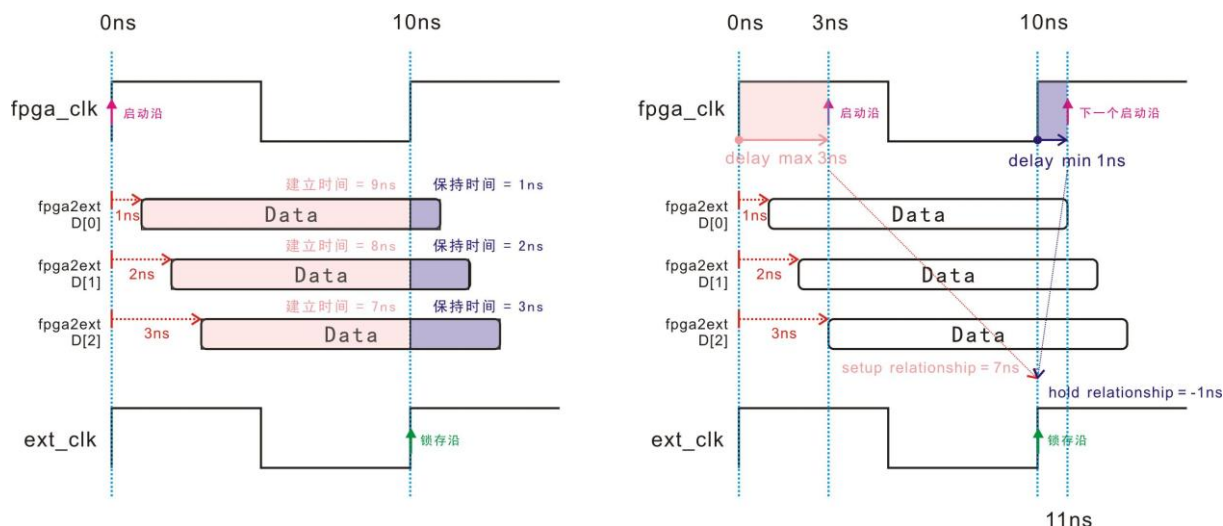


图 5.4.9 （左）建立时间与保持时间，（右）建立关系与保持关系，之间的等价图。

图 5.4.9 左边是建立时间与保持时间的时序图，换之右边是建立关系与保持关系的等价图。笔者曾讲过，**建立关系与保持关系的等价图事实上不怎么关心数据的死活，然而它们仅关心，启动沿，下一个启动沿，或者锁存沿的变动，还有建立关系和保持关系而已。**

从右图中我们可以了解到，3ns 延迟的 `delay max` 影响启动沿从原本 0ns 的位置，向右移至 3ns。换之，1ns 延迟的 `delay min` 影响下一个启动沿从原本的 10ns 位置，向右移至 11ns。最终导致建立关系为 7ns，保持关系为 -1ns ... 这个结果与 7ns 的建立时间，1ns 的保持时间不谋而合。

读到这里，笔者理解到许多同学可能一时之间无法消化这些莫名奇妙的东西 ... 说实话，笔者初次认识的时候也有同样的感觉，原本感觉莫名奇妙的东西经过多次研究以后，渐渐觉得“情有可原” ... 然后诚心接受了。从学习 Verilog 到至今再莫名奇妙的东西也遇过了，最终笔者领悟到一个道理，**要用那个世界的自然思考那个世界的现象，不要用这个世界的自然思考那个世界的现象。**

接下来让我们继续思考 clock skew 如何影响 `delay max` 与 `delay min` ？

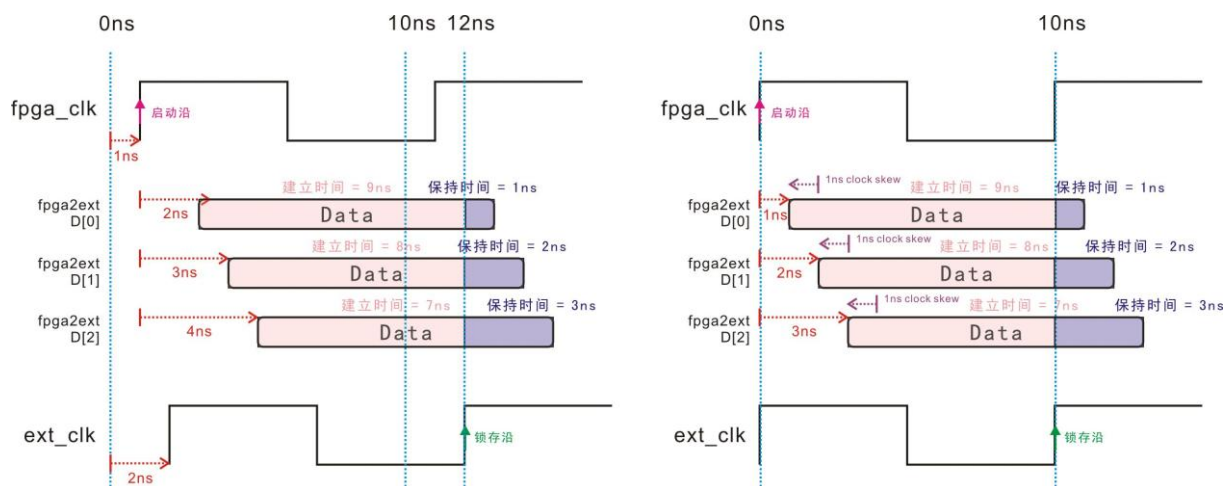


图 5.4.10 Clock skew 影响建立时间与保持时间的时序图。

在图 5.4.10 的左图中，fpga_clk 有 1ns 的延迟，ext_clk 有 2ns 的延迟，然而 D[0]有 2ns 的延迟，D[1]有 3ns 的延迟，D[2]有 4ns 的延迟。各种延迟与数据相互作用以后，D[0]取得 9ns 的建立时间，1ns 的保持时间；D[1]取得 8ns 的建立时间，2ns 的保持时间；D[2]取得 7ns 的建立时间，3ns 的保持时间。接下来，让我们先计算一下 clock skew ...

```
clock skew    = < destination reg clock delay > - < source reg clock delay >
clock skew    = <ext_clk delay> - < fpga_clk delay>
               = (2 - 1)ns
               = 1ns
```

clock skew 的结果为 1ns，结果 D[0..2]的延迟都被绊了一脚，通通向后移动 1ns，结果如图 5.4.10 的右图所示。虽然左图与右图拥有同样的建立时间与保持时间，不过它们的差别就在于左图的时钟跳来跳去，换之右图的时钟固步不动。

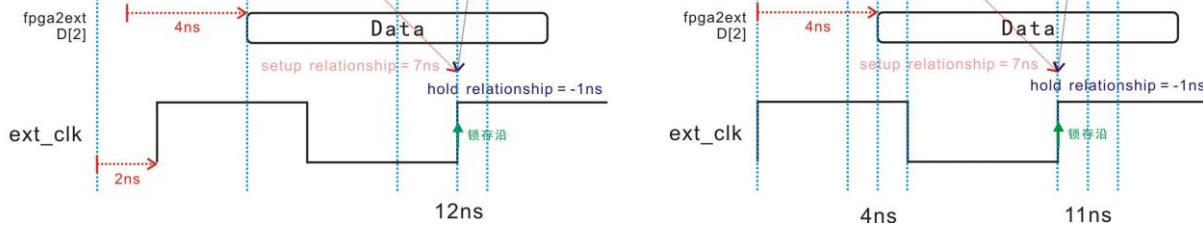


图 5.4.11 clock skew 影响建立关系与保持关系的时序图。

图 5.4.11 的左图是未曾受 clock skew 影响的建立关系与保持关系的时序图，其中 fpga_clk，ext_clk 与 D[0..2] 的延迟与上述一样。差别就在左图中，启动沿受 4ns 延迟的 delay_max 影响，再加上 fpga_clk 的 1ns 延迟，结果落在 5ns 的位置；然而下一个启动沿，受 2ns 延迟的 delay_min 影响，结果落在 13ns 的位置。ext_clk 有 2ns 的延迟，因此锁存沿的位置也在于 12ns。最后建立关系取得 7ns，保持关系取得 -1ns。

```
clock skew    = < destination reg clock delay > - < source reg clock delay >
clock skew    = < ext_clk delay > - < fpga_clk delay >
              = (2 - 1)ns
              = 1ns
```

clock skew 与上述一样是 1ns ... 右图中的 fpga_clk 与 ext_clk 固步不动在 0ns 位置，原本受 delay_max 影响的启动沿应该落在 4ns 的位置，可是它不小心撞到 1ns 的时间差，最后跌倒在 3ns 的地方；换之，原本受 delay_min 影响的的下一个启动沿，同样也不小心撞到 1ns 的时间差，从 12ns 的位置跌到 11ns 的位置。最终取得 7ns 的建立关系，还有 -1ns 的保持关系。左图与右图之间的差别就在于左图的 fpga_clk 与 ext_clk 都跳来跳去，换之右图的 fpga_clk 与 ext_clk 都乖乖呆在原地。

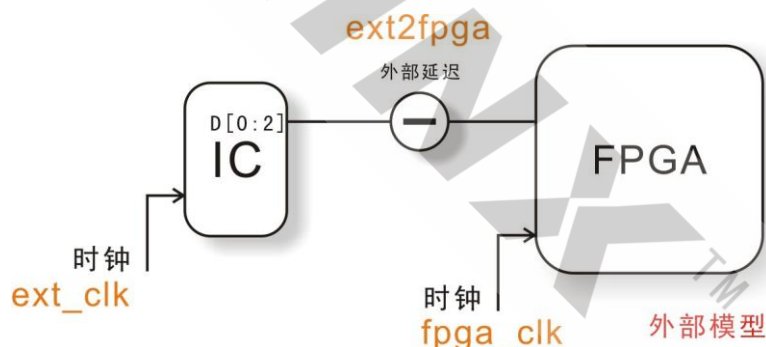


图 5.4.12 ic2fpga (ext2fpga) 多位宽的外部模型。

在前面我们解了有关 fpga2ic 与多位宽，这回合让我们来了解一下有关 ic2fpga 与多位宽的种种。图 5.4.12 是多位宽 ic2fpga 的外部模型，事实上它与多位宽 fpga2ic 的外部模型只是换个角色，换个位置而已，大体上都是换汤不换药。不过笔者还是惯例的讲一讲 ...

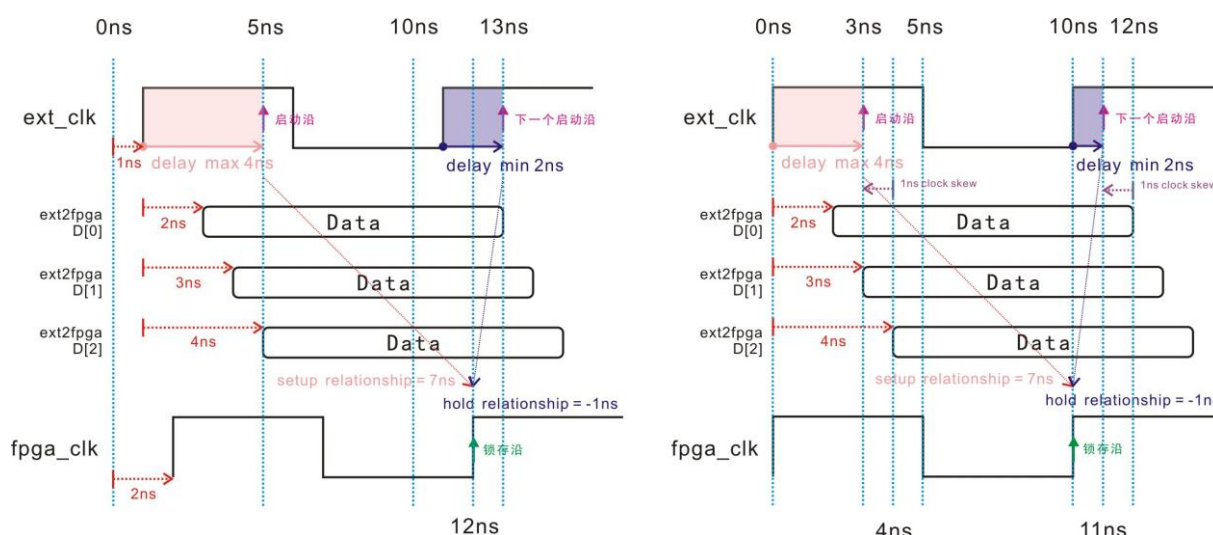


图 5.4.13 (ic2fpga) clock skew 影响建立关系与保持关系的时序图。

图 5.4.13 与图 5.4.12 基本上只是更换一下 D[0..2] 的名字还有 ext_clk 与 fpga_clk 的位置而已，其他都一模一样。在左图中，ext_clk 受 1ns 的延迟，fpga_clk 受 2ns 的延迟，然而 D[0..2] 接续有 2ns~4ns 的延迟。启动沿不仅受 ext_clk 延迟 1ns 的影响，它更受 delay max 的阻拦从而落在 5ns 的位置。至于下一个启动沿同样也受到 ext_clk 延迟 1ns 的影响，它接着又被 delay min 妨碍，结果落在 13ns 的位置。最终取得 7ns 的建立关系，还有 -1ns 的保持关系。

```
clock skew = < destination reg clock delay > - < source reg clock delay >
clock skew = < fpga_clk delay > - < ext_clk delay >
            = (2 - 1)ns
            = 1ns
```

clock skew 经过计算以后取值为 1ns。至于 5.4.13 的右图是启动沿与下一个启动沿受时钟差左右的时序图 ... 原本受 4ns delay max 延迟的启动沿，由于不小心被 1ns 的时钟差绊倒，从 4ns 的位置跌至 3ns 的位置。换之下一个启动沿原本受 2ns 的 delay max 延迟，同样也不小心撞到 1ns 的时钟差，结果从原本 12ns 的位置跌至 11ns 的位置。最后取得 7ns 的建立关系，还有 -1 的保持关系。

到目前位置，我们已经了解完相关的基础知识，接下来我们不仅会推导公式，还会学习有关的约束命令，亦即 set_input_delay 与 set_output_delay。同学们，笔者还是建议先歇一会吧，好让自己的脑袋冷静一下，就算战意高昂也不要勉强自己，任何时候都不要高估自己的能力。

5.5 推导外包资料的公式

想必任何人都不喜欢推导公式吧？说实话，笔者也是 ... 不过接下来我们要推导的不是数学公式，而是与外部模型有关的公式。这样做不是为了什么，而是为了让同学们进一步认识外部模型。

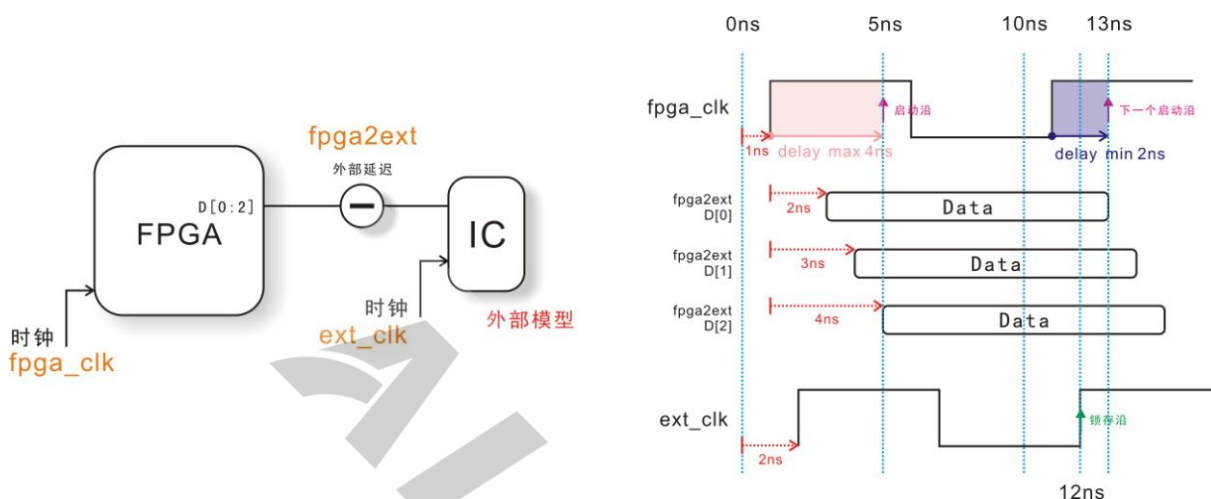


图 5.5.1 fpga2ic 的外部模型（左），有各种延迟的时序图。

图 5.5.1 的左图是 fpga2ic 的外部模型，然而右图是相关的时序图 ... 从右图中，我们可以取得以下信息：

```
fpga_clk delay 1ns
ext_clk delay 2ns

fpga2ext D[0] 2ns (delay min)
fpga2ext D[1] 3ns
fpga2ext D[2] 4ns (delay max)
```

信息取得以后，我们就要开始推导公式了 ... fpga2ic 的外部模型从另一个角度来看是 fpga 输出数据，ic 读取数据。然而公式的推导都以 fpga 为主要，因此 fpga2ic 的公式又称 output 公式，但是 output 公式又应该针对什么呢？很简单，output 公式主要是针对 5.5.1 右图中的启动沿，与下一个启动沿。

针对启动沿的 output 公式又名 output max，换之针对下一个启动沿的 output 公式又名 output min。

```
output max = <fpga2ext delay max>
output min = < fpga2ext delay min >
```

output max 与 min 的公式目前还是很单纯，因为我们还没有将 clock skew 考虑进去。

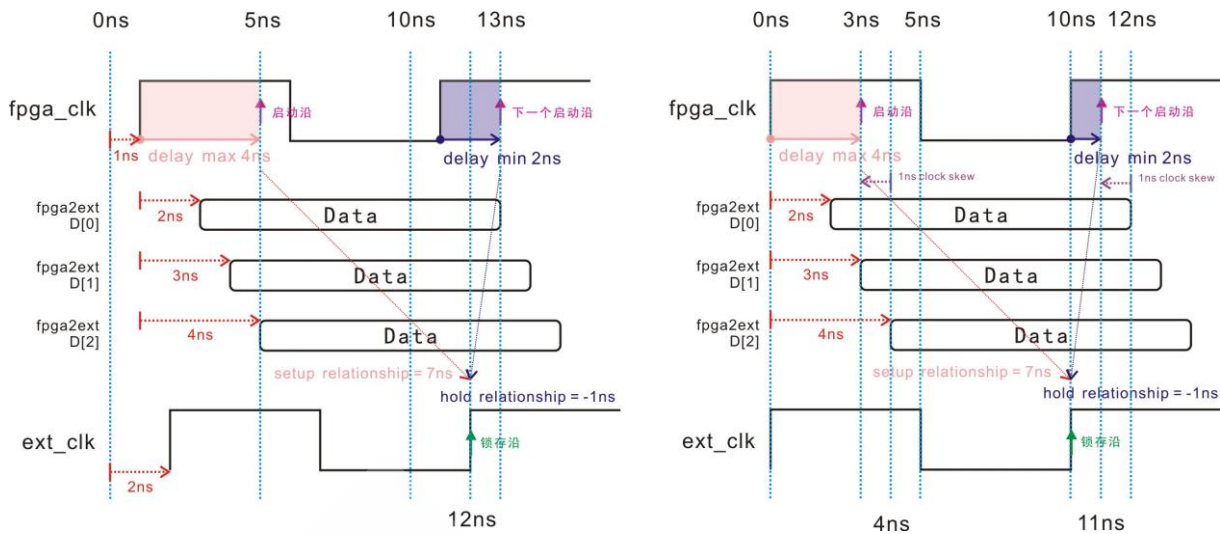


图 5.5.2 clock skew 与启动沿和下一个启动沿的关系。

图 5.5.2 的左图是还没有将 clock skew 考虑进去的时序图，其中 fpga_clk 的延迟有 1ns，ext_clk 的延迟有 2ns，经过计算以后 clock skew 值是 1ns。然而右图是左图与 clock skew 相互作用以后的结果，无论是启动沿还是下一个启动沿，它们都不小心撞到 1ns 的时钟差，然后向后移动 1ns。

clock skew 无差别对待启动沿与下一个启动沿，换句话说 clock skew 无论是 output max 或者 output min 它都看不顺眼，因此：

```

output max    = <fpga2ext delay max> - < destination reg clk delay - source reg clk delay >
               = <fpga2ext delay max> - < ext_clk delay - fpga_clk delay >
               = <fpga2ext delay max> - < clock skew >

output min    = < fpga2ext delay min > - < destination reg clk delay - source reg clk delay >
               = <fpga2ext delay min> - < ext_clk delay - fpga_clk delay >
               = <fpga2ext delay min> - < clock skew >

```

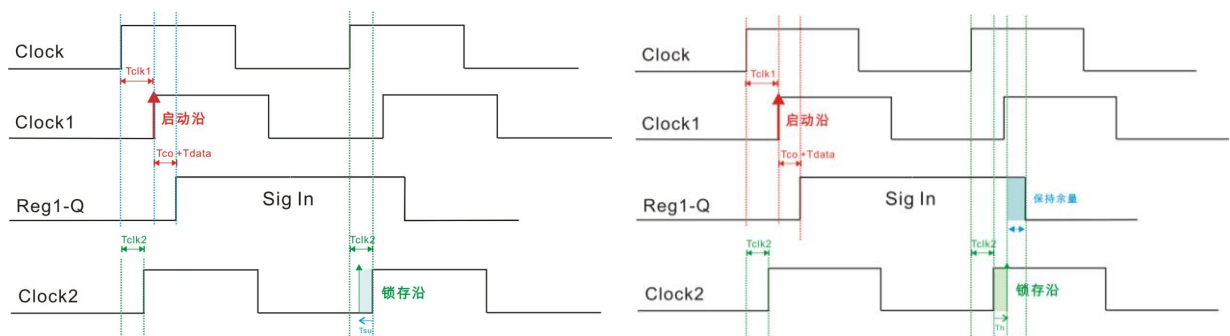


图 5.5.3 Tsu 影响建立关系（左），Th 影响保持关系（右）。

就算是外部 ic，凡是有 IO 口就有寄存器，凡是有寄存器一定有寄存器特性，因此外部 ic 的寄存器一定有参数 T_{su} 和 T_h 。 T_{su} 粗略点讲，它是影响建立关系的寄存器参数；换之， T_h 粗略点讲，它是影响保持关系的寄存器参数（如图 5.5.3 所示）。要将 T_{su} 与 T_h 加入 output min 与 output max 公式，建立关系和保持关系是关键。

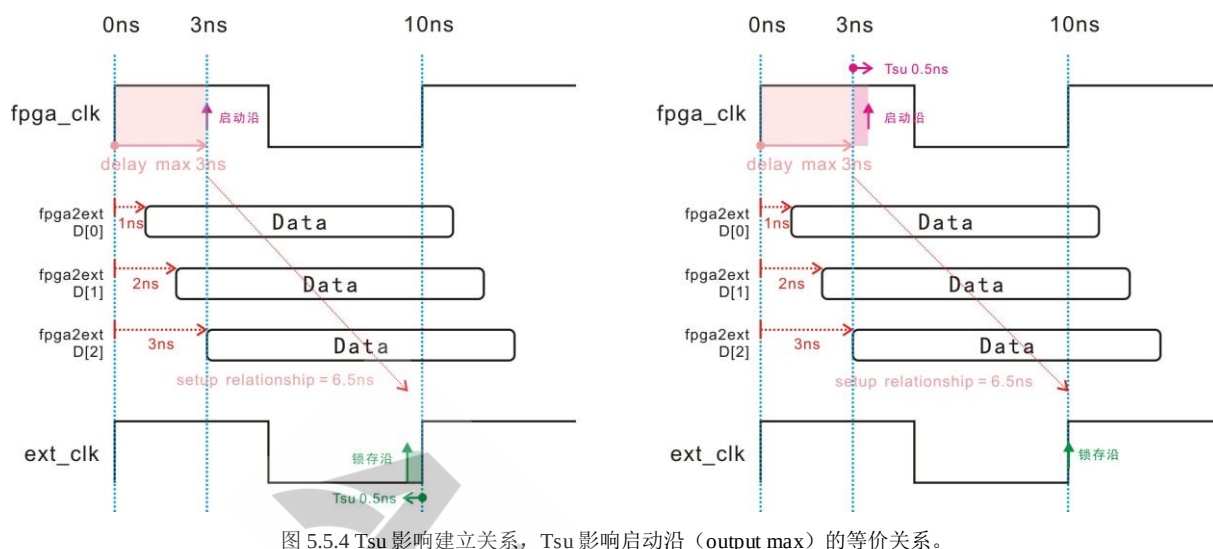


图 5.5.4 T_{su} 影响建立关系， T_{su} 影响启动沿（output max）的等价关系。

5.5.4 的左图是启动沿受 delay max 影响后落在 3ns 的位置，由于外部 ic 的寄存器也存有 0.5ns 的 T_{su} 参数，结果导致锁存沿向左移动 0.5ns，然后落在 9.5ns 的位置，最终得到 6.5ns 的建立关系。换之右图中的锁存沿固步不动，不过 0.5ns 的 T_{su} 却影响原先停留在 3ns 位置的启动沿，启动沿就这样迷迷糊糊的向右移动 0.5ns，然后落在 3.5ns 的位置。最终得到 6.5ns 的建立关系。

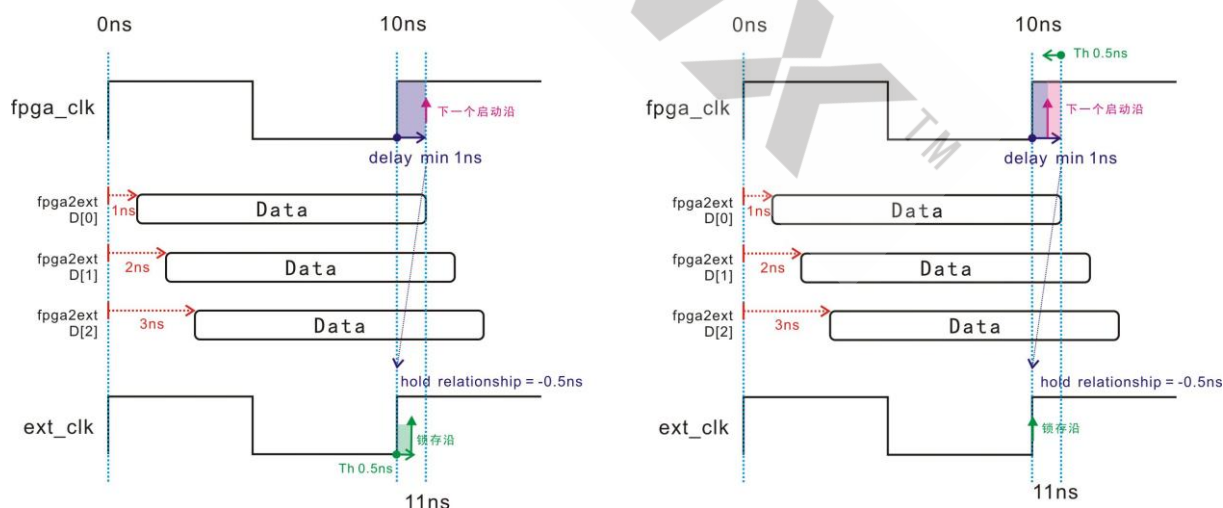


图 5.5.5 T_h 影响保持关系， T_h 影响下一个启动沿（output min）的等价关系。

图 5.5.5 的左图，下一个启动沿原本受 1ns 的 delay min 影响，接着移动到 11ns 的位置。此外，锁存沿不小心受到 0.5ns 的 T_h 影响，结果锁存沿移向 10.5ns 的位置，最终得到 -0.5ns 的保持关系。至于右图中的锁存沿却丝毫不动，可是原本已经受 delay min 影响的下一个启动沿，这下也受 T_h 那 0.5ns 的影响，然后落在 10.5ns 的地方，最终得到 -0.5ns 的保持关系。

的保持关系。

从上述的过程中，我们知道 T_{su} 影响启动沿也间接影响 $output\ max$ ； T_h 反而影响下一个启动沿也间接影响 $output\ min$... 因此， $output\ max$ 和 $output\ min$ 的公式可以这样更动：

```
output max    = <fpga2ext delay max> - < destination reg clk delay - source reg clk delay > + ext_Tsu
              = <fpga2ext delay max> - < ext_clk delay - fpga_clk delay > + ext_Tsu
              = <fpga2ext delay max> - < clock skew > + ext_Tsu

output min    = < fpga2ext delay min > - < destination reg clk delay - source reg clk delay > - ext_Th
              = <fpga2ext delay min> - < ext_clk delay - fpga_clk delay > - ext_Th
              = <fpga2ext delay min> - < clock skew > - ext_Th
```

ext_Tsu 别名 ic_Tsu 亦即外部 ic 寄存器的 T_{su}

ext_Th 别名 ic_Th 亦即外部 ic 寄存器的 T_h

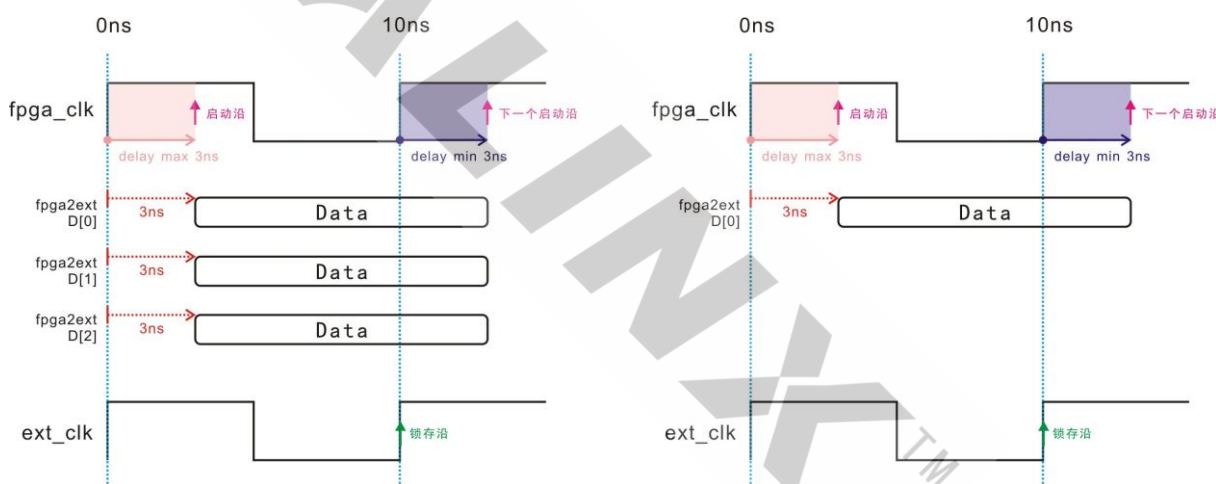


图 5.5.6 延迟压力均衡（左），当单位宽（右）。

图 5.5.6 是延迟压力均匀（作）与单位宽（右）的时序图 ... 对于均与的延迟压力来说，没有所谓的 $delay\ max$ 与 $delay\ min$ 之分，反过来说 $delay\ max$ 是 $delay\ min$ ，或者说 $delay\ min$ 是 $delay\ max$ 。至于单位宽来也没有所谓的 $delay\ max$ 与 $delay\ min$ ，或者说 $delay\ max$ 与 $delay\ min$ 相等。

这种情况我们称为 $delay\ both$ ，换句话说 $delay\ both$ 皆是 $delay\ max$ 与 $delay\ min$ ，因此 $output\ max$ 与 $output\ min$ 可以这样更动（ $both$ 英文是皆是的意思）：

```
output max    = <fpga2ext delay max> - < destination reg clk delay - source reg clk delay > + ext_Tsu
              = <fpga2ext delay max> - < ext_clk delay - fpga_clk delay > + ext_Tsu
              = <fpga2ext delay max> - < clock skew > + ext_Tsu
```

//等价//


```

output max    = <fpga2ext delay min> - < destination reg clk delay - source reg clk delay > + ext_Tsu
              = <fpga2ext delay min> - < ext_clk delay - fpga_clk delay > + ext_Tsu
              = <fpga2ext delay min> - < clock skew > + ext_Tsu
              //等价//

output max    = <fpga2ext delay both> - < destination reg clk delay - source reg clk delay > + ext_Tsu
              = <fpga2ext delay both> - < ext_clk delay - fpga_clk delay > + ext_Tsu
              = <fpga2ext delay both> - < clock skew > + ext_Tsu

output min    = < fpga2ext delay max > - < destination reg clk delay - source reg clk delay > - ext_Th
              = <fpga2ext delay max> - < ext_clk delay - fpga_clk delay > - ext_Th
              = <fpga2ext delay max> - < clock skew > - ext_Th
              //等价//

output min    = < fpga2ext delay min > - < destination reg clk delay - source reg clk delay > - ext_Th
              = <fpga2ext delay min> - < ext_clk delay - fpga_clk delay > - ext_Th
              = <fpga2ext delay min> - < clock skew > - ext_Th
              //等价//

output min    = < fpga2ext delay both > - < destination reg clk delay - source reg clk delay > - ext_Th
              = <fpga2ext delay both> - < ext_clk delay - fpga_clk delay > - ext_Th
              = <fpga2ext delay both> - < clock skew > - ext_Th

```

延迟压力均匀与单位宽是比较特殊的状况，不过道理上和不均匀的多位宽一样。

读到这里笔者猜想，一定有同学会觉得非常奇妙，fpga2ic 是指 fpga 发送数据给 ic “为什么在加入外部 ic 寄存器的 Tsu 与 Th 参数时，却没有考虑过 fpga 本身的 Tco 寄存器参数？”原因很单纯，不是我们不考虑事实上 TimeQuest 已经了解了，因为 output 公式是以 fpga 为主要，换句话说 Tco 是 fpga 的内部延迟因数，所以我们不用刻意告诉它。

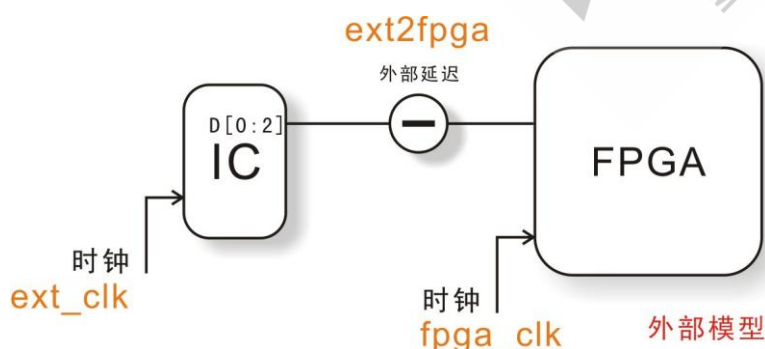


图 5.5.7 ic2fpga (ext2fpga) 的外部模型。

ic2fpga 是 ic 发送数据，fpga 读取数据的外部模型，不过不管位置怎么改变，大体上都是大同小异而已。ic2fpga 依然也是 **fpga 为主要**，亦即 fpga 读取数据，因此也称为 input 公式。同样，针对启动沿的 input 公式称为 input max；换之，针对下一个启动沿的 input 公式称为 input min。

```

input max      = <ext2fpga delay max> - < destination reg clk delay - source reg clk delay >
               = <ext2fpga delay max> - < fpga_clk delay - ext_clk delay >
               = <ext2fpga delay max> - < clock skew >

input min      = < ext2fpga delay min > - < destination reg clk delay - source reg clk delay >
               = <ext2fpga delay min> - < fpga_clk delay - ext_clk delay >
               = <ext2fpga delay min> - < clock skew >

```

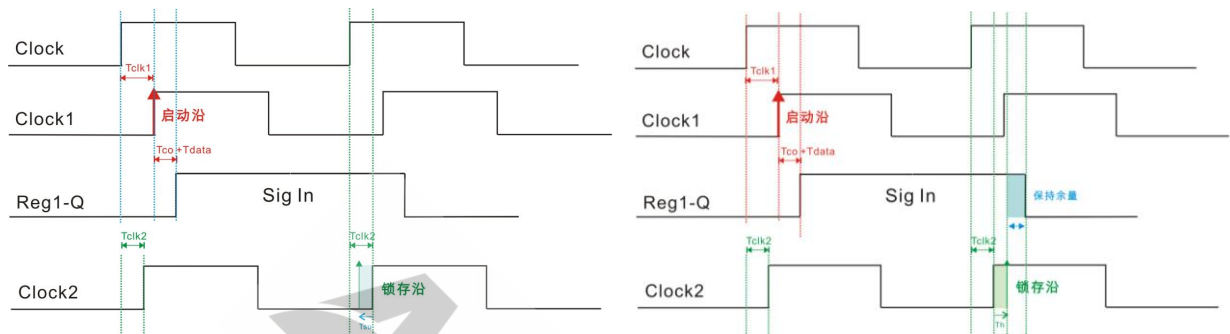


图 5.5.8 Tco 影响建立关系（左），Tco 影响保持关系（右）。

ic2fpga 的外部模型，外部 ic 是发送方，因此我们必须考虑外部 ic 的 Tco 寄存器参数。如图 5.5.8 所示，无论是建立关系还是保持关系都会路出 Tco 的笑脸。

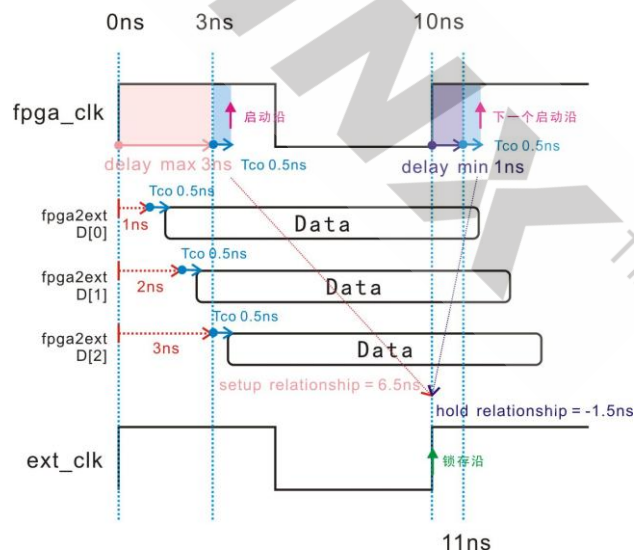


图 5.5.9 Tco 影响建立关系与保持关系。

Tco 是比较霸道的寄存器参数，它吃定建立关系与保持关系。从图 5.5.9 中我们可以理解到，0.5ns 的 Tco 助长了 1ns 的 delay min 成为 1.5ns，同样 Tco 也助长了 3ns 的 delay max 成为 3.5ns。以此类推，启动沿受 delay max 影响，而且 delay max 的新值是 3ns + 0.5ns，亦即 3.5ns。换之下一个启动沿受 delay min 影响，而且 delay min 的新值是 1ns + 0.5ns，亦即 1.5ns。最终导致建立关系为 6.5ns，保持关系为 -1.5ns。

因此 input max 与 input min 公式可以这样更动：

```
input max      = <ext2fpga delay max> - < destination reg clk delay - source reg clk delay > + ext_Tco
               = <ext2fpga delay max> - < fpga_clk delay - ext_clk delay > + ext_Tco
               = <ext2fpga delay max> - < clock skew > + ext_Tco

input min      = < ext2fpga delay min > - < destination reg clk delay - source reg clk delay > + ext_Tco
               = <ext2fpga delay min> - < fpga_clk delay - ext_clk delay > + ext_Tco
               = <ext2fpga delay min> - < clock skew > + ext_Tco
               //等价//

input min      = < ext2fpga delay min > - < destination reg clk delay - source reg clk delay > + ext_minTco
               = <ext2fpga delay min> - < fpga_clk delay - ext_clk delay > + ext_minTco
               = <ext2fpga delay min> - < clock skew > + ext_minTco

ext_Tco 别名 ic_Tco 亦即外部 ic 的 Tco 寄存器参数。
ext_minTco 别名 ic_minTco 亦即外部 ic 的最小 Tco 寄存器参数。
```

同学们可能会举得疑惑，为什么 input min 会有另一个等价的公式，还有 minTco 又是什么？minTco 是 Tco 的最小值，但是差别的比率，根据硬件的不同，结果也会不同。按照理论，input min 载入 ext_Tco 还是 ext_minTco 都没有问题。实际上，[载入 ext_Tco 比起载入 ext_minTco 有更好的保险作用](#)。既然如此，为什么 ext_minTco 还受到重用呢？关于这点笔者就不怎么清楚了，可能是近似作用或者其他的吧 ... 无论怎样选择都是见仁见智了。

5.6 外部模型的 input/output 约束指令

这一章我们将通过假想的实验来认识外部模型的 input/output 约束命令。

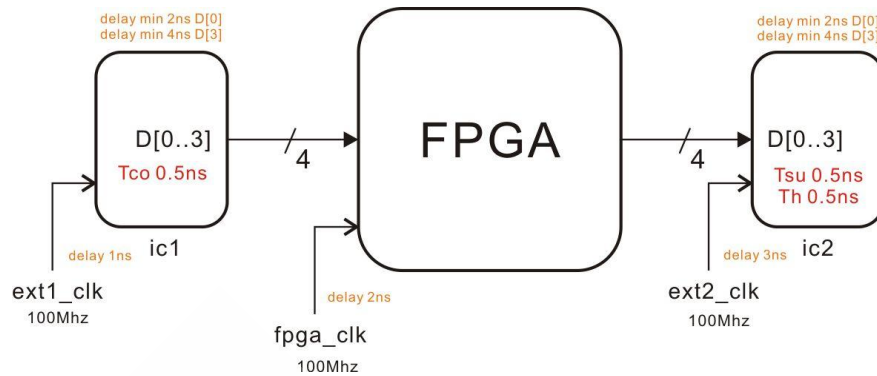


图 5.6.1 假想实验的外部模型。

图 5.6.1 是假想实验的外部模型，fpga 与 ic2 是 fpga2ic 的关系，亦即 fpga 经过 D[0:3] 发送数据给 ic2。然而 D[0..3]之间，delay min 是 2ns，然而 delay max 4ns，至于 ic2 的寄存器特性是 0.5ns 的 Tsu 和 0.5ns 的 Th。换之 ic1 与 fpga 是 fpga2ic 的关系，亦即 ic1 经过 D[0:3]发送数据给 fpga，其中 delay min 是 2ns，delay max 是 4ns，此外 ic1 有 0.5ns 的 Tco。

ext1_clk 是受 1ns 延迟的 100Mhz 时钟；fpga_clk 是受 2ns 延迟的 100Mhz 时钟；ext2_clk 是受 3ns 延迟的 100Mhz 时钟。然后让我们计算一下 input/output min/max 的各种取值：

```
delay min 2ns
delay max 4ns
ext1_clk delay 1ns
fpga_clk delay 2ns
ext2_clk delay 3ns
Tco/Tsu /Th 0.5ns
```

```
input max    = <ext2fpga delay max> - < destination reg clk delay - source reg clk delay > + ext_Tco
              = <ext2fpga delay max> - < fpga_clk delay - ext1_clk delay > + ext_Tco
              = <ext2fpga delay max> - < clock skew > + ext_Tco
              = 4ns - (2ns - 1ns) + 0.5ns
              = 3.5ns

input min    = < ext2fpga delay min > - < destination reg clk delay - source reg clk delay > + ext_Tco
              = <ext2fpga delay min> - < fpga_clk delay - ext1_clk delay > + ext_Tco
              = <ext2fpga delay min> - < clock skew > + ext_Tco
              = 2ns - (2ns - 1ns) + 0.5ns
              = 1.5ns
```

```

output max    = <fpga2ext delay max> - < destination reg clk delay - source reg clk delay > + ext_Tsu
              = <fpga2ext delay max> - < ext2_clk delay - fpga_clk delay > + ext_Tsu
              = <fpga2ext delay max> - < clock skew > + ext_Tsu
              = 4ns - (3ns - 2ns) + 0.5ns
              = 3.5ns

output min    = < fpga2ext delay min > - < destination reg clk delay - source reg clk delay > - ext_Th
              = <fpga2ext delay min> - < ext_clk1 delay - fpga_clk delay > - ext_Th
              = <fpga2ext delay min> - < clock skew > - ext_Th
              = 2ns - (3ns - 2ns) - 0.5ns
              = 0.5ns

```

实验六 假想实验 input/output delay 约束

center_module.v

```

1.  module center_module
2.  (
3.      input CLK,
4.      input RSTn,
5.      input [3:0]Din,
6.      output [3:0]Dout
7.  );
8.      /*****/
9.
10.     reg [3:0]rData;
11.
12.     always @ ( posedge CLK or negedge RSTn )
13.         if( !RSTn )
14.             begin
15.                 rData <= 4'd0;
16.             end
17.         else
18.             begin
19.                 rData <= Din;
20.             end
21.
22.     /*****/
23.
24.     assign Dout = rData;
25.
26.     /*****/

```

```
27.  
28. endmodule
```

上述是 center_module.v 的内容 ,乍看下 fpga 也只是充当 ic1 与 ic2 之间的间接寄存器而已。亦即从 ic1 读入数据致 rData , 然后将 rData 的数据发送给 ic2。

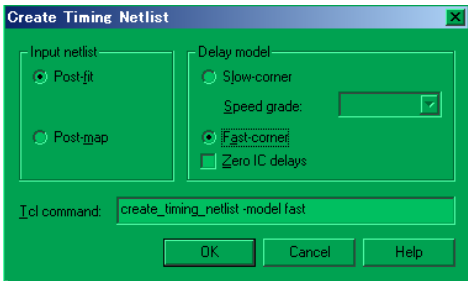


图 5.6.2 创建 best case 网表。

再来先创建同名的 center_module.sdc 文件 , 然后将它设置为本实验的约束文本 , 再来打开 TimeQuest。然后手动建立 best case 网表 , 如图 5.6.2 所示。

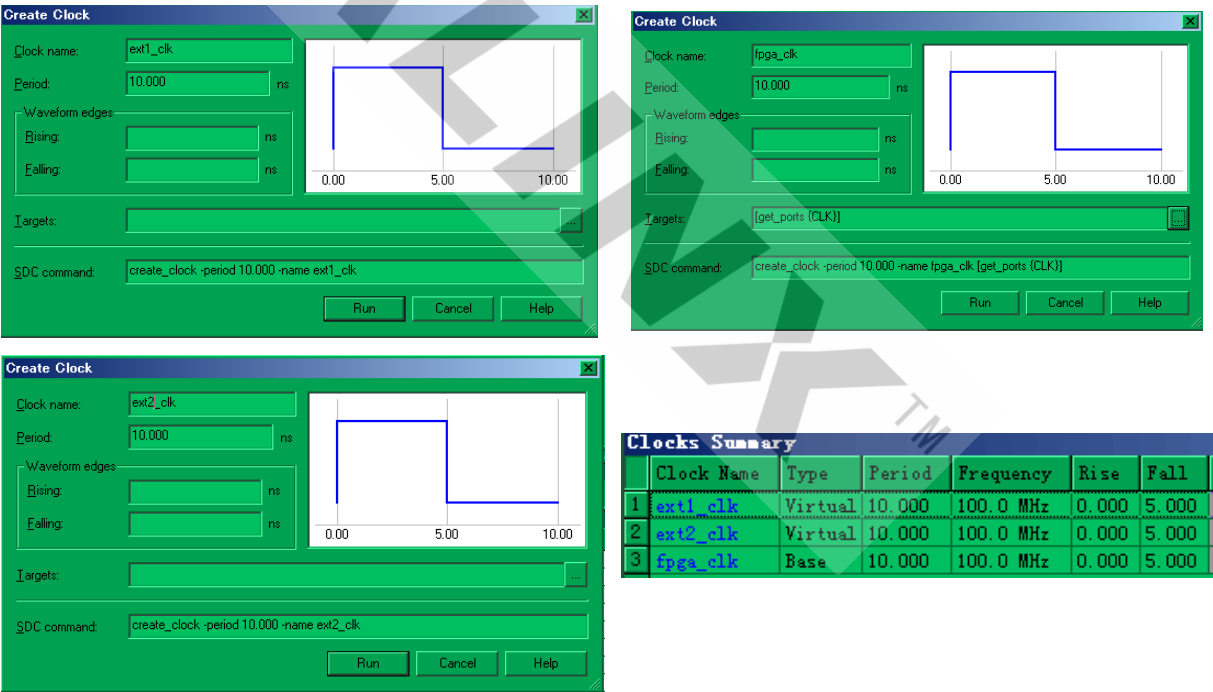


图 5.6.3 创建 3 个时钟。

再来建立 3 个时钟 (如图 5.6.3 所示) , 它们分别为是 10ns 周期的 fpga_clk , ext1_clk 还有 ext2_clk。除了 fpga_clk是拥有对应的 target 以外 (CLK 的 io) , ext1_clk 与 ext2_clk 都是没有对应目标的 virtual clock (虚拟时钟) 。

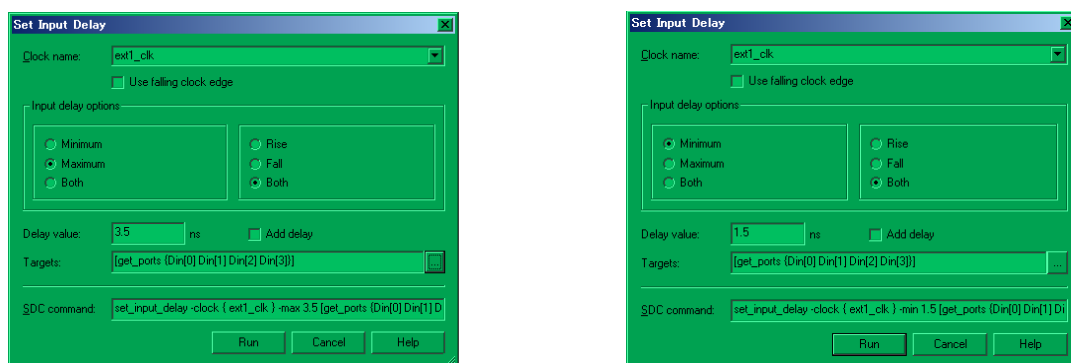


图 5.6.4 set_input_delay 界面。

接下来，从 Constraints 菜单中调出 set_input_delay 界面（图 5.6.4）。任何初次看到这张界面的同学，估计都会被奇奇怪怪的选项吓一跳，不过不要紧，让我们逐个来了解。clock name 指向 ic2fpga 外部模型中的 ic 时钟源，这里是 ext1_clk；至于 delay 选线中的 max 是指 input max，换之 min 是指 input min，而 both 是指 min/max，接着在 delay value 中输入相关的数值，min 是 1.5ns，max 是 3.5ns；然而 target 是属于 input 有关的输入口，在此是 Din[0:3]。

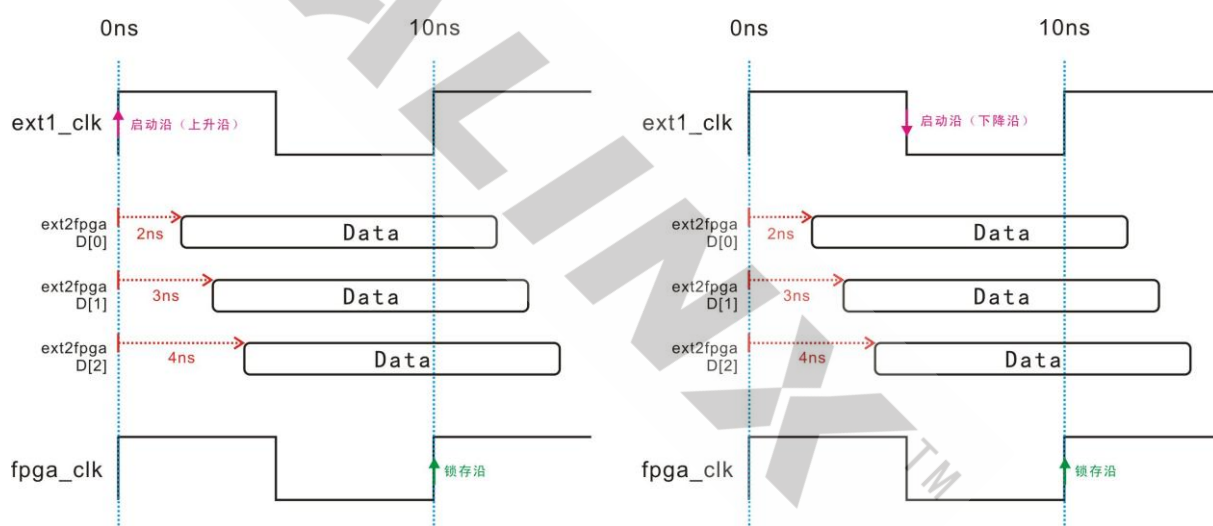


图 5.6.5 （左）默认，（右）use falling clock edge。

默认选项下 TimeQuest 都会认为被 clock name 指定的时钟都是上升沿触发（输出数据）。换之，使能 use falling clock edge，TimeQuest 就会知道被 clock name 指定的时钟由下降沿触发。至于这个选项的用处暂时没有实例讲解，同学们先弄个大概就可以了。

至于 add_delay 选项是比较模糊的东西，一般上一个 IO 口都由 1 个寄存器驱动。不过一些特殊的 IO 口可能由多个寄存器驱动，例如 DDIO，亦即双沿 IO 口 ... 它里边是由 2 个寄存器同时驱动一个 IO 口，使能 add_delay 就是告诉 TimeQuest 某个 IO 由 1 个以上的寄存器驱动。至于 rise/fall 或者 both 笔者就不清楚了，不过默认 both 就是了。

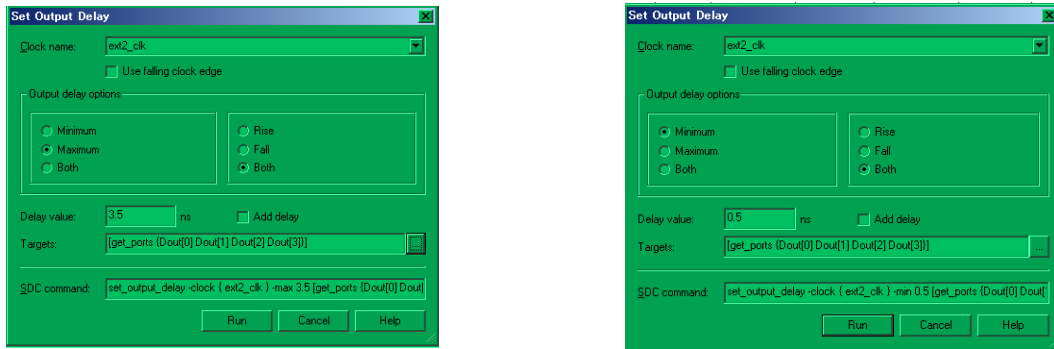


图 5.6.6 set output delay 界面。

同样在 Constraints 菜单下调出 set output delay 的界面，如图 5.6.6 所示。基本上 set input delay 与 set output delay 的意义差不多都一样，只是节点换了位置而已。其中 clock name 指定驱动外部 ic 的时钟，在这里是 ext2_clk。delay max 输入 3.5ns，delay min 则输入 0.5ns；target 则是 Dout[0:3]。完后，sdc 文件会自动添加以下的约束命令，

center_module.sdc

```

*****
# Create Clock
*****

create_clock -name {fpga_clk} -period 10.000 -waveform { 0.000 5.000 } [get_ports {CLK}]
create_clock -name {ext1_clk} -period 10.000 -waveform { 0.000 5.000 }
create_clock -name {ext2_clk} -period 10.000 -waveform { 0.000 5.000 }

*****
# Set Input Delay
*****

set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 3.500 [get_ports {Din[0]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 1.500 [get_ports {Din[0]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 3.500 [get_ports {Din[1]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 1.500 [get_ports {Din[1]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 3.500 [get_ports {Din[2]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 1.500 [get_ports {Din[2]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 3.500 [get_ports {Din[3]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 1.500 [get_ports {Din[3]}]

*****
# Set Output Delay
*****

```

```
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 3.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 0.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 3.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 0.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 3.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 0.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 3.500 [get_ports {Dout[3]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 0.500 [get_ports {Dout[3]}]
```

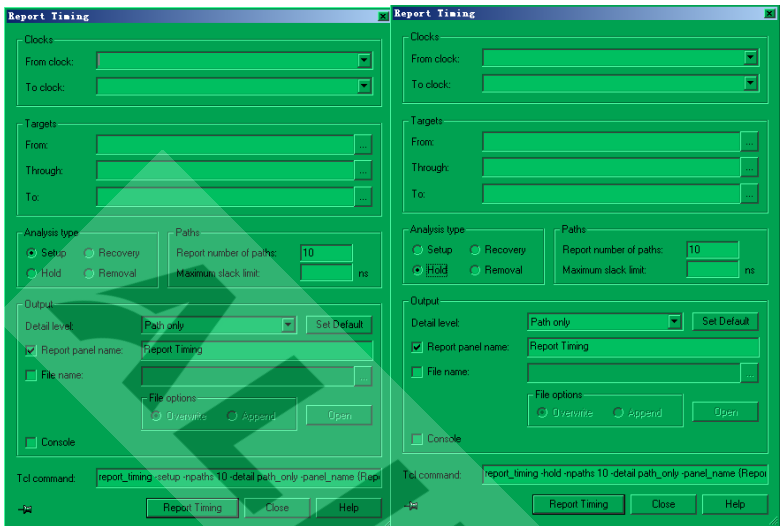


图 5.6.7 setup 与 hold 时序分析。

然后我们就可以开始分析时序了 ... 双击 Report Timing ,选择 setup 与 hold 分析 ,其他放空 , 然后点击 report timing 执行 , 如图 5.6.7 所示。

Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	3.512	rData[3]	Dout[3]	fpga_clk	ext2_clk	10.000	-1.471	1.517
2	3.530	rData[0]	Dout[0]	fpga_clk	ext2_clk	10.000	-1.473	1.497
3	3.564	rData[1]	Dout[1]	fpga_clk	ext2_clk	10.000	-1.449	1.487
4	3.749	rData[2]	Dout[2]	fpga_clk	ext2_clk	10.000	-1.364	1.387
5	5.210	Din[3]	rData[3]	ext1_clk	fpga_clk	10.000	1.471	2.723
6	5.274	Din[1]	rData[1]	ext1_clk	fpga_clk	10.000	1.449	2.637
7	5.288	Din[0]	rData[0]	ext1_clk	fpga_clk	10.000	1.473	2.647
8	5.467	Din[2]	rData[2]	ext1_clk	fpga_clk	10.000	1.364	2.362

图 5.6.8 setup 分析结果。

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	2.447	Din[2]	rData[2]	ext1_clk	fpga_clk	0.000	1.364	2.362
2	2.611	Din[0]	rData[0]	ext1_clk	fpga_clk	0.000	1.473	2.647
3	2.625	Din[1]	rData[1]	ext1_clk	fpga_clk	0.000	1.449	2.637
4	2.689	Din[3]	rData[3]	ext1_clk	fpga_clk	0.000	1.471	2.723
5	3.251	rData[2]	Dout[2]	fpga_clk	ext2_clk	0.000	-1.364	1.387
6	3.436	rData[1]	Dout[1]	fpga_clk	ext2_clk	0.000	-1.449	1.487
7	3.470	rData[0]	Dout[0]	fpga_clk	ext2_clk	0.000	-1.473	1.497
8	3.488	rData[3]	Dout[3]	fpga_clk	ext2_clk	0.000	-1.471	1.517

图 5.6.9 hold 分析结果。

经过分析以后 ,我们发现只有 8 对节点的 setup 分析与 hold 分析而已 ... 有 4 对节点是

rData[0:3]~Dout[0:3]，另外 4 对节点是 Din[0:3]~rData[0:3]。在此，rData 是 fpga 内的寄存器，Din 是 ic1 的寄存器，Dout 是 ic2 的寄存器。从图 5.6.8~9 中看见的 Clock Skew 或者 Data Delay 基本上是内部模型的延迟结果，与外部模型一丁儿关系也没有 ... 这话何说呢？

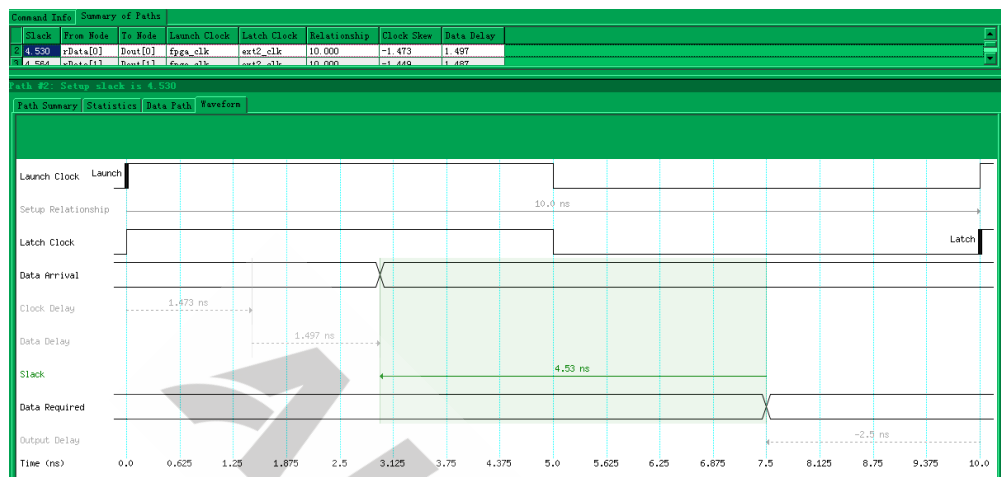


图 5.6.10 rData[0]~Dout[0] setup 分析的时序图。

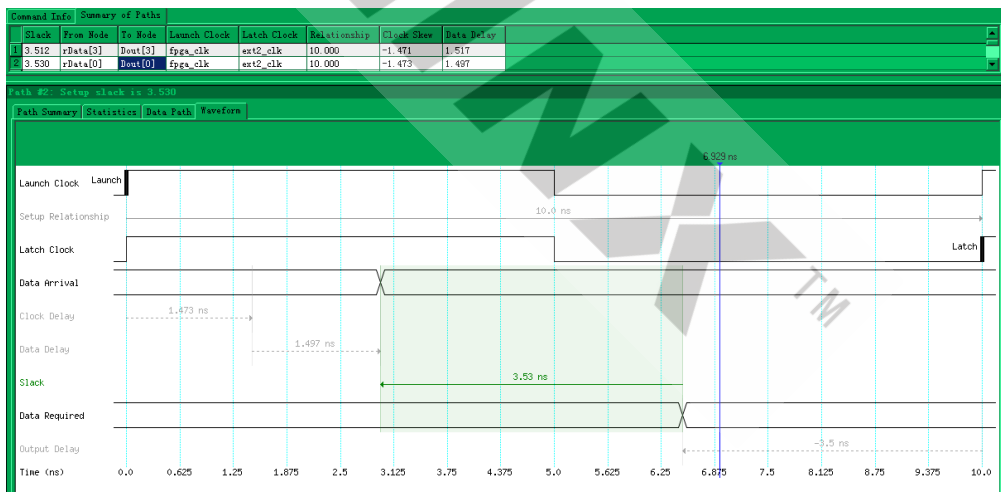


图 5.6.11 rDin[0]~rData[0] setup 分析的时序图。

笔者展开节点 rData[0]~Dout[0](图 5.6.10)还有节点 rData[0]~Dout[0](图 5.6.11)的 setup 分析过程，其中 3.5ns 还有 3.5ns 就是 set output delay max 与 set input delay max 的结果。

总结：

第五章终于写完了，这一章的实验数量不但少，而且约束命令的出现也不多，不过内容都是深入理解 TimeQuest 的要点。这个章节我们先了解网表，事实上网表也有极端与好之分，越是极端保险余量越大，可是性能限制却越大，结果越偏离实际的信息；换之，越是好保险余量越小，性能限制也越小，结果越接近实际的信息。默认下 TimeQuest 是一位极端的生存者，不过也可以手动要它成为乐天的屁民。

此外，笔者也稍微谈论一下 TimeQuest 的迷信，亦即 Fmax。笔者初学 TimeQuest 的时候，同样也认为 Fmax 是时序分析的重要标准，后来才知道 Fmax 是指某个“最差劲节点”的最高频率。Fmax 这东西不是越高越好，就像驾驶跑车那样，就算哪天死了也不会冲到最高速 ... 所以说，够用就好。提升 Fmax 有许多方法，除了选择较好的网表以外，还可以设置编译器的选项，当然最根本还是 Verilog 本身。Fmax 也可以是网表质量关系性能限制的评估值。

但是这章让笔者最着重内容 ... 亦即 TimeQuest 用非常巧妙的手段将外部信息包裹为“外部资料”，这外部资料有标志性的 max 与 min，而 max 是针对建立关系，min 则是针对保持关系。外部资料是由以下公式（手段）将外部延迟信息包裹而成：

```
input max      = <ext2fpga delay max> - < destination reg clk delay - source reg clk delay > + ext_Tco
               = <ext2fpga delay max> - < clock skew > + ext_Tco

input min      = < fpga2ext delay min > - < destination reg clk delay - source reg clk delay > + ext_Tco
               = <fpga2ext delay min> - < clock skew > + ext_Tco ( 或者 ext_minTco )

output max     = <fpga2ext delay max> - < destination reg clk delay - source reg clk delay > + ext_Tsu
               = <fpga2ext delay max> - < clock skew > + ext_Tsu

output min     = < fpga2ext delay min > - < destination reg clk delay - source reg clk delay > - ext_Th
               = <fpga2ext delay min> - < clock skew > - ext_Th
```

对笔者而言，最辛苦的差事就是推导以上的公式 ... 很久以前，笔者在接触 TimeQuest 不久，它们就出现在手册上。不过一般而言，绝对不会有人犯贱到折腾自己去推导它们。但直觉告诉笔者，推导公式可以让自己更了解 TimeQuest ... 就这样，笔者一度跌进学习的死胡同，然后灰心离开，学习进度从此就定格在哪里。

当《整合篇》完成，时间大约是一年后 ... 笔者再次打开当时的草稿，然后继续思考，不出几天的时间，许多东西就那样自然而然的理解起来 ... 推导整套公式的关键自然是那个不起眼的建立关系与保持关系。心想真妙，建立时间与保持时间既然与建立关系与保持关系有如此微妙的关系，这就是第五章的重点内容。

第六章：物理时钟与外部模型

6.1 物理时钟①

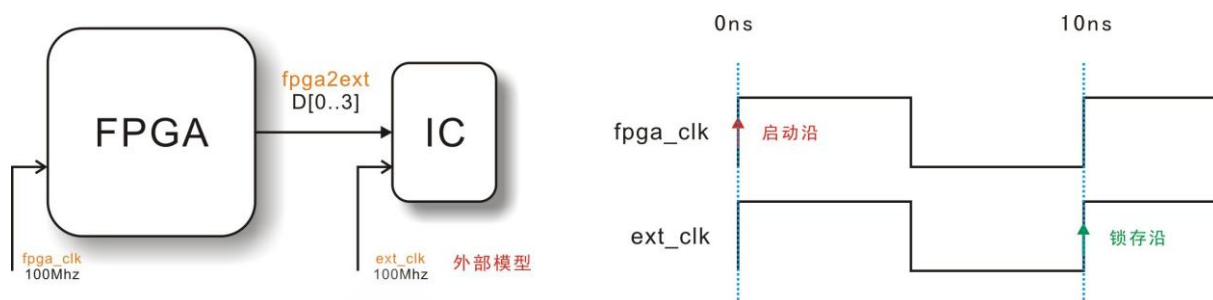


图 6.1.1 (左) fpga2ic 外部模型, (右) 理想时钟。

在理想时序中出现的时钟称为理想时钟，换之在物理时序中出现的时钟称为物理时钟。笔者知道自己在说废话 ... 不过理想时钟的特性如图 6.1.1 右图所示，fpga_clk 与 ext_clk 没有任何延迟。假设 fpga 发送数据，ic 读取数据的话，那么 fpga_clk 的启动沿必定在 0ns 触发，而 ext_clk 的锁存沿必定在 10ns 读取。

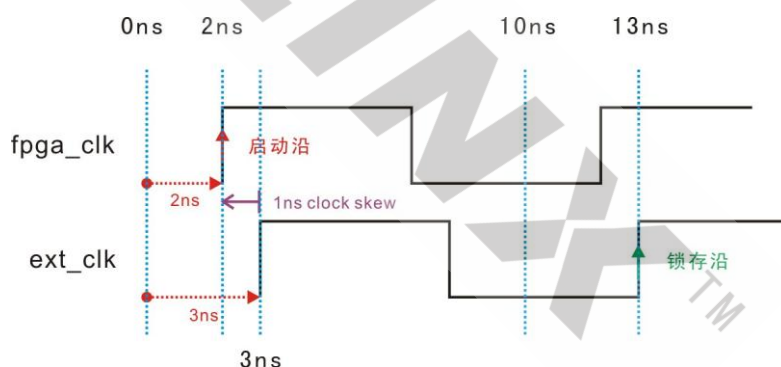


图 6.1.2 clock skew (时间差)。

相比之下，物理时钟就没有那么单纯了 ... 物理时钟有许多特性如：clock skew (时间差)，jitter (抖动)，或者 latency (潜伏) 等，这些物理时钟的特性又称为 **uncertainly** — 非定性，或者非理想。clock skew 我们已经在前面学习过，亦即 **fpga_clk 与 ext_clk 的抵达时间不一致所导致的时间差**，如 fpga_clk 有 2ns 的延迟，而 ext_clk 有 3ns 的延迟，那么时钟差就是 1ns，如图 6.1.2 所示。公式是：

$$\text{clock skew} = \langle \text{destination reg clock delay} \rangle - \langle \text{source reg clock delay} \rangle$$

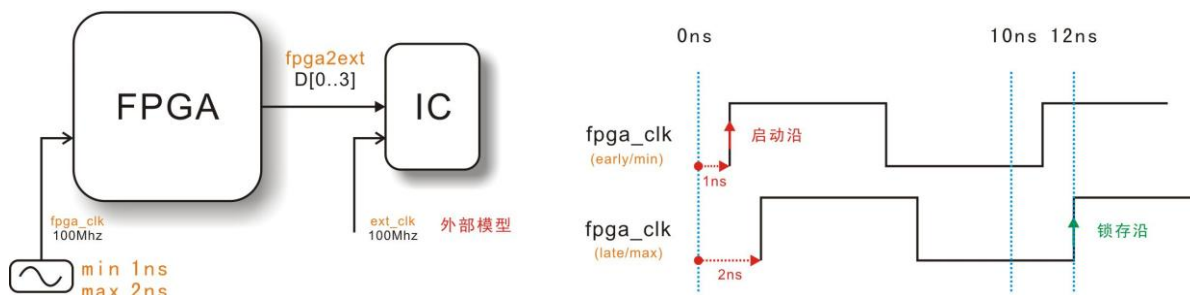


图 6.1.3 时钟抖动。

那么什么是抖动与潜伏呢？当时钟源产生时钟信号的时候，除了时钟路径的延迟以外，**时钟产生源本来也有延迟**，不过该死不死的是，这个产生源的延迟是非常暧昧的，它**有时大有时小** ... 如图 6.1.3 所示，假设倒霉的 fpga 连接到劣质的时钟源，该时钟源的最小延迟是 1ns，最大延迟是 2ns（**这时候还不包括 fpga_clk 路径本身的延迟**）。

结果，右图的时序被产生，亦即 fpga_clk 有时受 1ns 的 **min 延迟影响**，称为 **early**；fpga_clk 有时也受 2ns 的 **max 延迟影响**，称为 **late**。话中的 early/min 或者 late/max 也称为 **clock latency**（**潜伏时钟**）。clock latency 可说是非常恶趣味的玩笑，这种不确定的因数最要人摸不着，也让人最抓狂。

虽说 fpga 内部也有时钟抖动 ... 不过 fpga 内部有专门的硬件过滤时钟的抖动，以至于内部的时钟潜伏小到可以忽略它。所以说，时钟抖动就成为了外部模型的烦恼，不过也不是那么严重，以目前的科技而言，要把抖动控制在皮秒级别是绝对没问题的。此外，如果设计对时序不严谨的话，基本上也可以忽略时钟潜伏的问题。

话是这么说 ... 不过同学们也休想偷懒，因为 TimeQuest 部分约束命令是与物理时钟有关，所以不得不学习，哎 ...

时钟信号的抖动问题日最让人头疼的地方是 ... 它不像数据那样有多位宽，delay max 往往是针对有最大延迟的某位数据，反之 delay min 是针对某位有最小延迟的数据。至于时钟抖动都是朝向同一个时钟信号砸去，亦即 **delay min 也是该时钟信号**，**delay max 也是该时钟信号**。所以说，我们必须换个思路再思考。

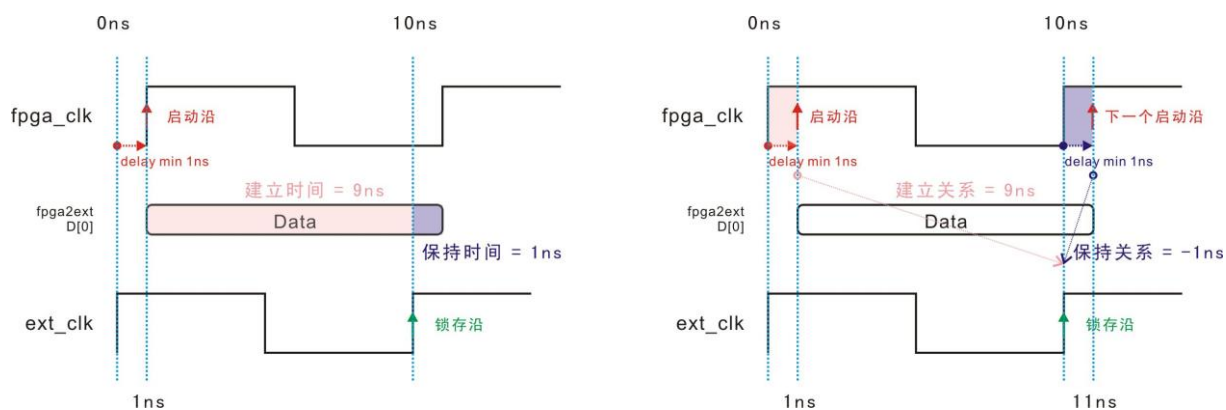


图 6.1.4 delay min 影响 fpga_clk 和 建立关系与保持关系的等价图。

如图 6.1.4 的左图所示，由于 fpga_clk 被 1ns 延迟的 delay min 影响，导致建立时间为 9ns，而保持时间为 1ns；换之，右图的启动沿与下一个启动沿也被 1ns 延迟的 delay min 影响，最后建立关系取得 9ns，保持关系取得 -1ns。

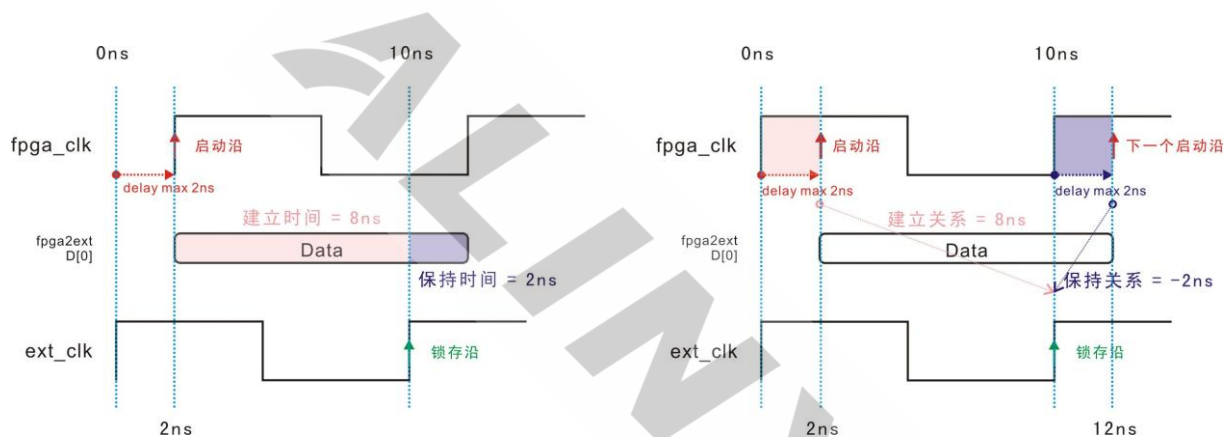


图 6.1.5 delay max 影响 fpga_clk 和建立关系与保持关系的等价图。

图 6.1.5 的左图中，fpga_clk 受 2ns 延迟的 delay max 影响，导致建立时间取得 8ns，而保持时间取得 2ns；换之，右图中的启动沿与下一个启动沿皆被 2ns 延迟的 delay max 影响，结果取得 8ns 的建立关系，还有-2ns 的保持关系。

根据图 6.1.4 与图 6.1.5 的各种信息，我们总结出 ... 在物理时序的角度上建立时间越短越危险，同样保持时间越短越危险，因此我们可以这样说：delay max (late) 影响启动沿，delay min (early) 影响下一个启动沿。因为 delay max (late) 会造就最小的建立时间，换之 delay min (early) 会造就最小的保持时间。

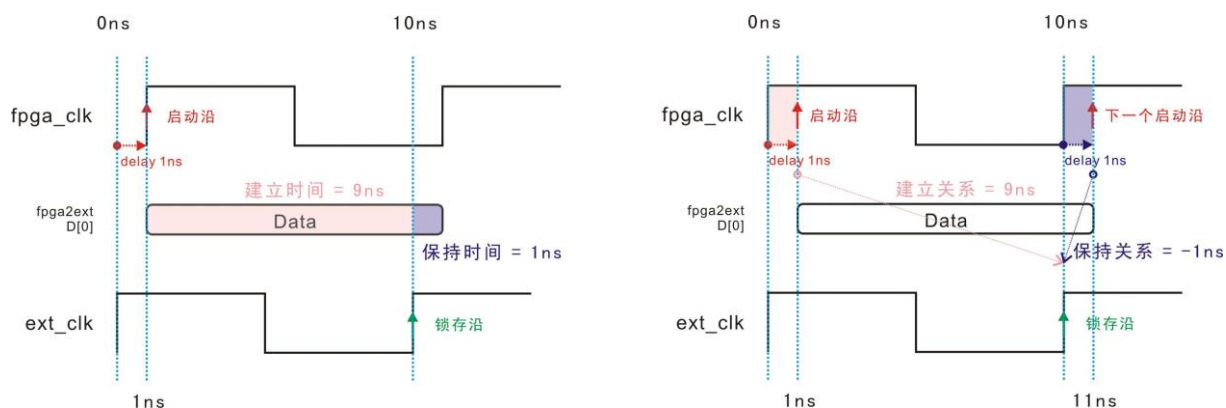


图 6.1.6 有抖动但是没有 min/max (early/late) 之分, 称为 both。

有时候时钟信号的产生源会有延迟但是没有抖动, 又或者说产生源有抖动但是没有 min/max (early/late) 之分, 这种情况下我们称为 both, 具体内容和 input/output delay both 差不多。如图 6.1.6 所示, 假设产生源有 1ns 的延迟, 但是没有抖动, 换句话说, 右图中的启动沿和下一个启动沿, 皆被 1ns 的延迟影响向右移至 1ns。

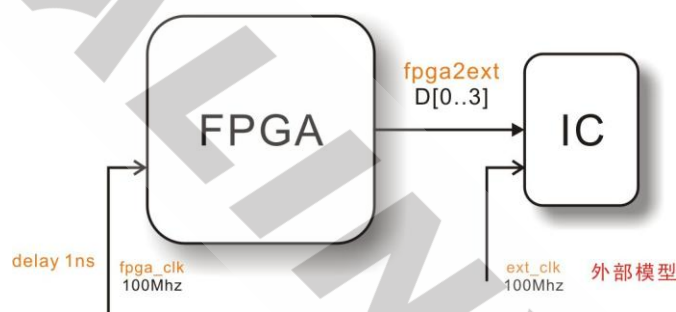


图 6.1.7 fpga_clk 时钟信号路径延迟 1ns。

这种情况与 fpga_clk 路径延迟为 1ns 等价, 如图 6.1.7 所示。如果反过来思考话, 在第五章出现的 input min/max 与 output min/max 公式, 我们可以除掉 clock ckew, 如下所示:

$$\begin{aligned}
 \text{input max} &= \langle \text{ext2fpga delay max} \rangle - \langle \text{destination reg clk delay} - \text{source reg clk delay} \rangle + \text{ext_Tco} \\
 &= \langle \text{ext2fpga delay max} \rangle - \langle \text{clock skew} \rangle + \text{ext_Tco} \\
 \\
 \text{input min} &= \langle \text{fpga2ext delay min} \rangle - \langle \text{destination reg clk delay} - \text{source reg clk delay} \rangle + \text{ext_Tco} \\
 &= \langle \text{fpga2ext delay min} \rangle - \langle \text{clock skew} \rangle + \text{ext_Tco} \text{ (或者 ext_minTco)} \\
 \\
 \text{output max} &= \langle \text{fpga2ext delay max} \rangle - \langle \text{destination reg clk delay} - \text{source reg clk delay} \rangle + \text{ext_Tsu} \\
 &= \langle \text{fpga2ext delay max} \rangle - \langle \text{clock skew} \rangle + \text{ext_Tsu} \\
 \\
 \text{output min} &= \langle \text{fpga2ext delay min} \rangle - \langle \text{destination reg clk delay} - \text{source reg clk delay} \rangle - \text{ext_Th} \\
 &= \langle \text{fpga2ext delay min} \rangle - \langle \text{clock skew} \rangle - \text{ext_Th}
 \end{aligned}$$

为 (除掉 clock skew 部分):

```
input max      = <ext2fpga delay max> + ext_Tco
input min      = <ext2fpga delay min> + ext_Tco ( 或者 ext_minTco )
```

```
output max     = <fpga2ext delay max> + ext_Tsu
output min     = < fpga2ext delay min > - ext_Th
```

同学们可能会觉得非常疑惑，然后问：“如果将外包资料的 clock skew 除掉的话，TimeQuest 又该如何知道外边有关 fpga_clk 延迟与 ext_clk 延迟的信息呢？”好问题，这就是接下来我们要学习的约束命令，亦即 set_clock_latency。

实验七: set_clock_latency 告诉 TimeQuest 外部时钟延迟信息

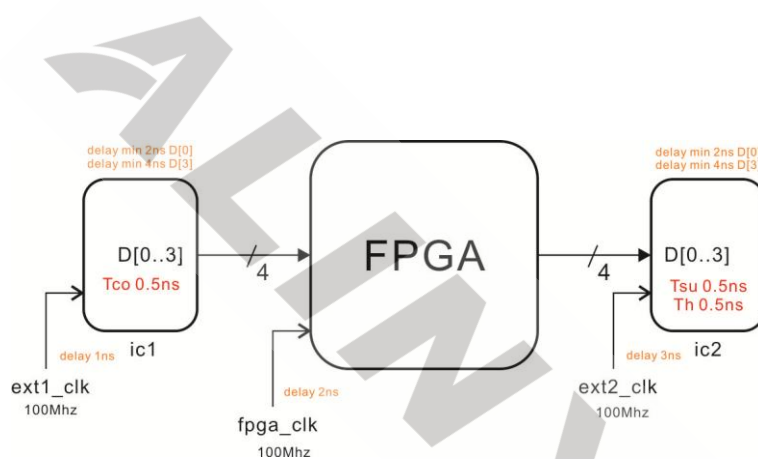


图 6.1.8 假想实验的外部模型。

图 6.1.8 是沿用实验六的假想外部模型，亦即 ic1 输出 fpga 读取，fpga 输出 ic2 读取。其中 ic1 致 fpga 的 D[0..3] 最小延迟是 2ns，最大延迟是 4ns；反之 fpga 致 ic2 的 D[0..3] 最小延迟是 2ns，最大延迟是 4ns。此外也有 100Mhz 的 ext1_clk，fpga_clk 与 ext2_clk 给相应的硬件提供时钟，其中 ext1_clk 有 1ns 延迟，fpga_clk 有 2ns 延迟，至于 ext2_clk 有 3ns 延迟。（注意，目前任何时钟信号的产生源还没有发生抖动）

接下来计算 input delay min/max 与 output delay min/max:

```
input max      = <ext2fpga delay max> + ext_Tco
                = 4ns + 0.5
                = 4.5ns

input min      = <fpga2ext delay min> + ext_Tco ( 或者 ext_minTco )
                = 2ns + 0.5ns
                = 2.5ns
```

```

output max    = <fpga2ext delay max> + ext_Tsu
               = 4ns + 0.5ns
               = 4.5ns

output min    = < fpga2ext delay min > - ext_Th
               = 2ns - 0.5ns
               = 1.5ns

```

接下来，让我们来认识一下 set_clock_latency 约束命令：

set_clock_latency 的约束行为不是为某个时钟设置延迟，而是告诉 TimeQuest 有关外边时钟信号的延迟信息。虽然说 set_clock_latency 约束命令，内外模型的时钟潜伏它都可以设置，不过 TimeQuest 会无视 set_clock_latency 对内部模型的一切影响，所以说它是半残废的约束命令，具体实例往后再谈。

不管怎么说，先打开实验七，然后建立和实验七同名的 .sdc 文件，然后将下列信息贴上：

```

*****
# Create Clock
*****

create_clock -name {fpga_clk} -period 10.000 -waveform { 0.000 5.000 } [get_ports {CLK}]
create_clock -name {ext1_clk} -period 10.000 -waveform { 0.000 5.000 }
create_clock -name {ext2_clk} -period 10.000 -waveform { 0.000 5.000 }

*****
# Set Input Delay
*****

set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[0]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[0]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[1]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[1]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[2]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[2]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[3]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[3]}]

*****
# Set Output Delay
*****

```

```

set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[3]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[3]}]

```

首先声明 3 个 create clock 亦即告诉 TimeQuest 有 100Mhz 的 fpga_clk 对应 CLK 口，还有 100Mhz 虚拟时钟的 ext1_clk 与 ext2_clk。至于 set input delay 与 set input delay 是针对 Din 与 Dout 的延迟声明。内容的大体上与实验六差不多一样，不过区别的只是 input/output delay 不考虑各个时钟信号的时间差（clock skew）。

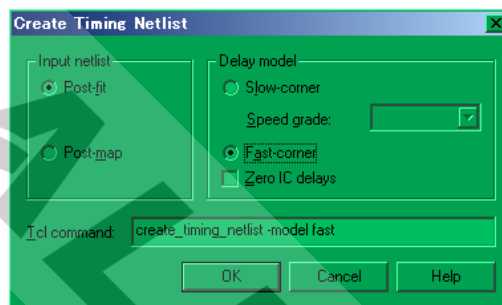
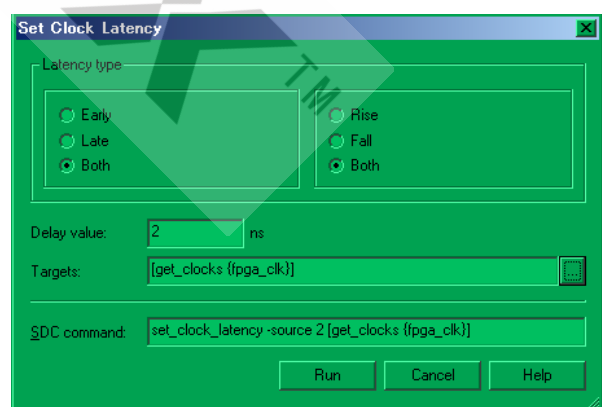
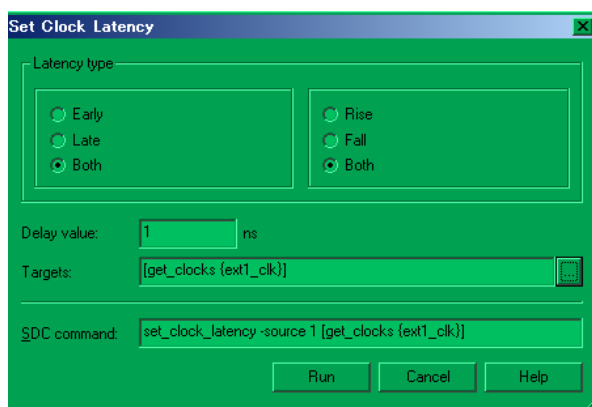


图 6.1.9 手动建立 Best Case 网表。

需要编译的同学先编译一下实验，然后打开 TimeQuest 手动声明 Best Case 网表，如图 6.1.9 所示。接着连打 Read SDC File 与 Update Timing Netlist。



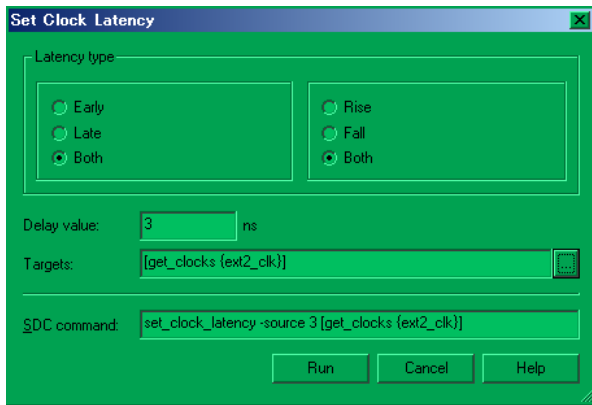


图 6.1.10 Set_Clock_Latency 的界面。

从 Constraint 的菜单下打开 set clock latency 的界面 ,如图 6.1.10 所示。Latency type 下的 Early ,Late 还有 Both ,就如上述解释的那样 ,early 对应 min ,late 对应 max ,而 both 对应 min/max 或者 early/late。至于左边的 Rise/Fall 与 Both 不用鸟它 ,我们现在用不到 ,总之默认下 Both 就是了。

Targets 是指对应的时钟信号 ,然而 delay value 是指该对应时钟信号的延迟值。同学只要根据图 6.1.8 假想实验的信息输入即可 ,亦即 ext1_clk 延迟 1ns ,fpga_clk 延迟 2ns , ext2_clk 延迟 3ns ... 结果如图 6.1.10 所示。完后 ,下面的代码就会自动添加在 sdc 文件中 :

```
*****
# Set Clock Latency
*****

set_clock_latency -source 2.000 [get_clocks {fpga_clk}]
set_clock_latency -source 1.000 [get_clocks {ext1_clk}]
set_clock_latency -source 3.000 [get_clocks {ext2_clk}]
```

接下来是习惯性的双击 Report Timing Netlist 更新网表 ,然后又习惯性的双击 Report Timing 调出全部 8 对节点的 Setup 时序报告 ,与 Hold 时序报告 ,接着再比较一下实验六的 8 对节点 Setup 时序报告与 Hold 时序报告。

Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	3.512	rData[3]	Dout[3]	fpga_clk	ext2_clk	10.000	-3.471	1.517
2	3.530	rData[0]	Dout[0]	fpga_clk	ext2_clk	10.000	-3.473	1.497
3	3.564	rData[1]	Dout[1]	fpga_clk	ext2_clk	10.000	-3.449	1.487
4	3.749	rData[2]	Dout[2]	fpga_clk	ext2_clk	10.000	-3.364	1.387
5	5.845	Din[2]	rData[2]	ext1_clk	fpga_clk	10.000	3.364	1.984
6	5.964	Din[3]	rData[3]	ext1_clk	fpga_clk	10.000	3.471	1.969
7	6.023	Din[1]	rData[1]	ext1_clk	fpga_clk	10.000	3.449	1.888
8	6.037	Din[0]	rData[0]	ext1_clk	fpga_clk	10.000	3.473	1.898

Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	3.512	rData[3]	Dout[3]	fpga_clk	ext2_clk	10.000	-1.471	1.517
2	3.530	rData[0]	Dout[0]	fpga_clk	ext2_clk	10.000	-1.473	1.497
3	3.564	rData[1]	Dout[1]	fpga_clk	ext2_clk	10.000	-1.449	1.487
4	3.749	rData[2]	Dout[2]	fpga_clk	ext2_clk	10.000	-1.364	1.387
5	5.210	Din[3]	rData[3]	ext1_clk	fpga_clk	10.000	1.471	2.723
6	5.274	Din[1]	rData[1]	ext1_clk	fpga_clk	10.000	1.449	2.637
7	5.288	Din[0]	rData[0]	ext1_clk	fpga_clk	10.000	1.473	2.647
8	5.467	Din[2]	rData[2]	ext1_clk	fpga_clk	10.000	1.364	2.362

图 6.1.11 实验七 (左) 与实验六 (右) 8 对节点的 setup 时序报告。

除了节点 rData~Dout 有相同的 setup 余量之外 ,似乎 Din~rData 的 setup 余量报告是不同。除此之外实验六与实验七的 clock skew 也有明显的差别 ,大约是 2ns ,不过先抛开 clock

skew 不讲 ... 为什么 Din~rData 会的建立余量会有所不同呢，差别大约是 0.5~0.8ns，其中关键就是 Data Delay。笔者尝试展开节点 Din[0]~rData[0]的详细来看看：

Path Summary Statistics Data Path Waveform							
Data Arrival Path							
Total	Incr	RF	Type	Fanout	Location	Element	
1	0.000	0.000				launch edge time	
2	1.000	1.000				clock path	
3	1.000	1.000	R			clock network delay	
4	5.500	4.500	R	iExt	1	PIN_90	Din[0]
5	7.398	1.898				data path	
6	8.015	0.515	RR	CELL	1	IOC_X28_Y0_N2	Din[0] combout
7	7.237	1.222	RR	IC	1	IOC_X25_Y0_N0	Bout[0] datain
8	7.398	0.161	RR	CELL	1	IOC_X25_Y0_N0	rData[0]
Data Required Path							
Total	Incr	RF	Type	Fanout	Location	Element	
1	10.000	10.000				latch edge time	
2	13.473	3.473				clock path	
3	13.473	3.473	R			clock network delay	
4	13.435	-0.038		uTsu	1	IOC_X25_Y0_N0	rData[0]

Path Summary Statistics Data Path Waveform							
Data Arrival Path							
Total	Incr	RF	Type	Fanout	Location	Element	
1	0.000	0.000				launch edge time	
2	0.000	0.000				clock path	
3	0.000	0.000	R			clock network delay	
4	3.500	3.500	R	iExt	1	PIN_90	Din[0]
5	6.147	2.647				data path	
6	4.015	0.515	RR	CELL	1	IOC_X28_Y0_N2	Din[0] combout
7	5.988	1.971	RR	IC	1	IOC_X25_Y0_N0	Bout[0] datain
8	6.147	0.161	RR	CELL	1	IOC_X25_Y0_N0	rData[0]
Data Required Path							
Total	Incr	RF	Type	Fanout	Location	Element	
1	10.000	10.000				latch edge time	
2	11.473	1.473				clock path	
3	11.473	1.473	R			clock network delay	
4	11.435	-0.038		uTsu	1	IOC_X25_Y0_N0	rData[0]

图 6.1.12 实验七（左）与实验六（右）节点 Din[0]~rData[0] 详细的 setup 时序报告。

6.1.12 左图中的 clock network delay 有 1ns 的延迟，这就是我们为 ext1_clk1 设置 1ns delay both 的结果，换之右图中的 clock network delay 则为 0ns。再接者左图 Din[0]的输入延迟是 4.5ns，而右图的 Din[0]是 3.5ns ... 这个信息告诉我们，实验七（左）不将 clock skew 计算在 input delay 中，而另用 set_clock_latency 命令告诉 TimeQuest ext1_clk 有 1ns 的延迟，换之实验六则不同。

继续看左图关键的 Dout[0]|datain 延迟是 1.222ns，右图则是 1.971，相差大约是 0.7ns，也就是这个原因，导致实验七（节点 rDin~rData）的 setup 余量比实验六的 setup 余量还要小的原因，亦即造就 Data Delay 的不同，至于为什么会多出 0.7ns 呢？笔者猜想应该是使用 set_clock_latency 的代价，又或者什么的 ... 真正的答案也只有 TimeQuest 自己知道吧！？

接着把焦点转向 Data Required Path 中，稍微注意左图的 clock path 延迟是 3.473ns，而右图的 clock path 是 1.473ns，两者相差了 2ns ... 这个信息告诉我们 1.473ns 是 fpga_clk 内部的路径延迟，换之相差的 2ns 是实验七用 set_clock_latency 告诉 TimeQuest 有关 fpga_clk 外部有 2ns 延迟的原因。

经过上述分析以后，我们可以得知实验七与实验六除了 Dout[0]|datain 有 0.7ns 的差别以外，其他结果经过计算以后都相互符合。说着说着我们将实验七与实验六的 hold 时序报告也来比较一下看看：

Command Info Summary of Paths							
Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1.882	Din[0]	rData[0]	ext1_clk	fpga_clk	0.000	3.473	1.888
1.876	Din[1]	rData[1]	ext1_clk	fpga_clk	0.000	3.449	1.888
1.935	Din[3]	rData[3]	ext1_clk	fpga_clk	0.000	3.471	1.969
2.069	Din[2]	rData[2]	ext1_clk	fpga_clk	0.000	3.364	1.984
3.251	rData[2]	Bout[2]	fpga_clk	ext2_clk	0.000	-3.364	1.387
3.436	rData[1]	Bout[1]	fpga_clk	ext2_clk	0.000	-3.449	1.487
3.470	rData[0]	Bout[0]	fpga_clk	ext2_clk	0.000	-3.473	1.497
3.488	rData[3]	Bout[3]	fpga_clk	ext2_clk	0.000	-3.471	1.517

Command Info Summary of Paths							
Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
2.447	Din[2]	rData[2]	ext1_clk	fpga_clk	0.000	1.364	2.362
2.611	Din[0]	rData[0]	ext1_clk	fpga_clk	0.000	1.473	2.647
2.625	Din[1]	rData[1]	ext1_clk	fpga_clk	0.000	1.449	2.637
2.689	Din[3]	rData[3]	ext1_clk	fpga_clk	0.000	1.471	2.723
3.251	rData[2]	Bout[2]	fpga_clk	ext2_clk	0.000	-1.364	1.387
3.436	rData[1]	Bout[1]	fpga_clk	ext2_clk	0.000	-1.449	1.487
3.470	rData[0]	Bout[0]	fpga_clk	ext2_clk	0.000	-1.473	1.497
3.488	rData[3]	Bout[3]	fpga_clk	ext2_clk	0.000	-1.471	1.517

图 6.1.13 实验七（左）与实验六（右）8 对节点点的 hold 时序报告。

从图 6.1.13 将实验七与实验六的 hold 余量比较一下，还是节点 Din~rData 在搞怪，用

狗眼一照又是 Data Delay 有问题，我们尝试展开节点 Din[0]~rData[0]的详细信息看看：

Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					launch edge time
2	1.000	1.000					clock path
3	1.000	1.000	R				clock network delay
4	3.500	2.500	R	iExt	1	PIN_90	Din[0]
5	5.398	1.898					data path
6	4.015	0.515	RR	CELL	1	IOC_X28_Y0_N2	Din[0] combout
7	5.237	1.222	RR	IC	1	IOC_X25_Y0_N0	Dout[0] datain
8	5.398	0.161	RR	CELL	1	IOC_X25_Y0_N0	rData[0]
Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					latch edge time
2	3.473	3.473					clock path
3	3.473	3.473	R				clock network delay
4	3.536	0.063		uTh	1	IOC_X25_Y0_N0	rData[0]

Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					launch edge time
2	0.000	0.000					clock path
3	0.000	0.000	R				clock network delay
4	1.500	1.500	R	iExt	1	PIN_90	Din[0]
5	4.147	2.647					data path
6	2.015	0.515	RR	CELL	1	IOC_X28_Y0_N2	Din[0] combout
7	3.986	1.971	RR	IC	1	IOC_X25_Y0_N0	Dout[0] datain
8	4.147	0.161	RR	CELL	1	IOC_X25_Y0_N0	rData[0]
Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					latch edge time
2	1.473	1.473					clock path
3	1.473	1.473	R				clock network delay
4	1.536	0.063		uTh	1	IOC_X25_Y0_N0	rData[0]

图 6.1.14 实验七（左）与实验六（右）节点 Din[0]~rData[0] 详细的 hold 时序报告。

图 6.1.14 左图中的 clock network delay 有 1ns ,换之右图的 clock network delay 是零值，这事情告诉我们，实验七用 set_clock_latency 告诉 TimeQuest ext1_clk 有 1ns 的延迟。接着注意左图的 Din[0] 输入延迟有 2.5ns，而右图的 Din[0] 输入延迟是 1.5ns，这种现象原来是实验七没有将 clock skew 加入 set input delay 公式里，而实验六则相反。

再者就是最可疑的 Dout[0]datain 延迟，左图是 1.222ns 而右图是 1.971ns，之间的相差有 0.7ns 左右，笔者猜这是有使用 set_clock_latency 与没有使用的差别吧？也就是这个 0.7ns 的差别，导致建立余量（节点 Din~rData）不相等。

接着请注意 Data Required Path 中的 clock path 延迟，左图是 3.473ns，右图是 1.473ns，很明显左图大右图 2ns，造就这个差别的原因绝对是实验七用 set_clock_latency 告诉 TimeQuest 那 fpga_clk 有 2ns 的延迟。至于导致两个实验的节点（Din~rData）的保持余量不同的凶手，估计就是那个充满阴谋的 Dout[0]datain ... 为什么它要犯罪呢？应该是吃 set_clock_latency 太多了吧？/

Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					launch edge time
2	3.473	3.473					clock path
3	3.473	3.473	R				clock network delay
4	4.970	1.497					data path
5	3.571	0.098	uTco	1	IOC_X25_Y0_N0	rData[0]	
6	4.970	1.399	RR	CELL	0	PIN_88	Dout[0]
Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	10.000	10.000					latch edge time
2	13.000	3.000					clock path
3	13.000	3.000	R				clock network delay
4	8.500	-4.500	R	oExt	0	PIN_88	Dout[0]

Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					launch edge time
2	1.473	1.473					clock path
3	1.473	1.473	R				clock network delay
4	2.970	1.497					data path
5	1.571	0.098	uTco	1	IOC_X25_Y0_N0	rData[0]	
6	2.970	1.399	RR	CELL	0	PIN_88	Dout[0]
Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	10.000	10.000					latch edge time
2	10.000	0.000					clock path
3	10.000	0.000	R				clock network delay
4	8.500	-3.500	R	oExt	0	PIN_88	Dout[0]

图 6.1.15 实验七（左）与实验六（右）节点 rData[0]~Dout[0] 详细的 setup 时序报告。

我们也顺便分析一下有关节点 rData[0]~Dout[0]的 setup 时序报告。首先注目一下 Data Arrival Path 中的 clock network delay，左图是 3.473ns，而右图是 1.473ns，相差是 2ns，其中的原因是实验七用 set_clock_latency 告诉 TimeQuest，fpga_clk 有 2ns 的缘故，换之实验六则没有。

再者是 Data Required Path 中的 clock network delay，左图是 3ns，右图则是 0ns，有 3ns 的区别，这是因为实验七用 set_clock_latency 告诉 TimeQuest，ext2_clk 有 3ns 的延迟，

反之实验六则没有。接下来就是 Dout[0]的输出延迟，左图是 4.5ns，右图是 3.5ns，其中的 1ns 相差是因为实验六有将 clock skew 计算在 set output delay 公式中，换之实验七则没有。

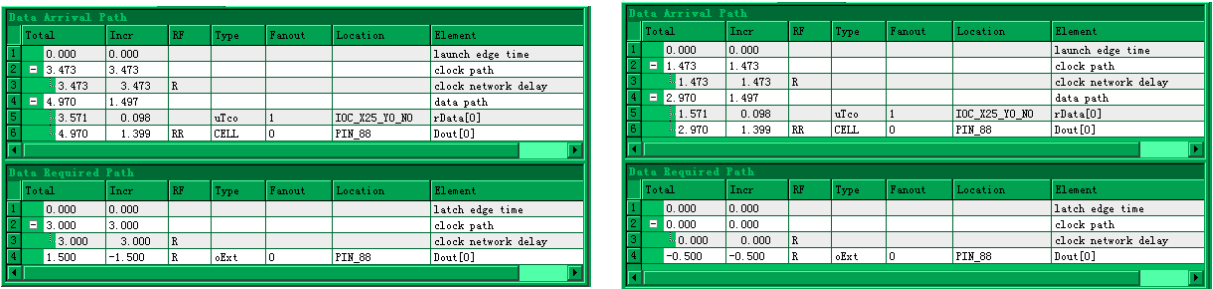


图 6.1.16 实验七（左）与实验六（右）节点 rData[0]~Dout[0] 详细的 hold 时序报告。

图 6.1.16 是节点 rData[0]~Dout[0]的详细 hold 时序报告，首先把注意力焦距在 Data Arrival Path 的 clock network delay，左图是 3.473ns，右图是 1.473ns，它们有 2ns 的相差，这是因为实验七用 set_clock_latency 告诉 TimeQuest 该 fpga_clk 有 2ns 的延迟，换之实验六则没有。

接着从 Data Required Path 中可以得知，左图的 clock network delay 有 3ns 的延迟，而右图是零值，这是因为实验七用 set_clock_latency 告诉 TimeQuest 该 ext2_clk 有 3ns 的延迟。至于 Dout[0]的输出延迟，左图是 1.5ns，而右图是 0.5ns，其中的 1ns 差别是实验六将 clock skew 计算在 set output delay 公式中，换之实验七则没有。

从这个实验中我们可以这样结论：

在计算 set_input_delay 与 set_output_delay 的时候，我们可以除掉 clock skew，取而代之用 set_clock_latency 约束命令告诉 TimeQuest 关于外部模型中到底什么时钟信号，然而又有多少的延迟。其中时钟信号的产生源没有抖动，又或者说抖动的 early 与 late 值 (both) 作为前提。此外，透过 set_clock_latency 约束命令，在 Data Arrival Path 与 Data Required Path 的信息中 clock network delay 也会显示对应的延迟。

6.2 物理时钟②

让我们继续时间抖动的问题吧 ... 在上一个章节里我们使用了 `set_clock_latency` 约束命令告诉 TimeQuest 有关外部时钟的各种延迟信息,然而这些时钟虽然有延迟但是没有发生抖动,结果造就 `delay both`。

笔者在前面讲过,时间抖动最让人头疼的问题既是时钟信号不像数据信号有多位宽, `delay min` 或者 `delay max` 都有对应的数据位。而抖动是造就单个时钟信号同时拥有 `delay min` 与 `delay max`,单是源寄存器的时钟信号抖来抖去已经够烦了,同学们请想象一下 ... 如果源时钟寄存器与目的寄存器的时钟信号同时抖来抖去,会是什么样的情形呢?

不过幸运的是, `set_clock_latency` 约束命令可以简化这个难题 ...

实验八 时钟抖动与 `set_clock_latency`

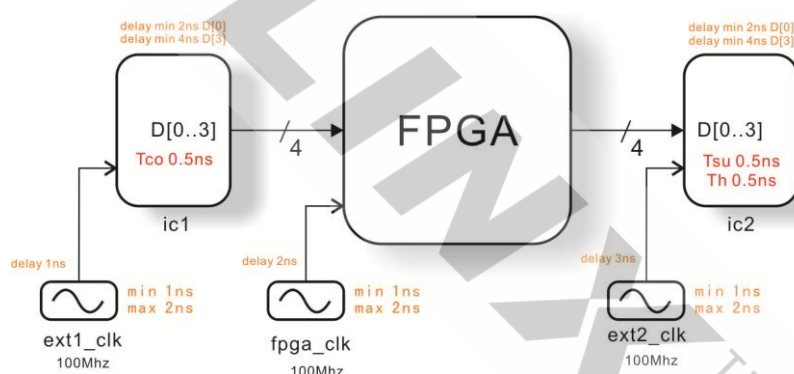


图 6.2.1 fpga_clk 与 ext_clk 时钟产生源发生抖动。

首先我们必须知道,时钟抖动的 `max` 对应 `late` 而 `min` 对应 `early`,此外 `max` 针对启动沿,也针对建立关系,`min` 针对下一个启动沿,也针对保持关系。从图 6.2.1 中我们可以知道 `ext1_clk`, `fpga_clk` 与 `ext2_clk` 的产生源都有 `min 1ns` 与 `max 2ns` 的抖动,时钟抖动再加上原本的路径延迟:

```
ext1_clk delay min : 1ns + 1ns = 2ns
ext1_clk delay max : 1ns + 2ns = 3ns
```

```
fpga_clk delay min : 2ns + 1ns = 3ns
fpga_clk delay max : 2ns + 2ns = 4ns
```

```
ext2_clk delay min : 3ns + 1ns = 4ns
ext2_clk delay max : 3ns + 2ns = 5ns
```

接着再计算一下不包括 clock skew 的 input/output delay :

```
input max      = <ext2fpga delay max> + ext_Tco
               = 4ns + 0.5
               = 4.5ns

input min      = <fpga2ext delay min> + ext_Tco ( 或者 ext_minTco )
               = 2ns + 0.5ns
               = 2.5ns
```

```
output max     = <fpga2ext delay max> + ext_Tsu
               = 4ns + 0.5ns
               = 4.5ns

output min     = < fpga2ext delay min > - ext_Th
               = 2ns - 0.5ns
               = 1.5ns
```

接着再打开实验八，建立同名的 sdc 文件，然后将下列代码添加进去，而上述代码的意义基本上与实验七一样，不要忘了网表是 create_timing_netlist -model fast。

```
#####
# Create Clock
#####

create_clock -name {fpga_clk} -period 10.000 -waveform { 0.000 5.000 } [get_ports {CLK}]
create_clock -name {ext1_clk} -period 10.000 -waveform { 0.000 5.000 }
create_clock -name {ext2_clk} -period 10.000 -waveform { 0.000 5.000 }

#####
# Set Input Delay
#####

set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[0]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[0]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[1]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[1]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[2]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[2]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[3]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[3]}]

#####
```

Set Output Delay

```
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[3]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[3]}]
```

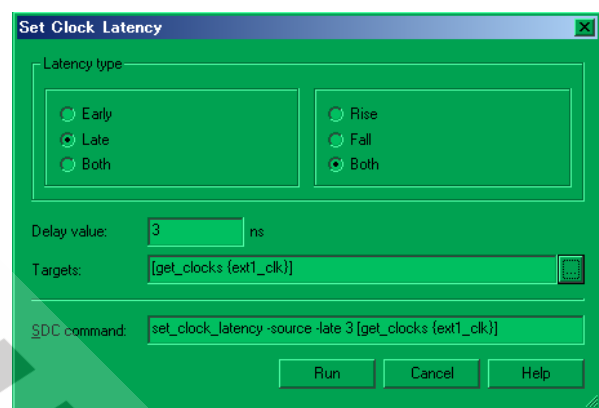
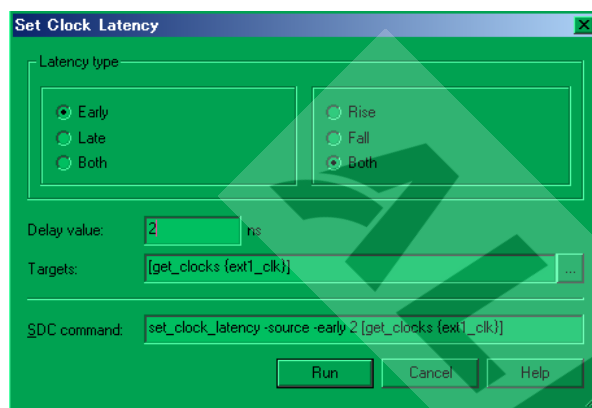


图 6.2.2 ext1_clk 输入 2ns 延迟的 early 与 3ns 延迟的 late。

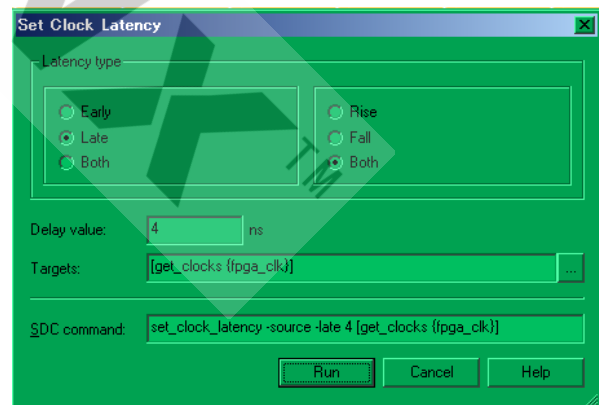
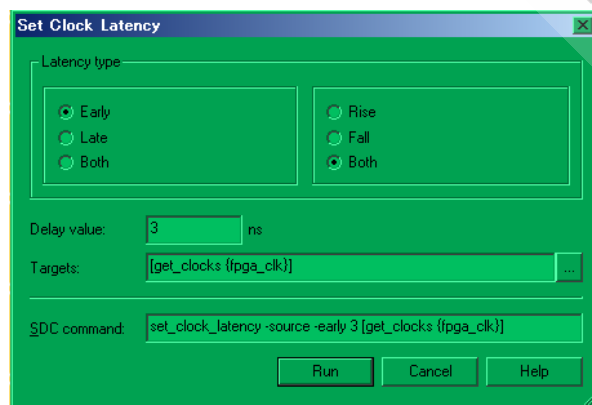


图 6.2.3 fpga_clk 输入 3ns 延迟的 early 与 4ns 延迟的 late。

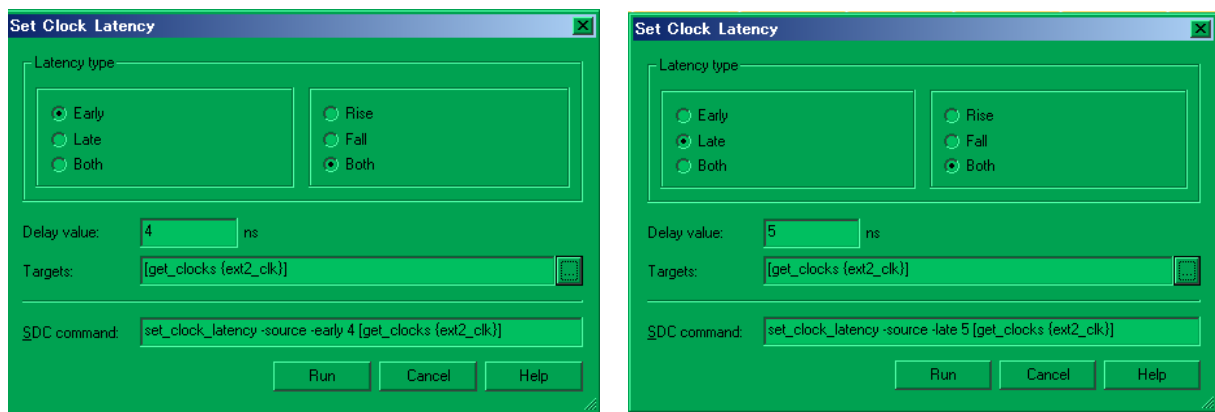


图 6.2.4 ext2_clk 输入 4ns 延迟的 early 与 5ns 延迟的 late。

接着打开 set clock latency 界面，按图 6.2.2~4 的步骤输入相对应的约束。

```
#####
# Set Clock Latency
#####

set_clock_latency -source -late 4.000 [get_clocks {fpga_clk}]
set_clock_latency -source -early 3.000 [get_clocks {fpga_clk}]
set_clock_latency -source -late 3.000 [get_clocks {ext1_clk}]
set_clock_latency -source -early 2.000 [get_clocks {ext1_clk}]
set_clock_latency -source -late 5.000 [get_clocks {ext2_clk}]
set_clock_latency -source -early 4.000 [get_clocks {ext2_clk}]
```

完后该 sdc 文件会自动添加上述的代码

事实上，透过 set_clock_latency 将各个外部时钟信号的抖动信息告诉 TimeQuest 是一种吃快餐，贪便利的方法 ... 因为我们对整个时序分析的过程是 100%的瞎子，完全不知道里边发生什么事。虽然这种做法没有不妥，还不如说非常好用，可是对于笔者这种喜欢吃妈妈料理的孩子而言，心里总是不舒服 ...

笔者编辑这本笔记的初衷不是解释 TimeQuest 的用法，而是思考它，探讨它 ... 打从骨子想知道它是什么样的一个东西。为此，笔者需要更进一步思考时钟抖动如何影响 clock skew，然而这种情况下的 clock skew 又会得到怎样的等价图（建立关系与保持关系）。

6.3 物理时钟③

时钟抖动之所以让人觉得忧郁，因为 1 个时钟信号抖动就会产生 2 个可能性的时序，2 个时钟信号同时抖动，就会产生 4 个可能性的时序，恰恰好 TimeQuest 模型是由两个寄存器，亦即两个时钟信号组成。所以说，用一般的方法理解时钟抖动会耗死自己不偿命的，不过 TimeQuest 有用例外的“手段”去解决它 ... 至于是什么样的“手段”呢？同学们一定很好奇吧！？

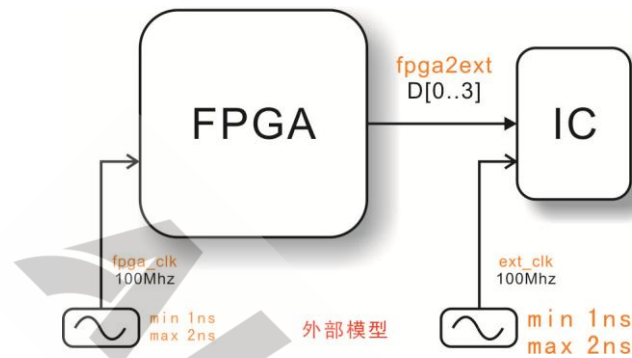


图 6.3.1 时钟抖动的外部模型。

假设我们有一个 fpga2ic 的外部模型，如图 6.3.1 所示。同是 100Mhz 频率的 fpga_clk 与 ext_clk 都有 min 1ns 与 max 2ns 的抖动延迟。我们先假设 fpga2ext 的 D[0] 有 2ns 的延迟，然后 fpga 向 ic 发送 10ns 周期的 D[0] 数据 ...

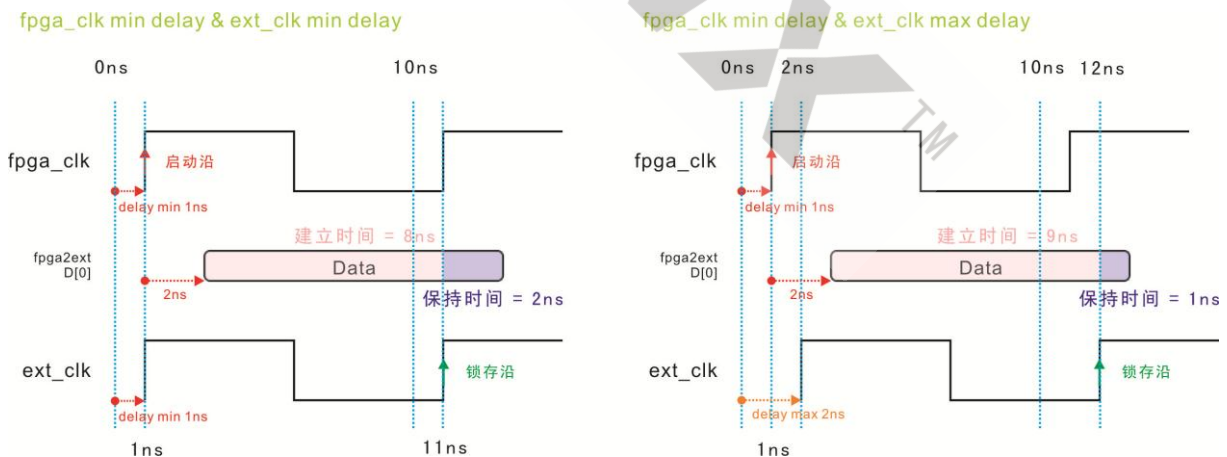


图 6.3.2 fpga_clk min delay & ext_clk min delay（左）与 fpga_clk min delay & ext_clk max delay（右）的第 1, 2 种可能性。

如图 6.3.2 所示，首先会产生前 2 种可能性，亦即 fpga_clk min & ext_clk min 与 fpga_clk min & ext_clk max，然而左图有 8ns 的建立时间与 2ns 的保持时间，至于右图有 9ns 的建立时间与 1ns 的保持时间。

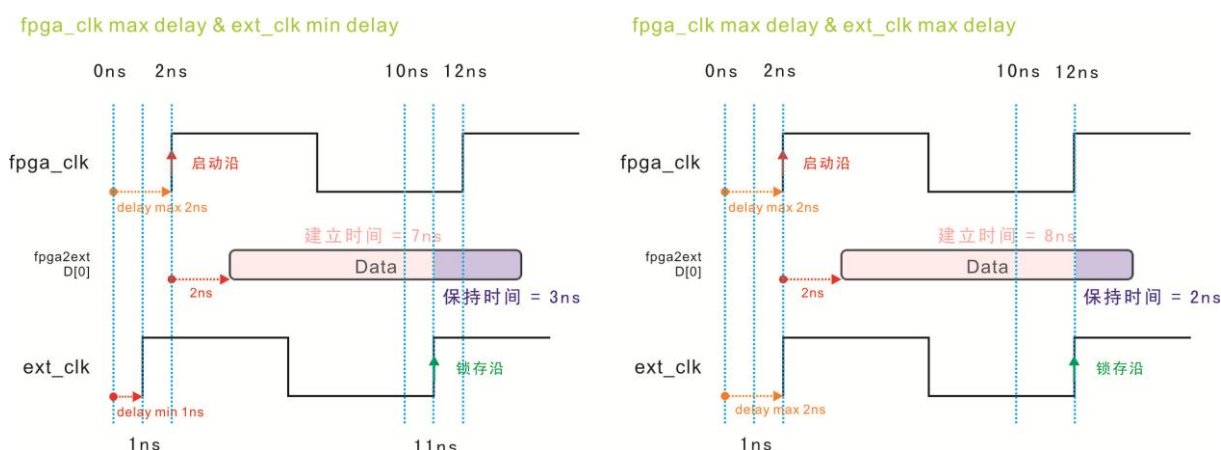


图 6.3.3 fpga_clk max delay & ext_clk min delay（左）与 fpga_clk max delay & ext_clk max delay（右）的第 3、4 种可能性。

如图 6.3.3 所示，接着会产生后 2 两种可能性，亦即 fpga_clk max & ext_clk min 与 fpga_clk max & ext_clk max，而左图有 7ns 的建立时间与 3ns 的保持时间，右图则有 8ns 的建立时间与 2ns 的保持时间。

图 6.3.1 都有可能出现这 4 种时序，那么我们又该选择哪一种可能性呢？无论怎么选择，哪一种可能性都是真确的，同学们是不是觉得很忧郁呢？呵呵，笔者也是 ... 不过我们可以这样想：“哪一种可能性会产生最小的建立时间？又哪一种可能性会产生最小的保持时间？”

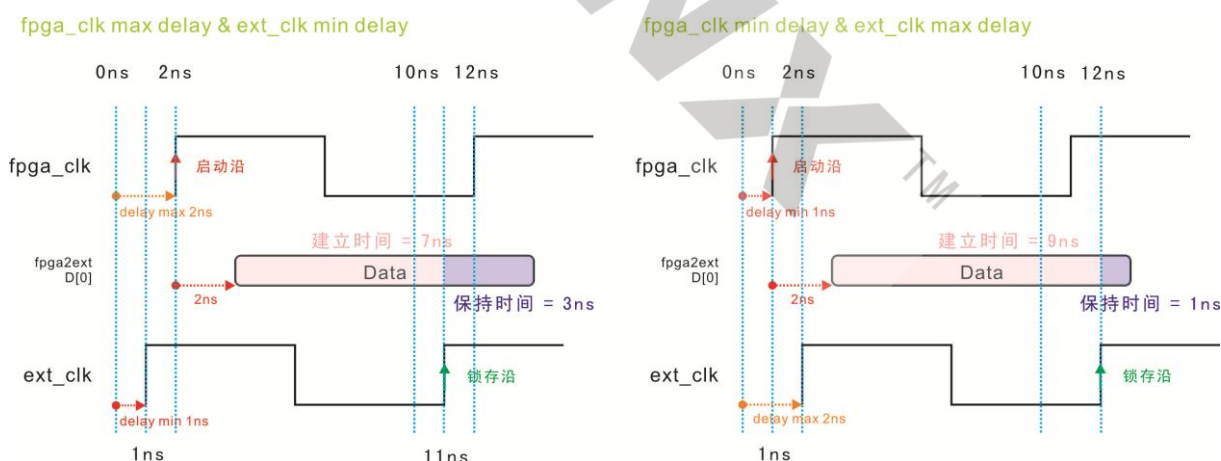


图 6.3.4 fpga_clk max delay & ext_clk min delay（左）与 fpga_clk min delay & ext_clk max delay（右）的可能性。

不用想答案是非常明显的，亦即 fpga_clk max & ext_clk min 会产生最小的建立时间，简称 S.L.D.E 可能性（source_reg_clk late, destination_reg_clk early）。换之，fpga_clk min & ext_clk max 会产生最小的保持时间，简称 S.E.D.L 可能性（source_reg_clk early, destination_reg_clk late）。然后我们可以无视余下的可能性。

接着让我们来计算一下 slde 与 sedl 的 clock skew：

```

clock skew      = < destination reg clk delay> - < source reg clk delay >
SLDE clock skew = < destination reg clk early/min delay > - < source reg clk late/max delay >
                = < ext_clk min delay > - < fpga_clk max delay >
                = 1ns - 2ns
                = -1ns

```

```

clock skew      = < destination reg clk delay> - < source reg clk delay >
SEDL clock skew = < destination reg clk late/max delay > - < source reg clk early/min delay >
                = < ext_clk max delay > - < fpga_clk min delay >
                = 2ns - 1ns
                = 1ns

```

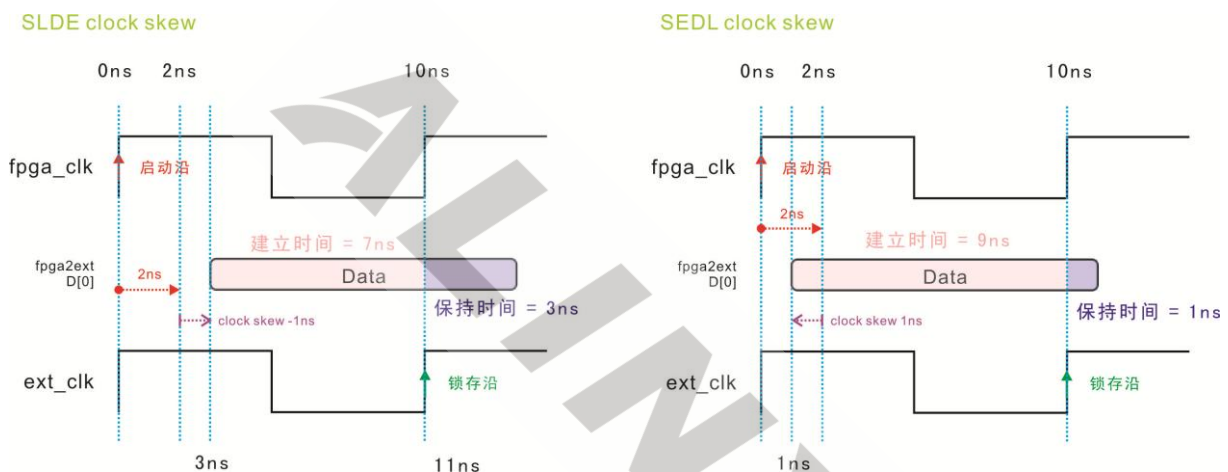


图 6.3.5 SLDE clock skew（左）与 SEDL clock skew 对数据的影响。

我们知道 clock skew 的定义就是将时钟差反映在数据的身上，然而 SLDE 时间差是 -1ns，则 SEDL 时间差是 1ns。如图 6.3.5 的左图所示，原本起跑线是 2ns 的数据，受到 -1ns 时间差的影响，然后移动到 3ns 的位置，最终得到 7ns 的建立时间与 3ns 的保持时间。

反之，左图的数据，原本也是在 2ns 的起跑线，不过受 1ns 时间差的影响，然后移动到 1ns 的位置，结果造就 9ns 的建立时间与 1ns 的保持时间。同学们稍微比较一下 图 6.3.4 与图 6.3.5 的建立时间与保持时间看看，是不是一样？

SLDE clock skew & SEDL clock skew

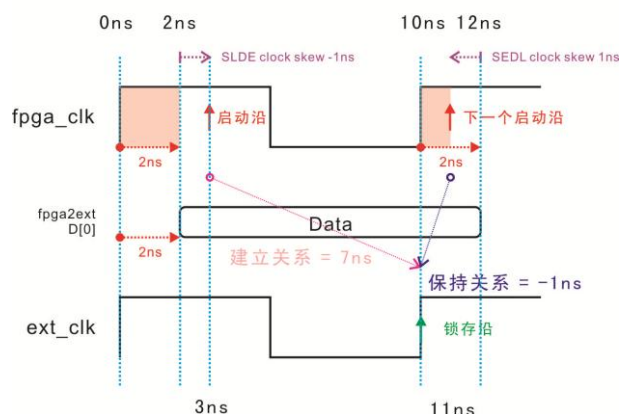


图 6.3.6 SLDE clock skew 与 SEDL clock skew，建立关系与保持关系的等价图。

此外我们也可以说 **SLDE 时间差针对建立时间，针对建立关系，针对启动沿**；换之 **SEDL 时间差针对保持时间，针对保持关系，针对下一个启动沿**。如图 6.3.6 所示，数据 D[0] 有 2ns 的延迟 ... 原本位置在 2ns 的启动沿，不小心被 -1ns 的 SLDE 时间差推了一把，然后仆街在 3ns 的位置，最终导致 7ns 的建立关系。换之，原本位置在 12ns 的下一个启动沿，不小心撞到 1ns 的 SEDL 时间差，然后跌在 11ns 的位置，最终导致 -1ns 的保持关系。

fpga_clk max delay & ext_clk min delay

fpga_clk min delay & ext_clk max delay

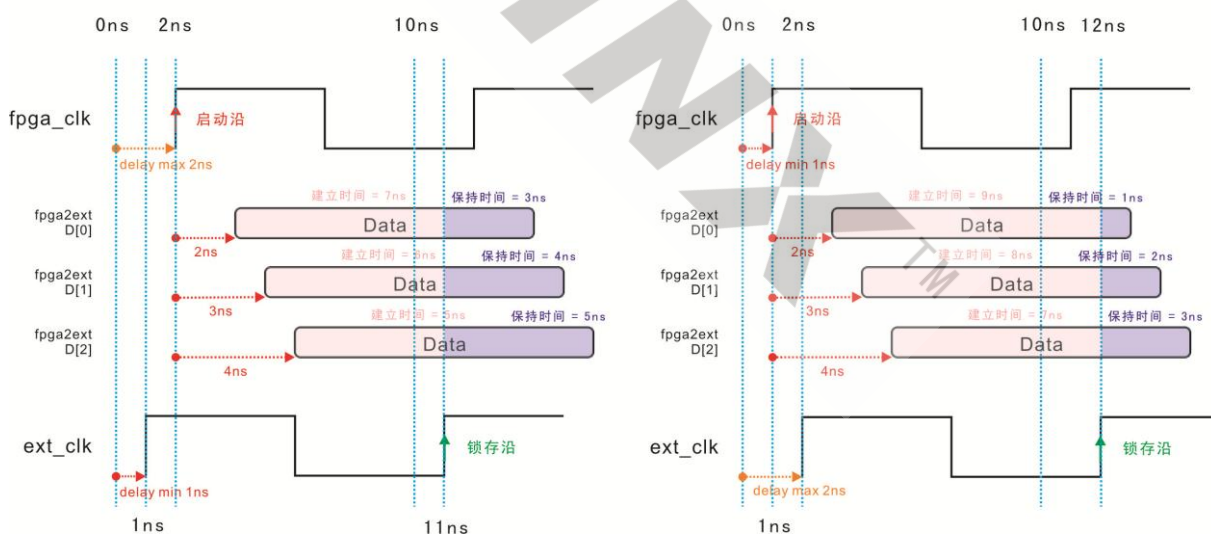


图 6.3.7 多位宽，fpga_clk max delay & ext_clk min delay（左）与 fpga_clk min delay & ext_clk max delay（右）的可能性。

图 6.3.7 是多位宽数据，其中 D[0] 有 2ns 的延迟，D[1] 有 3ns 的延迟，D[2] 有 4ns 的延迟 ... 接着 fpga_clk 与 ext_clk 同时抖动会产生 4 种可能性的时序，不过根据上述的理解，fpga_clk max & ext_clk min 可能性数据 D[2] 会有最小的建立时间，如左图所示是 5ns；反之 fpga_clk min & ext_clk max 可能性数据 D[1] 会有最小的保持时间，如右图所示是 1ns。

fpga_clk max & ext_clk min 可能性又叫 SLDE，其中的时间差称为 SLDE clock skew；换之，fpga_clk min & ext_clk max 可能性又叫 SEDL，其中的时间差称为 SEDL clock skew。根据图 6.3.7 中信息，SLDE 时间差是 -1ns，而 SEDL 时间差有 1ns。

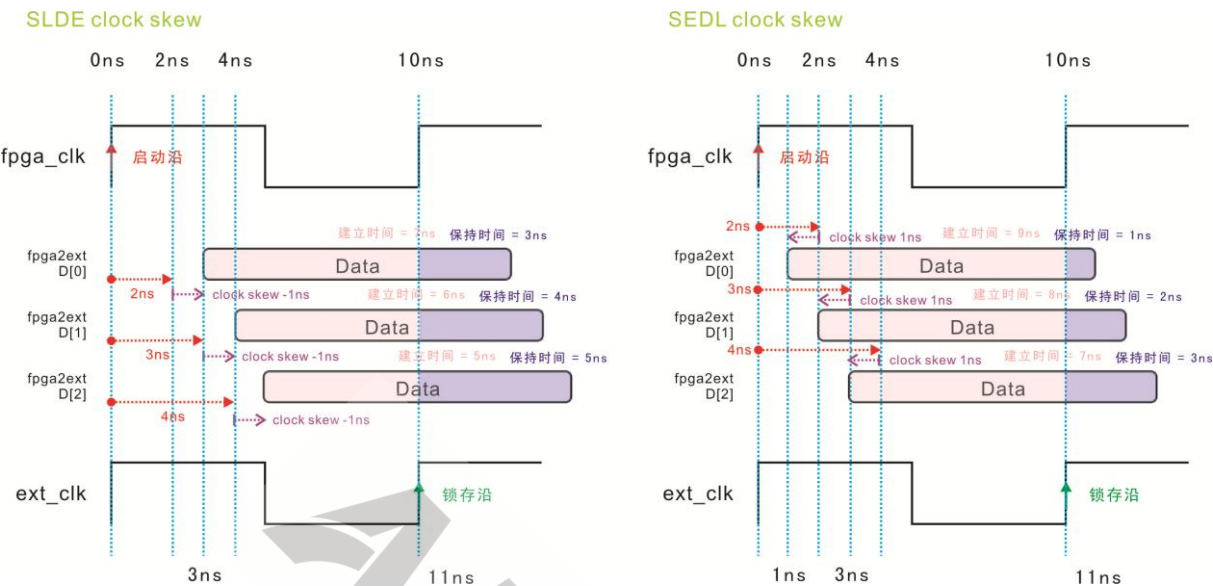


图 6.3.8 SLDE 时间差（左），SEDL 时间差（右）。

图 6.3.7 的数据经过各种可能性的时间差相互作用以后，结果会是如图 6.3.8 所示。左图中的 fpga_clk 与 ext_clk 不再跳来跳去，至于每一位数据 D 受 -1ns 的 SLDE 时间差影响以后，都向右移动 1ns；换之，右图的每一位数据 D 受 1ns 的 SEDL 时间差影响以后，都向左移动 1ns。不过不管怎么样，图 6.3.7 与图 6.3.8 的每一位数据 D 都有同样的建立时间与保持时间。

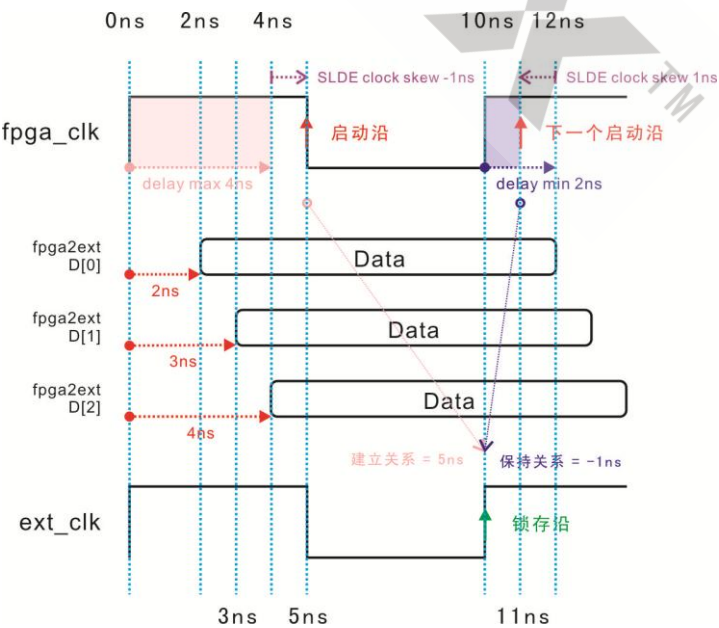


图 6.3.9 多位宽，SLDE clock skew 与 SEDL clock skew，建立关系与保持关系的等价图。

同样我们知道 SLDE 时间差 针对启动沿, 针对建立时间, 针对建立关系; 而 SEDL 时间差则针对下一个启动沿, 针对保持时间, 针对保持关系。此外, 我们也知道数据的 delay max 也针对启动沿, 数据的 delay min 也针对下一个启动沿。

如图 6.3.9 所示, 原本已经受数据 delay max 延迟的启动沿, 再加上-1ns SLDE 时间差的影响, 最终落在 5ns 的位置, 从而取得 5ns 的建立关系。至于下一个启动沿它先被 2ns 的 delay min 影响, 然后又受到 1ns SEDL 时间差的影响, 下一个启动沿最终落在 11ns 的位置, 从而取得 -1ns 的保持关系。

我们知道数据的 delay max 影响启动沿, 此外 SLDE clock skew 也是影响启动沿, 因此 output max 可以这样更动:

```
output max    = < fpga2ext delay max > - [< dest. reg clock delay min > - < source reg clock delay max >]
              = < fpga2ext delay max > - [ < ext_clk delay min > - < fpga_clk delay max > ]
              = < fpga2ext delay max > - < SLDE clock skew >
              = < fpga2ext delay max > - < SLDE clock skew > + ext_Tsu
```

至于数据的 delay min 影响下一个启动沿, 而 SEDL clock skew 同是影响下一个启动沿, 因此 output min 可以这样更动:

```
output min    = < fpga2ext delay min > - [< dest. reg clock delay max > - < source reg clock delay min >]
              = < fpga2ext delay min > - [ < ext_clk delay max > - < fpga_clk delay min > ]
              = < fpga2ext delay min > - < SEDL clock skew >
              = < fpga2ext delay min > - < SEDL clock skew > - ext_Th
```

接下来, 我们让我们来理解一下, 如果外部模型是 ic2fpga 的情况, 在两个时钟信号同时抖动的话, 到底会发生什么不一样的情形呢?

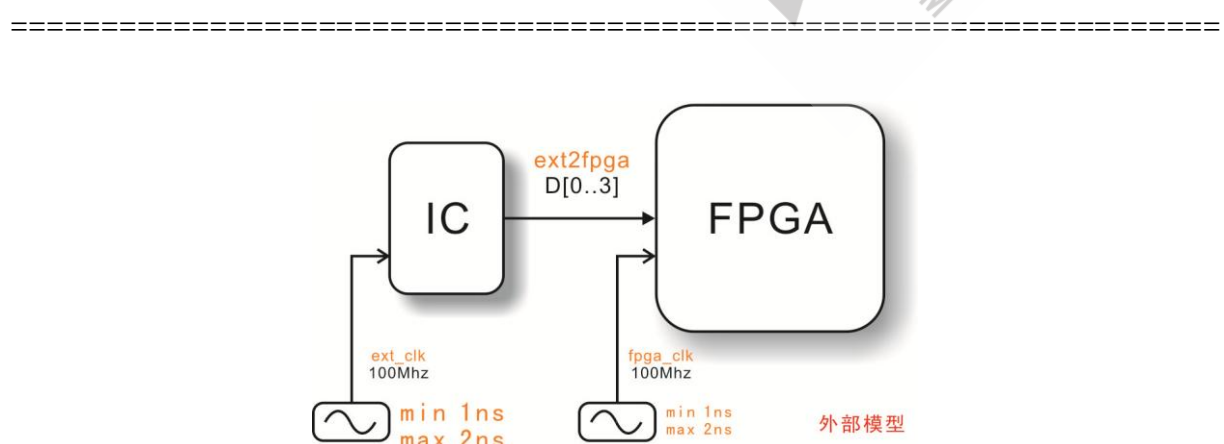


图 3.6.10 ic2fpga 的外部模型。

图 3.6.10 是 ic2fpga 的外部模型, 其中 ext_clk 和 fpga_clk 皆为 100Mhz, 两者的产生源都有 min 1ns 与 max 2ns 的抖动。然后我们也假设一下 D[0]有 2ns 延迟, D[1]有 3ns,

D[2]有 4ns 延迟。

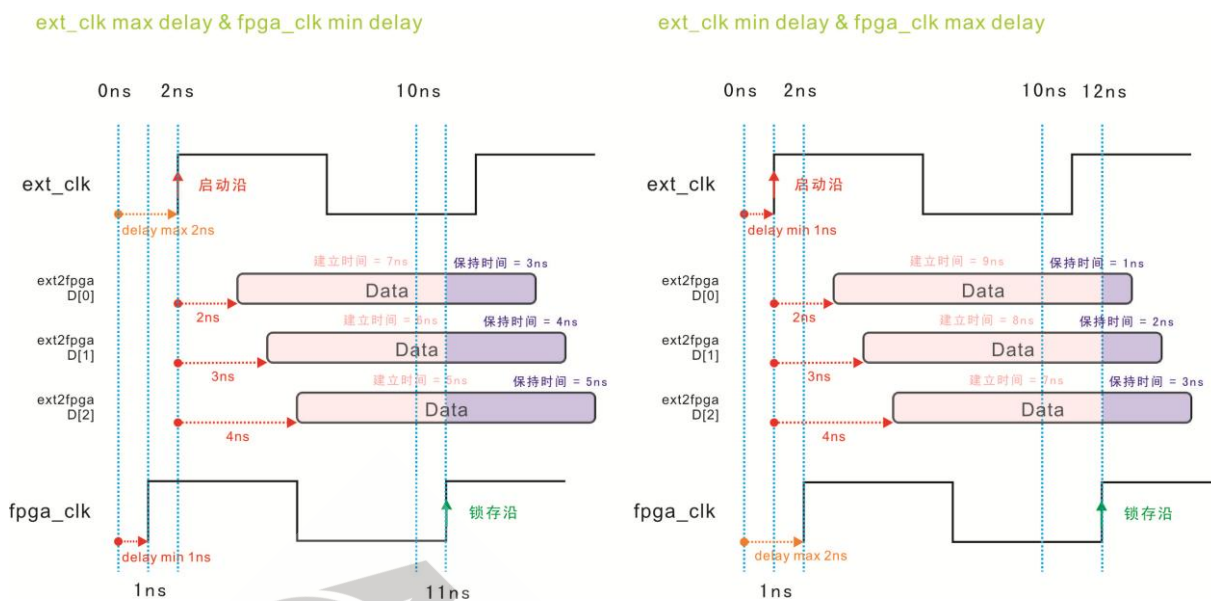


图 6.3.11 多位宽，ext_clk max delay & fpga_clk min delay（左）与 ext_clk min delay & fpga_clk max delay（右）的可能性。

同学们有没有觉得图 6.3.11 很眼熟，没错它是图 6.3.7 的换皮，只是稍微调动一下 fpga 与 ic 之间的角色位置。根据原理，2 个时钟信号的抖动会产生 4 个可能性的时序图，不过从图 6.3.11 的左图中可以发现 ext_clk max delay & fpga_clk min delay 可能性的数据 D[2]拥有最小的建立时间；换之，ext_clk min delay & fpga_clk max delay 可能性的数据 D[0]拥有最小的保持时间。

其中 SLDE clock skew 的取值是 (ext_clk max delay & fpga_clk min delay 的时间差)：

```
clock skew      = < destination reg clk delay > - < source reg clk delay >
SLDE clock skew = < destination reg clk early/min delay > - < source reg clk late/max delay >
                = < fpga_clk min delay > - < ext_clk max delay >
                = 1ns - 2ns
                = -1ns
```

而 SEDL clock skew 的取值是 (ext_clk min delay & fpga_clk max delay 的时间差)：

```
clock skew      = < destination reg clk delay > - < source reg clk delay >
SEDL clock skew = < destination reg clk late/max delay > - < source reg clk early/min delay >
                = < fpga_clk max delay > - < ext_clk min delay >
                = 2ns - 1ns
                = 1ns
```

然后将 SLDE clock skew 与 SEDL clock skew 加入图 6.3.11。

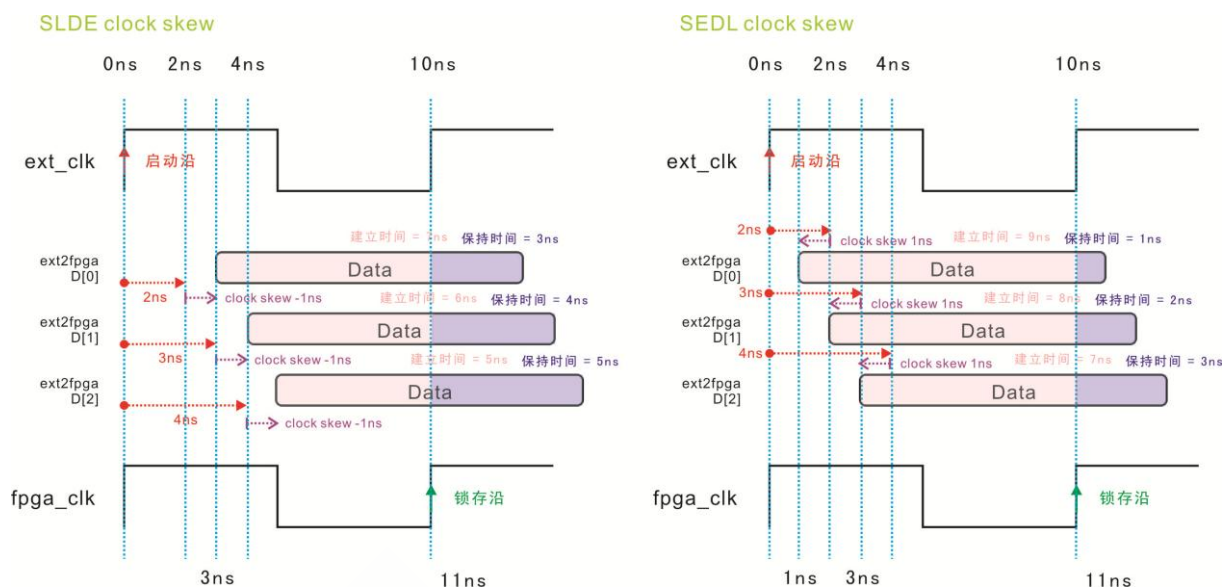


图 6.3.12 SLDE 时间差（左），SEDL 时间差（右）。

图 6.3.12 的 ext_clk 与 fpga_clk 不再跳来跳去，换之左图中的数据 D，皆被 -1ns 的 SLDE 时间差向右推一把。而右图中的数据 D，皆被 1ns 的 SEDL 时间差向左拉一把。完后可以发现图 6.3.11 与图 6.3.12 的各个数据都有相同的建立时间与保持时间。

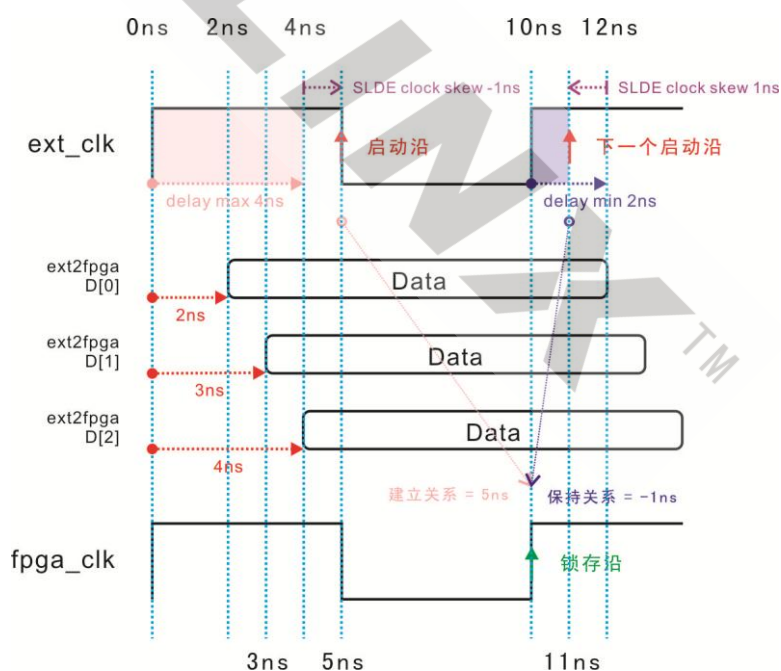


图 6.3.14 多位宽，SLDE clock skew 与 SEDL clock skew，建立关系与保持关系的等价图。

至于图 6.3.14 是建立关系与保持关系的等价图 ... 其中启动沿原先受 4ns 数据 delay max 的延迟影响，然后又被 -1ns 的 SLDE 时间差推一把，结果落在 6ns 的位置，最终取得 5ns 的建立关系。至于下一个启动沿，原先也是被 2ns 数据 delay min 的影响，接着它不小心被 1ns 的 SLDE 时间差绊了一跤，最终落在 11ns 的位置，结果也造就 -1ns 的保持关系。

我们知道数据的 delay max 影响启动沿，此外 SLDE clock skew 也是影响启动沿，因此 input max 可以这样更动：

```
input max      = < ext2fpga delay max > - [ < dest. reg clock delay min > - < source reg clock delay max > ]
               = < ext2fpga delay max > - [ < fpga_clk delay min > - < ext_clk delay max > ]
               = < ext2fpga delay max > - < SLDE clock skew >
               = < ext2fpga delay max > - < SLDE clock skew > + ext_Tco
```

至于数据的 delay min 影响下一个启动沿，而 SEDL clock skew 同是影响下一个启动沿，因此 input min 可以这样更动：

```
input min      = < ext2fpga delay min > - [ < dest. reg clock delay max > - < source reg clock delay min > ]
               = < ext2fpga delay min > - [ < fpga_clk delay max > - < ext_clk delay min > ]
               = < ext2fpga delay min > - < SEDL clock skew >
               = < ext2fpga delay min > - < SEDL clock skew > + ext_Tco
               //等价//
               = < ext2fpga delay min > - < SEDL clock skew > + ext_minTco
```

当我们成功推导出加入时间抖动的 input/output min/max 公式以后，我们可以用 set_input_delay 与 set_output_delay 约束命令，不仅将数据的延迟，还有寄存器特性，甚至是抖动的时间差，包裹成“外包资料”... 而不是用 set_clock_latency 约束命令告诉 TimeQuest 有关时间抖动的信息。

实验九 时钟抖动与 input/output 约束命令

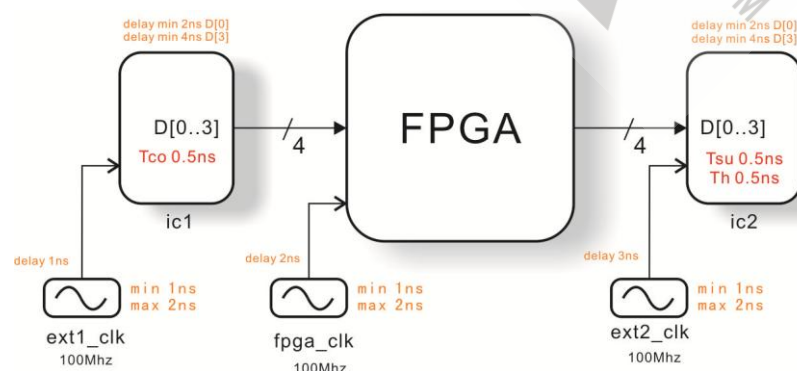


图 6.3.15 假想实验的外部模型。

图 6.3.15 的假想实验基本上和实验八一样，不过这一回我们不再使用 set_clock_latency 将时钟抖动的信息告诉 TimeQuest，而是直接透过 input/output 公式将时钟抖动包裹在一次成为外部资料。首先从图 6.3.15 取出各种延迟信息：

```

ext1_clk delay min : 1ns + 1ns = 2ns
ext1_clk delay max : 1ns + 2ns = 3ns

fpga_clk delay min : 2ns + 1ns = 3ns
fpga_clk delay max : 2ns + 2ns = 4ns

ext2_clk delay min : 3ns + 1ns = 4ns
ext2_clk delay max : 3ns + 2ns = 5ns

D[0..3] delay min : 2ns
D[0..3] delay max : 4ns

```

先计算 input min/max 的结果：

```

input max    = < ext2fpga delay max > - [ < dest. reg clock delay min > - < source reg clock delay max > ]
              = < ext2fpga delay max > - [ < fpga_clk delay min > - < ext1_clk delay max > ]
              = < ext2fpga delay max > - < SLDE clock skew >
              = < ext2fpga delay max > - < SLDE clock skew > + ext_Tco
              = 4ns - < 3ns - 3ns > + 0.5ns
              = 4.5ns

```

```

input min    = < ext2fpga delay min > - [ < dest. reg clock delay max > - < source reg clock delay min > ]
              = < ext2fpga delay min > - [ < fpga_clk delay max > - < ext1_clk delay min > ]
              = < ext2fpga delay min > - < SEDL clock skew >
              = < ext2fpga delay min > - < SEDL clock skew > + ext_Tco
              = 2ns - < 4ns - 2ns > + 0.5ns
              = 0.5ns

```

接着计算 output min/max 的结果：

```

output max    = < fpga2ext delay max > - [ < dest. reg clock delay min > - < source reg clock delay max > ]
              = < fpga2ext delay max > - [ < ext2_clk delay min > - < fpga_clk delay max > ]
              = < fpga2ext delay max > - < SLDE clock skew >
              = < fpga2ext delay max > - < SLDE clock skew > + ext_Tsu
              = 4ns - < 4ns - 4ns > + 0.5ns
              = 4.5ns

```

```

output min    = < fpga2ext delay min > - [ < dest. reg clock delay max > - < source reg clock delay min > ]
              = < fpga2ext delay min > - [ < ext2_clk delay max > - < fpga_clk delay min > ]
              = < fpga2ext delay min > - < SEDL clock skew >
              = < fpga2ext delay min > - < SEDL clock skew > - ext_Th
              = 2ns - < 5ns - 3ns > - 0.5ns
              = -0.5ns

```


然后，打开实验九然后建立同名的 sdc 文件，接着将下列的代码拷贝进去：

```
#####  
# Create Clock  
#####  
  
create_clock -name {fpga_clk} -period 10.000 -waveform { 0.000 5.000 } [get_ports {CLK}]  
create_clock -name {ext1_clk} -period 10.000 -waveform { 0.000 5.000 }  
create_clock -name {ext2_clk} -period 10.000 -waveform { 0.000 5.000 }  
  
#####  
# Set Input Delay  
#####  
  
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[0]}]  
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 0.500 [get_ports {Din[0]}]  
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[1]}]  
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 0.500 [get_ports {Din[1]}]  
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[2]}]  
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 0.500 [get_ports {Din[2]}]  
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[3]}]  
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 0.500 [get_ports {Din[3]}]  
  
#####  
# Set Output Delay  
#####  
  
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[0]}]  
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] -0.500 [get_ports {Dout[0]}]  
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[1]}]  
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] -0.500 [get_ports {Dout[1]}]  
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[2]}]  
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] -0.500 [get_ports {Dout[2]}]  
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[3]}]  
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] -0.500 [get_ports {Dout[3]}]
```

代码的内容和实验八差不多，不同的只是少了 set_clock_latency 约束命令，换之就是 set_input_delay 与 set_output_delay 被更改了。如果同学不喜欢直接粘贴代码，也可以透过 TimeQuest 手动输入相关的内容。

接下来打开 TimeQuest，然后手动建立网表 create_timing_netlist -model fast，再者就是

管理的双击 Read SDC File , 双击 Update Timing Netlist ... 完后举例所有节点的 setup 时序报告与 hold 时序报告。

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	2.512	rData[3]	Dout[3]	fpga_clk	ext2_clk	10.000	-1.471	1.517
2	2.530	rData[0]	Dout[0]	fpga_clk	ext2_clk	10.000	-1.473	1.497
3	2.564	rData[1]	Dout[1]	fpga_clk	ext2_clk	10.000	-1.449	1.487
4	2.749	rData[2]	Dout[2]	fpga_clk	ext2_clk	10.000	-1.364	1.387
5	4.210	Din[3]	rData[3]	ext1_clk	fpga_clk	10.000	1.471	2.723
6	4.274	Din[1]	rData[1]	ext1_clk	fpga_clk	10.000	1.449	2.637
7	4.288	Din[0]	rData[0]	ext1_clk	fpga_clk	10.000	1.473	2.647
8	4.467	Din[2]	rData[2]	ext1_clk	fpga_clk	10.000	1.364	2.362

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	2.512	rData[3]	Dout[3]	fpga_clk	ext2_clk	10.000	-5.471	1.517
2	2.530	rData[0]	Dout[0]	fpga_clk	ext2_clk	10.000	-5.473	1.497
3	2.564	rData[1]	Dout[1]	fpga_clk	ext2_clk	10.000	-5.449	1.487
4	2.749	rData[2]	Dout[2]	fpga_clk	ext2_clk	10.000	-5.364	1.387
5	4.845	Din[2]	rData[2]	ext1_clk	fpga_clk	10.000	4.364	1.984
6	4.964	Din[3]	rData[3]	ext1_clk	fpga_clk	10.000	4.471	1.969
7	5.023	Din[1]	rData[1]	ext1_clk	fpga_clk	10.000	4.449	1.888
8	5.037	Din[0]	rData[0]	ext1_clk	fpga_clk	10.000	4.473	1.898

图 6.3.16 实验九（左）与实验八（右）的 setup 时序报告。

图 6.3.16 是实验九与实验八的 setup 时序报告比较,节点 rData~Dout 有相同的 setup 余量,而节点 Din~rData 也有相近的 setup 余量,但是左图与右图之间有 0.5~0.8ns 的区别,估计是 DataIn 那里出现差别。此外同学们稍微注意一下两者之间的 clock skew,其中有 4ns 与 3ns 的差别,这就表示实验九有将“时钟抖动”信息打包在 input/output 公式里,而实验八则透过 set_clock_latency 将“时钟抖动”信息告诉 TimeQuest。

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	1.447	Din[2]	rData[2]	ext1_clk	fpga_clk	0.000	1.364	2.362
2	1.611	Din[0]	rData[0]	ext1_clk	fpga_clk	0.000	1.473	2.647
3	1.625	Din[1]	rData[1]	ext1_clk	fpga_clk	0.000	1.449	2.637
4	1.689	Din[3]	rData[3]	ext1_clk	fpga_clk	0.000	1.471	2.723
5	2.251	rData[2]	Dout[2]	fpga_clk	ext2_clk	0.000	-1.364	1.387
6	2.436	rData[1]	Dout[1]	fpga_clk	ext2_clk	0.000	-1.449	1.487
7	2.470	rData[0]	Dout[0]	fpga_clk	ext2_clk	0.000	-1.473	1.497
8	2.488	rData[3]	Dout[3]	fpga_clk	ext2_clk	0.000	-1.471	1.517

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	0.862	Din[0]	rData[0]	ext1_clk	fpga_clk	0.000	5.473	1.888
2	0.876	Din[1]	rData[1]	ext1_clk	fpga_clk	0.000	5.449	1.888
3	0.935	Din[3]	rData[3]	ext1_clk	fpga_clk	0.000	5.471	1.969
4	1.069	Din[2]	rData[2]	ext1_clk	fpga_clk	0.000	5.364	1.984
5	2.251	rData[2]	Dout[2]	fpga_clk	ext2_clk	0.000	-4.364	1.387
6	2.436	rData[1]	Dout[1]	fpga_clk	ext2_clk	0.000	-4.449	1.487
7	2.470	rData[0]	Dout[0]	fpga_clk	ext2_clk	0.000	-4.473	1.497
8	2.488	rData[3]	Dout[3]	fpga_clk	ext2_clk	0.000	-4.471	1.517

图 6.3.17 实验九（左）与实验八（右）的 hold 时序报告。

图 6.3.17 是实验九与实验八的 hold 时序报告比较,节点 rData~Dout 拥有一样的 hold 余量,而节点 Din~rData 则又 0.5~0.8ns 的差别,不用差凶手也是 DataIn。同样两者之间的 clock skew 有 4ns~3ns 的区别,原因与上述所述的一样。

实验九的重点是要证明:除了用 set_latency_clock 可以将物理时钟的抖动信息告诉 TimeQuest 以外,我们也可以透过 input/output 公式将时钟的抖动信息包裹在同一个“外包资料”里。至于哪一个办法比较好用 ... 这就见仁见智了。

6.4 物理时钟④

笔者曾在前面讨论过,物理时钟既是不理想时钟,其中 clock skew, clock jitter 或者 clock latency 等物理时钟的特性,称为 clock uncertainly。然而 TimeQuest 也有专门的约束命令用来针对 clock uncertainly,约束命令名为 set_clock_uncertainly。该约束命令的行为[仅告诉 TimeQuest 保险信息而已](#),而该约束命令有两种功能:

- (一) 设置而外的 setup 或者 hold 保险余量。
- (二) 设置物理时钟的不确定性。

功能一很容易理解... 假设节点 A 有 5ns 建立余量与 2ns 保持余量,不过笔者却很怕死的为节点 A 设置而外 1ns 的保险余量,结果节点 A 有 4ns 的建立余量与 1ns 的保险余量。从另一个角度来思考“提升保险余量有也是提升环境的极端性”,这种思路和网表的质量一样,亦即越极端的网表,保险余量与大。(功能一也可以用来约束 tRise 与 tFall 延迟)

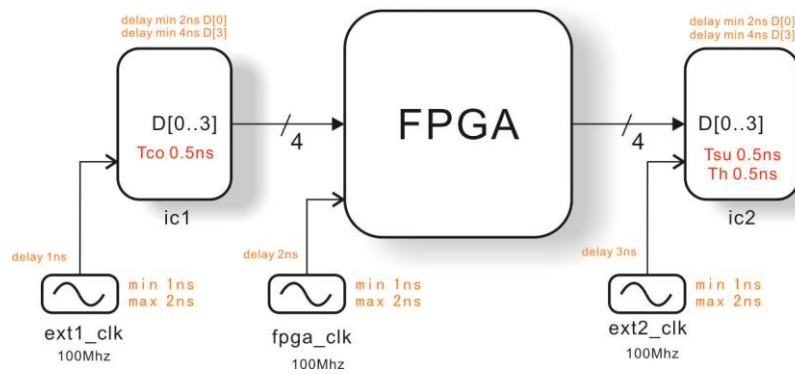
至于功能二中所谓的物理时钟不确定性,就是我们之前所曾学过的,[时钟抖动所产生的各种可能性时序](#)。我们知道 TimeQuest 模型至少需要 2 个寄存器,换句话一个节点需要 2 个时钟信号,如果两个时钟信号同时发生抖动,根据概率论会产生 4 种可能性的时序。不过,这 4 种可能性的时序有何 set_clock_uncertainly 约束命令有什么关系呢?让笔者拿 input/output 公式来解释:

```
input max      = < ext2fpga delay max > - < SLDE clock skew > + ext_Tco
input min      = < ext2fpga delay min > - < SEDL clock skew > + ext_Tco
output max     = < fpga2ext delay max > - < SLDE clock skew > + ext_Tsu
output min     = < fpga2ext delay min > - < SEDL clock skew > - ext_Th
```

input/output 公式如果有考虑时钟抖动的可能性,就会有标志性的 SLDE 时间差与 SEDL 时间差,而 set_clock_uncertainly 约束命令就是针对它们。此外,我们也知道 [SLDE 时间差是针对建立关系](#),而 [SEDL 时间差是针对保持关系](#)。在此之前我们先将它们从 input/output 公式中除掉:

```
input max      = < ext2fpga delay max > + ext_Tco
input min      = < ext2fpga delay min > + ext_Tco
output max     = < fpga2ext delay max > + ext_Tsu
output min     = < fpga2ext delay min > - ext_Th
```

然后再将 [SLDE 与 SEDL 的时间差透过 set_clock_uncertainly 告诉 TimeQuest 即可](#)。



实验十的内容是基于实验八 ... 不管怎么样,我们先求出各种延迟信息:

```
ext1_clk delay min : 1ns + 1ns = 2ns
ext1_clk delay max : 1ns + 2ns = 3ns
```

```
fpga_clk delay min : 2ns + 1ns = 3ns
fpga_clk delay max : 2ns + 2ns = 4ns
```

```
ext2_clk delay min : 3ns + 1ns = 4ns
ext2_clk delay max : 3ns + 2ns = 5ns
```

计算不包括 SEDL 与 SLDE 时间差的 input/output delay :

```
input max      = <ext2fpga delay max> + ext_Tco
               = 4ns + 0.5
               = 4.5ns
```

```
input min      = <fpga2ext delay min> + ext_Tco ( 或者 ext_minTco )
               = 2ns + 0.5ns
               = 2.5ns
```

```
output max = <fpga2ext delay max> + ext_Tsu
            = 4ns + 0.5ns
            = 4.5ns
```

```
output min    = < fpga2ext delay min > - ext_Th
               = 2ns - 0.5ns
               = 1.5ns
```

再者将 input max/min 的 SEDL 与 SLDE 时间差另外算出：

```
input max SLDE      = < dest. reg clock delay min > - < source reg clock delay max >
                    = < fpga_clk delay min > - < ext1_clk delay max >
                    = 3ns - 3ns
```

```
= 0ns
```

```
input min SEDL      = < dest. reg clock delay max > - < source reg clock delay min >
                    = < fpga_clk delay max > - < ext1_clk delay min >
                    = 4ns - 2ns
                    = 2ns
```

接着另外算出 output max/min 的 SEDL 与 SLDE 时间差：

```
output max SLDE      = < dest. reg clock delay min > - < source reg clock delay max >
                    = < ext2_clk delay min > - < fpga_clk delay max >
                    = 4ns - 4ns
                    = 0ns
```

```
output min SEDL      = < dest. reg clock delay max > - < source reg clock delay min >
                    = < ext2_clk delay max > - < fpga_clk delay min >
                    = 5ns - 3ns
                    = 2ns
```

完后打开实验十，然后建立同名 sdc 文件，随之将下面的代码拷贝进去：

```
*****
# Create Clock
*****

create_clock -name {fpga_clk} -period 10.000 -waveform { 0.000 5.000 } [get_ports {CLK}]
create_clock -name {ext1_clk} -period 10.000 -waveform { 0.000 5.000 }
create_clock -name {ext2_clk} -period 10.000 -waveform { 0.000 5.000 }

*****
# Set Input Delay
*****

set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[0]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[0]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[1]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[1]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[2]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[2]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[3]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[3]}]
```

```

*****

# Set Output Delay

*****

set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[3]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[3]}]

```

接着打开 TimeQuest 随之手动建立网表 `create_timing_netlist -model fast`，然后惯例的双击 Read SDC File 与 Update Timing Netlist。当网表更新结束以后，从 Constraint 菜单下打开 Set Clock Uncertainty。

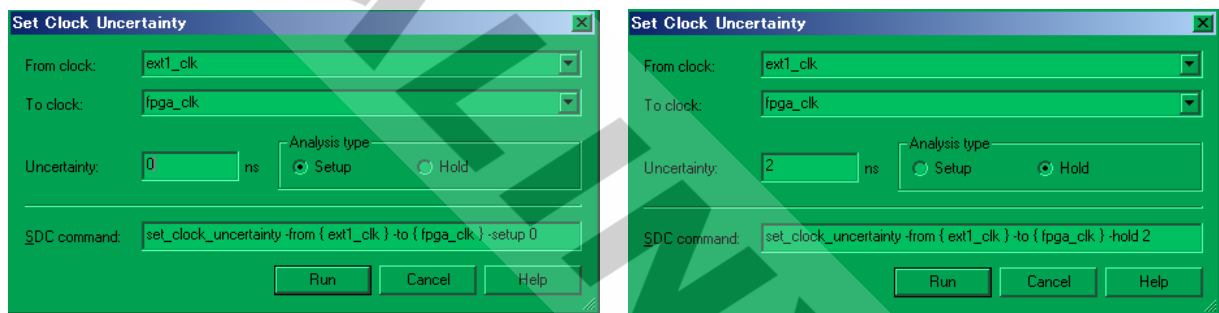


图 6.4.2 set clock uncertainty 界面，针对 source clock ext1_clk 与 destination clock fpga_clk。

我们知道 input 公式的源寄存器的时钟信号是 ext1_clk 所以 From Clock 对应 ext1_clk，换之我们知道目的寄存器的时钟信号是 fpga_clk 所示 to clock 对应 fpga_clk。此外我们也知道 input max 中的 SLDE 时间差针对建立关系，而该值是 0ns，结果如 6.4.2 左图所示，Uncertainty 是 0ns，而 analysis type 是 setup。反之，input min 中的 SEDL 时间差针对保持关系，结果如右图所示，uncertainty 是 2ns，而 analysis type 是 hold。

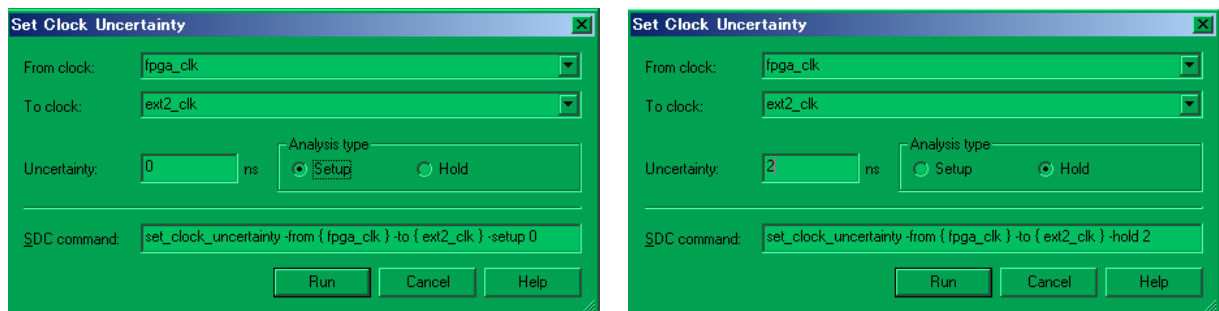


图 6.4.3 set clock uncertainty 界面，针对 source clock fpga_clk 与 destination clock ext2_clk。

我们也知道 output 公式的源寄存器时钟信号是 ext1_clk 所以 from Clock 输入 ext1_clk , 而我们知道目的寄存器的时钟信号是 fpga_clk 所示 to clock 输入 fpga_clk。另外我们也知道 output max 中的 SLDE 时间差针对建立关系, 而该值是 0ns , 结果如 6.4.3 左图所示, Uncertainly 是 0ns , 而 analysis type 是 setup。反之, output min 中的 SEDL 时间差针对保持关系, 结果如右图所示, uncertainly 是 2ns , 而 analysis type 是 hold。

```
#####
# Set Clock Uncertainty
#####

set_clock_uncertainty -rise_from [get_clocks {fpga_clk}] -rise_to [get_clocks {ext2_clk}] -setup 0.000
set_clock_uncertainty -rise_from [get_clocks {fpga_clk}] -rise_to [get_clocks {ext2_clk}] -hold 2.000
set_clock_uncertainty -rise_from [get_clocks {fpga_clk}] -fall_to [get_clocks {ext2_clk}] -setup 0.000
set_clock_uncertainty -rise_from [get_clocks {fpga_clk}] -fall_to [get_clocks {ext2_clk}] -hold 2.000
set_clock_uncertainty -fall_from [get_clocks {fpga_clk}] -rise_to [get_clocks {ext2_clk}] -setup 0.000
set_clock_uncertainty -fall_from [get_clocks {fpga_clk}] -rise_to [get_clocks {ext2_clk}] -hold 2.000
set_clock_uncertainty -fall_from [get_clocks {fpga_clk}] -fall_to [get_clocks {ext2_clk}] -setup 0.000
set_clock_uncertainty -fall_from [get_clocks {fpga_clk}] -fall_to [get_clocks {ext2_clk}] -hold 2.000
set_clock_uncertainty -rise_from [get_clocks {ext1_clk}] -rise_to [get_clocks {fpga_clk}] -setup 0.000
set_clock_uncertainty -rise_from [get_clocks {ext1_clk}] -rise_to [get_clocks {fpga_clk}] -hold 2.000
set_clock_uncertainty -rise_from [get_clocks {ext1_clk}] -fall_to [get_clocks {fpga_clk}] -setup 0.000
set_clock_uncertainty -rise_from [get_clocks {ext1_clk}] -fall_to [get_clocks {fpga_clk}] -hold 2.000
set_clock_uncertainty -fall_from [get_clocks {ext1_clk}] -rise_to [get_clocks {fpga_clk}] -setup 0.000
set_clock_uncertainty -fall_from [get_clocks {ext1_clk}] -rise_to [get_clocks {fpga_clk}] -hold 2.000
set_clock_uncertainty -fall_from [get_clocks {ext1_clk}] -fall_to [get_clocks {fpga_clk}] -setup 0.000
set_clock_uncertainty -fall_from [get_clocks {ext1_clk}] -fall_to [get_clocks {fpga_clk}] -hold 2.000
```

完后上述代码会自动添加在 sdc 文件下。接下来将所有节点的 setup 时序报告与 hold 时序报告都一举列出。

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	2.512	rData[5]	Dout[3]	fpga_clk	ext2_clk	10.000	-1.471	1.517
2	2.530	rData[0]	Dout[0]	fpga_clk	ext2_clk	10.000	-1.473	1.497
3	2.564	rData[1]	Dout[1]	fpga_clk	ext2_clk	10.000	-1.449	1.487
4	2.749	rData[2]	Dout[2]	fpga_clk	ext2_clk	10.000	-1.364	1.387
5	4.945	Din[2]	rData[2]	ext1_clk	fpga_clk	10.000	1.364	1.984
6	4.984	Din[3]	rData[3]	ext1_clk	fpga_clk	10.000	1.471	1.969
7	5.023	Din[1]	rData[1]	ext1_clk	fpga_clk	10.000	1.449	1.888
8	5.037	Din[0]	rData[0]	ext1_clk	fpga_clk	10.000	1.473	1.898

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	2.512	rData[5]	Dout[3]	fpga_clk	ext2_clk	10.000	-5.471	1.517
2	2.530	rData[0]	Dout[0]	fpga_clk	ext2_clk	10.000	-5.473	1.497
3	2.564	rData[1]	Dout[1]	fpga_clk	ext2_clk	10.000	-5.449	1.487
4	2.749	rData[2]	Dout[2]	fpga_clk	ext2_clk	10.000	-5.364	1.387
5	4.945	Din[2]	rData[2]	ext1_clk	fpga_clk	10.000	4.364	1.984
6	4.984	Din[3]	rData[3]	ext1_clk	fpga_clk	10.000	4.471	1.969
7	5.023	Din[1]	rData[1]	ext1_clk	fpga_clk	10.000	4.449	1.888
8	5.037	Din[0]	rData[0]	ext1_clk	fpga_clk	10.000	4.473	1.898

图 6.4.4 实验十（左）与实验八（右）的 setup 时序报告。

图 6.4.4.是实验十与实验八的 setup 时序报告比较 不过双方也有同样的 setup 余量结果。

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	0.862	Din[0]	rData[0]	ext1_clk	fpga_clk	0.000	1.473	1.898
2	0.876	Din[1]	rData[1]	ext1_clk	fpga_clk	0.000	1.449	1.888
3	0.835	Din[3]	rData[3]	ext1_clk	fpga_clk	0.000	1.471	1.969
4	1.069	Din[2]	rData[2]	ext1_clk	fpga_clk	0.000	1.364	1.984
5	2.251	rData[2]	Dout[2]	fpga_clk	ext2_clk	0.000	-1.364	1.387
6	2.436	rData[1]	Dout[1]	fpga_clk	ext2_clk	0.000	-1.449	1.487
7	2.470	rData[0]	Dout[0]	fpga_clk	ext2_clk	0.000	-1.473	1.497
8	2.488	rData[3]	Dout[3]	fpga_clk	ext2_clk	0.000	-1.471	1.517

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	0.862	Din[0]	rData[0]	ext1_clk	fpga_clk	0.000	5.473	1.898
2	0.876	Din[1]	rData[1]	ext1_clk	fpga_clk	0.000	5.449	1.888
3	0.835	Din[3]	rData[3]	ext1_clk	fpga_clk	0.000	5.471	1.969
4	1.069	Din[2]	rData[2]	ext1_clk	fpga_clk	0.000	5.364	1.984
5	2.251	rData[2]	Dout[2]	fpga_clk	ext2_clk	0.000	-4.364	1.387
6	2.436	rData[1]	Dout[1]	fpga_clk	ext2_clk	0.000	-4.449	1.487
7	2.470	rData[0]	Dout[0]	fpga_clk	ext2_clk	0.000	-4.473	1.497
8	2.488	rData[3]	Dout[3]	fpga_clk	ext2_clk	0.000	-4.471	1.517

图 6.4.5 实验十（左）与实验八（右）的 hold 时序报告。

图 6.4.5 是实验十与实验八的 hold 时序报告比较，而双方有同样的 hold 余量，笔者稍微手痒将节点 Din[0]~rData[0] 的详细信息展开 ...

Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					launch edge time
2	0.000	0.000					clock path
3	0.000	0.000	R				clock network delay
4	2.500	2.500	R	iExt	1	PIN_90	Din[0]
5	4.398	1.898					data path
6	3.015	0.515	RR	CELL	1	IOC_X28_Y0_N2	Din[0]combout
7	4.237	1.222	RR	IC	1	IOC_X25_Y0_M0	Dout[0]datain
8	4.398	0.161	RR	CELL	1	IOC_X25_Y0_M0	rData[0]
Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					latch edge time
2	1.473	1.473	R				clock path
3	1.473	1.473	R				clock network delay
4	3.473	2.000					clock uncertainty
5	3.536	0.063		uTh	1	IOC_X25_Y0_M0	rData[0]

Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					launch edge time
2	2.000	2.000					clock path
3	2.000	2.000	R				clock network delay
4	4.500	2.500	R	iExt	1	PIN_90	Din[0]
5	6.398	1.898					data path
6	5.015	0.515	RR	CELL	1	IOC_X28_Y0_N2	Din[0]combout
7	6.237	1.222	RR	IC	1	IOC_X25_Y0_M0	Dout[0]datain
8	6.398	0.161	RR	CELL	1	IOC_X25_Y0_M0	rData[0]
Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000					latch edge time
2	5.473	5.473	R				clock path
3	5.473	5.473	R				clock network delay
4	5.536	0.063		uTh	1	IOC_X25_Y0_M0	rData[0]

图 6.4.6 实验十（左）与实验八（右）的节点 Din[0]~rData[0] hold 信息报告的详细信息。

我们稍微比较一下图 6.4.6 的两张图，右图的 clock network delay 之所以有延迟是因为实验八用 set_clock_latency 告诉 TimeQuest 有关时钟的抖动信息。换之左图的 clock uncertainty 有延迟信息是因为实验九用 set_clock_uncertainly 告诉 TimeQuest 各种 SLDE 与 SEDL 时间差的信息。

至于 set_clock_uncertainly 的功能一笔者就不演示了因为太单纯了，而实验十我们可以这样结论：

set_clock_uncertainly 所谓的 uncertainly 是指时钟信号抖动的信息，不过更正确一点说就是各种时钟抖动所产生的可能性时序里所隐藏的 SLDE 时钟差与 SEDL 时钟差，前者是针对建立关系，后者则是针对保持关系。此外，如果使用 set_clock_uncertainly 该约束命令，在详细的时序报告中 clock uncertainty 列表就会留下设置的痕迹。

6.5 无奈的 set_max_delay 与 set_min_delay

首先，先让我们瞧瞧官方如何解释 set_max_delay 与 set_min_delay?

```
set_max_delay: Specifies a maximum delay exception for a given path.  
set_min_delay: Specifies a minimum delay exception for a given path.
```

set_max_delay 大意是为某个路径（实际上是指节点）指定自定义最大延迟，换言之 set_min_delay 是为某个路径（实际上是指节点）指定自定义最小延迟。这两个约束命令容易让人误会的原因是英文单词的 path，这个英文单词不禁让人联想 clock path 或者 data path。以此类推，set_max_delay 或者 set_min_delay 仿佛就有能力自定义 clock path 或者 data path，不过这是美丽的误会。

所谓的 clock path 或者 data path 是指内部延迟，然而决定内部延迟的动作除了 HDL 的书写以外，还有编译器设置，而且网表的质量也有关系。所以说该约束命令与内部延迟是完全没有关系，再次强调！没有关系！

用 TimeQuest 的话来说 path 往往是指某对节点，接着有 max 与 min 这两个关键字，就一定与时钟有关，而建立关系与保持关系式绝对逃不了。不过为什么说 set_max_delay 与 set_min_delay 是无奈的东西呢？

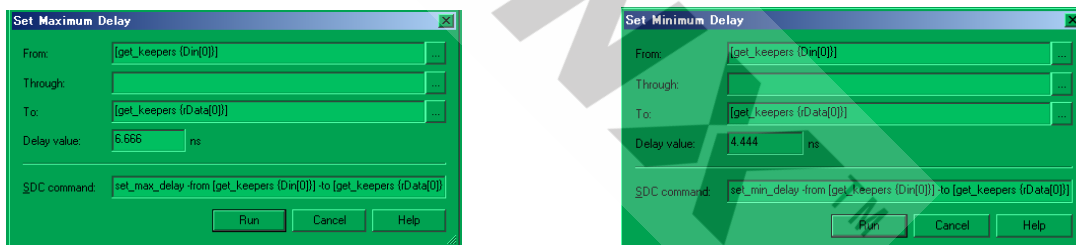


图 6.5.1 set_max_delay 与 set_min_delay 的界面。

以实验十为例，网表为 fast model，从 Constraint 菜单下打开 set maximum delay 与 set minimum delay，然后针对节点 Din[0]~rData[0]，maximum delay 设置为 6.666ns，而 minimum 设置为 4.444ns，如图 6.5.1 所示。

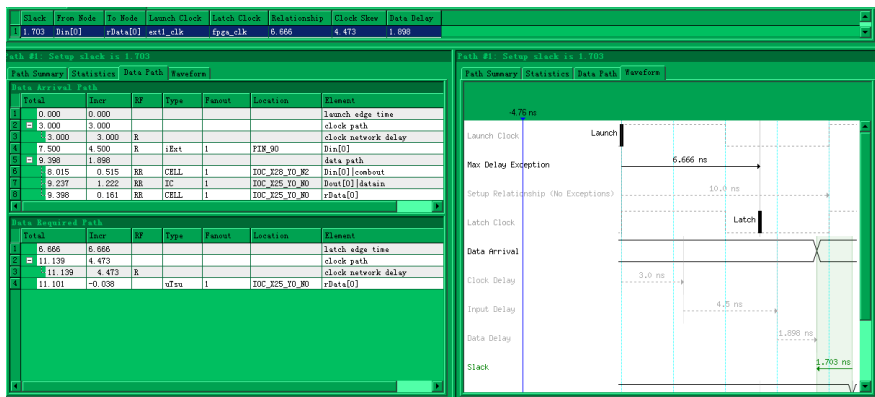


图 6.5.2 节点 Din[0]~rData[0]的 setup 详细报告。

然后打开节点 Din[0]~rData[0]的详细 setup 时序报告，如图 6.5.2 所示。我们可以看到该节点的 setup relationship 不是其它，而是我们之前用 set maximum delay 设置的 6.666ns。接着注意一下右边的 waveform，很明显锁存沿向左移动大约 3.334ns，结果造就 6.666ns 的建立关系。

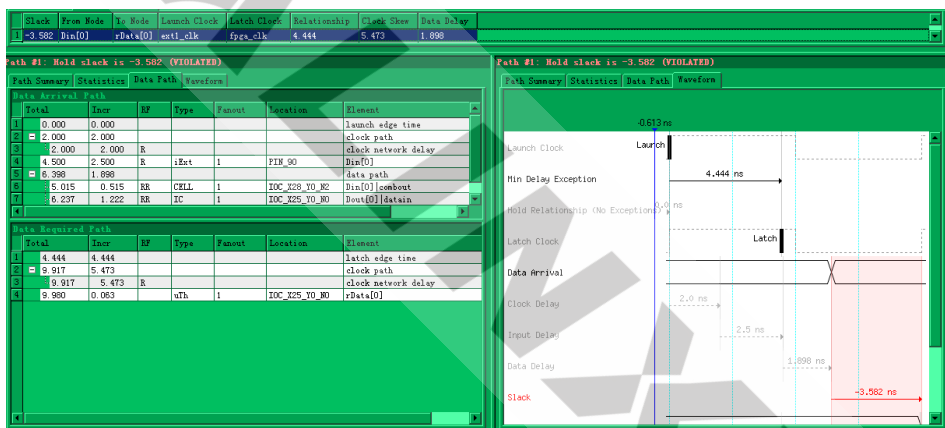


图 6.5.3 节点 Din[0]~rData[0]的 hold 详细报告。

接下来，打开节点 Din[0]~rData[0]的详细 hold 时序报告，如图 6.5.3 所示。同样 hold relationship 不是其它而是我们经 set minimum delay 设置的 4.444ns。从右边的 waveform 那里我们可以看到下一个启动沿与锁存沿有 4.444ns 的距离（保持关系）。

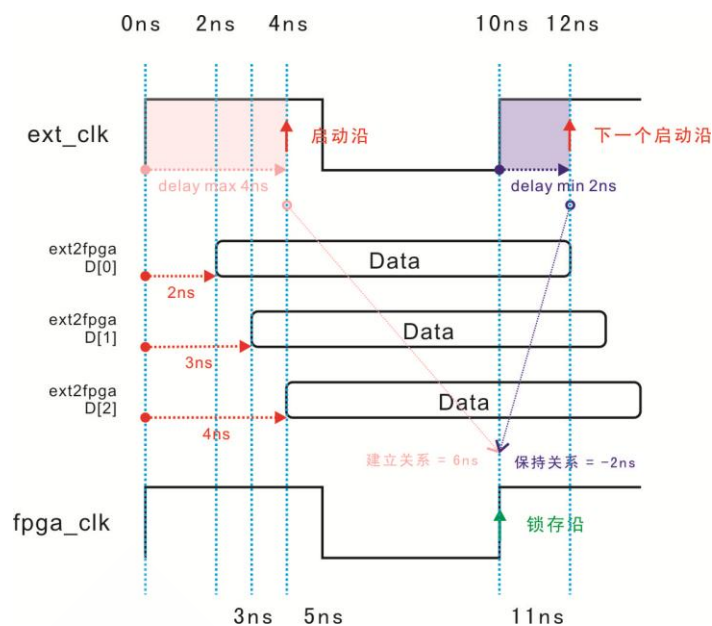


图 6.5.4 建立关系与保持关系的等价图。

根据上述的内容，我们知道这样一个结果，亦即 `set maximum delay` 直接自定义建立关系，而 `set minimum delay` 直接定义保持关系。有关这一点与之前所学过的 `delay max` 与 `delay min` 有不同的表现 ... 如图 6.5.4 所示，以往的 `delay max` 与 `delay min` 是取 D 数据 D 的最大和最小延迟，然后直接反映在启动沿与下一个启动沿的身上，从而取得等价图。然后再将这些信息告诉 TimeQuest，让它去计算。

相反的，`set maximum delay` 与 `set minimum delay` 的约束行为实际上是各种乱七八糟的约束合成体，设置时钟位移也可，设置多这种路径也可以。既然 `set maximum delay` 与 `set minimum delay` 那么糟糕，为什么还要它干嘛呢？根据官方的资料 ... 在很早以前 TimeQuest 还小的时候，静态时序分析的工作是由传统的工具（Classic Timing Analysis Tool）负责，其中就用 `set maximum delay` 和 `set minimum delay` 约束外部 ic 各种寄存器特性，如 `Tsu` 和 `Th`。

不过 TimeQuest 出现以后，随之 `set input delay` 与 `set output delay` 也跟着出现，该约束命令用“外包资料”的方式，不仅可以将时钟抖动也包装进去，而且也能将外部 ic 的寄存器特性也包裹进去 ... 慢慢的 `set input delay` 与 `set output delay` 就替代了 `set maximum delay` 与 `set minimum delay` 这部分的工作。

至于要不要使用 `set maximum delay` 与 `set minimum delay` 结果是见仁见智了。

总结：

第六章的重点内容基本上都是围绕着“物理时钟”展开，笔者想说物理时钟是外部模型是最关键，其中物理时钟所拥有的物理特性，亦即时钟抖动，潜伏和时间差等，被 TimeQuest 视为 Uncertainly。至于这些问题的重要性，单是约束命令的数量就可以看出它有多受重视。

时钟抖动是物理时钟最常见的一个表现，虽说目前的科技可以将时钟抖动控制在一定范围，不过对那些时序非常挑剔的设计而言，一天不把时钟抖动算清楚，一天都不得安宁。时钟抖动最大的难题就是产生多个可能性时序，以 TimeQuest 的模型而言，最多会产生 4 种可能性的时序。时钟抖动如果按一般的手段去解决是非常烦恼的。

时钟抖动所产生最大的延迟称为 max 别名 late，最小延迟则是 min 别名 early，这些抖动延迟又称为时钟潜伏。如果同学是懒人的话，TimeQuest 有专门的约束命令用来取得时钟抖动的信息，亦即 `set_clock_latency`。

任时钟抖动的四种可能性中，SLDE 的数据拥有最小的建立时间，SEDL 拥有最小的保持时间。其中取得的 SLDE 时间差与 SEDL 时间差是理解相关约束命令的关键所在。不仅 SLDE 时间差与 SEDL 时间差可以加入 input/output 公式里，而且也有专门的约束命令用来取得它们，亦即 `set_clock_uncertainly`。

笔者原本打算只讲 `set_clock_latency` 而已，可是犯贱的性子就喜欢往更深处钻。不过不管怎么样，收获也挺多的，以往累积的许多问题，就这样一扫而空了。到目前位置，constrainy 菜单下的约束命令，只剩下 Create generate clock 还未讨论，下一章我们就拿它来开刀吧。

第七章：供源时钟与其他

7.1 供源时钟①

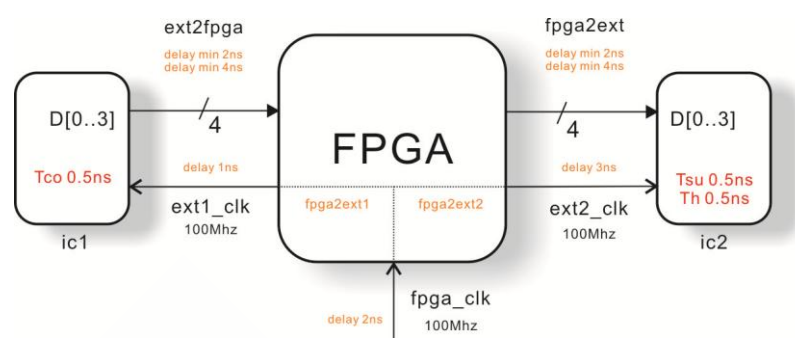


图 7.1.1 供源时钟的外部模型。

什么是供源时钟？想必同学们一定不陌生，fpga 是一个无私的孩子，有时候会牺牲一个管脚，将自己的时钟源透过该管脚，输出给外部 ic，如图 7.1.1 所示 ... 频率为 100Mhz 的 fpga_clk 作为 fpga 的时钟信号，该 fpga_clk 经过 fpga 内部以后，再路由某个管脚将新命名的 ext1_clk 与 ext2_clk 输出给外部 ic，100Mhz 的 ext1_clk 与 ext2_clk 因此称为供源时钟。（我们先假设任何时钟信号都没有发生抖动）

供源时钟既是内部时钟信号，也是外部时钟信号，因此它夹杂了内部延迟因数，与外部延迟因数。到底为什么供源时钟会那么大牌，以至于我们要特别关注它呢？同学们有所不知了，图 7.1.1 中的 fpga2ext1 与 fpga2ext2 是提供源，事实上它们的存在感很弱，如果我们不强调它们的存在，TimeQuest 不容易发现它们。

供源时钟基本的问题是如何将它存在感加强以至于 TimeQuest 认识到呢？这时候我们就要使用 create_generated_clock 这个约束命令了。

实验十一 供源时钟与 Create Generated Clock

实验十一是基于实验七，两者之间的区别就在于，实验十一的 ext1_clk 与 ext2_clk 是供源时钟而实验七不是。先让我们分析一下实验十一的 HDL 代码：

center_module.v

```
1. module center_module
2. (
3.     input CLK,
4.     input RSTn,
5.     input [3:0]Din,
```

```

6.     output [3:0]Dout,
7.     output ext1_clk,ext2_clk
8. );
9.     /*****/
10.
11.     reg [3:0]rData;
12.
13.     always @ ( posedge CLK or negedge RSTn )
14.         if( !RSTn )
15.             begin
16.                 rData <= 4'd0;
17.             end
18.         else
19.             begin
20.                 rData <= Din;
21.             end
22.
23.     /*****/
24.
25.     assign Dout = rData;
26.     assign ext1_clk = CLK;
27.     assign ext2_clk = CLK;
28.
29.     /*****/
30.
31. endmodule

```

第 7 行声明 ext1_clk 与 ext2_clk 的输出 然后在第 26~27 行设置 ext1_clk 与 ext2_clk 的驱动源，亦即 CLK。

再者求出图 7.11 的各种延迟信息：

```

fpga_clk delay 2ns
ext1_clk delay 1ns
ext2_clk delay 3ns
data delay min 2ns
data delay max 4ns
Tsu/Th/Tsu 0.5ns
fpga2ic1 ?
fpga2ic2 ?

```

```

input max      = <ext2fpga delay max> + ext_Tco
               = 4ns + 0.5

```



```

        = 4.5ns
input min    = <fpga2ext delay min> + ext_Tco ( 或者 ext_minTco )
              = 2ns + 0.5ns
              = 2.5ns

```

```

output max    = <fpga2ext delay max> + ext_Tsu
              = 4ns + 0.5ns
              = 4.5ns
output min    = < fpga2ext delay min > - ext_Th
              = 2ns - 0.5ns
              = 1.5ns

```

寻找 fpga2ext1 与 fpga2ext2 的延迟信息是我们在实验十一的首要任务，实际上 fpga2ext1 是 ext1 的一部分，换之 fpga2ext2 是 ext2_clk 的一部分，不过 fpga2ext1 与 fpga2ext2 是内部时钟信号，也是内部延迟因数，而 ext1_clk 与 ext_clk 是外部时钟信号，也是外部延迟因数。外部延迟信息我们可以直观晓得，反之内部延迟信息必须透过 TimeQuest 计算。

结果而言，如何告诉 TimeQuest 该 fpga2ext1 与 fpga2ext2 是存在，我们就要透过约束命令 create_generated_clock 的力量，然后再寻找它们的延迟值。首先打开实验十一，然后建立同名的 sdc 文件，再者将下述代码拷贝进去：

```

*****
# Create Clock
*****

create_clock -name {fpga_clk} -period 10.000 -waveform { 0.000 5.000 } [get_ports {CLK}]

*****
# Set Clock Latency
*****

set_clock_latency -source 2.000 [get_clocks {fpga_clk}]
set_clock_latency -source 1.000 [get_clocks {ext1_clk}]
set_clock_latency -source 3.000 [get_clocks {ext2_clk}]

*****
# Set Input Delay
*****

set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[0]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[0]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[1]}]

```

```

set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[1]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[2]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[2]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[3]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[3]}]

#####

# Set Output Delay
#####

set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[3]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[3]}]

```

上述的代码行基本上和实验七差不多，区别仅是实验十一将其中 ext1_clk 与 ext2_clk 的虚拟时钟拿掉而已。接下来打开 TimeQuest，手动建立 model fast 的网表，然后就是惯例的双击 Read SDC File，还有 Update Timing Netlist。

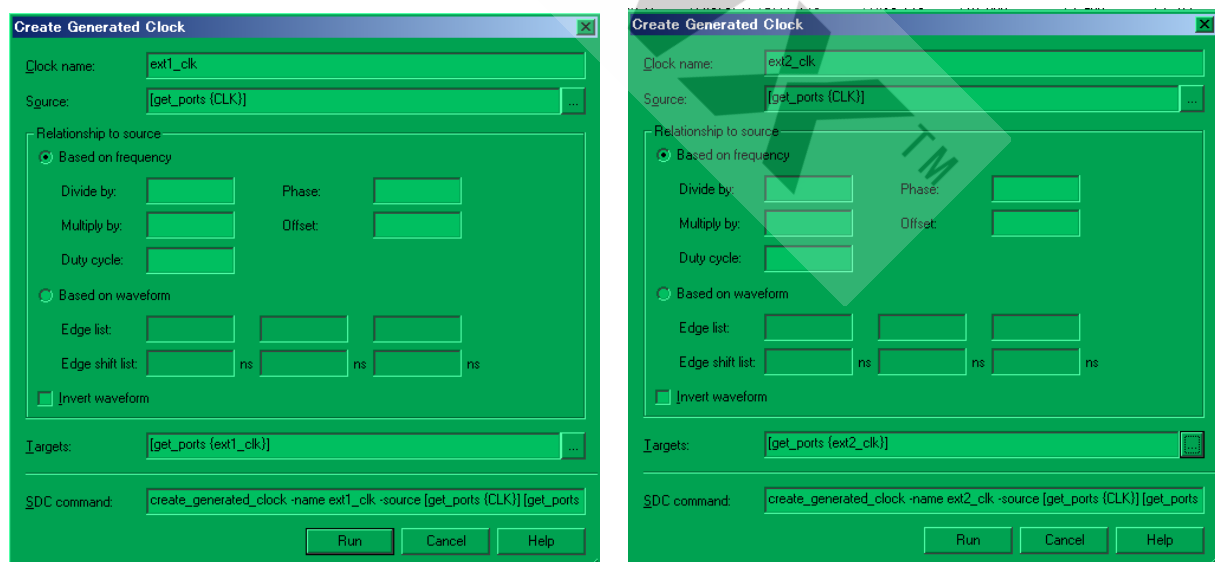


图 7.1.2 create generated clock 界面。

从 Constraint 菜单下打开 create generated clock 界面 ... Clock name 是供源时钟的命名，在此是 ext1_clk 与 ext2_clk；source 对应时钟源，在此是 CLK，余下的 Relationship to source 选项是改变供源时钟的行为，我们暂时先不管。然而 Targets 是供源时钟的输

出口，在此是 ext1_clk 与 ext_clk2。

```
#####  
# Create Generated Clock  
#####  
  
create_generated_clock -name {ext1_clk} -source [get_ports {CLK}] -master_clock {fpga_clk} [get_ports  
{ext1_clk}]  
  
create_generated_clock -name {ext2_clk} -source [get_ports {CLK}] -master_clock {fpga_clk} [get_ports  
{ext2_clk}]
```

完后，上述代码会自添加在 sdc 文件下，然后双击 Update Timing Netlist 来更新网表。



图 7.1.3 Report Path 界面（左），路径结果（右）。

接下来双击 Report Path 快捷，然后会跳出图 7.1.3 左图的界面，任所有选项放空，而 Path 深度设置 1000，上述的设置亦即将所有路径列出。完后会跳出右图，其中 CLK~ext1_clk 与 CLK~ext2_clk 路径就是我们要寻找的结果。两者路径有 2.562ns 的延迟，不过这是内部数据路径与内部时钟路径总和值，实际的 setup 余量与 hold 余量计算中，它们会个别列出。

Command Info		Summary of Paths						
Slack	From	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay	
2.511	Din[0]	rData[0]	ext1_clk	fpga_clk	10.000	3.522	3.543	
2.643	Din[1]	rData[1]	ext1_clk	fpga_clk	10.000	3.518	3.407	
2.378	Din[2]	rData[2]	ext1_clk	fpga_clk	10.000	3.521	3.675	
2.709	Din[3]	rData[3]	ext1_clk	fpga_clk	10.000	3.488	3.311	
4.729	rData[0]	Dout[0]	fpga_clk	ext2_clk	10.000	-3.522	2.249	
4.932	rData[1]	Dout[1]	fpga_clk	ext2_clk	10.000	-3.518	2.050	
4.889	rData[2]	Dout[2]	fpga_clk	ext2_clk	10.000	-3.521	2.090	
5.153	rData[3]	Dout[3]	fpga_clk	ext2_clk	10.000	-3.488	1.859	

Command Info		Summary of Paths						
Slack	From	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay	
6.037	Din[0]	rData[0]	ext1_clk	fpga_clk	10.000	3.473	1.898	
6.023	Din[1]	rData[1]	ext1_clk	fpga_clk	10.000	3.449	1.888	
5.845	Din[2]	rData[2]	ext1_clk	fpga_clk	10.000	3.364	1.984	
5.964	Din[3]	rData[3]	ext1_clk	fpga_clk	10.000	3.471	1.969	
5.530	rData[0]	Dout[0]	fpga_clk	ext2_clk	10.000	-3.473	1.497	
5.564	rData[1]	Dout[1]	fpga_clk	ext2_clk	10.000	-3.449	1.487	
5.749	rData[2]	Dout[2]	fpga_clk	ext2_clk	10.000	-3.364	1.387	
5.512	rData[3]	Dout[3]	fpga_clk	ext2_clk	10.000	-3.471	1.517	

图 7.1.4 实验十一（左）与实验七（右）的 setup 时序报告。

图 7.1.4 是实验实验十一与实验七的 setup 报告比较，同学们请注意 Data Delay，很明显左图的结果必右图的结果还要大上记号，为什么会这样呢？原因是供源时钟 ext1_clk 与 ext2_clk 的内部延迟信息已经计算进去。不管怎么样，我们还是将先展开节点 Din[0]~rData[0] 的详细信息，看看里边发生什么了。

Data Arrival Path						
Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000				launch edge time
2	3.000	3.000				clock path
3	3.000	3.000				clock network delay
4	7.500	4.500	A	iExt	1	PIN_60
5	11.043	3.543				Din[0]
6	3.035	0.535	RR	CELL	1	IOC_X3_Y0_M2
7	10.858	2.823	RR	IC	1	LCFF_X5_Y1_M17
8	11.043	0.185	RR	CELL	1	LCFF_X5_Y1_M17
9						rData[0]
Data Required Path						
Total	Incr	RF	Type	Fanout	Location	Element
1	10.000	10.000				latch edge time
2	13.522	3.522				clock path
3	13.522	3.522	R			clock network delay
4	13.554	0.032		ufsu	1	LCFF_X5_Y1_M17
5						rData[0]

Data Arrival Path						
Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000				launch edge time
2	1.000	1.000				clock path
3	1.000	1.000				clock network delay
4	5.500	4.500	A	iExt	1	PIN_90
5	7.398	1.898				Din[0]
6	6.015	0.515	RR	CELL	1	IOC_X25_Y0_M2
7	7.237	1.222	RR	IC	1	IOC_X25_Y0_M0
8	7.398	0.161	RR	CELL	1	IOC_X25_Y0_M0
9						rData[0]
Data Required Path						
Total	Incr	RF	Type	Fanout	Location	Element
1	10.000	10.000				latch edge time
2	13.473	3.473				clock path
3	13.473	3.473	R			clock network delay
4	13.435	-0.038		ufsu	1	IOC_X25_Y0_M0
5						rData[0]

图 7.1.4 实验十一（左）与实验七（右）节点 Din[0]~rData[0]的 setup 详细信息。

实验十一与实验七之间的区别就在于实验十一的 ext1_clk 与 ext2_clk 是供源时钟，而实验七不是。同学们请注意一下 clock network delay，左图是 3ns 右图是 1ns，其中的差别是 2ns，这是供源时钟造就而外的时钟路径延迟。再者是 data path，左图是 3.543，而右图是 1.898ns，其中有 1.645ns 的差别，这是供源时钟造就而外的数据延迟。接着将焦点切换到 Data Required Path 下，很明显左图的各种延迟数值比起右图高出一，这一切也是供源时钟造成的而外延迟。

Data Arrival Path						
Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000				launch edge time
2	3.522	3.522				clock path
3	3.522	3.522	R			clock network delay
4	5.771	2.249				data path
5	3.663	0.141	ufco	1	LCFF_X5_Y1_M17	rData[0]
6	3.663	0.000	RR	CELL	1	LCFF_X5_Y1_M17
7	4.178	0.515	RR	IC	1	IOC_X12_Y0_M0
8	5.771	1.593	RR	CELL	0	PIN_69
9						Dout[0]
Data Required Path						
Total	Incr	RF	Type	Fanout	Location	Element
1	10.000	10.000				latch edge time
2	15.000	5.000				clock path
3	15.000	5.000	R			clock network delay
4	10.500	-4.500	A	oExt	0	PIN_69
5						Dout[0]

Data Arrival Path						
Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000				launch edge time
2	3.473	3.473				clock path
3	3.473	3.473	R			clock network delay
4	4.970	1.497				data path
5	3.571	0.098	ufco	1	IOC_X25_Y0_M0	rData[0]
6	4.970	1.399	RR	CELL	0	PIN_88
7						Dout[0]
Data Required Path						
Total	Incr	RF	Type	Fanout	Location	Element
1	10.000	10.000				latch edge time
2	13.000	3.000				clock path
3	13.000	3.000	R			clock network delay
4	8.500	-4.500	R	oExt	0	PIN_88
5						Dout[0]

图 7.1.5 实验十一（左）与实验七（右）节点 rData[0]~Dout[0]的 setup 详细信息。

图 7.1.5 是节点 rData[0]~Dout[0]的详细 setup 信息，其中 Data Arrival Path 的各种信息，很明显左图比右图更多一些，这些都是供源时钟的错。换言之，Data Required Path 的 clock network delay，左图有 5ns 延迟，而右图有 3ns 延迟，其中有 2ns 的差别，这也是供源时钟造就的而外时钟路径延迟。

Slack	From	Δ	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	5.369	Din[0]	rData[0]	ext1_clk	fpga_clk	0.000	3.522	3.543
2	5.237	Din[1]	rData[1]	ext1_clk	fpga_clk	0.000	3.518	3.407
3	5.502	Din[2]	rData[2]	ext1_clk	fpga_clk	0.000	3.521	3.675
4	5.171	Din[3]	rData[3]	ext1_clk	fpga_clk	0.000	3.488	3.311
5	2.271	rData[0]	Dout[0]	fpga_clk	ext2_clk	0.000	-3.522	2.249
6	2.069	rData[1]	Dout[1]	fpga_clk	ext2_clk	0.000	-3.518	2.050
7	2.111	rData[2]	Dout[2]	fpga_clk	ext2_clk	0.000	-3.521	2.090
8	1.847	rData[3]	Dout[3]	fpga_clk	ext2_clk	0.000	-3.488	1.859

Slack	From	Δ	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	1.862	Din[0]	rData[0]	ext1_clk	fpga_clk	0.000	3.473	1.898
2	1.876	Din[1]	rData[1]	ext1_clk	fpga_clk	0.000	3.449	1.888
3	2.069	Din[2]	rData[2]	ext1_clk	fpga_clk	0.000	3.364	1.984
4	1.935	Din[3]	rData[3]	ext1_clk	fpga_clk	0.000	3.471	1.969
5	3.470	rData[0]	Dout[0]	fpga_clk	ext2_clk	0.000	-3.473	1.497
6	3.436	rData[1]	Dout[1]	fpga_clk	ext2_clk	0.000	-3.449	1.487
7	3.251	rData[2]	Dout[2]	fpga_clk	ext2_clk	0.000	-3.364	1.387
8	3.488	rData[3]	Dout[3]	fpga_clk	ext2_clk	0.000	-3.471	1.517

图 7.1.6 实验十一（左）与实验七（右）的 hold 时序报告。

图 7.1.6 是实验十一与实验七的 hold 报告比较，至于 data delay 左图比起右图高出不少，不用说都是供源时钟惹的祸。

Data Arrival Path						
Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000				launch edge time
2	3.000	3.000				clock path
3	3.000	3.000	R			clock network delay
4	5.500	2.500	R	iExt	PIN_60	Din[0]
5	9.043	3.543				data path
6	6.035	0.535	RR	CELL	IOC_X3_Y0_M2	Din[0] combout
7	8.858	2.823	RR	IC	LCFF_X5_Y1_M17	rData[0] ddata
8	9.043	0.185	RR	CELL	LCFF_X5_Y1_M17	rData[0]

Data Required Path						
Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000				latch edge time
2	3.522	3.522				clock path
3	3.522	3.522	R			clock network delay
4	3.674	0.152		uTh	LCFF_X5_Y1_M17	rData[0]

图 7.1.7 实验十一（左）与实验七（右）节点 Din[0]~rData[0]的 Hold 详细信息。

图 7.1.7 是节点 Din[0]~rData[0]详细的 hold 信息比较，先是 Data Arrival Path 的 clock network delay，左图有 3ns 延迟，而右图有 1ns 延迟，其中有 2ns 的时钟路径的差别。此外左图的数据 path 比起右图的数据 path 同样也增加不少。

Data Arrival Path						
Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000				launch edge time
2	3.522	3.522				clock path
3	3.522	3.522	R			clock network delay
4	5.771	2.249				data path
5	3.663	0.141	uTco	1	LCFF_X5_Y1_M17	rData[0]
6	3.663	0.000	RR	CELL	LCFF_X5_Y1_M17	rData[0] regout
7	4.178	0.515	RR	IC	IOC_X12_Y0_M0	Dout[0] datain
8	5.771	1.593	RR	CELL	PIN_68	Dout[0]

Data Required Path						
Total	Incr	RF	Type	Fanout	Location	Element
1	0.000	0.000				latch edge time
2	5.000	5.000				clock path
3	5.000	5.000	R			clock network delay
4	3.500	-1.500	R	oExt	PIN_68	Dout[0]

图 7.1.8 实验十一（左）与实验七（右）节点 rData[0]~Dout[0]的 Hold 详细信息。

图 7.1.8 是节点 rData[0]~Dout[0]详细的 hold 信息比较，同学们注意 Data Required Path 的 clock network delay，左图有 5ns 延迟，而右图有 3ns 延迟，其中有 2ns 的时钟路径延迟差别。至于 Data Arrival path 的数据 path，左图比右图多出一点。

经过上述的内容，我们可以这样结论：

实验十一与实验七相比较，实验十一的 ext1_clk 与 ext2_clk 是供源时钟，而实验七却不是。供源时钟造就了而外的 data path 延迟，此外实验十一的 ext1_clk 与 ext2_clk 与实验七相比多出 2ns 的时钟路径延迟。因此我们可以说，图 7.1.1 中的 fpga2ext1 与 fpga2ext2 都有 2ns 的延迟。

同学们请注意，实验十一的重点再也不是时序是否合格，也不是 fpga2ext1 与 fpga2ext2 的延迟信息，而是：当我们使用 create_generated_clock 约束命令告诉 TimeQuest 相关供源时钟的信息以后，TimeQuest 就会将供源时钟造成的而外延迟信息也计算进去。结果而言，实验十一与实验七会取得不同的结果，那怕 ext1_clk 与 ext2_clk 的角色改变一点点而已。

7.2 供源时钟②

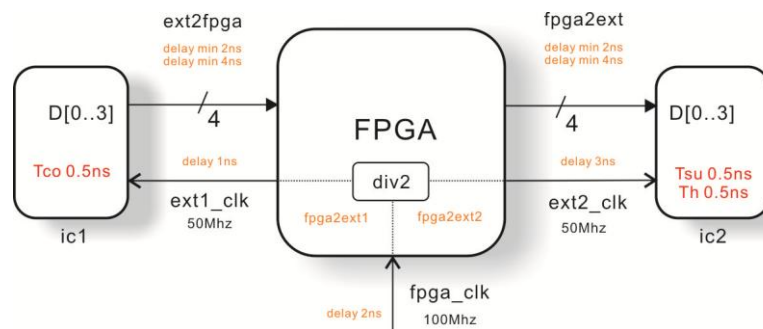


图 7.2.1 供源时钟的外部模型。

图 7.2.1 与图 7.1.1 的外部模型相比较起来，图 7.2.1 的 fpga2ext1 与 fpga2ext2 多出了减频 1/2 的模块。结果，这一回我们必须更动一下 create_generated_clock 的用法，不然的话 TimeQuest 是无法知晓 fpga2ext1 与 fpga2ext2 是否存在。让我们先瞧一瞧到底，究竟 center_module 更动了什么？

center_module.v

```
1. module center_module
2. (
3.     input CLK,
4.     input RSTn,
5.     input [3:0]Din,
6.     output [3:0]Dout,
7.     output ext1_clk,ext2_clk
8. );
9.     /*****/
10.
11.     reg ext_div2;
12.
13.     always @ ( posedge CLK or negedge RSTn )
14.         if( !RSTn )
15.             begin
16.                 ext_div2 <= 1'b0;
17.             end
18.         else
19.             begin
20.                 ext_div2 <= ~ext_div2;
21.             end
22.     /*****/
```

```

23.
24.     reg [3:0]rData;
25.
26.     always @ ( posedge CLK or negedge RSTn )
27.         if( !RSTn )
28.             begin
29.                 rData <= 4'd0;
30.             end
31.         else
32.             begin
33.                 rData <= Din;
34.             end
35.
36.     /*****/
37.
38.     assign Dout = rData;
39.     assign ext1_clk = ext_div2;
40.     assign ext2_clk = ext_div2;
41.
42.     /*****/
43.
44. endmodule

```

第 7 行声明了 ext1_clk 与 ext2_clk 的输出口，然而第 11~21 行描述了 ext_div2 寄存器的 1/2 分频行为，最后在第 40~41 行将 ext_div2 驱动 ext1_clk 与 ext2_clk。

实验十二： 供源时钟与寄存器与 create_generated_clock

打开实验是十二，然后建立同名的 sdc 文件，接着将下面的代码拷贝进去。

```

*****
# Create Clock
*****

create_clock -name {fpga_clk} -period 10.000 -waveform { 0.000 5.000 } [get_ports {CLK}]

```

再者，打开 TimeQuest 手动建立 model fast 网表，然后惯例的连打 Read SDC File 与 Update Timing Netlist。在此我们只有 fpga_clk 的系统时钟而已，我们必须经 create generated clock 告诉 TimeQuest 还有 ext1_clk 与 ext2_clk 这两个时钟。

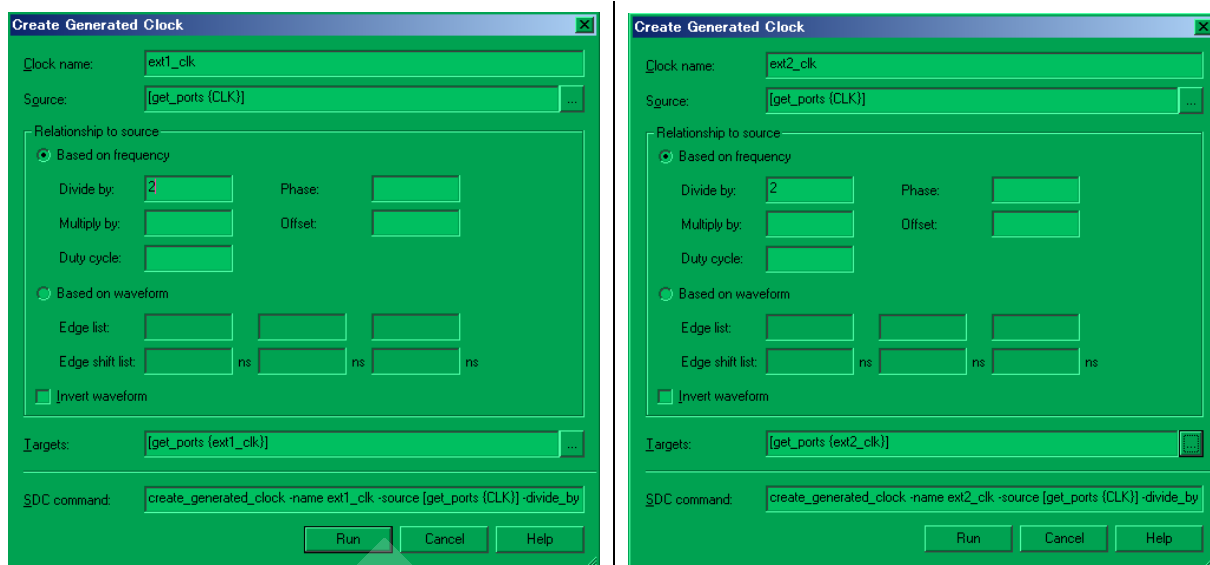


图 7.2.2 模糊的供源时钟 ext1_clk 与 ext2_clk 声明。

一般上我们都会直接从 create generated clock 界面下，命名 ext1_clk 与 ext2_clk，时钟源为 CLK，然后在 Divide by 的选项下输入 2，最后的 Targets 分别对应 ext1_clk 与 ext2_clk 输出口，如图 7.2.2 所示。

Warning: No paths exist between clock target "ext1_clk" of clock "ext1_clk" and its clock source. Assuming zero source clock latency.
Warning: No paths exist between clock target "ext2_clk" of clock "ext2_clk" and its clock source. Assuming zero source clock latency.

不过当我们再度双击 Update Timing Netlist 更新网表，此时 TimeQuest 就会反馈找不到 ext1_clk 与 ext2_clk 驱动源的信息，如上述信息所示。这就俗称模糊供源时钟声明，因为图 7.2.2 的告知表现让 TimeQuest 这个笨蛋仅晓得 ext1_clk 有 1/2 分频，时钟源是 CLK，对应输出口是 ext1_clk；或者，ext2_clk 有 1/2 分频，时钟源也是 CLK，对应输出口则是 ext2_clk。可是 TimeQuest 却不清楚中间的经过，也不会自己去寻找 ... 真是蠢死了！因此，我们必须手动建立 ext1_clk 与 ext2_clk 的完全经过 ...

CLK 输入口 → ext_div2 寄存器 → ext1_clk 输出口
→ ext2_clk 输出口

在这里，我们知道 ext1_clk 与 ext2_clk 供源时钟的声明，各自都有两个箭头，然而每一个箭头就代表一次 create generate clock 声明。ext1_clk 供源时钟的声明先是 CLK 输入口到 ext_div2 寄存器，然后 ext_div2 到 ext1_clk 输出口；而 ext2_clk 供源时钟的声明可是 ext_div 寄存器致 ext2_clk 输出口。这里有 3 个箭头，亦即我们需要声明 create generate clock 三次。

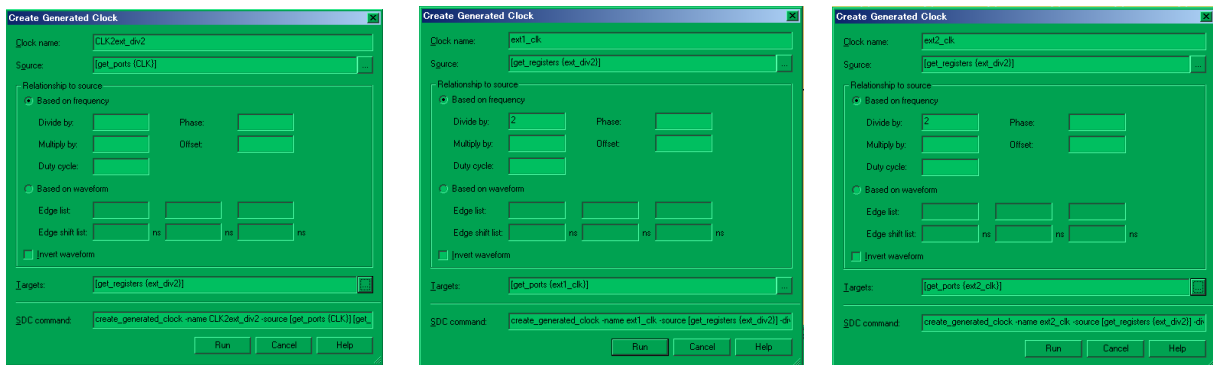


图 7.2.3 清晰的供源时钟 ext1_clk 与 ext2_clk 声明。

不保存之前的约束过程，然后重新打开 TimeQuest 再手动建立 model fast 网表，随之惯例的连打操作，亦即先双击 Read SDC File，然后又在双击 Update Timing Netlist。接下来从 Constraint 菜单下调出 create generated clock 界面，按图 7.2.3 声明。左图是声明名为 CLK2ext_div2 的时钟路径，时钟源为 CLK，对应目标是 ext_div2 寄存器。

中图是声明名为 ext1_clk 的供源时钟，时钟源为 ext_div2 寄存器，对应输出口是 ext1_clk；至于右图是声明名为 ext2_clk 的供源时钟，时钟源为 ext_div2 寄存器，对应输出口是 ext2_clk。完后下列代码会自动添加在 sdc 文件下。

```
*****
# Create Generated Clock
*****

create_generated_clock -name {CLK2ext_div2} -source [get_ports {CLK}] -master_clock {fpga_clk} [get_registers {ext_div2}]
create_generated_clock -name {ext1_clk} -source [get_registers {ext_div2}] -divide_by 2 -master_clock {CLK2ext_div2} [get_ports {ext1_clk}]
create_generated_clock -name {ext2_clk} -source [get_registers {ext_div2}] -divide_by 2 -master_clock {CLK2ext_div2} [get_ports {ext2_clk}]
```

Clocks Summary												
	Clock Name	Type	Period	Frequency	Rise	Fall	Duty Cycle	Divide by	Multiply by	Master	Source	Targets
1	CLK2ext_div2	Generated	10.000	100.0 MHz	0.000	5.000		1	1	fpga_clk	CLK	{ ext_div2 }
2	ext1_clk	Generated	20.000	50.0 MHz	0.000	10.000		2	1	CLK2ext_div2	ext_div2	{ ext1_clk }
3	ext2_clk	Generated	20.000	50.0 MHz	0.000	10.000		2	1	CLK2ext_div2	ext_div2	{ ext2_clk }
4	fpga_clk	Base	10.000	100.0 MHz	0.000	5.000						{ CLK }

图 7.2.4 时钟简报。

然后双击 Update Timing Netlist 更新网表，接着顺便双击 Report All Summaries 调出所有相关的时钟声明，结果如图 7.2.4 所示。其中 CLK2ext_div2 的作用仅告知 TimeQuest 有一个频率为 100Mhz 的时钟路径从 CLK 输入口连接至 ext_div2 寄存器。至于 ext1_clk 供源时钟则是 50Mhz 的频率，从 ext_div2 寄存器连接至 ext1_clk 输出口；换言之 50Mhz 频率的供源时钟也是从 ext_div2 寄存器连接至 ext2_clk 输出口。

```
*****
# Set Clock Latency
*****
```

```

set_clock_latency -source 2.000 [get_clocks {fpga_clk}]
set_clock_latency -source 1.000 [get_clocks {ext1_clk}]
set_clock_latency -source 3.000 [get_clocks {ext2_clk}]

#####

# Set Input Delay
#####

set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[0]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[0]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[1]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[1]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[2]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[2]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[3]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[3]}]

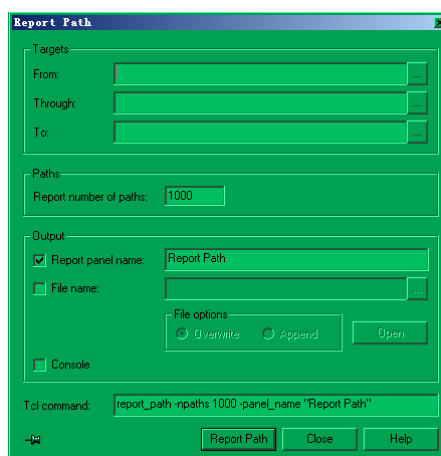
#####

# Set Output Delay
#####

set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[3]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[3]}]

```

完后将上述各种约束信息手动添加致 sdc 文件下，然后重新打开 TimeQuest。继续惯例的连打操作，知道网表更新。



	Delay	From ...	To Node
1	1.519	CLK	rData[1]
2	1.518	CLK	rData[3]
3	1.517	CLK	ext_div2
4	1.515	CLK	rData[2]
5	1.468	CLK	rData[0]
6	3.616	Din[0]	rData[0]
7	3.747	Din[1]	rData[1]
8	3.415	Din[2]	rData[2]
9	3.537	Din[3]	rData[3]
10	1.631	RSTn	rData[1]
11	1.630	RSTn	rData[3]
12	1.629	RSTn	ext_div2
13	1.627	RSTn	rData[2]
14	1.580	RSTn	rData[0]
15	1.738	ext_div2	ext2_clk
16	1.708	ext_div2	ext1_clk
17	0.226	ext_div2	ext_div2
18	2.030	rData[0]	Dout[0]
19	1.748	rData[1]	Dout[1]
20	2.612	rData[2]	Dout[2]
21	1.893	rData[3]	Dout[3]

图 7.2.5 路径报告。

接下来双击 Report Path，就会弹出图 7.2.5 的左图，选项任空，深度为 1000 ... 执行以后就会弹出右图。同学们稍微注意第 3 路径 CLK~ex_div，和 15 路径 ext_div2~ext2_clk 还有第 16 路径 ext_div2~ext1_clk，都有延迟信息。结果而言，TimeQuest 已经知晓 ext1_clk 与 ext2_clk 供源时钟的存在。

到目前位置实验十一的目的已经完成，我们先用 create generated clock 将 CLK 输入口连接至 ext_div2 寄存器，声明为 CLK2ext_div2。接着再用同样的约束命令将 ext_div2 寄存器连接至 ext1_clk 输出口，还有 ext2_clk 输出口。然后再经过 Report Path 取得相关路径的延迟信息，亦即 TimeQuest 已经认识 ext1_clk 与 ext2_clk 供源时钟。

7.3 供源时钟③

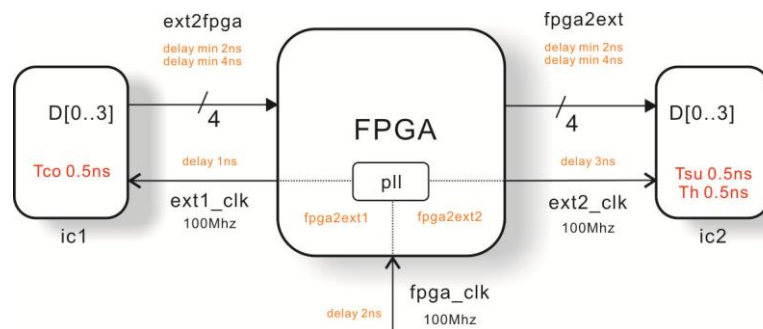


图 7.3.1 供源时钟的外部模型。

最后一种供源时钟就是借助 pll 的力量，这也是供源时钟最常的产生方法。如图 7.3.1 所示，fpga_clk 进入 fpga 内部，接着经过 pll 后产生 ext1_clk 与 ext2_clk 供源时钟。让我们先瞧瞧 hdl 的描述内容：

center_module.v

```
1.  module center_module
2.  (
3.      input CLK,
4.      input RSTn,
5.      input [3:0]Din,
6.      output [3:0]Dout,
7.      output ext1_clk,ext2_clk
8.  );
9.      /*****/
10.
11.     wire ext_CLK;
12.
13.     pll_module pll
14.     (
15.         .inclk0 ( CLK ),
16.         .c0 ( ext_CLK )
17.     );
18.
19.     /*****/
20.
21.     reg [3:0]rData;
22.
23.     always @ ( posedge CLK or negedge RSTn )
```

```

24.         if( !RSTn )
25.             begin
26.                 rData <= 4'd0;
27.             end
28.         else
29.             begin
30.                 rData <= Din;
31.             end
32.
33.         /*****/
34.
35.         assign Dout = rData;
36.         assign ext1_clk = ext_CLK;
37.         assign ext2_clk = ext_CLK;
38.
39.         /*****/
40.
41.     endmodule

```

第 7 行声明 ext1_clk 与 ext2_clk 的输出口，第 11~17 行实例化 pll 模块，而 pll 的输入是 CLK 输出是 ext_clk。最后在第 36~37 行将 ext_clk 驱动 ext1_clk 与 ext2_clk。

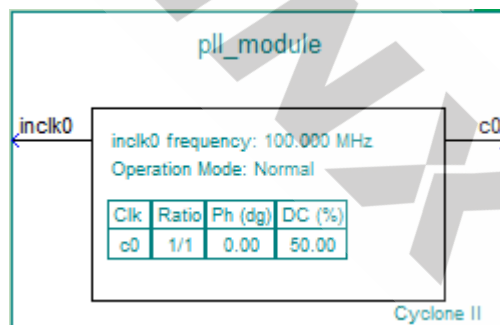


图 7.3.2 pll 设置。

图 7.3.2 是 pll 的设置内容，inclk0 是 100Mhz 的时钟信号，而 c0 也是 100Mhz 的时钟信号。

实验十三 供源时钟与 pll 与 create generated clock

首先打开实验十三，然后建立同名 sdc 文件，接着将下面的约束代码拷贝进去。

```

*****
# Create Clock
*****

```

```
create_clock -name {fpga_clk} -period 10.000 -waveform { 0.000 5.000 } [get_ports {CLK}]

#####

# Create Generated Clock

#####

derive_pll_clocks
```

接下来打开 TimeQuest，手动建立 model fast 网表，随之执行惯例的连打操作，亦即双击 Read SDC File 与 Update Timing Netlist。

Clocks Summary									
Clock Name	Type	Period	Frequency	Rise	Fall	Master	Source	Targets	
1 pll altpll_component pll clk[0]	Generated	10.000	100.0 MHz	0.000	5.000	fpga_clk	pll altpll_component pll inclclk[0]	{ pll altpll_component pll clk[0] }	
2 fpga_clk	Base	10.000	100.0 MHz	0.000	5.000			{ CLK }	

图 7.3.3 时间简报。

完后双击 Report All Summaries，就会弹出图 7.3.3 的时间简报，此时只有 pll 的 clk[0]，和 fpga_clk，却没有供源时钟 ext1_clk 与 ext2_clk。

Report Path

Targets

From:

Through:

To:

Paths

Report number of paths: 1000

Output

☒ Report panel name: Report Path

☐ File name:

File options: ☐ Overwrite ☐ Append

☐ Console

Tcl command: report_path -npaths 1000 -panel_name "Report Path"

	Delay	From Node	To Node
1	1.864	rData[3]	Dout[3]
2	2.448	rData[2]	Dout[2]
3	1.708	rData[1]	Dout[1]
4	2.030	rData[0]	Dout[0]
5	2.627	pll altpll_component pll clk[0]	ext2_clk
6	1.961	pll altpll_component pll clk[0]	ext1_clk
7	1.847	RSTn	rData[2]
8	1.640	RSTn	rData[3]
9	1.629	RSTn	rData[1]
10	1.580	RSTn	rData[0]
11	3.640	Din[3]	rData[3]
12	3.746	Din[2]	rData[2]
13	3.721	Din[1]	rData[1]
14	3.425	Din[0]	rData[0]
15	1.534	CLK	rData[2]
16	1.527	CLK	rData[3]
17	1.516	CLK	rData[1]
18	1.467	CLK	rData[0]
19	-1.439	CLK	pll altpll_component pll clk[0]

图 7.3.4 路径报告。

再者双击 Report Path 任选项放空，执行以后就会出现图 7.3.4 右图的情形 ... 亦即第 5 与第 6 的路径，同学们有没有看到呢？pll 致 ext1_clk 还有 ext2_clk，至于第 19 的路径则是 CLK 致 pll。

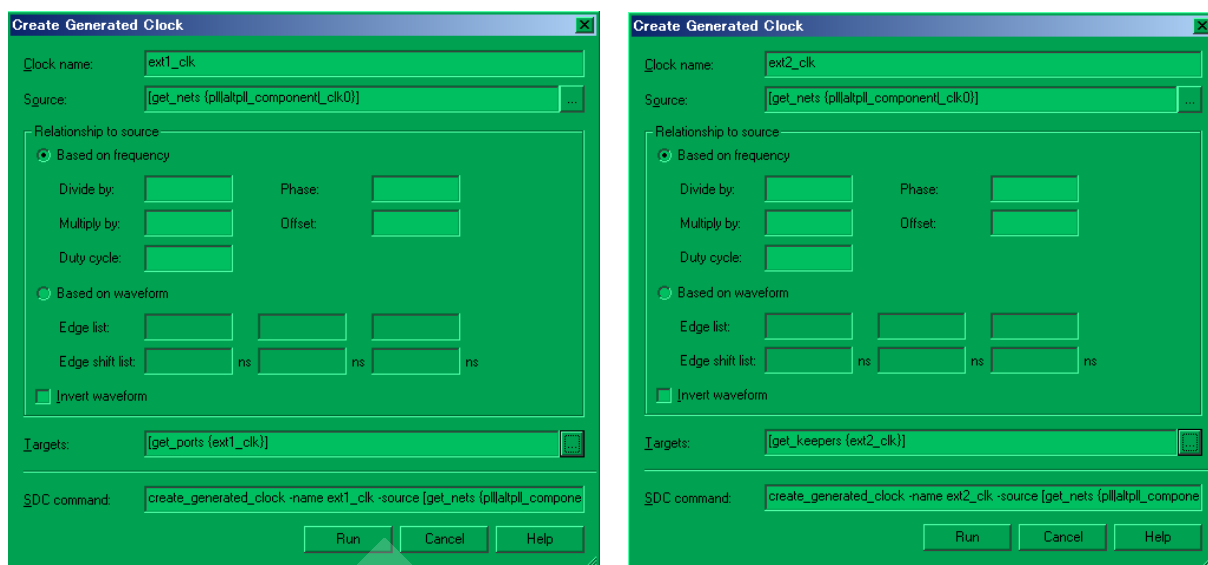


图 7.3.5 声明 ext1_clk 与 ext2_clk 供源时钟。

接着从 Constraint 菜单下打开 create generated clock ,然后根据图 7.3.5 的选择设置并且声明 ext1_clk 与 ext2_clk 供源时钟,不过此时的 source 对应 net 资源内的 pll_clk[0]。完后,下面的代码会自动添加在 sdc 文件下 ...

```
#####
# Create Generated Clock
#####

create_generated_clock -name {ext1_clk} -source [get_nets {pll|altpll_component|_clk0}] -master_clock
{pll|altpll_component|pll|clk[0]} [get_ports {ext1_clk}]
create_generated_clock -name {ext2_clk} -source [get_nets {pll|altpll_component|_clk0}] -master_clock
{pll|altpll_component|pll|clk[0]} [get_ports {ext2_clk}]
```

Clocks Summary									
	Clock Name	Type	Period	Frequency	Rise	Fall	Master	Source	Targets
1	ext1_clk	Generated	10.000	100.0 MHz	0.000	5.000	pll altpll_component pll clk[0]	pll altpll_component pll clk[0]	{ ext1_clk }
2	ext2_clk	Generated	10.000	100.0 MHz	0.000	5.000	pll altpll_component pll clk[0]	pll altpll_component pll clk[0]	{ ext2_clk }
3	fpga_clk	Base	10.000	100.0 MHz	0.000	5.000			{ CLK }
4	pll altpll_component pll clk[0]	Generated	10.000	100.0 MHz	0.000	5.000	fpga_clk	pll altpll_component pll inclk[0]	{ pll altpll_component pll clk[0] }

图 7.3.6 时间简报。

同学们让我们乘胜追击,再双击 Report All Summaries 调出时间简报,结果如图 7.3.6 所示,此时已经出现 ext1_clk 与 ext2_clk 的供源时钟,换句话说 ... TimeQuest 已经认识成功,接下来我们就可以进行其他约束了 ... 同学们麻烦将下列的代码拷贝进去 sdc 文件里。

```
#####
# Set Clock Latency
#####
```

```

set_clock_latency -source 2.000 [get_clocks {fpga_clk}]
set_clock_latency -source 1.000 [get_clocks {ext1_clk}]
set_clock_latency -source 3.000 [get_clocks {ext2_clk}]

#####

# Set Input Delay
#####

set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[0]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[0]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[1]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[1]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[2]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[2]}]
set_input_delay -add_delay -max -clock [get_clocks {ext1_clk}] 4.500 [get_ports {Din[3]}]
set_input_delay -add_delay -min -clock [get_clocks {ext1_clk}] 2.500 [get_ports {Din[3]}]

#####

# Set Output Delay
#####

set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[0]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[1]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[2]}]
set_output_delay -add_delay -max -clock [get_clocks {ext2_clk}] 4.500 [get_ports {Dout[3]}]
set_output_delay -add_delay -min -clock [get_clocks {ext2_clk}] 1.500 [get_ports {Dout[3]}]

```

最后使出决定性攻击，亦即双击 Update Timing Netlist 更新网表。

	Slack	From	Δ	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	4.584	Din[0]		rData[0]	ext1_clk	fpga_clk	10.000	3.336	2.656
2	4.761	Din[1]		rData[1]	ext1_clk	fpga_clk	10.000	3.466	2.606
3	4.463	Din[2]		rData[2]	ext1_clk	fpga_clk	10.000	3.352	2.793
4	4.716	Din[3]		rData[3]	ext1_clk	fpga_clk	10.000	3.357	2.545
5	4.338	rData[0]		Dout[0]	fpga_clk	ext2_clk	10.000	-3.336	1.387
6	4.078	rData[1]		Dout[1]	fpga_clk	ext2_clk	10.000	-3.466	1.517
7	4.342	rData[2]		Dout[2]	fpga_clk	ext2_clk	10.000	-3.352	1.367
8	4.317	rData[3]		Dout[3]	fpga_clk	ext2_clk	10.000	-3.357	1.387

	Slack	From	Δ	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	2.511	Din[0]		rData[0]	ext1_clk	fpga_clk	10.000	3.522	3.543
2	2.643	Din[1]		rData[1]	ext1_clk	fpga_clk	10.000	3.518	3.407
3	2.378	Din[2]		rData[2]	ext1_clk	fpga_clk	10.000	3.521	3.675
4	2.709	Din[3]		rData[3]	ext1_clk	fpga_clk	10.000	3.488	3.311
5	4.729	rData[0]		Dout[0]	fpga_clk	ext2_clk	10.000	-3.522	2.249
6	4.932	rData[1]		Dout[1]	fpga_clk	ext2_clk	10.000	-3.518	2.050
7	4.889	rData[2]		Dout[2]	fpga_clk	ext2_clk	10.000	-3.521	2.090
8	5.153	rData[3]		Dout[3]	fpga_clk	ext2_clk	10.000	-3.488	1.859

图 7.3.7 实验十三（左）与实验十一（右）的 setup 时序报告。

	Slack	From	Δ	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	3.330	Din[0]		rData[0]	ext1_clk	fpga_clk	0.000	3.336	2.656
2	3.139	Din[1]		rData[1]	ext1_clk	fpga_clk	0.000	3.466	2.606
3	3.451	Din[2]		rData[2]	ext1_clk	fpga_clk	0.000	3.352	2.793
4	3.198	Din[3]		rData[3]	ext1_clk	fpga_clk	0.000	3.357	2.545
5	2.682	rData[0]		Dout[0]	fpga_clk	ext2_clk	0.000	-3.336	1.387
6	2.922	rData[1]		Dout[1]	fpga_clk	ext2_clk	0.000	-3.466	1.517
7	2.859	rData[2]		Dout[2]	fpga_clk	ext2_clk	0.000	-3.352	1.367
8	2.683	rData[3]		Dout[3]	fpga_clk	ext2_clk	0.000	-3.357	1.387

	Slack	From	Δ	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	5.369	Din[0]		rData[0]	ext1_clk	fpga_clk	0.000	3.522	3.543
2	5.237	Din[1]		rData[1]	ext1_clk	fpga_clk	0.000	3.518	3.407
3	5.502	Din[2]		rData[2]	ext1_clk	fpga_clk	0.000	3.521	3.675
4	5.171	Din[3]		rData[3]	ext1_clk	fpga_clk	0.000	3.488	3.311
5	2.271	rData[0]		Dout[0]	fpga_clk	ext2_clk	0.000	-3.522	2.249
6	2.068	rData[1]		Dout[1]	fpga_clk	ext2_clk	0.000	-3.518	2.050
7	2.111	rData[2]		Dout[2]	fpga_clk	ext2_clk	0.000	-3.521	2.090
8	1.847	rData[3]		Dout[3]	fpga_clk	ext2_clk	0.000	-3.488	1.859

图 7.3.8 实验十三（左）与实验十一（右）的 hold 时序报告。

图 7.3.7 是实验十三与实验十一之间的 setup 时序报告比较,反之图 7.3.8 是 hold 时序报告的比较,笔者虽然知道此刻的比较没有任何实际的意义,不过我们看看也无妨。

到目前为止,我们已经知道分别有 3 个情况的供源时钟会出现在 TimeQuest 里,亦即 CLK 直接驱动供源时钟;CLK 驱动寄存器,寄存器再驱动供源时钟;或者 CLK 驱动 PLL,再由 PLL 驱动供源时钟。不过不管哪一样,将供源时钟告诉 TimeQuest 的方法都大同小异。



7.4 引脚电容性与上下坡信号

Warning: Found N output pins without output pin load capacitance assignment

每当我们编译完毕某个模块的时候，Quartus 就会反馈给我们知道，某个输出管脚忘记设置电容性，如上所示。事实上电容性，比起延迟特性，它更接近电器特性，不过本身也夹杂一些延迟特性 ... 用语言描述会很抽闲，笔者还是用图来说话吧。

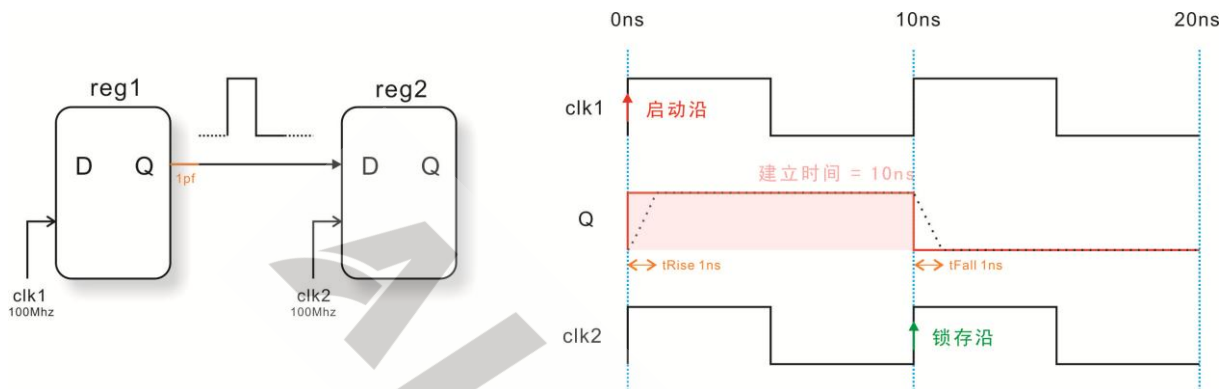


图 7.4.1 1pf 输出口的寄存器（左），有 tRise 与 tFall 的时序图（右）。

为了方面解释我们先假设 $1\text{pf} \approx 1\text{ns}$ ，如图 7.4.1 的左图所示有一对节点，reg1 向 reg2 先发送一个时钟周期的拉高，然后有发送一个周期的拉低，然而右图就是时序图。对 TimeQuest 而言，它是没有办法绘出斜坡的黑色虚线，而是红色的实线。只有电器特性的信号才能使信号上坡或者下坡，虽然 TimeQuest 是针对物理时序的数学工具，但它本身却不参与电器特性的活动。

虽说如此，信号的上下坡却会影响物理时序，因此 TimeQuest 必须采取特殊的手段。tRise 与 tFall 延迟信息，事实上它们与实际的延迟信息却有不同行为，这话何解呢？一般的延迟信息会使信号向右移动，然而 tRise 与 tFall 却像寄生虫一样寄生在数据里，使部分数据无效化。如右图所示，数据 D 有 10ns 的建立时间，不过 tRise 却吃掉开始的 1ns，结果为 9ns。

此外，图 7.4.1 还隐藏一个重要信息，亦即 tRise 与 tFall 仅对建立时间有效，换句话说 tRise 与 tFall 也只是针对发送方，如左图的 reg1。至于 reg2 作接收方，它会免疫 tRise 与 tFall 的寄生攻击。

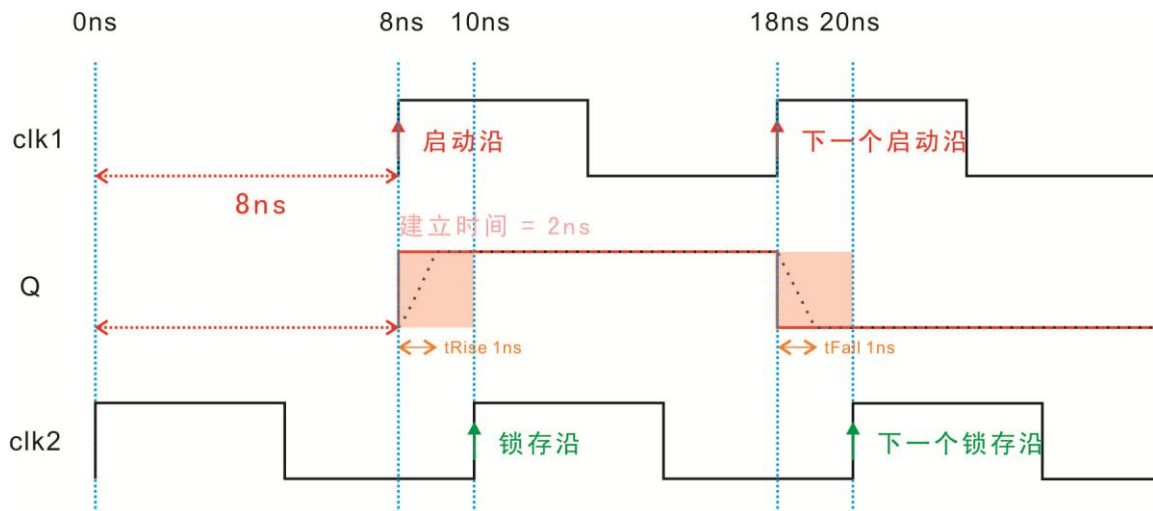


图 7.4.3 clk1 有 8ns 延迟的时序图。

我们再假设 clk1 有 8ns 的延迟，自然而然数据 Q 也会向右边移动 8ns，从而导致建立时间为 2ns，不过再加上 t_{Rise} 的 1ns 寄生攻击，数据前面的 1ns 部分已经失效，因此建立时间为 1ns。在第 18ns 的位置，数据 Q 由高变低，此时 t_{Fall} 寄生物攻击数据，数据失去 1ns 的建立时间，让原本 2ns 的建立时间成为 1ns。

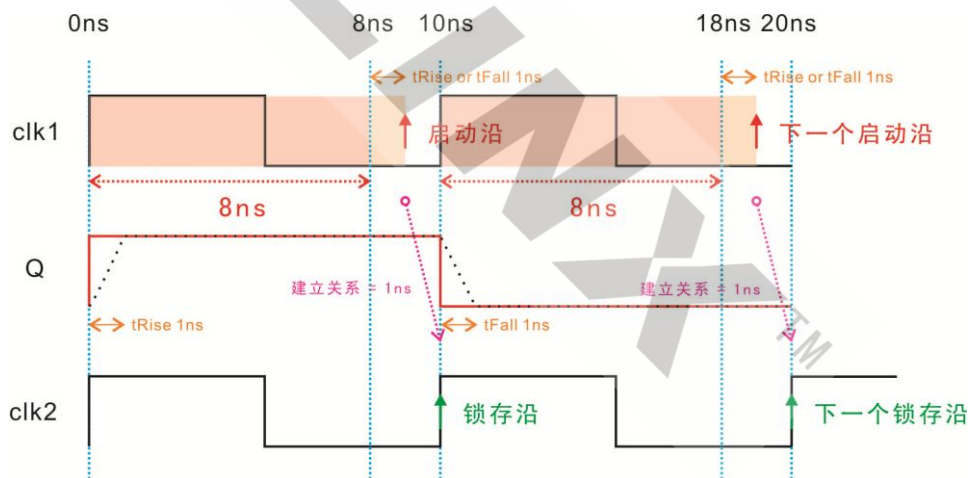


图 7.4.4 建立关系的等价图。

图 7.4.4 是建立关系的等价图 我们知道数据有延迟，启动沿也会跟着移动，结果说 ... 原本在 0ns 的启动沿受 8ns 的延迟后，落在 8ns 的位置，不过再加上 t_{Rise} 或者 t_{Fall} 的寄生攻击，启动沿最后倒在 9ns 的位置，导致建立关系为 1ns。然而时间为 10ns~20ns 的时候，数据 Q 也遇见同样的状况。

认真思考一下，图 7.4.3 与图 7.4.4 它们之间的建立时间与建立关系都为 1ns。结果我们可以如此断定， t_{Rise} 或者 t_{Fall} 的寄生攻击，针对建立时间也针对建立关系，换之它们同样也针对启动沿和下一个启动沿（但不是与保持关系有关的下一个启动沿）。只要理清上述的重点，我们变可以采取的措施。

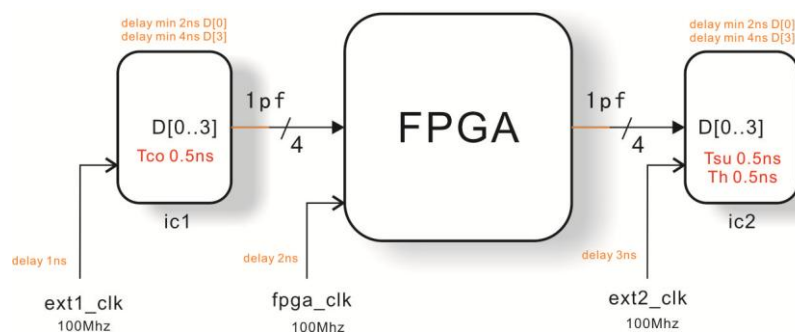


图 7.4.5 假想外部模型。

假设有一个假想的外部模型，其中 ic1 的 io 口有 1pf 的电容性，而 fpga 的 io 口也有 1pf 的电容性。此外我们也知道 ic1 致 fpga 是 ic2fpga 的外部模型，亦即 ic1 发送 fpga 接收；换之 fpga 致 ic2 是 fpga2ic 的外部模型，亦即 fpga 发送 ic2 读取。而 fpga 的 hdl 内容如下：

```

1. module center_module
2. (
3.     input CLK,
4.     input RSTn,
5.     input [3:0]Din,
6.     output [3:0]Dout
7. );
8.     /*****/
9.
10.    reg [3:0]rData;
11.
12.    always @ ( posedge CLK or negedge RSTn )
13.        if( !RSTn )
14.            begin
15.                rData <= 4'd0;
16.            end
17.        else
18.            begin
19.                rData <= Din;
20.            end
21.
22.    /*****/
23.
24.    assign Dout = rData;
25.
26.    /*****/
27.
28. endmodule

```

在第 5 行声明的 Din 是 ic1 给 fpga 的输入口，而第 6 行声明的 Dout 则是 fpga 给 ic2 的输出口。换句话说，Din~rData 有 4 个节点，而 rData~Dout 有又 4 个节点。首先我们可以使用 set_clock_uncertainly 的约束命令告诉 TimeQuest 相关的电容性信息 ...

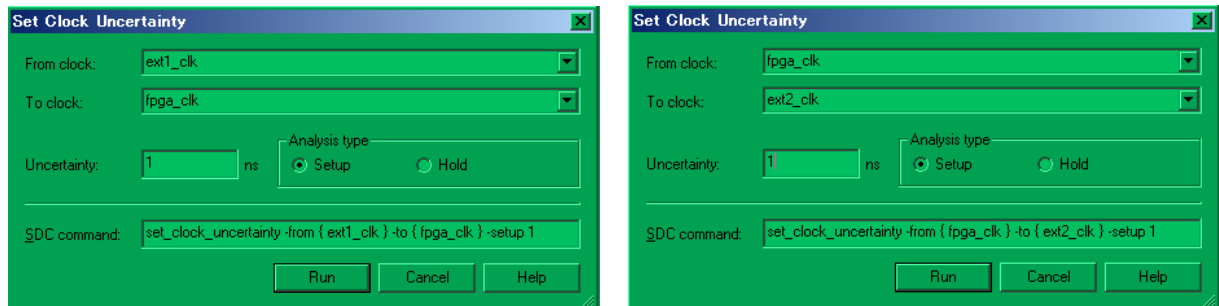


图 7.4.6 使用 set_clock_uncertainly 确保电容性。

如图 7.4.6 所示，左图是 ic2fpga 的外部模型，亦即针对节点 Din~rData，我们在 from clock 选择 ext1_clk 然后 to clock 选择 fpga_clk，至于 analysis type 使能 Setup 最后在 Uncertainty 中输入 1ns 的值。同样，右图是 fpga2ic 的外部模型，亦即针对节点 rData~Dout，而 from clock 选择 fpga_clk，to clock 则是 ext2_clk2，analysis type 也是 setup，uncertainty 也是 1ns。

完后，节点 Din~rData 与节点 rData~Dout 的 setup 分析都会保留 1ns 的 setup 余量，只要余量一天为正直，tRise 与 tFall 的寄生攻击也不会坏掉数据的有效性。

除了使用 set_clock_uncertainly 约束命令以外，我们还可以使用 input/output 公式来确保 tRise 和 tFall ... 首先我们必须知道电容性仅影响建立关系，而 input/output max 也是针对建立关系而已，因此我们可以这样更改 input/output 公式。

```
input max      = <ext2fpga delay max> - < destination reg clk delay - source reg clk delay > + ext_Tco
               = <ext2fpga delay max> - < clock skew > + ext_Tco + < tRise/tFall fpga io >
```

```
output max     = <fpga2ext delay max> - < destination reg clk delay - source reg clk delay > + ext_Tsu
               = <fpga2ext delay max> - < clock skew > + ext_Tsu + < tRise/tFall ic io >
```

input 是指 fpga2ic 外部模型，亦即 fpga 输出数据而 ic 读取，其中 fpga 的 io 口的电容性是关键，所以 input max 加上 tRise 或者 tFall 的时间；而 output 是指 ic2fpga 外部模型，亦即 ic 输出数据而 fpga 读取，其中 ic 的 io 口电容性是关键，所以 output max 加上 tRise 或者 tFall 的时间。

同学们一定觉得很好奇 ... 到底 fpga 引脚是如何换取电容性的 tRise 或者 tFall 呢？在这里有一个非常粗略也很简单的办法。

Symbol	Parameter	Typical	Unit
C _{IO}	Input capacitance for user I/O pin.	6	pF
C _{LVDS}	Input capacitance for dual-purpose LVDS/user I/O pin.	6	pF
C _{VREF}	Input capacitance for dual-purpose VREF pin when used as VREF or user I/O pin.	21	pF
C _{CLK}	Input capacitance for clock pin.	5	pF

图 7.4.5 Cyclone II 的引脚电容性。

图 7.4.5 所示是 Cyclone II 家族 fpga 的 IO 引脚有 6pF，而 Time Constant 的公式下：

$$t = RC$$

t 固定时间

r 引脚阻值

c 引脚电容

一般上 IO 的阻值非常小 ... 都是 1 Ohm 以下，不过为了方便计算，也为了增加保险余量，IO 的阻值都取为 1 Ohm，此外 IO 电容性的 F 单位直接对应 s (秒)，因此：

$$\begin{aligned}
 t &= RC \\
 &= 1(6pf) \\
 &= 6ps \\
 t_{Rise}/t_{Fall} &= 6ps
 \end{aligned}$$

以上纯属粗略的算法而已，实际的 tRise 与 tFall 取值还要根据相关的手册。虽说信号的上下山会影响时序的合格化，不过在默认的情况下我们一般都会无视它们，因为它们实在小到可怜。除非是时序非常挑剔的设计，否则我们看也不看它们一眼。

7.5 介电参数与延迟时间

笔者曾在第三章说过，粗略点认为介电参数就像 PCB 大道上的柏油路，还是沙石路 ... 如果一台汽车在这条大道上行驶，很明显柏油路比起砂石路更顺畅，因为砂石路给予更大的行驶阻碍。介电参数就是这么一回事，不同材料的 PCB，会有不同数值的介电参数，越高的数值，走线就有越高的阻值。

一般经济型的 PCB，原材料都是 FR4，而 FR4 的介电参数是 4.4~5.2 之间，至于详细的介电参数信息可以向 PCB 厂商要。笔者自认是懒人，所以笔者也不打算解释介电参数换算延迟时间的过程，换之我们要懂得享用前人种好的大树。

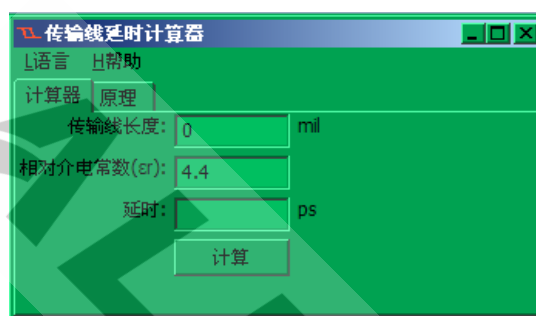


图 7.5.1 传输线延迟计算器。

图 7.5.1 出现的小软件是网友 Aaron Lee “开发”的超方便应用程序，只要根据走线的长度，再输入相关的介电参数，然后轻轻点击一下《计算》就会自动求出走线的延迟时间。看吧，是不是很傻瓜？是不是很方便？至少笔者是这么认为 ...

7.6 IO 口

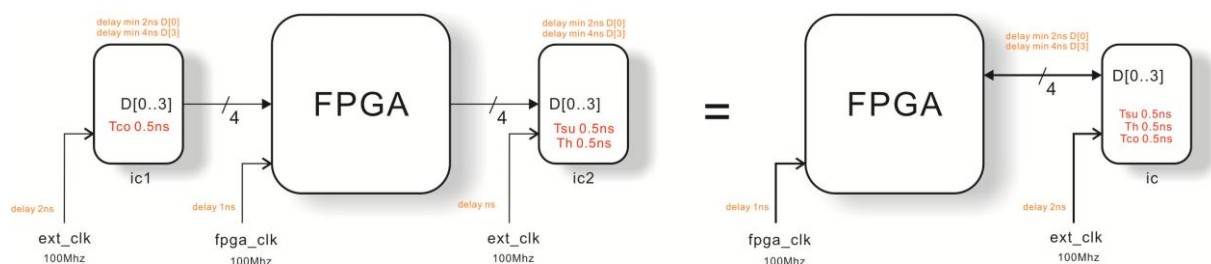


图 7.6.1 IO 口的外部模型（右）。

有关 IO 口的外部模型有如图 7.6.1 所示，实际上它就是输出方的 ic1 与输入方的 ic2 结为一体，此时的外部模型既是 fpga2ic 也是 ic2fpga。然而在约束的执行上倒是没有巨大不同，不同只是就在于针对对象已经二合为一，不管怎么说，我们还是做实验比较实际。

实验十四 分析和约束 io 口外部模型

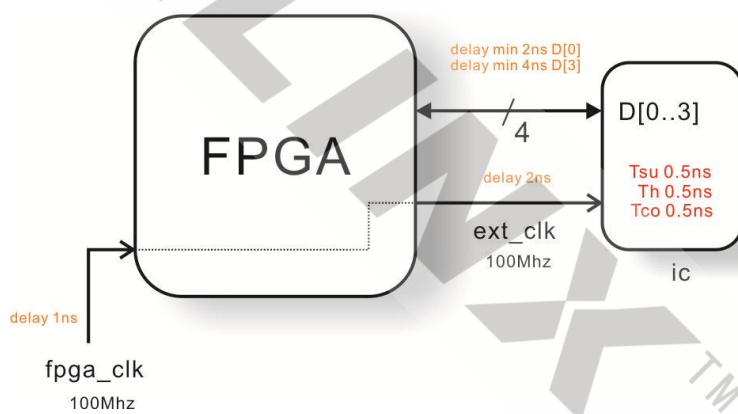


图 7.6.2 供源时钟与 io 口的外部模型。

如图 7.6.2 所示，图中是一个有供源时钟与 io 口的外部模型，它虽然看似有点难度的样子，不过不用担心，我们会一步步分析它。事不宜迟，让我们先瞧瞧 fpga 的 hdl 模块如何描述呢？

center_module.v

```
1. module center_module
2. (
3.     input CLK,
4.     input RSTn,
5.     inout [3:0]Data,
6.     output ext_clk
```

```

7. );
8.  /*****/
9.
10.    reg [3:0]rData;
11.    reg isOut;
12.    reg i;
13.
14.    always @ ( posedge CLK or negedge RSTn )
15.        if( !RSTn )
16.            begin
17.                rData <= 4'd0;
18.                isOut <= 1'b0;
19.                i <= 1'b0;
20.            end
21.        else
22.            case( i )
23.
24.                0:
25.                    begin isOut = 1'b0; rData <= Data; i <= 1'b1; end
26.
27.                1:
28.                    begin isOut = 1'b1; i <= 1'b0; end
29.
30.            endcase
31.
32.    /*****/
33.
34.    assign Data = isOut ? rData : 4'dz;
35.    assign ext_clk = CLK;
36.
37.    /*****/
38.
39. endmodule

```

上述是一个没什么大不了的模块 第5行声明了名为 Data 的 io 口 ,而第6行声明 ext_clk 输出时钟。第 22~30 行是该模块的核心操作 ... 现在步骤 0 从 ic 读取数据 ,然后在步骤 1 将数据输出 ,其中 isOut 寄存器成为 io 口开关的关键。再者计算和举例相关的延迟信息 :

```

data delay min 2ns
data delay max 4ns
fpga_clk delay 1ns
ext_clk delay 2ns

```

Tsu/Th/Tco 0.5ns

```
input max      = <ext2fpga delay max> + ext_Tco
               = 4ns + 0.5
               = 4.5ns

input min      = <fpga2ext delay min> + ext_Tco (或者 ext_minTco)
               = 2ns + 0.5ns
               = 2.5ns
```

```
output max     = <fpga2ext delay max> + ext_Tsu
               = 4ns + 0.5ns
               = 4.5ns

output min     = < fpga2ext delay min > - ext_Th
               = 2ns - 0.5ns
               = 1.5ns
```

接着为实验十四建立同名 sdc 文件，然后将下列代码拷贝进去：

```
*****
# Create Clock
*****

create_clock -name {fpga_clk} -period 10.000 -waveform { 0.000 5.000 } [get_ports {CLK}]
```

打开 TimeQuest 手动建立 model fast 网表，然后执行管理的连打 ...

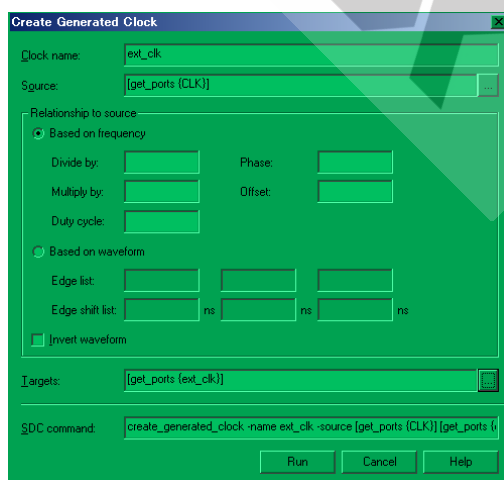


图 7.6.3 建立 ext_clk 供源时钟。

先透过 create generated clock 建立 ext_clk 供源时钟，操作如图 7.6.3 所示，完后请记得更新网表，让 ext_clk 生效。

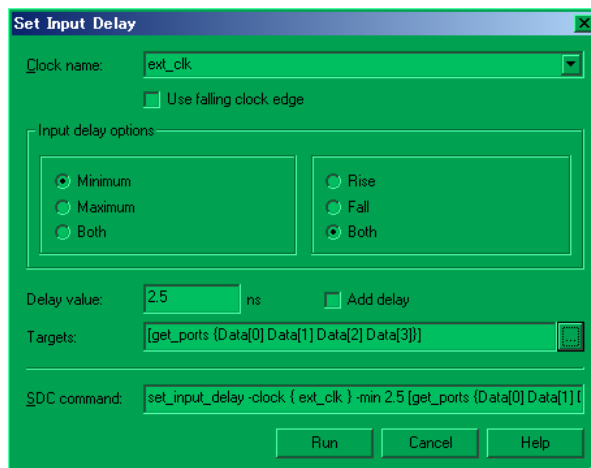
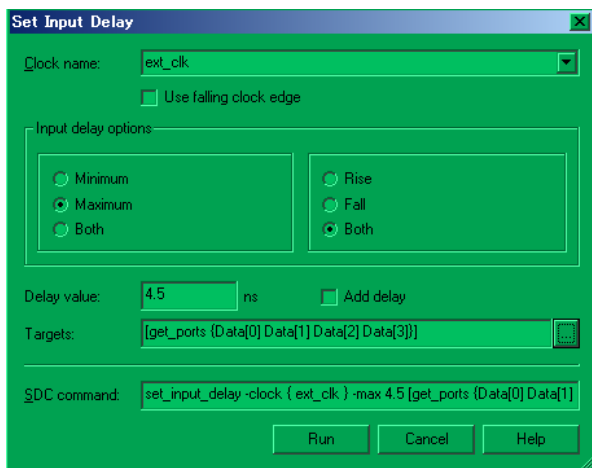


图 7.6.4 为 io 的 input 方设置约束。

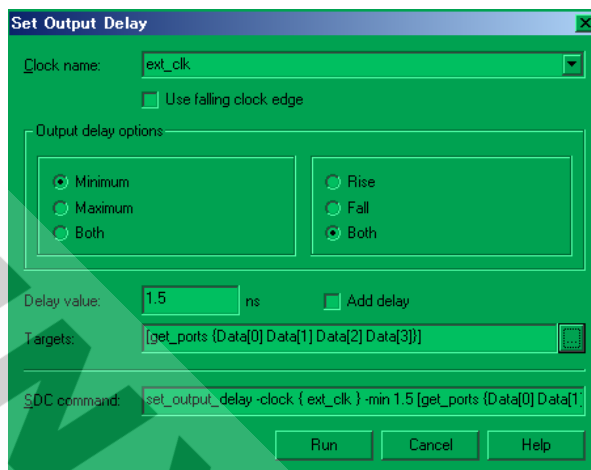
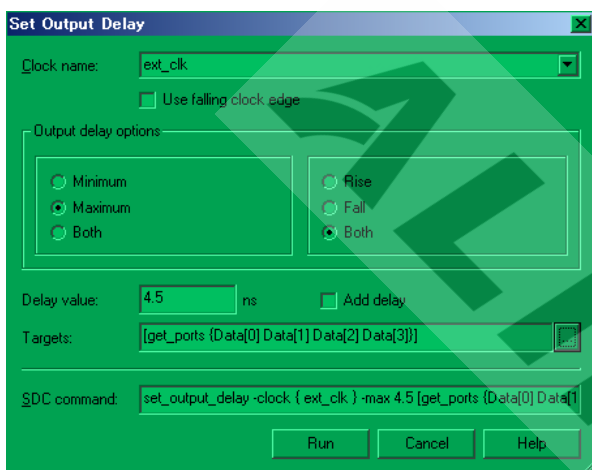


图 7.6.5 为 io 的 output 方设置约束。

不管 input 方或者 output 方，是 io 都有同样的 clock name 对象，与 Targets 对象，不同的只是 input 时的约束行为，或者 output 时的约束行为。如图 7.6.4 与图 7.6.5 所示，各种 input/output 的 min/max 设置够根据之前都计算得到的延迟信息。完后再更新一下网表，以便生效。

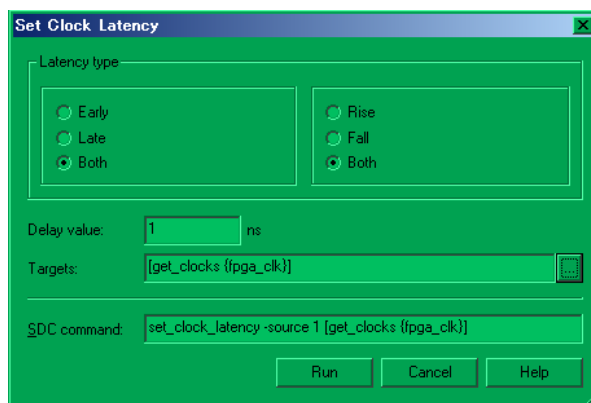
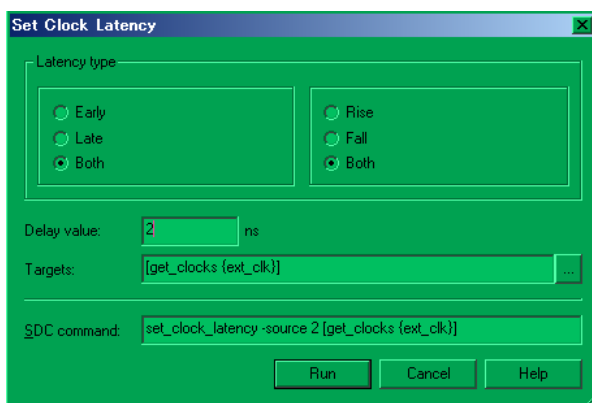


图 7.6.5 设置外部时钟路径的延迟。

接下来设置 `fpga_clk` 与 `ext_clk` 的外部路径的延迟信息，操作如图 7.6.5 所示。完后也记得更新一下网表，就这样，实验十四的约束过程就完结了，看不看时序结果一点都不重要。

实验十四的要点除了让同学们认识 io 口的约束方法以外，笔者也先想让同学们热身热身一下 ... 因为从下一章开始，就是我们久违的目标，亦即 `sdram` 实例。实际上 `sdram` 的约束设置和实验十四差不了多少而已，不过不同的是实验十四时钟是假想实验而已，换之 `sdram` 是真实的实例，其中有许多细节是假想实验不能体现的。



7.7 SDRAM 实例

假如笔者说，我们之前的努力都是为了挑战 sdr 而作准备，同学们可否相信吗？笔者就是这样的男人，为了一件不解的小事，就会从最基本做起 ... 虽然旁人会笑过程慢，耗精力。不过让笔者告诉同学们，这一切的付出一定是十分有价值 ... 感动的言辞先放在一旁，让我们开始步入冒险！

有关 sdr 的驱动原理，麻烦同学回顾《Verilog HDL 那些事儿》—《整合篇》，在这里我们要将全部精力放在约束的操作上。

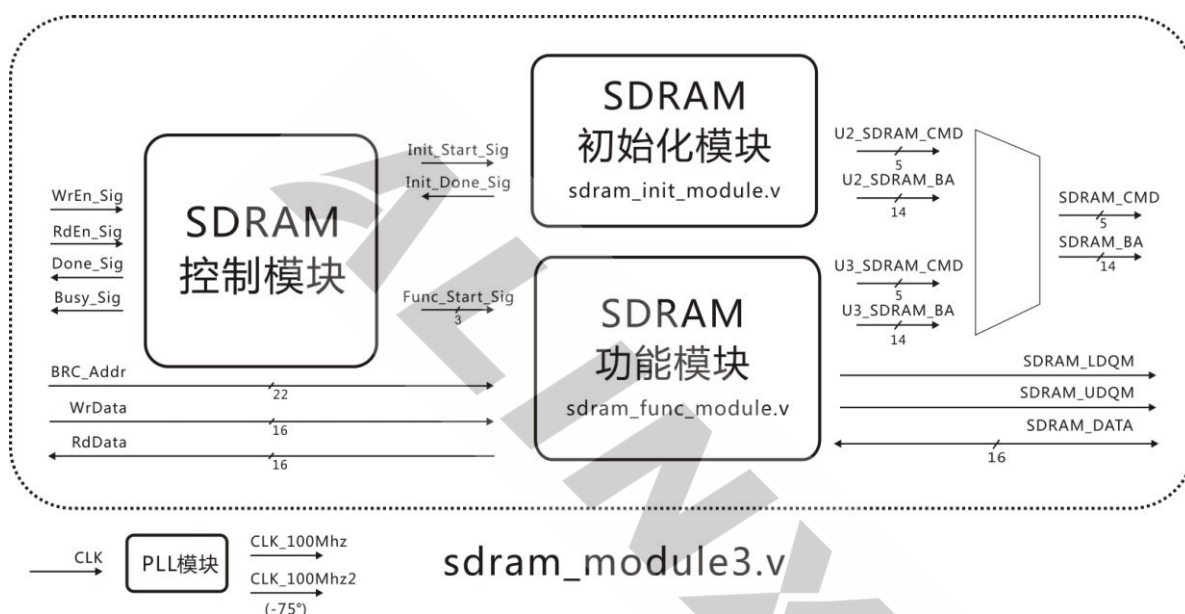


图 7.7.1 sdr 的建模图。

但是看图 7.7.1，笔者想是会吓跑一群小朋友吧，实际上模块的内容不是那么复杂，只是 sdr 的驱动原理烦人一点。让我们看看模块的内容：

```
1. module sdr_demo3
2. (
3.     input CLK,
4.     input RSTn,
5.
6.     output SDRAM_CLK,
7.     output [4:0]SDRAM_CMD,
8.     output [13:0]SDRAM_BA,
9.     inout [15:0]SDRAM_DATA,
10.
11.     output SDRAM_UDQM,
```

```

12.     output SDRAM_LDQM
13. );
14.
15.     /*****/
16.
17.     wire CLK_100Mhz;
18.     wire CLK_100Mhz2;
19.
20.     pll_module U1
21.     (
22.         .inclk0 ( CLK ),
23.         .c0 ( CLK_100Mhz ),
24.         .c1 ( CLK_100Mhz2 )
25.     );
26.
27.     /*****/
28.
29.     wire Done_Sig;
30.     wire Busy_Sig;
31.     wire [15:0]RdData;
32.
33.     sdram_module3 U2
34.     (
35.         .CLK( CLK_100Mhz ),
36.         .RSTn( RSTn ),
37.         .WrEN_Sig( 1'b1 ),
38.         .RdEN_Sig( 1'b1 ),
39.         .Done_Sig( Done_Sig ),
40.         .Busy_Sig( Busy_Sig ),
41.         .BRC_Addr( 22'd1 ),
42.         .WrData( 16'd1 ),
43.         .RdData( RdData ),
44.         .SDRAM_CMD( SDRAM_CMD ),
45.         .SDRAM_BA( SDRAM_BA ),
46.         .SDRAM_DATA( SDRAM_DATA ),
47.         .SDRAM_UDQM( SDRAM_UDQM ),
48.         .SDRAM_LDQM( SDRAM_LDQM )
49.     );
50.
51.     /*****/
52.
53.     assign SDRAM_CLK = CLK_100Mhz2;
54.

```

```

55.      /*****/
56.
57. endmodule

```

第 6 行声明 SDRAM_CLK 的供源时钟，它主要由 pll 的 c1 在 24 行经由 CLK_100Mhz 连线，最后在第 53 行驱动。此外第 7~8 行是 5 位的 SDRAM_CMD 输出，还有 14 为的 SDRAM_BA 输出；第 9 行是 inout 的 SDRAM_DATA；第 11~12 行则是 SDRAM_LQM 与 SDRAM_UQM 输出声明。至于其他的内容不关紧要 ...

实验十五 SDRAM 约束实例

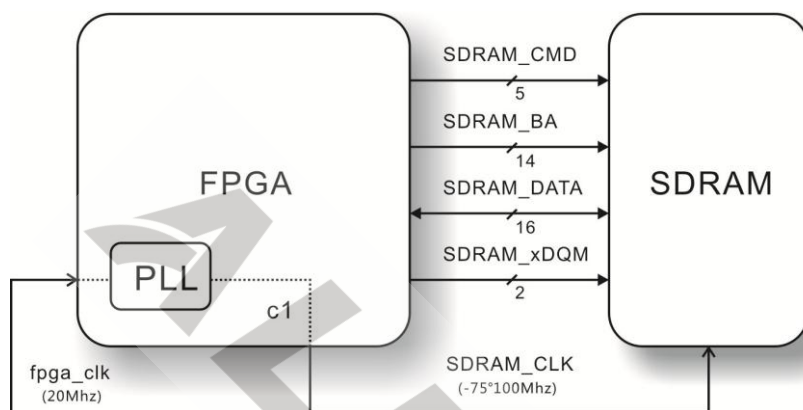


图 7.7.2 fpga 与 sdram 的外部模型。

图 7.7.2 是 fpga 与 sdram 的外部模型 其中 fpga 有 20Mhz 的 fpga_clk 时钟信号 而 sdram 是由 fpga_clk 经 pll 的 c1 输出而成 有-75 度位移 100Mhz 频率的供源时钟 SDRAM_CLK 驱动。此外还有 fpga 输出的 5 位宽 SDRAM_CMD，14 位宽 SDRAM_BA，各自 1 位宽的 SDRAM_LDQM 还有 SDRAM_UDQM。接下来，我们则要取出相关的延迟信息 ...

S_A7	2	1118.484mil	0mil
S_A8	2	1074.951mil	0mil
S_A9	2	1022.89mil	0mil
S_A10	2	1047.163mil	0mil
S_A11	2	983.622mil	0mil

S_DB14	2	439.371mil	0mil
S_DB15	2	383.97mil	0mil

S_NCS	2	1296.516mil	0mil
S_NRAS	2	957.242mil	0mil
164 Nets (1 Highlighted)			
Name	Node Count	Routed	Un-Routed ...
164 Nets (1 Highlighted)			
Name	Node Count	Routed	Un-Routed ...
S_LDQM	2	851.31mil	0mil

图 7.7.3 SDRAM 的各种走线长度。

图 7.7.3 是从核心版的原理图中取得的相关路径信息，由于商业关系笔者就不公开相关的原理图文件了，有望同学们见谅。其中 CLOCK 对应 fpga_clk，S_CLK 对应 SDRAM_CLK，B_Ax 对应 SDRAM_BA，S_DBx 对应 SDRAM_DATA，紧接着的 S_CKE 与 S_Nx 对应 SDRAM_CMD，S_UDQM 对应 SDRAM_UQDM，S_LDQM 对应 SDRAM_LQDM。

除了 fpga_clk，SDRAM_CLK，还有 SDRAM_UDQM 与 SDRAM_LQDM 以外的路径都是多位宽数据。前面我们曾经学过，对多位宽的数据而言，我们仅取最大延迟 (max) 和最小延迟 (min) 的路径即可。然后，用章节 7.5 的小软件，介电参数设置为 4.4，将

路径长度换算为延迟时间，结果如下：

```
fpga_clk delay 0.022ns
SDRAM_CLK delay 0.158ns (-75度)
SDRAM_BA max delay 0.218ns
SDRAM_BA min delay 0.172ns
SDRAM_DATA max delay 0.136ns
SDRAM_DATA min delay 0.068ns
SDRAM_CMD max delay 0.227ns
SDRAM_CDM min delay 0.156ns
SDRAM_UQDM delay 0.152ns
SDRAM_LQDM delay 0.149ns
SDRAM 相关的寄存器特性，相关手册查不到 ...
```

其中让读者觉得困惑的就是附有-75°的 SDRAM_CLK 路径，关于这点同学们就不用多心了，SDRAM_CLK 的供源是 pll 的 c1，而且 c1 原有-75 度位移的设置。接着打开实验十五的 sdr_demo3，然后建立同名的 sdc 文件，然后先将下列的代码拷贝进去：

```
#####
# Create Clock
#####

create_clock -name {CLK} -period 50.000 -waveform { 0.000 25.000 } [get_ports {CLK}]

#####
# Create Generated Clock
#####

derive_pll_clocks
```

原时钟是频率 20Mhz 的时钟信号，对应 CLK 输入口，而 derive_pll_clocks 就是声明 pll 的时钟源。接着打开 TimeQuest，然后手动建立 create_timing_netlist -model fast 网表，然后惯例的执行双击 Read SDC File 和 Update Timing Netlist。

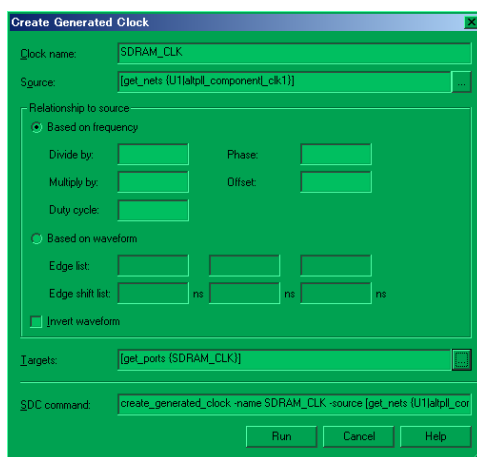


图 7.7.4 建立 SDRAM_CLK 供源时钟。

打开 create generated clock 界面，声明供源时钟 SDRAM_CLK，源对象是 net 资源的 pll|clk1，而目前是 SDRAM_CLK 输出口，如图 7.7.4 所示。

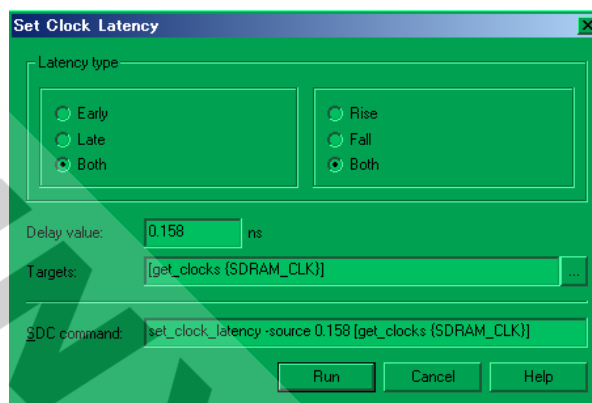
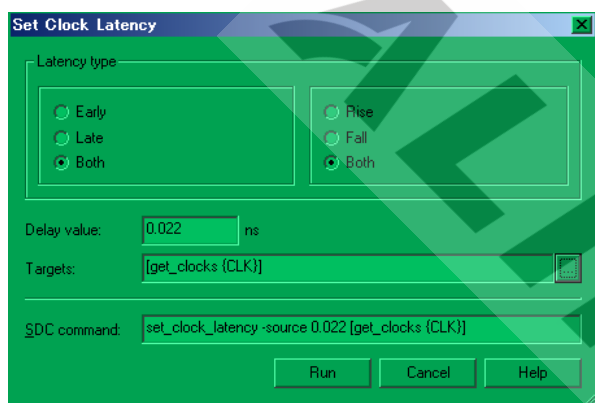


图 7.7.5 声明外部时钟信号的延迟。

打开 set clock latency 界面，声明 CLK 时钟路径有 0.022ns 延迟，SDRAM_CLK 时钟路径有 0.158ns 延迟，如图 7.7.5 所示。

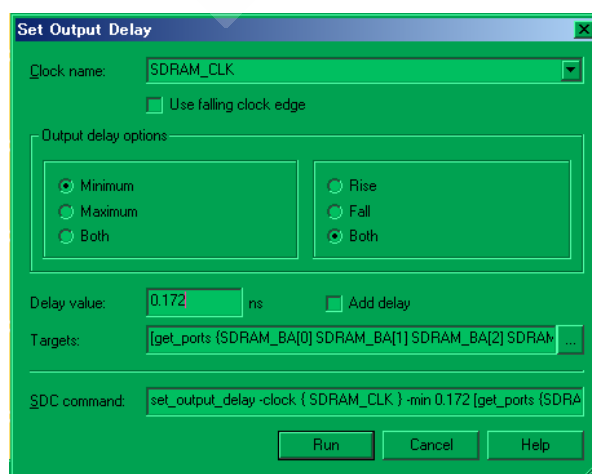
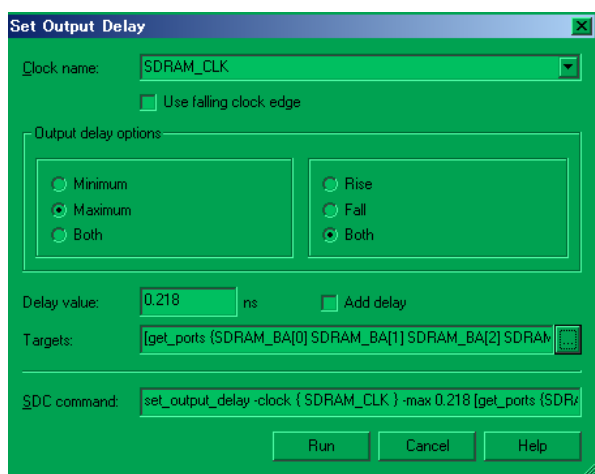


图 7.7.7 声明 SDRAM_BA 的输出延迟。

打开 set output delay 界面，对象是 SDRAM_BA 输出口而最大延迟是 0.218ns，最小延迟是 0.172ns，如图 7.7.7 所示。

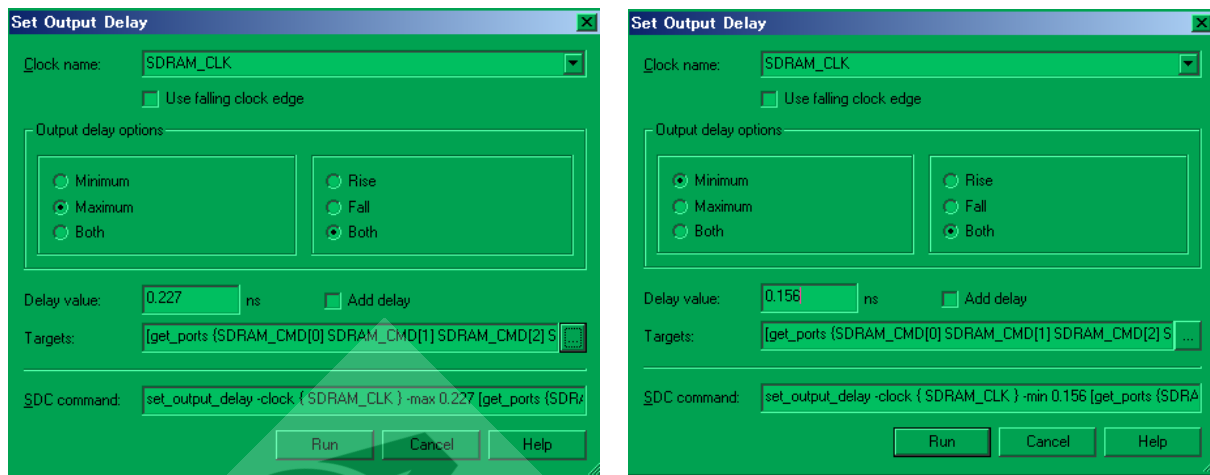


图 7.7.8 声明 SDRAM_CMD 的输出延迟。

打开 set output delay 界面，对象是 SDRAM_CMD 输出口而最大延迟是 0.227ns，最小延迟是 0.156ns，如图 7.7.8 所示。

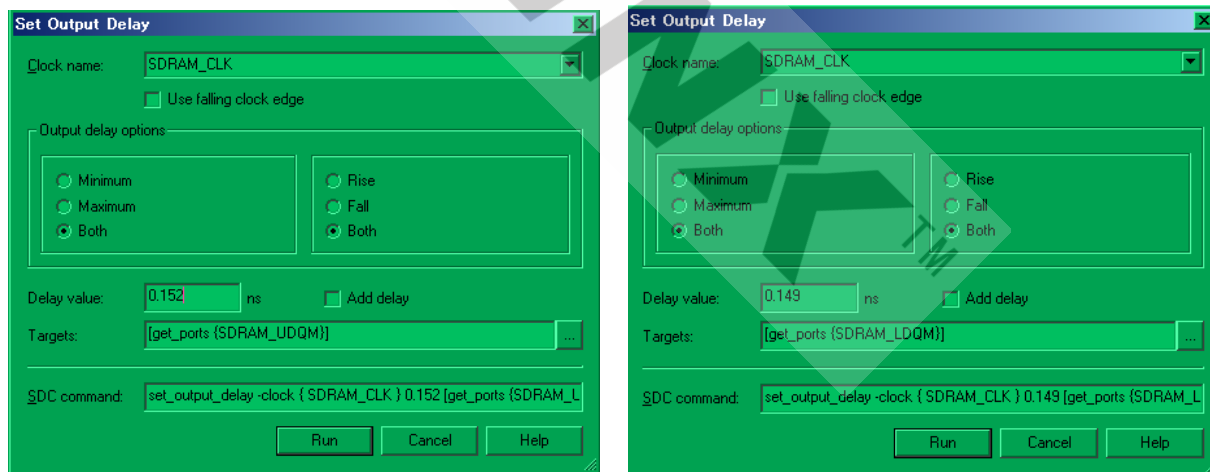


图 7.7.9 声明 SDRAM_xDQM 的输出延迟。

打开 set output delay 界面，对象是 SDRAM_UQDM 与 SDRAM_LUDQM，它两都是单位宽信号，所以为 Both ... 前者的延迟是 0.152ns，后者的延迟是 0.149ns，如图 7.7.9 所示。

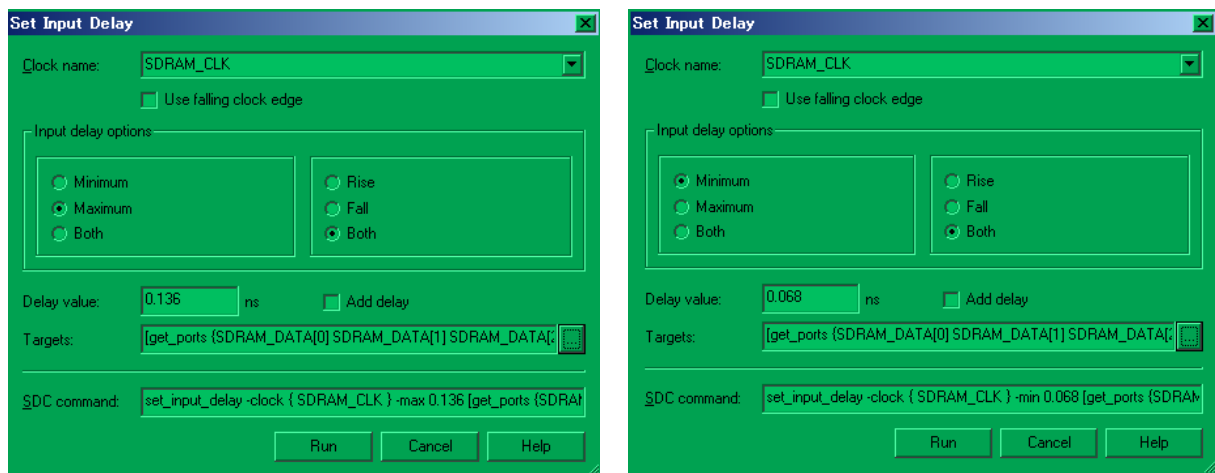


图 7.7.9 声明 SDRAM_DATA 的输入延迟。

打开 set input delay 界面，对象是 SDRAM_DATA 的输入口而最大延迟是 0.136ns，最小延迟是 0.068ns，如图 7.7.9 所示。

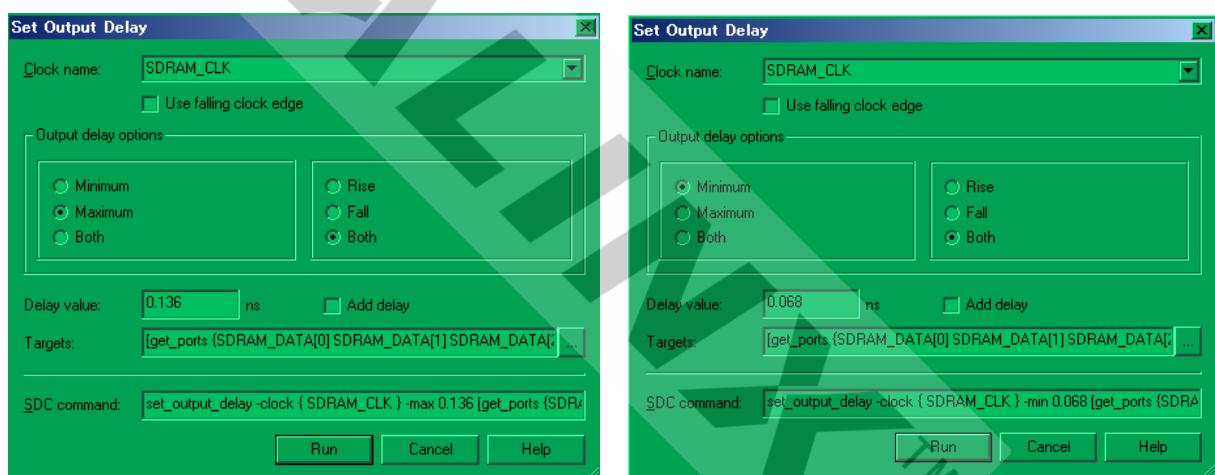


图 7.7.10 声明 SDRAM_DATA 的输出延迟。

打开 set output delay 界面，对象是 SDRAM_DATA 的输出口而最大延迟是 0.136ns，最小延迟是 0.068ns，如图 7.7.10 所示。同学们用不着怀疑，为什么图 7.7.9 和图 7.7.10 的约束行为都针对 SDRAM_DATA，因为 SDRAM_DATA 是 IO 口。

完后，sdc 文件打开是这个样子：

```
*****
# Create Clock
*****

create_clock -name {CLK} -period 50.000 -waveform { 0.000 25.000 } [get_ports {CLK}]
```

```

#####

# Create Generated Clock

#####

derive_pll_clocks

create_generated_clock -name SDRAM_CLK -source [get_nets {U1|altpll_component|_clk1}] [get_ports
{SDRAM_CLK}]

#####

# Set Clock Latency

#####

set_clock_latency -source 0.022 [get_clocks {CLK}]
set_clock_latency -source 0.158 [get_clocks {SDRAM_CLK}]

#####

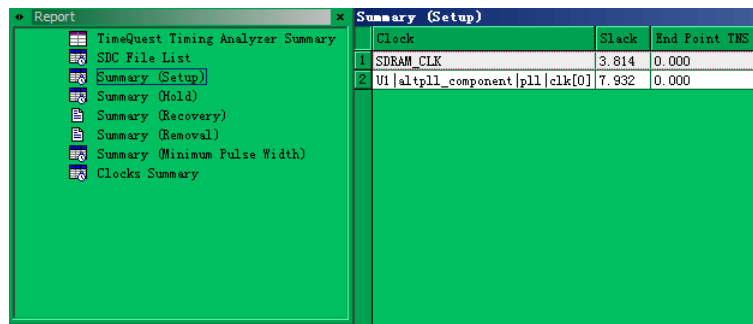
# Set Input Delay

#####

set_input_delay -add_delay -max -clock [get_clocks {SDRAM_CLK}] 0.136 [get_ports {SDRAM_DATA[0]}]
set_input_delay -add_delay -min -clock [get_clocks {SDRAM_CLK}] 0.068 [get_ports {SDRAM_DATA[0]}]
set_input_delay -add_delay -max -clock [get_clocks {SDRAM_CLK}] 0.136 [get_ports {SDRAM_DATA[1]}]
set_input_delay -add_delay -min -clock [get_clocks {SDRAM_CLK}] 0.068 [get_ports {SDRAM_DATA[1]}]
set_input_delay -add_delay -max -clock [get_clocks {SDRAM_CLK}] 0.136 [get_ports {SDRAM_DATA[2]}]
set_input_delay -add_delay -min -clock [get_clocks {SDRAM_CLK}] 0.068 [get_ports {SDRAM_DATA[2]}]
set_input_delay -add_delay -max -clock [get_clocks {SDRAM_CLK}] 0.136 [get_ports {SDRAM_DATA[3]}]
set_input_delay -add_delay -min -clock [get_clocks {SDRAM_CLK}] 0.068 [get_ports {SDRAM_DATA[3]}]
set_input_delay -add_delay -max -clock [get_clocks {SDRAM_CLK}] 0.136 [get_ports {SDRAM_DATA[4]}]
set_input_delay -add_delay -min -clock [get_clocks {SDRAM_CLK}] 0.068 [get_ports {SDRAM_DATA[4]}]
set_input_delay -add_delay -max -clock [get_clocks {SDRAM_CLK}] 0.136 [get_ports {SDRAM_DATA[5]}]
set_input_delay -add_delay -min -clock [get_clocks {SDRAM_CLK}] 0.068 [get_ports {SDRAM_DATA[5]}]
set_input_delay -add_delay -max -clock [get_clocks {SDRAM_CLK}] 0.136 [get_ports {SDRAM_DATA[6]}]
set_input_delay -add_delay -min -clock [get_clocks {SDRAM_CLK}] 0.068 [get_ports {SDRAM_DATA[6]}]
set_input_delay -add_delay -max -clock [get_clocks {SDRAM_CLK}] 0.136 [get_ports {SDRAM_DATA[7]}]
set_input_delay -add_delay -min -clock [get_clocks {SDRAM_CLK}] 0.068 [get_ports {SDRAM_DATA[7]}]
set_input_delay -add_delay -max -clock [get_clocks {SDRAM_CLK}] 0.136 [get_ports {SDRAM_DATA[8]}]
set_input_delay -add_delay -min -clock [get_clocks {SDRAM_CLK}] 0.068 [get_ports {SDRAM_DATA[8]}]
set_input_delay -add_delay -max -clock [get_clocks {SDRAM_CLK}] 0.136 [get_ports {SDRAM_DATA[9]}]
set_input_delay -add_delay -min -clock [get_clocks {SDRAM_CLK}] 0.068 [get_ports {SDRAM_DATA[9]}]
set_input_delay -add_delay -max -clock [get_clocks {SDRAM_CLK}] 0.136 [get_ports {SDRAM_DATA[10]}]
set_input_delay -add_delay -min -clock [get_clocks {SDRAM_CLK}] 0.068 [get_ports {SDRAM_DATA[10]}]
set_input_delay -add_delay -max -clock [get_clocks {SDRAM_CLK}] 0.136 [get_ports {SDRAM_DATA[11]}]
set_input_delay -add_delay -min -clock [get_clocks {SDRAM_CLK}] 0.068 [get_ports {SDRAM_DATA[11]}]

```

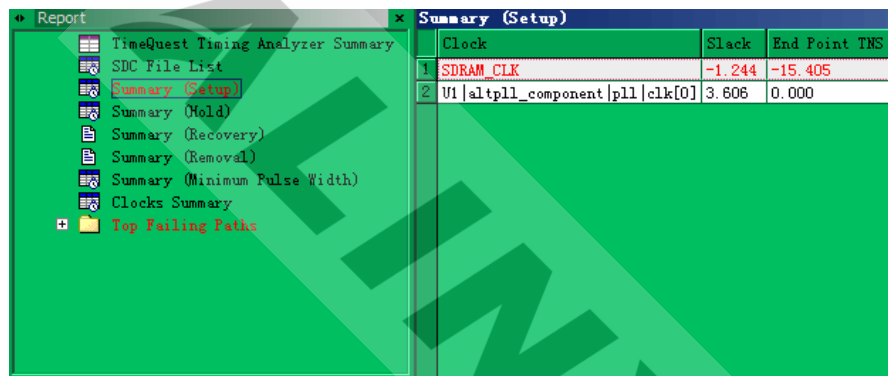

[illegible]



Clock	Slack	End Point TNS
1 SDRAM_CLK	3.814	0.000
2 U1 altpll_component pll clk[0]	7.932	0.000

图 7.7.11 best case 网表造就的 setup 结果。

完后，当我们再次更新网表的时候 ... 一切都是预期的合格结果，最差的建立余量也有 3.814ns，不过 best case 网表的保险余量很小而已 ... 让我们重新打开 TimeQuest，然后使用默认网表亦即 `create_timing_netlist -model slow`，再次分析看看 ... 想定会看见 TimeQuest 反馈红色警报！如图 7.7.12 所示，最差建立余量既然是 -1.244ns。



Clock	Slack	End Point TNS
1 SDRAM_CLK	-1.244	-15.405
2 U1 altpll_component pll clk[0]	3.606	0.000

图 7.7.12 worst case 网表导致 setup 不足的红色警报（编译器优化之前）。

让我们仔细思考看看 ... worst case 网表比起 best case 网表，前者的保险余量虽然比后者高出许多，但是性能限制也因此高出不少，结果而言 best case 网表可以合格的时序，却在 worst case 网表无法合格。不过只要 best case 网表的执行环境不极端的话，想必运行不会崩溃，但是 worst case 网表无法合格，这点多少要人放不下心 ... 如果希望 worst case 网表可以合格我们又该如何呢？

在此，我们需要用到第五章，关于 Fmax 的优化设置了，虽说是优化设置 ... 但是优化对象是编译器而不是 hdl 内容本身，不管怎么样！同学将所有编译器的优化选项开启，然后重新编译，然后打开 TimeQuest 又一次选择 `create_timing_netlist -model slow` 默认网表看看 ...

Clock	Slack	End Point TNS
SDRAM_CLK	-0.791	-6.645
U1 altpll_component pll clk[0]	3.612	0.000

图 7.7.13 worst case 网表导致 setup 不足红色警报（编译器优化之后）。

与优化之前的结果相比，图 7.7.13 所示的最差建立余量已经减少许多，亦即-0.791ns。事实上，如果实验对时序不挑剔的话，这种结果已经是接受了，毕竟建立余量不足在容差在±1.0ns 的范围，不过红色始终是红色，完美主义的我们又该怎么办呢？

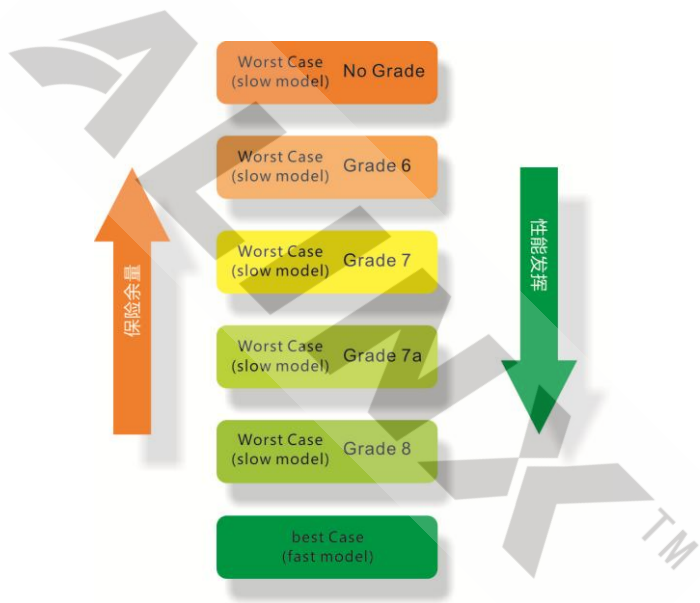


图 7.7.14 不同 grade 等级的 worst case 网表。

根据 TimeQuest 还有 altera 的 fpga 而言，不同家族的 fpga 都有可支持 grade 等级，以 cyclone II 为例，它可以支持的 grade 等级如图 7.7.14 所示。笔者曾说过，越极端的网表质量保险余量越大，亦即内部延迟因数的取值也会越大，同时越是限制性能。反之，越好的网表质量保险余量越小，亦即内部延迟因数的取值越接近实体，同时性能限制越小。

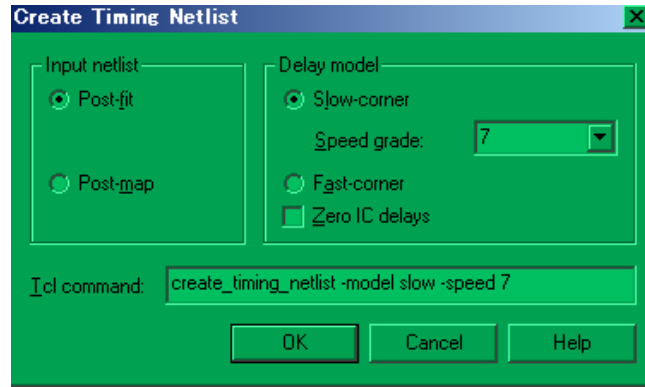


图 7.7.15 worst case 网表 grade 7。

实验十五告诉我们，即使是优化以后的 worst case 网表，时序也不一定合格 ... 此外我们也没有理由选择 best case 网表，因为保险余量很小，因此我们可以选择 worst case，grade 7 看看（ep2c8q208c8 不支持 grade 6），如图 7.7.15 所示。

	Clock	Slack	End Point TNS
1	SDRAM_CLK	0.142	0.000
2	U1 altp11_component pll clk[0]	4.588	0.000

图 7.7.15 worst case 网表 grade 7 造就的 setup 结果（编译器优化之后）。

完后，我们会发现 TimeQuest 不再爆红，而最差的建立余量也是 0.142ns，如图 7.7.15 所示。就这样我们可以安心结束分析了。

实验十五既是一个综合练习，也是一个总结，为了分析 sdram 模块的时序是否合格，首先我们收集各种外部延期信息，然后将它们转换为延迟时间。过后再用各种约束命令告诉 TimeQuest 相关的延迟信息。虽说优化编译器的设置会稍微提高时序的合格率，不过最关键的还是不同质量的网表，没有 grade 等级的 worst case 网表是最保险的网表，但是时就是不容易合格，不过我们不一定要选择 best case 网表，结果最佳的选择自然而然就是含有 grade 等级的 worst case 网表，最终我们可以取得五十五十的平衡结果。

总结：

这一章的前面虽大部分都在讨论供源时钟的声明方法 ... 笔者认为供源时钟是学习 TimeQuest 的其中一个重点吧，因为很多时候为了经济，外部模型的时钟源都由 fpga 提供，不过以笔者看来供源时钟至少有 3 个种类，亦即直接驱动，寄存器驱动，还是 pll 驱动。

嘛 ... TimeQuest 本来就是笨蛋 如果不告诉他有关供源时钟的存在，它就会看不见 ... 除了直接驱动的供源时钟以外，寄存器驱动或者 pll 驱动的供源时钟，声明过程会比较猥琐一点，而声明供源时钟的相关约束命令就是 `create_generated_clock`。

除此之外，我们还学习到一些零碎的内容，如 `tRise` 与 `tFall` 或者有关 io 口的电容性。一般上相关的硬件手册取出引脚的电容性之余，同时也会取出 `tRise` 与 `tFall` 的时间。有 `tRise` 和 `tFall` 的实际时间当然是最好不过的，不过也有例外的时候 ... 那时候，我们就要自己手动将电容性计算出 `tRise` 与 `tFall` 的实际时间。不过笔记取出的粗略方法是没有任何正确性可言，呵呵呵！

这一章重头戏当然是 sdram 的约束实例了。菜鸟时候的笔者，这关是跨不了的一道巨门，没办法之下笔者也只能一步一步往上爬，最后发觉到 ... 原来也不过如此而已。做人最重要就是要有耐性，耐性才是成功的关键，呵呵！有点老王卖瓜了。实验十五有一点遗憾，那就是没有对 `tRise` 与 `tFall` 进行约束，不这样做是有诸多原因的 ... 不过一般上，`tRise` 与 `tFall` 都不会过度考虑在约束中，即使有，也是像笔者这种贱骨头才会行动而已。

基本上从第一章到第七篇为止，TimeQuest 的正篇我们已经学习完毕，所谓正篇除了 TimeQuest 的基本知识以外，还有就是 Constraint 菜单下的相关约束命令，不过笔者被迫除去 `set_clock_group`，没法子呀 ... 就是没有可举例的实例，因为 `set_clock_group` 必须关系到多时钟设计。

后语

为时数年的学习旅行终于告一段落了，为了理解 TimeQuest 笔者不知道走过多少知识的城镇小乡，入眠时也不知看无数次夜中漫天的概念星星，它们不停一眨一闪，耀眼的星光不停诱出心底深处的思路与念头。电脑为伴的旅程，它会帮助笔者记录所有灵感，就这样字字成行，行行成段，段段成页，页页成章，章章成本，这本笔记就这样编辑而成。

完后，巨大的疲惫感虽然胜过一时的兴奋感，不过很欣慰自己想法自然以黑字白字储存着，这是真实的成长记录，笔者感觉自己不知不觉之间更加接近 FPGA 的世界，因为“静态时序分析”这块遮盖眼前的面纱已经脱落，因此视觉又一步扩大和清晰。笔者真想大声叫喊，然后将心中所有的感动，经由大气传达致世界各处。

呵呵！笔者偶尔也想这样表达心情，读者看着笑笑就好。这本笔记终于编辑完毕了，笔记的内容虽然只包含 TimeQuest 的篇内容，不过已经足够了，因为我们已经经由 TimeQuest 去了解物理时序的种种，好使自己更进一步触及 FPGA。这本笔记的前生是当初一时心血来潮所写过的瞎搞 TimeQuest。瞎搞 TimeQuest 事实上称不上学习笔记，顶多只是笔者的涂鸦笔记而已。

笔者经过思虑以后，总觉得 ... 既然有闲情写涂鸦笔记，还不如又认真一回写出像样一点的学习笔记。这本笔记有许多内容都是笔者在一年前累计的涂鸦和素描，经过重新编辑一番以后，再稍微添加几笔，有模有样的学习笔记就出来了，不过笔记的内容完全不按官方的资料去编辑，呵呵！管它什么马，只要跑得快就是好马，不是吗？

不管读者讨厌或者喜欢，想必与笔者无关吧？笔者完全也是按照自己的心情去编辑而已，简单说笔记是为自己而编辑。内容喜欢与否都请笑笑而过吧，笔者的所作所为一切都是为了 Verilog HDL 的宏愿而已，所以说 TimeQuest 不过是在学习 Verilog HDL 的路途中，顺手牵一把的东西而已。不知这是不是所谓的醉翁之意吗？

好了，好了 ... 感叹就到此为止吧！这本笔记只是描述 TimeQuest 的正篇内容而已，不过对付一般的应用已经却却有余了。看见后语这也意味着笔记的完结，想必读者会感觉一丝寂寞和舍不得吧？读者千万别这样感触。笔者只是一支指明灯，作用就是照耀迷途的路人还有为他们指向大概的方位而已，未来的路上读者还要靠自己努力。

往后如果足够的资料的话，再考虑编辑副篇也不迟 ... 不过未来的事情不是人人都可以预测的，借老子的话说：“让河水顺其自然的流动吧，少年！”。有机会的话，再实现当初的想法也不迟。

就这样，有缘再见。